Thesis for The Degree of Doctor of Philosophy

Haskell for Secure, Embedded, and Distributed Programming

High-level abstractions for low-level platforms

Robert Krook

Department of Computer Science and Engineering
Chalmers University of Technology | University of Gothenburg
Gothenburg, Sweden, 2026

Haskell for Secure, Embedded, and Distributed Programming
High-level abstractions for low-level platforms

Robert Krook

© Robert Krook, 2026
except where otherwise stated.
All rights reserved.

Acknowledgements, dedications, and similar personal statements in this thesis, reflect the author's own views.

ISBN 978-91-8103-496-7
Doktorsavhandlingar vid Chalmers tekniska högskola, Ny serie nr 5953.
ISSN 0346-718X
https://doi.org/10.63959/chalmers.dt/5953

Department of Computer Science and Engineering
Division of Computing Science
Chalmers University of Technology | University of Gothenburg
SE-412 96 Göteborg,
Sweden
Phone: +46(0)31 772 1000

Printed by Chalmers Digitaltryck,
Gothenburg, Sweden 2026.

“När en blomma bryts av i sin vackraste blom,
då blir marken så trist och så fruktansvärt tom.

Men kanske ändå att det någonstans finns
en äng eller undangömd skreva,
dit blommor som brutits för tidigt får komma
och fortsätta växa och dofta och leva. . . ”
- Atle Burman



Haskell for Secure, Embedded, and Distributed Programming

High-level abstractions for low-level platforms

Robert Krook

Department of Computer Science and Engineering
Chalmers University of Technology | University of Gothenburg

Abstract

Can Haskell's abstractions and compiler infrastructure target execution environments the language was not designed for, such as resource-constrained microcontrollers, trusted execution environments, and distributed systems?

The programming language Haskell has many appealing qualities: a type system that lets programmers state and check invariants, purity that lets them reason about program transformations, and abstractions that express their intent concisely. Compiled with GHC, Haskell achieves excellent performance, supported by an advanced runtime system. That runtime, however, assumes a rich operating system and plentiful resources. A microcontroller offers neither, an enclave restricts which system services its code may use, and a distributed system needs to move computations between machines, something GHC cannot do. In each case, a boundary separates what Haskell expects from what the platform provides.

The four papers in this thesis move Haskell across this boundary with two complementary strategies. The first transforms the program before deployment. Scoria is an embedded language for real-time programs whose compiler generates C, so that microcontrollers run Haskell programs without hosting a Haskell runtime. HasTEE describes both sides of an enclave application in one program and uses the type system to partition it and to keep confidential data inside the enclave. The third paper extends partitioning to distributed programs with many participants, using only GHC's rewrite rules and specialisation. The second strategy adapts the runtime. HasTEE combines the two strategies: its trusted part runs on a modified GHC runtime inside the enclave. CloudMicroHaskell builds on the MicroHs runtime, which represents code and data uniformly, to send computations between heterogeneous devices.

Together, the papers show that Haskell can target all three environments. The boundary does not disappear, but each paper makes it something the language, compiler, or runtime can manage.

Keywords

Haskell, Functional Programming, Distributed Programming, Program Partitioning, Code Mobility, Embedded Programming, Synchronous Programming, Trusted Execution Environments, Confidential Computing

List of Publications

Appended publications

This thesis is based on the following publications:

[**Paper I**] **R. Krook**, J. Hui, BJ. Svensson, S.A. Edwards, K. Claessen, *Creating a Language for Writing Real-Time Applications for the Internet of Things*
Proceedings of the 20th ACM-IEEE International Conference on Formal Methods and Models for System Design (MEMOCODE) (October 2022), 1-20.

[**Paper II**] A. Sarkar, **R. Krook**, A. Russo, K. Claessen, *HasTEE: Programming Trusted Execution Environments with Haskell*
Proceedings of the 16th ACM SIGPLAN International Haskell Symposium (Haskell) (September 2023), 72-88.

[**Paper III**] **R. Krook**, S. Hammersberg, *Welcome to the Parti(tioning) (Functional Pearl)*
Proceedings of the 17th ACM SIGPLAN International Haskell Symposium (Haskell) (September 2024), 27-40.

[**Paper IV**] **R. Krook**, L. Augustsson, *CloudMicroHaskell: Direct-Style Distributed Haskell via Runtime Graph Serialisation*
Proceedings of the 19th ACM SIGPLAN International Haskell Symposium (Haskell) (August 2026), 34-64.

Other publications

The following publications were published during my PhD studies, or are currently in submission/under revision. However, they are not appended to this thesis, due to contents overlapping that of appended publications or contents not related to the thesis.

[a] N. Valliappan, **R. Krook**, A. Russo, K. Claessen, *Towards Secure IoT Programming in Haskell*
Proceedings of the 13th ACM SIGPLAN International Symposium on Haskell (September 2020), 136-150.

[b] A. Sarkar, **R. Krook**, BJ. Svensson, M. Sheeran, *Higher-Order Concurrency for Microcontrollers*
Proceedings of the 18th ACM SIGPLAN International Conference on Managed Programming Languages and Runtimes (September 2021), 26-35.

[c] **R. Krook**, N. Smallbone, BJ. Svensson, K. Claessen, *Quickercheck: Implementing and Evaluating a Parallel Run-Time for QuickCheck*
Proceedings of the 35th Symposium on Implementation and Application of Functional Languages (IFL) (June 2024), 1-12.

Authorship Statement

The following statement summarises my personal contribution to the papers included in this thesis. Where relevant, co-authors have agreed with it.

[**Paper I**] Creating a Language for Writing Real-Time Applications for the Internet of Things.

This paper was co-authored with John Hui, Bo Joel Svensson, Stephen A. Edwards, and Koen Claessen. Stephen had designed and implemented a small runtime system for the sparse synchronous model, but had no compiler to write programs for it with. Additionally, he had not designed an IO interface for the runtime. I implemented most of the Scoria compiler. The group discussed and designed an IO interface, which I then implemented. I also tested the compiler using property-based testing. I implemented a generator for arbitrary, well-formed programs, and a shrinker that produces syntactically smaller but well-formed programs. John Hui and I implemented and ran the experiments, and wrote the evaluation section together. I wrote most of the remaining sections.

[**Paper II**] HasTEE: Programming Trusted Execution Environments with Haskell.

This paper was co-authored with Abhiroop Sarkar, Koen Claessen, and Alejandro Russo. The research idea came from Abhiroop. Abhiroop implemented the patched GHC runtime that executes Haskell inside the enclave. Before we settled on this approach, I explored two alternatives: running the Hugs interpreter inside the enclave, and running code from GHC's WebAssembly backend on a WebAssembly interpreter inside the enclave. I implemented the encrypted password wallet and data clean room case studies, and worked on the federated learning one together with Abhiroop. I implemented the restricted IO monad, including support for random number generation and for reading and writing trusted and untrusted files. Abhiroop and I ran the evaluation and wrote the paper together. Koen and Alejandro gave input throughout the project and provided feedback on the draft paper.

[**Paper III**] Welcome to the Parti(tioning): (Functional Pearl).

This paper was co-authored with Samuel Hammersberg. The research idea came from me, and I did the entire implementation. I did most of

the evaluation, and Samuel helped me write and run examples. I wrote the paper on my own.

[**Paper IV**] CloudMicroHaskell: Direct-Style Distributed Haskell via Runtime Graph Serialisation.

This paper was co-authored with Lennart Augustsson. I came up with the idea of sending functions and built a simple prototype, after which my advisor, Koen Claessen, pointed out the connection to Cloud Haskell. I implemented CloudMicroHaskell, with periodic feedback from Lennart. Lennart guided me through the serialisation mechanism in MicroHs, and through making my own additions to MicroHs, such as non-blocking IO. He also implemented other changes to the compiler that I needed. I implemented the case studies and did all of the evaluation. I wrote most of the paper, and Lennart wrote the sections on graph serialisation and MicroHs.

Acknowledgement

There are countless people who deserve my gratitude and without whose support I would not have gotten here. For fear of forgetting anyone, I leave most of you out. If you feel that you should have been named here, rest assured that I absolutely agree, and that I am endlessly grateful for your mentorship, collaboration, or friendship. There are, however, some people that I want to acknowledge explicitly.

First, I want to thank my friend and advisor, Koen Claessen. You have made these years some of the most enjoyable of my life. I have felt seen and supported since day one, and I cannot imagine a better advisor. I admire your curiosity and ability to find interesting problems to solve, and you can rest assured that I will keep showing you the fun things that I hack on.

Bo Joel Svensson, my co-advisor. I refer to you as Hackerman, because aside from being a strong researcher, you are an incredible engineer. I have learned so much from you, and hope to continue learning. You have helped me turn research ideas into working systems in ways I could not have managed alone.

John Hughes, my co-advisor. You have inspired me to do good research and to enjoy doing it. You have also taught me to take pleasure in writing clearly about what I have found.

Mary Sheeran, my co-advisor. You have helped me look at the bigger picture of my academic career, something that is easily forgotten in the midst of doing a PhD. You have inspired me to improve myself not just as an academic, but also as a teacher.

Lennart Augustsson, for inspiring me. Your curiosity and enthusiasm have always made it a joy to show you what I have been working on. Thank you for your endless patience in answering my many questions about MicroHs.

To my fellow PhD students. Whether our time at CSE overlapped for years or only briefly, I am glad we shared it. You have been wonderful friends as well as colleagues, and you have made these years happier.

I want to thank three of you in particular. Abhiroop Sarkar, for blinking *so* many LEDs with me. No one blinks LEDs quite like you do.

Henrik Jansson Valter, thank you for being such a wonderful officemate and friend. I have been lucky to share both an office and these years with you.

Prabhat Kumar Jha, for your good friendship and interesting discussions. I

look forward to having many more dosas together.

To the Octopi students: Nachi, Agustin, Abhiroop, and Jeremy. Thank you for being such amazing friends and talented colleagues. I miss working together.

To every visiting researcher. Meeting you has left a lasting impression on me, and I feel fortunate to have had the chance. I want to single out Joachim "Scoobert" Kristensen, whose contagious enthusiasm has led to many interesting discussions.

To everyone at CSE, thank you for making this such a wonderful place to work. I have always looked forward to coming in, even on Mondays. Conversations by the coffee machine have moved easily from exciting research problems to what we did over the weekend. Whenever I needed to learn something, someone nearby would gladly make time to help. I could turn to senior colleagues as readily as to my supervisors. Your curiosity, generosity, and friendship have made CSE much more than a workplace.

To my parents, thank you for your endless support, even though neither of you is entirely sure what I do. To my twin brother and best friend, Christoffer, thank you for always being there for me, even when "there" means Stockholm.

Finally, I want to thank my wife and children, who are dearest to me.

Therese, my wife. You have supported me without hesitation throughout these years. You keep my feet on the ground, and I would surely have gone crazy without you. I miss you every minute that I am not near you. This thesis is as much yours as it is mine. I love you.

Astrid, marrying your mother was the best decision I ever made. I love her dearly, and I feel lucky and honoured to be your bonus father. I fondly remember how every visit you made to see me at work had to include a hot chocolate from the coffee machine.

My son, Olof. Even though we never got to take you home from the hospital, I carry you in my heart and think about you every day. I don't have the words to describe what you mean to me or how much I wish you were here with us.

To my daughter Ingrid, Olof's little sister. These words are for you.

Ingrid. You came when I needed you most. You taught me that grief and joy can coexist in my heart. You have reminded me that there is more to life than work. I am grateful for everything you have already given me and for everything we have yet to share.

Right now, as I write the final words of a thesis that has occupied more than half a decade of my life, you are sleeping soundly on my chest. The sound and rhythm of your breathing make this moment more meaningful than I could ever tell you.

I love you with all my heart.

Contents

Abstract iii

List of Publications v

Authorship Statement vii

Acknowledgement ix

I Summary 1

1 Introduction 3

2 Background 7
2.1 Haskell 7
2.2 Embedded Domain-Specific Languages 12
2.3 Synchronous Programming 14
2.4 Security & Confidentiality 20
2.5 Multiparty Programming 25
2.6 Distributed Haskell 29
2.7 Embedded Haskell 39

3 Contributions 45
3.1 Transforming Programs Before Deployment 45
3.2 Adapting Runtime Systems 47

4 Conclusions 49

5 Chronology 51

6 Future Work 55
6.1 Multiparty Haskell 55
6.2 Haskell on Microcontrollers with Trusted Execution Environments 56
6.3 Performance and Resource Predictability in MicroHs 57
6.4 Partial Evaluation of Combinator Graphs 59
6.5 Preserving Scoria's Guarantees in a Common Runtime 60

Bibliography **63**

II Appended Papers 71

Paper I - Creating a Language for Writing Real-Time Applications for the Internet of Things **73**

Paper II - HasTEE - Programming Trusted Execution Environments with Haskell **115**

Paper III - Welcome to the Parti(tioning) (Functional Pearl) **155**

Paper IV - CloudMicroHaskell - Direct-Style Distributed Haskell via Runtime Graph Serialisation **187**

Part I

Summary

Chapter 1

Introduction

Can a single high-level program be written once and automatically partitioned, compiled, and deployed across a heterogeneous network of devices consisting of microcontrollers, trusted execution environments, and cloud nodes, with the type system statically enforcing the safety properties of each node?

This question is easy to pose but difficult to answer, and it has driven much of the work in this thesis. Before examining what it entails, we must explain why it is worth asking.

Many application domains require a network of devices for the application to be executable. As an example, consider a home alarm system. There are sensors distributed throughout a home to detect physical violations of some property, such as "*is the door closed?*". With the sensors being distributed, they must form a network to inform each other of violations. The homeowner typically also has a terminal from which they can configure the sensors without having to physically visit them.

If one sensor fails, the other sensors may remain operational, so the failure is isolated rather than causing the entire network to fail. Nevertheless, the application can no longer monitor the property assigned exclusively to the failed sensor. Fault detection allows the system to notify the homeowner instead of failing silently, while true fault tolerance for that property would require some recovery mechanism, such as restarting the sensor or transferring its responsibility to a redundant component.

This example shows why an application may require a network of devices and why detecting and responding to faults matters.

Often devices in a network are heterogeneous. By this we mean that the devices are not all identical. Even devices serving as sensors may differ considerably. Some have plentiful memory and powerful processors, whereas others have little memory and much slower processors. Additionally, some devices are powered by a battery and must remain operational for perhaps years at a time. Those devices need to manage power consumption carefully.

Developing the software to run on a device requires knowledge of the capabilities of that device's hardware, as well as how the other devices in

the network operate and communicate. The software for a device is typically implemented as a stand-alone application, with tools and languages tailored to the type of device. The software for a microcontroller is typically written in C, whereas the software for a larger, more powerful machine may be written in a higher-level language. The software also has to implement the communication protocol, being able to produce and receive the structured messages that the network agrees to use. Even if the communication protocol is correctly implemented, at runtime, devices may enter incompatible states where they are unable to communicate. There are many opportunities for bugs to sneak into the software.

By heterogeneous devices we may refer not just to differences in memory and CPU capabilities, but also to differences in trust boundaries. During deployment, a single device can use hardware-supported *Trusted Execution Environments* to ensure confidentiality of code and data, keeping secure data and the operations over that data in an *enclave*, while the rest of the application executes in normal memory. While such hardware enclaves are good for protecting data and code, they are only as good as the programmer. It is very possible that the programmer accidentally writes software with bugs, where the secrets may leak to a consumer outside of the enclave. These enclaves are typically programmed in C, where the programmer needs to interact with many low-level details, increasing the chance that the final software contains bugs.

These problems bring us back to the opening question. The question asks whether we can write a single program that is then deployed on a network, on several devices that communicate. If we can write a single program that describes the behaviours of several endpoints in a network, we can use static analysis techniques to e.g. verify that the devices will never enter a state where they are deadlocked. Furthermore, the interactions are type-checked, ensuring that the producers and consumers of messages agree on the type of messages being sent.

While there are several upsides to this kind of programming, there are significant downsides as well. Unless we employ program partitioning, the compiled program will contain the code for all endpoints in the network. This is a problem for devices without a lot of memory, where there is not enough space for a binary that contains the instructions for more than one program. As a second example of why this is a problem, we reiterate that some devices may have strict requirements regarding confidentiality of both data and code, and that one device should not have access to the code that was loaded onto another device.

The opening question has guided the work in this thesis. This thesis does not answer it definitively, but it addresses several of its central technical problems. This motivates the following narrower thesis statement:

> *This thesis investigates the feasibility of using Haskell's abstractions and compiler infrastructure to target execution environments the language was not necessarily intended for, such as resource-constrained microcontrollers, trusted execution environments, and distributed systems, and demonstrates that tools such as embedded domain-specific languages, type-directed partitioning, and graph serialisation are sufficient to bridge the gap.*

Why Haskell? The suitability of Haskell for the problems studied in this thesis comes not just from its static types or support for higher-order functions, but from the combination of its expressive type system, its pure functional core, and its compiler infrastructure. With its purity we can separate computation from effects, which lets us reason about program transformations with confidence. The type system allows us to encode domain-specific properties, such as where a computation is executed or which operations are allowed, and check them statically. Its support for higher-order functions, algebraic data types, type classes, and monadic notation makes it possible to provide domain-specific abstractions while still benefiting from the type checker, module system, and abstraction mechanisms.

The compiler ecosystem provides mechanisms that we in this thesis repurpose for unconventional execution environments. We use GHC's rewrite rules and specialisation syntax to partition programs. We deploy a modified GHC runtime on SGX-enabled Intel CPUs, and we leverage MicroHs's graph-reduction runtime to deploy Haskell programs on microcontrollers. The MicroHs runtime additionally lets us serialise both code and data, allowing us to move a suspended computation from one machine to another, in the pursuit of distributed Haskell programming.

Haskell is not just a language that we use for the implementations of the artefacts in this thesis. Its abstractions, type system, and compiler architectures are the tools being investigated.

The chapters that follow develop this argument in three steps. The background chapter introduces the language concepts and execution environments needed to understand the papers. The contributions chapter then relates the papers to the thesis statement through two complementary strategies: transforming programs before deployment and adapting runtime systems to new targets. The conclusions chapter gives a qualified answer to the thesis statement. Chapter 5 then steps outside this argument and recounts, in the first person, how the work came about and where it is heading. The final chapter identifies what remains before the broader question that motivated this thesis can be answered.

Chapter 2

Background

2.1 Haskell

This thesis centres on Haskell, a statically typed, purely functional programming language with non-strict semantics. This section introduces the language properties and implementation techniques needed to understand the work presented in the thesis.

Haskell is pure, so evaluating an expression cannot itself produce an externally observable effect. A pure expression may safely be replaced by its value without changing the observable behaviour of the program, a property known as *referential transparency*. Referential transparency allows programmers and compilers to transform programs by equational reasoning while preserving their meaning. Effectful computations are instead represented explicitly by types such as `IO a`, which describes a computation that may perform input or output before producing a value of type `a`.

Haskell compilers statically check the type of every expression, although they can infer most types without explicit annotations. Features such as algebraic data types, parametric polymorphism, type classes, and higher-order functions allow types to describe domain-specific interfaces and constraints. Later chapters use these features to express properties such as where a computation may execute and which operations it may perform.

Haskell has non-strict semantics, which implementations ordinarily realise using call-by-need evaluation. An expression is evaluated only when its result is demanded, and that result is then shared between subsequent uses. Implementations generally realise this sharing with heap-allocated closures connected by pointers, forming a graph of the live computation. For example, computing the length of a list requires evaluating the structure of the list, but not its elements. This evaluation model is relevant to both of the implementations discussed below. GHC implements laziness using the Spineless Tagless G-machine, while MicroHs reduces a mutable combinator graph.

Since Haskell was conceived, there have been many different compilers and interpreters, each with its own approach to implementation. However, rather than describing them here, we focus primarily on the two Haskell

implementations that matter to the work described in this thesis, GHC and MicroHs. The remainder of this section introduces the features of each compiler that are relevant to the included papers.

GHC The *Glasgow Haskell Compiler* [1] is the de facto standard Haskell compiler. It is a heavily optimising compiler that has been in development for over thirty years. GHC is used in both academia and industry. For example, Mercury maintains roughly two million lines of Haskell in its production codebase [2].

GHC achieves good performance on two fronts: the code it generates is tailored to its runtime system, and the compiler applies a battery of non-trivial optimisations before that code is generated. GHC compiles a Haskell program to native machine code executed by a runtime system based on the *Spineless Tagless G-machine*. After optimisation, top-level functions appear as closed STG bindings, traditionally called *supercombinators*. Functions and thunks are represented as closures containing a native-code entry point and, for local closures, any captured free variables. Evaluation proceeds by entering that code through the closure.

The generated code is intimately coupled to a runtime system of over fifty thousand lines of code. The runtime implements everything that a lazy functional language needs. Because lazy functional programs allocate many short-lived heap objects, the garbage collector is tuned for high allocation rates. The scheduler multiplexes Haskell's green threads onto a collection of operating system threads, with an IO manager that makes sure that blocking calls only stall a Haskell thread rather than an OS thread.

GHC produces high-performance code not only because of a clever runtime system, but also because the *compiler* applies many non-trivial optimisations before generating code. Describing all of these optimisations is beyond the scope of this thesis, but we do want to highlight one pass in particular: the pass that applies rewrite rules.

Rewrite Rules are used by GHC to rewrite terms during compilation. A library developer can write their own rewrite rules, injecting optimisation behaviour into the compiler. A common use case for rewrite rules is to do short-cut fusion, where intermediate data structures are eliminated. The composition law for `map` tells us that `map f (map g xs)` is equivalent to `map (f . g) xs`, but GHC cannot infer this law from the definition of `map` alone. We can inject this knowledge into the compiler by stating it as a rewrite rule in our source code:

```
1  {-# RULES "map/map" forall f g xs .
2        map f (map g xs) = map (f . g) xs #-}
```

The rule instructs the compiler to replace any instance of the left-hand side with the corresponding instance of the right-hand side. GHC checks that the rule is well typed, but not that its two sides have the same meaning. Phase annotations coordinate rule activation with inlining, which may otherwise prevent a rule from matching. Rewrite rules need not be confluent, so when several rules match, their order of application may affect the final term. GHC

also does not check that rules terminate. Cyclic rules can therefore make optimisation loop. The paper in this thesis on program partitioning, *Welcome to the Parti(tioning)* [3], describes rewrite rules and their constraints in much greater detail.

MicroHs MicroHs [4] is a Haskell compiler developed by Lennart Augustsson. It implements an extended subset of Haskell 2010: some parts of the standard are omitted, while several features beyond the standard are supported.

Unlike GHC, MicroHs does not compile user-defined functions to native machine code. Instead, it emits a serialised combinator graph that is loaded and evaluated by a small runtime written in C. This design sacrifices execution speed in exchange for a substantially smaller and more portable runtime, making it possible to run Haskell programs on resource-constrained platforms such as microcontrollers.

The combinator representation has another property that is important to this thesis. Because the graph represents both code and data, subgraphs representing values and suspended computations can be serialised and reconstructed by another compatible runtime. This property provides the code-mobility mechanism used by the fourth paper in this thesis [5].

The use of combinators as an implementation technique is not new. Combinatory logic originated in a lecture given by Schönfinkel to the Göttingen Mathematical Society in 1920. The lecture was prepared for publication by Heinrich Behmann and appeared in 1924 [6]. Curry developed the theory further in 1930 [7]. In 1979, Turner demonstrated that functional programs could be translated into combinators and evaluated by reducing a shared graph in place [8]. Subsequent work pursued faster execution through increasingly specialised graph-reduction machines, eventually leading to the Spineless Tagless G-machine used by GHC. MicroHs instead remains closer to Turner's direct combinator-based approach.

To illustrate this representation, we use the following small expression language. The constructors `Var`, `App`, and `Lam` represent lambda terms, while `Lit` and `Prim` represent integer literals and primitive operations.

```
data Exp
  = Var String
  | App Exp Exp
  | Lam String Exp

  | Lit Int
  | Prim String
  deriving (Eq)
```

Bracket abstraction eliminates a lambda abstraction by translating its body into applications of primitive combinators. The following simplified transformation, based on Turner's presentation, uses the combinators `S`, `K`, and `I`.

```
bracket :: String -> Exp -> Exp
bracket x e = case e of
```

```
      Var y | x == y -> i
      App e1 e2      -> App (App s (bracket x e1)) (bracket x e2)
      Lam y e        -> bracket x (bracket y e)
      e              -> App k e
  where
    s :: Exp
    s = Prim "S"

    k :: Exp
    k = Prim "K"

    i :: Exp
    i = Prim "I"
```

Applying `bracket x e` eliminates `x` from `e`, although other free variables may remain. Eliminating every abstraction from a closed program eventually produces an expression containing neither `Var` nor `Lam` nodes. The resulting representation requires no variable names and therefore avoids variable capture.

Consider the input program:

```
suc :: Int -> Int
suc x = 1 + x
```

Represented using the data type `Exp`, the body of `suc` is

```
App (App (Prim "+") (Lit 1)) (Var "x")
```

Abstracting the body over `x` produces an equivalent expression that does not mention the variable. For readability, we omit the `App` constructor and write application by juxtaposition, adding parentheses where necessary. We also write `S` rather than `Prim "S"`.

```
S (S (K +) (K 1)) I
```

The behaviour of the three combinators is given by the following reduction rules:

$$I\ x \longrightarrow x,$$
$$K\ x\ y \longrightarrow x,$$
$$S\ f\ g\ x \longrightarrow f\ x\ (g\ x).$$

In a term-rewriting implementation, the final rule appears to duplicate `x`. In a graph-reduction implementation, both occurrences instead point to the same node. This sharing is essential for implementing Haskell's call-by-need evaluation strategy.

MicroHs uses more than the three combinators shown here and serialises the resulting graph in postfix form. For this example, it produces the following output:

```
v8.4
0
S S K + @@ K #1 @@@ I @
```

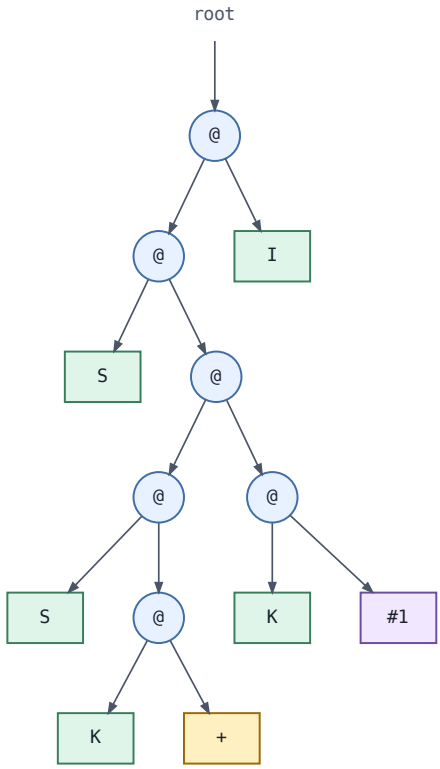


Figure 2.1: The combinator graph the MicroHs runtime builds in memory from the serialised representation of `suc`. Application nodes are drawn as circles; combinators and primitive operations are drawn as leaves.

Here, `v8.4` is the format version and `0` is the number of labels used (`suc` is not recursive, so no labels are needed). The symbol `@` denotes postfix application. `f x @` represents the application of `f` to `x`. Thus, `S K + @@ K #1 @@@` represents `S (K +) (K 1)`, and the final `I @` applies that expression to `I`.

The MicroHs runtime, written in C, parses this output and constructs the graph shown in fig. 2.1.

The constructed graph represents both executable code and data. During execution, the runtime reduces the graph in place, replacing nodes with their results while preserving sharing between subgraphs.

The runtime can also perform the inverse of loading a combinator graph: it can traverse the subgraph reachable from a particular node and serialise it. Since such a subgraph may contain both executable combinators and captured data, this allows values such as functions and unevaluated computations to be transferred to another compatible MicroHs runtime. The receiving runtime can reconstruct the graph and continue its evaluation.

This mechanism does not amount to checkpointing an arbitrary running

process. Runtime-owned resources, such as threads, open handles, and foreign pointers, cannot in general be transferred as part of the graph. Nevertheless, pure values and suspended computations can be moved without requiring the programmer to perform closure conversion explicitly. The fourth paper in this thesis uses this property to implement remote spawning and message passing in a distributed Haskell system.

2.2 Embedded Domain-Specific Languages

A domain-specific language (DSL) [9] provides abstractions and notation tailored to a particular class of problems, rather than attempting to support general-purpose programming. SQL provides notation for querying and modifying relational databases. LaTeX describes the structure and typesetting of documents such as this thesis. Make describes dependencies between software components and the commands required to build them.

Why use a DSL when the same task could be programmed in a general-purpose language? The advantage lies not in additional computational power, but in making domain concepts explicit. By restricting programs to domain-relevant operations, a DSL can enable specialised validation, analysis, optimisation, and code generation. By adopting notation and concepts from the application domain, it can also let programmers express solutions in the language of that domain.

Implementing a standalone DSL, however, is costly. It typically requires a parser and either a compiler or an interpreter. A practical language may also need a type checker, module system, debugger, and editor support. Developing this infrastructure may be disproportionate to a small collection of domain-specific abstractions. An EDSL avoids much of this cost by reusing the compiler and tools of an existing host language.

An *Embedded Domain-Specific Language* (EDSL) is implemented within a host language, usually as a library. Its chief benefit is that it can reuse the host language's parser, type checker, and other infrastructure. Programmers can also use the abstraction mechanisms of the host language to compose and extend embedded programs. EDSLs therefore support rapid prototyping before deciding whether a bespoke language and toolchain are worth implementing.

Two examples illustrate the range of applications and implementation strategies that embedding supports. QuickCheck [10] is a Haskell EDSL for specifying and randomly testing properties of programs. Its interface provides combinators for defining both generators of test data and properties that the generated values should satisfy. QuickCheck repeatedly samples the generators and evaluates the properties on the resulting inputs. When a property fails, QuickCheck reports a counterexample and attempts to reduce it to a simpler one. Copilot [11] is a Haskell EDSL for specifying stream-based runtime monitors. Its compiler translates these specifications into constant-time and constant-space C programs suitable for embedded systems. These examples also lead into a central design choice, whether embedded constructs are given their meaning directly in Haskell or used to build a representation that can be

analysed and compiled.

EDSLs are commonly classified as shallow, deep, or a mixture of the two [12].

To distinguish shallow from deep embeddings, it is useful to separate a program's representation from its meaning. A syntactic representation is inspectable data that describes a program, such as source text, an abstract syntax tree, or a compiler intermediate representation. The program's semantics determines the value it denotes or the behaviour it exhibits. A compiler translates one syntactic representation into another while preserving that meaning, whereas an interpreter realises the meaning by executing the representation.

In a shallow embedding, each construct of the embedded language is represented directly by its meaning in the host language. In a deep embedding, the constructs instead build an explicit syntactic representation of the embedded program. That representation can subsequently be inspected, transformed, interpreted, or compiled.

Haskell is particularly well suited as a host language for EDSLs. Algebraic data types provide a natural representation of embedded syntax, while Haskell's type system can rule out certain malformed programs as they are constructed. Purity supports equational reasoning about interpreters and transformations defined over this representation. A domain-specific monad can distinguish embedded computations from ordinary host-language computations and expose only the operations permitted by the EDSL. Monadic notation can additionally hide the mechanics of constructing and sequencing syntax, allowing a deep embedding to present an imperative-looking interface when that suits the domain.

Embedding a DSL entails costs as well as benefits. Because an EDSL must conform to the syntax of its host language, its notation may express the concepts of the domain less naturally than a standalone language could. Users must also understand enough of the host language to use its abstraction mechanisms effectively. In a deep embedding, the boundary between computations performed by the host language and programs constructed in the embedded language may be difficult to recognise. Moreover, errors are reported by the host compiler and may expose implementation details rather than explain the problem in terms of the application domain. Deep embeddings incur an additional implementation cost because their explicit syntax must be interpreted, analysed, or compiled. Shallow embeddings avoid much of this machinery, but generally provide fewer opportunities to inspect, transform, and compile embedded programs.

EDSLs appear throughout this thesis. The first paper presents Scoria [13], a deep EDSL that generates C code for a small runtime on resource-constrained MCUs. The second paper presents HasTEE [14], which combines deep and shallow embedding techniques to express programs with confidentiality constraints. The third paper [3] studies how GHC can partition programs written using the Haste App and HasChor EDSLs. The fourth paper presents a shallow EDSL for concurrent, communicating processes [5].

2.3 Synchronous Programming

This section introduces the execution model needed to understand the first paper of this thesis, which presents Scoria. In particular, the distinction between logical and physical time explains both the guarantees Scoria aims to provide and the timing behaviour examined in its evaluation.

Imagine designing software that controls a physical system, such as a vehicle or an industrial machine. You begin with a deterministic model that describes precisely how the system should respond to each input. Translating that model into conventional software introduces concerns that are absent from the model. You must implement interrupts, threads, and scheduling priorities, while the resulting behaviour may still depend on subtle timing details outside your control. This gap between model and implementation motivated the development of synchronous programming languages in the early 1980s, and it remains relevant to the implementation of reactive systems today.

Two ideas from the 1970s laid the groundwork for these languages, in particular for the dataflow languages Signal and Lustre. In 1974, Gilles Kahn introduced what are now called Kahn process networks (KPNs) as a model of concurrent computation [15]. Kahn wanted a parallel language simple enough to be given a precise mathematical semantics. In his model, sequential processes communicate over unbounded first-in first-out (FIFO) channels. The sequence of values that passes through a channel is called its *history*.

Because channels are unbounded, a process never blocks when it writes. A process that reads from an empty channel blocks until a value arrives, and it cannot test whether a channel is empty before it reads. A process can therefore never react to a value being late. It can only wait, so a slow writer delays its reader but never changes what the reader does next. Every channel also has exactly one writer and one reader, so no two processes ever compete to write to a channel or to read from it. As a result, the values a process writes depend only on the values it has read, and never on how fast the other processes run. This makes a network deterministic: however the processes are scheduled, every channel carries the same history.

Around the same time, and independently of Kahn, Edward Ashcroft and William Wadge developed the language Lucid [16]. They wanted a language in which programs could be verified, and they believed that a declarative language suited this task best. In particular, they wanted to prove properties of iterative algorithms, which compute by repeatedly updating variables in a loop. Their key insight was that a loop variable can be identified with the sequence of values it assumes during the iteration. A Lucid variable therefore denotes an infinite stream of values, again called a *history*, and a program is a set of equations that define these streams. Such a program can be read as a Kahn network in which the operators are processes and the variables are channels.

Lucid builds streams with a small set of operators. The equations below use two of them, written in the notation of the later Lucid book [17].

```
n = 0 fby n + 1
e = n whenever n mod 2 eq 0
s = n + e
```

The operator `fby`, read "followed by", puts a value in front of a stream, so `n` is the stream of natural numbers. The operator `whenever` keeps only those values of its first argument for which its second argument is true, so `e` is the stream of even numbers. The last equation adds the two streams pointwise, pairing the first value of `n` with the first value of `e`, the second with the second, and so on. This is where memory becomes a problem. Producing one value of `e` consumes two values of `n`, while the addition consumes only one value of `n` for each result. Read as a Kahn network, the channel that carries `n` to the addition fills faster than it empties, and it keeps growing for as long as the program runs.

Kahn networks and Lucid both let a programmer write programs that are deterministic by construction. Neither says anything about how much memory those programs need. A channel in a Kahn network may grow without bound, and the example above shows that even a three-line Lucid program can require this. As the rest of this section explains, the synchronous languages keep this determinism and restrict programs enough that they also run in bounded memory.

A synchronous program structures execution as a sequence of logical instants in which it reacts to inputs and produces outputs. Computation within each instant is treated as taking no logical time. For a given initial state and sequence of timed inputs, the synchronous languages considered here require the program to produce the same outputs at the same logical instants. Depending on the language, this requirement is enforced through causality analysis, clock checking, or a deterministic scheduling discipline.

To illustrate this model, consider a sample-driven implementation governed by a global clock that "ticks" periodically. Computations that occur within an instant are deterministic and logically happen at the same time, in a `step` function. Only when we move from one instant to another do we consider the advance of time. We model this `step` function as

```
step :: Input -> State -> (Output, State)
```

Upon every tick, the `step` function is invoked, and the state is threaded through. This is illustrated in fig. 2.2.

Moving from the simple synchronous setting we have described so far into a more realistic setting brings us to one of the canonical synchronous languages, *Esterel*. Esterel was presented by Berry et al. [18] in 1983 and uses imperative syntax to describe nested concurrent threads and control flow. These threads communicate with each other and with their environment using *signals*. At each logical instant, a signal is either present or absent and may carry a value. When a signal is emitted, its presence is immediately visible to every thread during that instant. A thread may pause and resume in a later instant, for example when a particular signal is present.

When compiling Esterel programs, the compiler will reject programs with causality problems [19]. Communicating via signals is instantaneous, and consequently the presence of one signal can depend on another, whose presence in turn depends on the first signal. Since the signal status cannot be determined without circular reasoning, Esterel rejects the program.

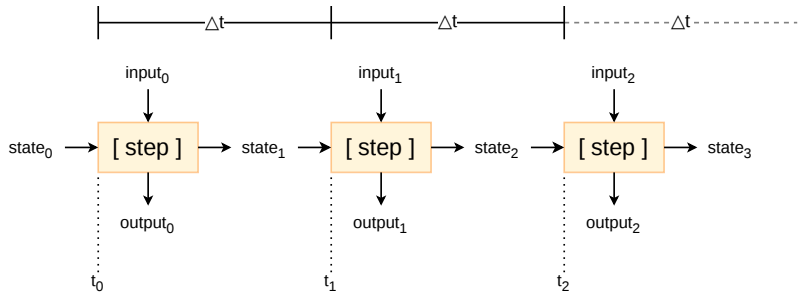


Figure 2.2: The figure illustrates how the `step` function is invoked periodically with Δt time elapsing between invocations. Within an instant, time does not advance, so throughout the first invocation of `step` the time is always t_0. As long as the invocation of `step` finishes before the next time of invocation, the observed behaviour of the system will be faithful to the synchronous model, with computation appearing to be instantaneous.

In 1986, *Signal* [20] was introduced. Signal is a *declarative*, synchronous *dataflow* language. Its intended application domain is signal processing and real-time applications.

A Signal program consists of equations relating signals. These equations are interpreted simultaneously as constraints, rather than as assignments executed in a specified order. Each signal is a potentially infinite sequence of typed values paired with a clock that specifies the logical instants at which its values are present. Different signals may have different clocks, so Signal is a *multiclock* language. Operators such as sampling introduce relationships among these clocks. The compiler applies a clock and causality calculus to determine whether these relationships admit a consistent, deterministic execution schedule.

Signal allows the programmer to describe values from previous instants. Consider the equations below:

```
y := x + 1
z := y $1 init 0
a := x when b
```

`y` is a signal that depends on the `x` signal, meaning that it inherits the clock from `x`. `z`, however, says that it holds the value of `y` in the *previous* instant. Because `y` has no preceding value in the first instant, `z` is initialised to `0`. `a` is more interesting. `a` is present only in instants in which both `x` and `b` are present and `b` has the value `True`. We say that `a` runs on a subclock of both `x` and `b`.

Our last example of a classical synchronous programming language is *Lustre*. Lustre was introduced in 1987 [21], and a more comprehensive presentation followed in 1991 [22]. It shares many similarities with Signal, but there are also differences. First, Lustre is also a *declarative* language. Like Signal, Lustre describes potentially infinite sequences of typed values, which it calls *streams*. Each stream has a clock that specifies the logical instants at which its values

are present. The compiler checks that the relationships between these clocks are consistent and rejects ill-clocked programs.

In Lustre, the programmer defines *nodes* that take the role of reusable stream transformers. Consider the node definition below:

```
node counter(reset: bool) returns (n: int);
let
  n = 0 -> if reset then 0 else pre(n) + 1;
tel
```

The input stream `reset` has the node's clock and therefore provides a Boolean value at every activation. The output stream `n` is initially `0`, returns to `0` whenever `reset` is `true`, and otherwise increments by one. The equation defining `n` is recursive because it refers to `n` itself. However, `pre(n)` denotes the value of `n` at the preceding instant of its clock, introducing a one-instant delay. Without this delay, the equation `n = n + 1` would make the current value of `n` depend on itself in the same instant, creating a causality cycle that the compiler would reject. The `pre` operator makes persistent data state explicit, while `->` specifies its initial value. Because the structure and types of a Lustre program are known statically, the compiler can determine the storage required for this retained state. Predictable memory requirements are particularly valuable in resource-constrained and safety-critical real-time systems.

Clocks are also what rule out the Lucid program we saw earlier. Written in Lustre, it becomes:

```
n = 0 -> pre(n) + 1;
e = n when (n mod 2 = 0);
s = n + e;
```

The stream `n` is present at every instant of the base clock, from which all other clocks in a program are derived. Sampling a stream with `when` produces a stream on a slower clock, so `e` is present only at the instants where `n` is even. Both `whenever` and `when` discard the odd values of `n`, but they treat the remaining values differently. In Lucid, the remaining values are renumbered, so the even numbers become the first, second, and third values of `e`. In Lustre, each remaining value stays at the instant where it was produced, and `e` has no value in the instants in between.

instant	0	1	2	3	4	···
`n`	0	1	2	3	4	···
`e` in Lucid	0	2	4	6	8	···
`e` in Lustre	0		2		4	···

The addition requires both of its arguments to be on the same clock, and since `n` and `e` are not, the compiler rejects the last equation. Because operators only combine values that are present in the same instant, no value ever has to wait for another, and no buffer is needed.

Lustre was later extended with ideas from functional programming. Beginning in 1996, Paul Caspi and Marc Pouzet developed *Lucid Synchrone* [23], a functional language in the style of ML that is built on Lustre's programming

model. Its clock calculus is expressed as a type system, so an ill-clocked program is rejected in the same way as an ill-typed one. Caspi and Pouzet used this type system to state precisely how synchronous programs relate to Kahn networks. A Kahn network needs buffers because a value may be written long before it is read. In a well-clocked program this never happens, since every value is read in the same instant in which it is written. Its channels therefore need no buffers, and behave as wires rather than queues. Caspi and Pouzet call such programs *synchronous Kahn networks*. The only memory that remains is the state a programmer requests explicitly, as with `pre`. Seen this way, a synchronous program is a Kahn network that never needs its buffers.

Lucid Synchrone served primarily as a vehicle for research into how functional and synchronous programming combine. It showed that the analyses a synchronous compiler performs, such as clock and causality checking, can be expressed as type systems. It also showed which functional features suit the synchronous domain. Polymorphism, type inference, and higher-order functions do. Unbounded recursion does not, because a program with real-time constraints must finish each instant in bounded time. Several of the features that do were later adopted by SCADE 6, the industrial successor of Lustre.

Esterel, Signal, Lustre, and Lucid Synchrone share one foundational pillar: the synchronous hypothesis. Execution moves through a sequence of logical instants, and computation within an instant is assumed to be instantaneous. Their compilers carefully analyse the source programs to ensure that concurrent or unordered computations can be executed deterministically.

The clocks discussed above are logical in that they specify the instants at which a component is active, but not necessarily how they relate to the passage of physical time. In the periodically driven execution scheme illustrated in fig. 2.2, the `step` function that drives the entire program is invoked at every tick of a fixed base clock. If the ticks arrive fairly frequently, the sequence of instants becomes quite dense and we risk invoking the program too often. If no new inputs have arrived, there might be nothing to react to, and we will unnecessarily run the step function.

The *Sparse Synchronous Model* [24] (SSM) tackles this problem. The underlying properties of the model are similar in that execution is deterministic and reactions are logically instantaneous. However, rather than being driven periodically by a base clock, the system reacts *aperiodically* to explicitly timed events. The system knows the logical instant at which the next event will occur and a reaction must take place. The runtime can therefore advance directly from one active instant to the next without executing every intervening tick.

SSM uses a concept of model time when progressing between instants. Within an instant, the system can ask *What is the model time now?* and get some value back. The unit of this value is not fixed by the runtime and may represent seconds, milliseconds, or another measure. Within one instant, this value never changes, regardless of how many times the system is queried for the time. Only when we move to the next instant does model time change. SSM allows the change in model time between instants to be dynamic, strictly distancing itself from a model where a periodic base clock ticks. Consider the example below:

```
do r <- ref 0
   t1 <- now
   -- some computation which takes time
   t2 <- now
```

The example is presented in a Haskell-style syntax. `now` is the function with which we ask the runtime system for the current model time. Since we have not used any of SSM's temporal abstractions, the computations above all occur within the same instant. There is an unspecified computation that takes *physical* time to perform, naturally, but the values of `t1` and `t2` are equal, as model time has not advanced.

In SSM, *scheduled variables* play an analogous communication role to Lustre's streams and Esterel's signals. They always carry a value, but that value may not necessarily have been updated in the current instant. The runtime system lets a program ask whether a scheduled variable has been updated in the current instant, but will not otherwise say in *which* instant it received its most recent update. Most notably, rather than directly assigning a value to a scheduled variable, SSM additionally allows a programmer to *schedule* an assignment for the future. "In 2 seconds, assign this value to this scheduled variable". How the notion of two seconds relates to model time is left to the runtime system to decide and is not something the programmer worries about. We modify the example above to schedule an event for the future and then suspend while waiting for it to happen:

```
do r <- ref 0
   t1 <- now
   after (secs 1) r 1
   wait r
   t2 <- now
```

In this modified example, we do use temporal abstractions between the two calls to `now`. We schedule an event to happen after one second, and then wait for that event to happen. If compared, `t1` and `t2` would not be equal, as model time will have advanced.

When an instant ends, the runtime will inspect the events scheduled for the future, and suspend until the next event is supposed to fire. It then applies the update to the scheduled variable, potentially scheduling any concurrent thread that was waiting for it, and then enters the next instant.

Within an instant, execution is deterministic. Execution is driven by concurrent threads, which are totally ordered. A thread can create one or more concurrent child threads and suspend until all of them have terminated. The child threads derive priorities from their parent thread's priority, ensuring deterministic scheduling.

Keeping the SSM model time aligned with physical time is the concern of the step function described above. When inspecting the future scheduled events, the runtime system knows at what model time the event is supposed to happen, as well as what the model time was at the last instant. The difference between the two model times denotes a duration, and the runtime system knows how to convert from model time to physical time. It puts itself to sleep

after scheduling a wake-up when the next event is supposed to happen. This is analogous to setting an alarm for a specified time, rather than setting a timer. If a timer were set to go off after the delta between the two instants had elapsed, it would miss the next instant since physical time actually *has* advanced, even though model time has not. Setting an alarm for the actual time that the next instant starts is absolute, and does not depend on the previous instant. If the implementation is fast enough that the previous instant finishes before the next instant should occur, the illusion of real-time behaviour is not broken.

Lingua Franca [25] occupies a similar design space. It is a reactor-oriented coordination language in which timestamped events trigger reactions whose declared dependencies impose a deterministic execution order. Like SSM, it makes logical time explicit, relates it to physical time, and advances between relevant events rather than executing every tick of a periodic base clock. However, Lingua Franca structures computation as a declared network of reactors, whereas SSM combines scheduled variables with imperative control flow and dynamically created threads.

The synchronous languages discussed above provide context for paper one, but Scoria builds specifically on the Sparse Synchronous Model. SSM supplies the notions of logical instants, scheduled variables, and deterministic scheduling, while Scoria contributes a deep Haskell embedding, an interface connecting scheduled variables to input and output peripherals, and a compiler targeting a lightweight C runtime.

The distinction between model time and physical time is also central to the evaluation of Scoria. The paper tests the functional behaviour of generated programs against an interpreter and measures response time, timing stability, and overload behaviour on physical hardware. These experiments examine whether the implementation preserves the useful properties of the synchronous model when it is executed on a resource-constrained microcontroller.

In terms of the thesis statement, Scoria demonstrates one way of bridging the gap between Haskell and an execution environment that cannot accommodate GHC. Haskell is used to describe and compile the embedded program, while the generated C program and its runtime provide the representation required by the target device.

2.4 Security & Confidentiality

Data at rest is stored, for example, in files or databases and can be protected through storage encryption. Data in transit moves between systems and can be protected through secure communication protocols such as TLS. Data in *use* is being processed and must ordinarily be available to the executing software, exposing it to the operating system, hypervisor, or other privileged components.

This section introduces two related boundaries needed to understand the second paper, which presents HasTEE. Trusted execution environments provide a hardware-enforced boundary around code and data, while information-flow control concerns how information may cross security boundaries. HasTEE combines these perspectives by using Haskell abstractions to describe which

computations execute inside an enclave and which operations may expose or consume data across the enclave boundary.

Trusted Execution Environments A trusted execution environment (TEE) is a hardware-backed execution environment that isolates code, data, and execution state from the rest of a system. Within a specified threat model, this isolation protects the confidentiality and integrity of the protected state even when privileged software outside the TEE, such as the operating system or hypervisor, is compromised. The hardware mechanisms, firmware, runtime system, libraries, and application code on which these guarantees depend form the Trusted Computing Base (TCB). That said, the word *trusted* in trusted execution environment does not imply that the software inside is always correct. The software is trusted in the sense that the claimed security guarantees depend on its correctness.

Several processor architectures provide mechanisms for constructing TEEs. HasTEE targets Intel Software Guard Extensions (SGX) [26], which provides application-level enclaves. Arm TrustZone for Armv8-M (TrustZone-M) [27] provides a different security boundary on microcontrollers and is relevant to the ongoing work discussed in chapters 5 and 6.

SGX allows a user to create *enclaves* within a host application's address space. Enclave memory is encrypted when it is outside the processor, and the host application is not trusted. The processor interprets specific SGX instructions to create, manage, enter, and leave enclaves. The untrusted host can enter an enclave only via explicitly declared entry points, which may in turn invoke enclave-internal functions without ceremony.

TrustZone-M divides execution into Secure and Non-secure states. Secure code can access both Secure and Non-secure memory, whereas Non-secure code cannot directly access Secure memory. Device-specific hardware can also restrict access to peripherals and route interrupts to the appropriate state. Non-secure code can invoke Secure services through predefined *Non-secure Callable* entry points. Unlike SGX's multiple application-level enclaves, TrustZone-M provides one Secure/Non-secure boundary. Secure firmware must configure this boundary and provide the runtime and services needed to construct a TEE.

These isolation mechanisms enforce a boundary, but do not ensure that the software uses it correctly. Programming a TEE therefore requires the developer to decide which code and data belong on either side of that boundary and to define the interface between them. TEE SDKs commonly expose this interface through low-level C or C++ APIs, making the representation and ownership of data crossing the boundary security-critical. For example, a pointer supplied by untrusted code may refer to memory that remains under an adversary's control. If code inside the TEE validates the referenced data and later reads it again, the adversary may change it between the two reads, creating a time-of-check-to-time-of-use vulnerability. To avoid this race, the code can copy the data into protected memory and validate only the protected copy.

Porting existing software to SGX presents a different difficulty. Code inside an enclave cannot treat services provided by the untrusted operating system

as trusted local operations. The Intel SGX SDK therefore provides `tlibc`, a restricted system C library that omits many interfaces available through libraries such as `glibc`, including `mmap`. Large language runtimes, such as the GHC runtime, depend on such interfaces and therefore cannot run in an enclave unchanged. Library operating systems such as Gramine [28] recover much of the expected environment by mediating access to services outside the enclave. This improves compatibility at the cost of adding code to the TCB. In HasTEE, we combined Gramine with modifications to the GHC runtime to execute Haskell programs inside SGX enclaves.

On TrustZone-M, the developer must also configure memory, peripheral, and interrupt attribution consistently with the intended security policy. The hardware enforces this configuration, but cannot determine whether it matches the assumptions of the Secure and Non-secure programs.

A TEE boundary induces a coarse classification with two security levels: protected and unprotected. Code and data are placed on one side or the other, and the hardware restricts how the unprotected domain may access the protected domain. This resembles a security policy enforced over labels, although it governs access to memory rather than every way in which information can propagate. In particular, code in the protected domain may still reveal confidential data through an authorised interface, while useful applications may need to release selected results deliberately. The more general discipline of assigning security labels and controlling the permitted flows between them is known as *information-flow control*.

Information-Flow Control *Information-Flow Control* (IFC) is a mechanism by which we can assign security labels [29] to information and restrict the permitted flow of that information. In the simplest case, IFC defines two labels, L for public and H for secret, which can be used to classify information. The allowed flow of information is from L to H, but not from H to L. This is typically illustrated with the relations $L \sqsubseteq H$ ("L flows to H") and $H \not\sqsubseteq L$ ("H does not flow to L").

The core idea is that any result that depends on secret information must itself be classified as secret. This idea can be stated as a security property that forms a central pillar of IFC.

Noninterference [30].
The public output of a program does not depend on its secret input.

This is enforced by observing how secret data participates in operations and flows through a program's control flow. Consider the simple example below, where we describe the type and initial label of a variable by its declaration.

```
int H x; /* with some initial value */
int L y; /* with some initial value */

if(x == 1) {
    y = 0;
} else {
```

```
    y = 1;
}
```

This program computes an assignment to y based on the value of x, forming a dependency from x to y. If y has the fixed label L, the assignment must be rejected because it would make public information depend on secret information. A flow-sensitive system may instead raise the label of y from L to H, reflecting that its value now depends on secret data.

An explicit operation that lowers the confidentiality label of a value from H to L is called *declassification* [31]. Declassification identifies a point at which the programmer deliberately releases information that would otherwise remain secret. For example, if print represents a public output channel, and y has been raised to H via its dependency on x, printing y should be rejected, but may be permitted following an explicit declassification.

```
int H x; /* with some initial value */
int L y; /* with some initial value */

if(x == 1) {
    y = 0;
} else {
    y = 1;
}

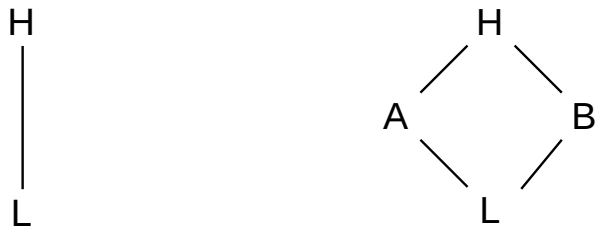
print(declassify(y));
```

Endorsement [32] is a distinct operation concerned with integrity rather than confidentiality. It marks information originating from an untrusted source as trustworthy enough for use in a trusted context. Both concepts are relevant to HasTEE, where information may be deliberately released from an enclave, while data read from outside the enclave may need to be explicitly trusted before it is used by enclave computations.

So far, we have considered a simplified setting with only the two labels L and H. More realistic policies may distinguish several security domains and organise their labels into a *security lattice*. The lattice is ordered by the flows-to relation $\sqsubseteq$, so $\ell_1 \sqsubseteq \ell_2$ means that information classified as ℓ_1 may flow to a context classified as ℓ_2.

For every pair of labels, a lattice provides a least upper bound, called their *join* and written $\ell_1 \sqcup \ell_2$, and a greatest lower bound, called their *meet* and written $\ell_1 \sqcap \ell_2$. The join is particularly important for tracking information flow because it gives the least label capable of protecting information derived from both operands. If a computation combines values labelled ℓ_1 and ℓ_2, its result must therefore be protected at least at $\ell_1 \sqcup \ell_2$. Lattices are often illustrated as Hasse diagrams. Figure 2.3 shows Hasse diagrams for our simple 2-label lattice and a 4-label lattice. $\sqsubseteq$ is a transitive relation, so we also have $L \sqsubseteq H$ in the 4-label example.

Coming back to the example above, we describe its information flow using joins. The context of the program is initially L, while it changes when we enter the branches of the if statement. The condition depends on the secret x, so the context in which y is assigned is $L \sqcup H$, which is H. If the label of y is fixed



(a) The simple 2-label lattice becomes a line, with the H label on top and the L label below, illustrating $L \sqsubseteq H$.

(b) The security lattice shown in the figure represents four security labels, with two domains. Aside from the least secure L and most secure H, there are two added labels A and B. Reading the diagram, we see that the permitted information flow is $L \sqsubseteq A$, $L \sqsubseteq B$, $A \sqsubseteq H$, and $B \sqsubseteq H$.

Figure 2.3: Hasse diagrams for our previously presented 2-label lattice and a new 4-label lattice. The orientation of the diagram matters, in that the higher up in the diagram a label appears, the higher its security position in the lattice.

to be L, the assignment is rejected because $H \not\sqsubseteq L$. A flow-sensitive system may instead raise the label of `y` to H.

There are two major flavours of IFC systems: static and dynamic enforcement [32]. Static systems will reject invalid flows at compile time, whereas dynamic systems track the labels at runtime. If a violation happens during runtime, the program faults.

Haskell is a useful host language for library-based IFC because purity and abstraction can restrict how protected values are observed. Monads and abstract data types [33] can distinguish computations operating at different security levels and expose only the operations through which information is permitted to cross between them. The strength of these guarantees nevertheless depends on which operations the library exports and on whether they can be used to construct illegal programs.

HasTEE applies these ideas to the trusted and untrusted components of an enclave application. The `Enclave` and `Client` monads form two coarse security domains corresponding approximately to secret and public computation. HasTEE does not implement the arbitrary security lattices described above; instead, it specialises the IFC vocabulary to the two sides of an enclave boundary.

The operation `onEnclave` is an explicit release point through which a result is returned from an enclave computation to the client. The released type must have a `Binary` instance, making its ability to cross the boundary explicit, although this requirement does not by itself determine whether releasing the value is semantically safe. The `RestrictedIO` constraint limits the effects available to enclave computations, reducing the channels through which confidential information may escape. Conversely, the `trust` operation is an endorsement

point at which the programmer accepts data obtained from an untrusted source for use inside the enclave.

This background therefore explains both the protection provided by HasTEE and its limitations. The API makes declassification, endorsement, and effectful operations visible, but it still relies on the programmer to use these escape hatches correctly and on the correctness of the trusted runtime and libraries. The contribution of the second paper is not a general-purpose IFC language, but the integration of these language-based controls into a practical, high-level programming model for trusted execution environments.

Information-flow control addresses which information may cross the trusted boundary. A separate question is which computations and code should be placed on each side of that boundary. The following section introduces multiparty programming and program partitioning, which provide the concepts used by HasTEE and the partitioning paper to address this placement problem.

2.5 Multiparty Programming

When developing a distributed application, every participating program is typically developed independently of the others. Each program has a *local view* of what that specific program is doing in the distributed application. Communication protocols define the boundary between it and the other programs, and every participant must agree on the message formats and interaction protocol. Mistakes in these protocols can cause mismatched communication actions, data races, or deadlocks.

Multiparty Languages [34] aim to alleviate some of these concerns by generally allowing a programmer to describe all programs in a network at the same time, in the same compilation unit. This provides a *global view* of the application, allowing the interactions among participants to be analysed together. Depending on the language and the restrictions it imposes, this global view can support guarantees such as communication safety and deadlock freedom. This thesis describes work that relates to two subparadigms of multiparty programming: *multitier* [35], [36] programming (or *tierless*) and *choreographic* [37] programming.

The two paradigms differ primarily in what they emphasise in the source program. A multitier program is organised around the placement of computations, with communication commonly inferred when data crosses from one tier to another. A choreographic program is organised around the interactions among participants, making communication explicit and obtaining the individual endpoint programs through projection.

Consider the simple multitier program in fig. 2.4, written in a made-up syntax.

The program is written from the viewpoint of the client. It first prompts the user for a name and then temporarily enters the server tier using `at @server`. The server-side computation refers to `name`, which was produced by the client, so the implementation must transfer the name to the server. Likewise, the result `known` must be returned to the client before it can be inspected by the

```
greet : () -> ()
greet () @client =
  let name = prompt "your name?"
      known = at @server
                (doesUserExist name db)
  in print (if known
              then "Welcome back!"
              else "Nice to meet you!")
```

Figure 2.4: A multitier description of the greeting application, written from the client's viewpoint. The database lookup is placed on the server, while the communication required to supply its argument and return its result remains implicit.

```
greet_client : () -> ()
greet_client () =
  let name = prompt "your name?"
  send server name
  known <- receive server
  print (if known
           then "Welcome back!"
           else "Nice to meet you!")
```

(a) The client projection prompts the user, sends the name to the server, receives the result, and prints the greeting.

```
greet_server : () -> ()
greet_server () =
  name <- receive client
  let known = doesUserExist name db
  send client known
```

(b) The server projection receives the name, queries the database, and returns the result to the client.

Figure 2.5: The client and server programs obtained by splitting the multitier example. Communication that was implicit in the source program appears as matching `send` and `receive` operations.

client. Neither transfer is stated explicitly: the required communication follows from the placement of the computations and their data dependencies.

Splitting the multitier program produces the client and server programs shown in fig. 2.5.

This example illustrates the principal convenience of multitier programming. The programmer specifies where computations take place, while the implementation determines which values must cross tier boundaries. For a direct client-server data flow such as this one, the necessary transfers are easy to infer and the resulting source program is concise.

Choreographic programming makes a different trade-off. Rather than asking the implementation to reconstruct the interaction protocol from placement and data dependencies, a choreography records that protocol explicitly. It is written from an objective viewpoint, whereby there is no distinguished current tier, and each communication names the participants involved.

The choreography in fig. 2.6 describes the same application. Because this example contains only two participants and a straight-line exchange of two

```
greet : () -> ()
greet () =
  let name = prompt "your name?" @client
      name' = client.name ~> @server
      known = doesUserExist name' db @server
      known' = server.known ~> @client
  in print (if known'
              then "Welcome back!"
              else "Nice to meet you!") @client
```

Figure 2.6: A choreographic description of the greeting application, written from an objective viewpoint. The transfers of the name to the server and the lookup result back to the client are explicit in the source program.

values, its communication protocol is deliberately simple.

The operator `~>` describes one logical communication from a source endpoint to a destination endpoint. It does not manually implement a network transfer. During *endpoint projection*, the communication becomes a `send` operation at the source, a matching `receive` operation at the destination, and no operation at endpoints that are not involved. A single communication in the choreography therefore generates the complementary local actions required at both ends.

Projecting this choreography produces the same client and server behaviours shown in fig. 2.5. The difference lies not in what the resulting programs can compute, but in which information the source program preserves. The multitier program exposes placement and allows communication to be inferred, whereas the choreography exposes the communication protocol and allows it to be inspected and analysed directly.

In this small example, the multitier representation is shorter because the required data transfers are unambiguous. The explicit structure of a choreography becomes more valuable when a protocol contains several participants, branching communication, or choices made at one endpoint that must be observed by others. In such cases, endpoint projection uses the global protocol to ensure that the generated endpoint programs agree on the communication actions and control-flow decisions they must perform.

A multiparty program describes the behaviour of several participants, but an executable for a particular endpoint needs only the behaviour of that participant. The transformation that derives this local behaviour is called *endpoint projection* (EPP) in choreographic programming and is often called *splitting* in multitier programming. Projection can be deferred until runtime, in which case the executable retains enough of the global program to select the relevant endpoint behaviour during execution. Alternatively, a multiparty compiler can perform projection statically and directly generate a separate program for each endpoint, ensuring that computations belonging exclusively to other endpoints are not included.

In this thesis, *program partitioning* refers to the outcome in which the

multiparty program is compiled separately for each endpoint and code intended exclusively for other endpoints is absent from the resulting executable. Static partitioning has several benefits.

- Program partitioning helps prevent one endpoint from inheriting platform-specific requirements from another endpoint. An endpoint should not need libraries, foreign code, hardware interfaces, or build tools used only by another endpoint.
- If one endpoint executes inside a TEE, it is paramount to keep the trusted computing base as small as possible. Removing unnecessary code statically reduces the trusted computing base of an endpoint executing in a TEE.
- Partitioning can produce smaller binaries, which can make a difference if the execution platform is resource constrained, such as an MCU.
- If a multiparty program has been partitioned, each endpoint would reveal much less of the overall application if it were decompiled or inspected. In this sense, partitioning reduces code disclosure. However, this does not by itself guarantee confidentiality of data or protocols.

The second paper in this thesis presents HasTEE, a Haskell framework for confidential computing. A program written using HasTEE describes the behaviour of a client and a server application, with the server component executed in an Intel SGX enclave. The location of computation is described using the type system, with two different monads used for describing the location of computation. The program is compiled separately for the two sides, assigning different semantics to a single high-level API. Unreachable code for the other side is expected to be eliminated by GHC's optimisations, thereby removing client code from the enclave and reducing the trusted computing base. The general technique was inspired by Haste.App's approach to client-server partitioning.

The third paper investigates how GHC's rewrite rules and specialisation can be used to partition programs written using Haste.App and HasChor. It first reformulates Haste App's client-server partitioning technique, which was later adopted by HasTEE, using rewrite rules that encode the endpoint-specific semantics of its API. It then applies the same idea to *HasChor* [38], a Haskell framework for choreographic programming. In standard HasChor, a choreography is represented as a Haskell value, and the existing endpoint-projection function interprets this value at runtime for a selected endpoint. Consequently, each executable can retain computations and dependencies belonging to all endpoints. The paper instead uses endpoint-specific rewrite rules to encode the equations of endpoint projection, eliminating the intermediate choreographic representation and constructing the local network program directly. When location polymorphism hides the identities of participants from these rules, specialisation exposes the missing type or term information so that the rules can fire. The rewrites can make computations belonging to other endpoints unreachable, after which GHC's dead-code elimination can remove them. Exports

and separate compilation can, however, prevent this removal. A Core plugin examines the program after optimisation and reports an error if high-level operations such as `locally`, `~>`, or `cond` remain, checking that the expected partitioning rewrites have occurred.

The paper implements a case study where one endpoint in a choreographic program requires bindings for the BlueZ Bluetooth stack. Without partitioning, even the participants that do not need to know anything about Bluetooth must carry this dependency, whereas after partitioning only the one endpoint that uses Bluetooth must supply it.

In both of these papers, a high-level program describes more than one execution context, while language abstractions and compiler infrastructure derive the code appropriate to one context.

2.6 Distributed Haskell

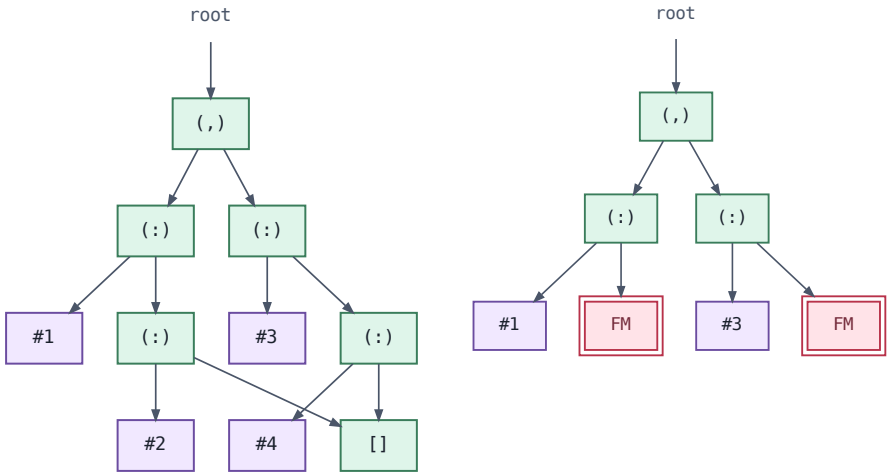
Distributed Haskell systems pursue two distinct aims. One is to accelerate a single computation by evaluating parts of it across several processors or machines. The other is to describe an application whose components must execute at different locations, for example close to particular devices, data, or services. These aims lead to different programming models: distributed evaluation in the former case, and independently communicating components in the latter. The challenges both parallel and distributed Haskell systems must solve significantly overlap, so this section discusses both parallel and distributed Haskell systems.

We use the systems below to examine four recurring design questions. What are the independently executing units, who decides where they run, what information can move between them, and which part of the language implementation provides that mobility?

GUM is a portable parallel implementation of Haskell presented by Trinder et al. [39]. It extends the STG runtime with a collection of logical processing elements (PEs), which may be mapped to processors on either shared- or distributed-memory machines. Each PE maintains its own local heap and schedules a pool of Haskell threads. Together, these local heaps implement a global virtual heap containing the program's shared graph.

When evaluation on one PE demands an object owned by another, the runtime requests it from the owning PE. Rather than returning only the requested object, the owning PE also packs a bounded portion of the nearby reachable graph. This bulk transfer amortises communication latency by reducing the likelihood of another remote request. When the packet boundary cuts an edge in the graph, the omitted target is represented by a Fetch-Me node, as illustrated in fig. 2.7. If that node is subsequently demanded, GUM retrieves the missing graph fragment from its owning PE.

GUM transfers heap closures and their relationships, but it does not transfer executable code. The closures describe the current state of a computation while referring to code already available to every PE. GUM therefore moves work and data within a common compiled program, rather than providing code



(a) The full subgraph reachable from the requested root on the owning PE.

(b) The bounded graph packet sent to the requesting PE.

Figure 2.7: Graph packing in GUM. The full subgraph reachable from the requested root (a) is truncated to form a bounded transfer packet (b). Edges crossing the packet boundary are replaced by Fetch-Me (FM) nodes, allowing the omitted subgraphs to be retrieved later if demanded.

mobility between independently deployed programs.

GUM is important here because it demonstrates that a distributed runtime can move the state of a lazy computation by operating directly on its heap graph. It also exposes a central limitation that will recur throughout this section, that moving a graph of closures is not the same as moving the executable code to which those closures refer.

While GUM achieved true parallelism by running multiple PEs, a single Haskell thread still drove the evaluation, with the runtime system using additional PEs to accelerate it when possible. A related line of work is that of Concurrent Haskell [40], which focused on concurrent programming primitives for Haskell. Its implementation used just one PE to run multiple threads, which communicated via messaging primitives such as `MVar`s. We will not elaborate much on Concurrent Haskell, but mention it here to form a segue to Glasgow Distributed Haskell (GdH) [41].

GdH built on the results of both the GUM runtime and Concurrent Haskell. It retained GUM's multiple PEs and added a thread vocabulary much like that of Concurrent Haskell. The GUM runtime was extended with support for talking about the PEs of a running program, and creating threads on them. The core of the API is listed in fig. 2.8. Notably, a thread on one PE can spawn a thread on another PE. The `MVar`s of Concurrent Haskell were extended to work across the multiple PEs of GUM and were the primary mechanism by which threads exchanged data. A simple example of a GdH program, taken

```
myPEId :: IO PEId
allPEId :: IO [PEId]

rforkIO :: IO () -> PEId -> IO ThreadId
revalIO :: IO a -> PEId -> IO a

class Immobile a where
  owningPE :: a -> IO PEId

```

Figure 2.8: The cornerstones of the GdH API are facilities for identifying underlying PEs at runtime and creating threads on distant PEs. Furthermore, some resources must remain at a particular PE, so the `Immobile` type class can identify which PE owns a particular resource.

```
main = do
  p:ps <- allPEId
  m <- newEmptyMVar
  rforkIO (putMVar m 42) p
  r <- takeMVar m
  putStrLn (show r)

```

Figure 2.9: The example program queries the runtime for all available PEs and spawns a thread on the first one using `rforkIO`. The main thread then waits until the created thread populates the `MVar`, at which point it reads its value and prints it.

from [41], is shown in fig. 2.9.

As GdH built upon the GUM runtime, it inherited its characteristics. Code never actually crosses between PEs, and all PEs are loaded up with the same compiled executable. In fig. 2.9, the body of the created thread is represented by a closure that models the applications involved, but the code for actually evaluating, e.g., `putStrLn`, is not sent between PEs.

Eden [42] is another Haskell implementation for distributing evaluation. It takes a different approach to the implicit distribution of the GUM runtime, and to the explicit thread vocabulary of GdH. Eden provides *processes*, which are remotely instantiated functions from `a` to `b`. The core API is, once again, quite small. It is shown in fig. 2.10.

Eden separates three distinct things. Computations themselves are expressed as ordinary Haskell functions. The process abstraction is created with `process`, and finally the parallel instantiation is performed with `(#)`. The type class `Trans` is instantiated by every type that can cross between processes. The `NFData` constraint is used to ensure that values are evaluated before being communicated. Because an Eden process takes an input and produces an output when it terminates, it implicitly communicates with its parent process.

```
class NFData a => Trans a where (...)
data (Trans a, Trans b) => Process a b = ...

process :: (Trans a, Trans b) => (a -> b) -> Process a b
( # )   :: (Trans a, Trans b) => Process a b -> a -> b

type ChanName a = ...
new     :: Trans a => (ChanName a -> a -> b) -> b
parfill :: Trans a => ChanName a -> a -> b -> b
```

Figure 2.10: The core API of Eden. A `Process a b` type models a process that takes a value of type `a` as input and returns a value of type `b`. Processes are declared via `process`, but actually started by #, which applies it to its input. Channels can be used to send data between arbitrary processes, using the `new` and `parfill` functions.

However, for dynamic communication with arbitrary processes a user must create named channels with `new` and write to them with `parfill`. A restriction on channels is that they can only be used *once*. Subsequent writes to the same channel result in an error.

Reading from a channel is done implicitly. A process that creates a channel might execute

```
new (\chanName val -> body)
```

While not obvious, `body` now has access to *both* the write-end and read-end of the channel. `chanName` can be used by any process to send a value down the channel. A process might write to the channel like so:

```
parfill chanName x cont
```

`parfill` creates a concurrent thread that evaluates `x` and sends it down the channel, while the current process immediately continues with `cont`. The read-end of the channel is the value `val :: a`. It is a lazy placeholder for a value that might arrive through the channel at some point. Trying to force `val` will block that process until the value arrives.

GdH and Eden give the programmer more explicit ways to describe remote activity and communication than GUM. Nevertheless, they retain the same basic deployment assumption: computation may be placed remotely, but the code required to perform it must already be installed there. The next question is therefore whether a Haskell system can move not only the state of a computation, but also the code required to continue it. *Mobile Haskell* (mHaskell) [43] explores this possibility. It extends Concurrent Haskell with higher-order communication channels called `MChannel`s. An `MChannel` can transmit Haskell values, including functions, `IO` actions, and other channels. Channels can be registered by name and discovered by independently executing programs, allowing mHaskell to support open distributed systems rather than only a fixed collection of processing elements. Remote evaluation is implemented

by sending an `IO` action through such a channel and starting it on the receiving machine.

The interface for remote evaluation is particularly direct:

```
1  rforkIO :: IO () -> HostName -> IO ()
2
3  greet :: HostName -> String -> IO ()
4  greet host name =
5    rforkIO (putStrLn ("Hello, " ++ name)) host
```

The action passed to `rforkIO` closes over `name`. mHaskell serialises the graph representing this action, including the captured string and any required mobile bytecode, and reconstructs it on `host`. References to native code, such as library routines used by `putStrLn`, are instead resolved against code already installed at the receiving machine.

mHaskell is implemented as an extension of GHC. It was designed for heterogeneous networks whose machines may use different processor architectures and operating systems. Its runtime extensions are implemented in portable C, with communication between machines implemented using TCP sockets. To make executable code portable between architectures, mHaskell combines native code produced by GHC with machine-independent bytecode produced by GHCi.

Modules compiled by GHCi are represented by ByteCode Objects and can be included in a transmitted graph. Modules compiled to native code are not transmitted and must instead be available for linking at the destination. By compiling shared modules to native code and potentially mobile modules to bytecode, the programmer controls how much code is transmitted and how much must already be installed at each node.

To serialise a computation, mHaskell adapts GUM's graph-packing algorithm and extends it to recognise ByteCode Objects. Unlike GUM, which may terminate a graph packet with Fetch-Me nodes referring back to the source PE, mHaskell continues until the entire graph reachable from the transmitted value has been serialised. Sharing and cycles within that graph are preserved, but no references back into the sender's heap remain. The receiver therefore obtains an independent copy rather than participating in a global shared heap.

mHaskell's mobility therefore depends on a shared software base. Participating machines must run compatible versions of the modified GHC runtime and provide any native libraries required by received bytecode. Furthermore, any user-defined data types used by a transmitted computation must already have compatible compiled definitions at the receiving machine, since the prototype transmits their values but not their definitions.

mHaskell demonstrates that code mobility is possible when the transferred representation includes portable bytecode. Its implementation also depends directly on GHC's internal representations of heap closures, ByteCode Objects, and linker information. This illustrates the engineering cost of implementing distribution inside a large and evolving compiler runtime.

GdH, Eden, and mHaskell all obtain their distributed behaviour through specialised compiler or runtime support. Cloud Haskell explores a different

point in the design space by implementing distributed coordination primarily as a library on top of an ordinary GHC runtime.

Cloud Haskell [44] (CH) is a shallowly embedded domain-specific language for distributed programming on an ordinary GHC runtime. It follows Erlang's model of lightweight processes that communicate by passing messages rather than through shared memory. A *node* represents a participating runtime system at a particular network location. Each node can execute many concurrent processes, and each process has a process identifier and a mailbox. Processes can create further processes locally or remotely, exchange messages, and monitor one another for termination.

The types of the basic message-passing operations expose an important restriction:

```
send   :: Serializable a
       => ProcessId -> a -> ProcessM ()

expect :: Serializable a
       => ProcessM a
```

A value sent between processes must satisfy the `Serializable` constraint, which provides the binary representation needed to transmit it. Function values present a particular difficulty because a function's type describes only its argument and result, not the environment captured by that function. A closed function and a function of the same type that captures an `Int` may both have type `Int -> Int`, while a third function of that type might capture another function. Their types are identical, but their environments have different serialisation requirements. Consequently, whether a function can be serialised cannot be determined structurally from its type.

Cloud Haskell addresses the code part of this problem by introducing the type `Static a`. The proposed typing rule distinguishes top-level bindings, which are classified as static, from locally bound variables, which are classified as dynamic. An expression `static e` is accepted only when every free variable of `e` refers to a static, top-level binding. It may therefore refer to other top-level definitions, but it cannot capture a value bound by a surrounding function or local `let`. A top-level supercombinator, meaning a function with no dynamically bound free variables, is the simplest example of such a static expression, although the rule also permits expressions composed from several top-level definitions.

Under Cloud Haskell's assumption that participating nodes run the same program, a static function can be represented by a symbolic code label. The receiving node can use that label to locate its own copy of the function. A value of type `Static a` is therefore serialisable without requiring `a` itself to satisfy `Serializable`. The static machinery does not make an arbitrary captured environment serialisable; instead, it guarantees that the code component has no hidden dynamic environment.

Remote process creation makes this boundary visible in its type:

```
spawn :: NodeId
      -> Closure (ProcessM ())
```

```
3           -> ProcessM ProcessId
```

The caller explicitly chooses the destination node, but cannot pass an arbitrary `ProcessM ()` action. Instead, the action must be represented by a value of type `Closure (ProcessM ())`.

The word *closure* requires some care here. The GHC heap closures discussed earlier in this section are low-level runtime objects representing functions, applications, constructors, thunks, and other heap values. Cloud Haskell's `Closure a` is not an exposed GHC heap closure. It is an explicitly constructed, source-level Haskell value that describes how a value of type `a` can be reconstructed on another node. Cloud Haskell serialises this description rather than traversing and copying the arbitrary runtime graph reachable from the original value.

Conceptually, Cloud Haskell uses the following representation:

```
1   data Closure a where
2     MkClosure
3       :: Static (ByteString -> a)
4       -> ByteString
5       -> Closure a
```

The two fields separate the statically identifiable code from its dynamically supplied environment. The `Static` component identifies a reconstruction function whose definition is available on every participating node and whose lack of dynamically bound free variables has been checked. The `ByteString` contains the environment, serialised explicitly while its type and corresponding `Serializable` instance are still known at the construction site. This timing matters because the type of the environment is hidden inside `Closure a`, and cannot be recovered from `a` at the receiving node. The appropriate environment decoder is therefore included in the static reconstruction function.

When a `Closure` is transmitted, Cloud Haskell sends the static code identifier and the already serialised environment. The receiving node looks up its local reconstruction function and applies it to the received environment. Cloud Haskell therefore does not transmit executable code in the manner of mHaskell. It transmits a reference to installed code together with the data needed to reconstruct its environment.

The distinction becomes visible when attempting to send a function that captures a local variable. One might like to write:

```
1   sendFunc :: ProcessId -> Int -> ProcessM ()
2   sendFunc p x =
3     send p (\y -> x + y + 1)
```

This program expresses the intended computation directly, but the type `Int -> Int` neither identifies the function's code nor reveals that its environment contains `x`, so it cannot provide the required `Serializable` instance. In Cloud Haskell, we instead need to write this:

```
1   sendFunc :: ProcessId -> Int -> ProcessM ()
2   sendFunc p x = send p clo
3     where
```

```
4         clo :: Closure (Int -> Int)
5         clo = MkClosure (static sfun) (encode x)
6
7     sfun :: ByteString -> Int -> Int
8     sfun bs y =
9       let x = decode bs
10      in x + y + 1
```

This transformation performs closure conversion explicitly. The code of the local function is moved into the top-level function `sfun`. Because `sfun` has no dynamically bound free variables, the static typing rule permits the expression `static sfun`. The local value `x`, which cannot occur inside that static expression, is instead moved into the explicitly serialised environment. The reconstruction function `sfun` decodes this environment before evaluating the original function body.

The original Cloud Haskell implementation used Template Haskell to generate the closure-construction, decoding, and registration machinery. However, the programmer still had to perform the essential closure conversion by defining a top-level function whose arguments represented the captured environment.

Cloud Haskell therefore places the mobility boundary in its source-level API. Processes are the units of distribution, the programmer chooses their placement, messages must be serialisable, and remotely spawned computations must be represented by explicit `Closure` values referring to code already installed at the destination. This boundary is particularly relevant to this thesis because CloudMicroHaskell retains the process and message-passing model while moving the serialisation mechanism into the runtime representation.

A project closely related to Cloud Haskell is *Haskell distributed parallel Haskell* (HdpH) [45]. Like Cloud Haskell, HdpH is implemented primarily as a library and uses static references to identify code that is available on every node. The two systems are intended for different forms of distributed computation. Cloud Haskell describes applications consisting of independently executing processes, whereas HdpH distributes tasks in order to accelerate a parallel computation.

HdpH builds on the `Par` monad, a library interface for deterministic parallelism. Computations in the `Par` monad can create parallel tasks and communicate their results through single-assignment variables called `IVar`s. HdpH extends this model from shared-memory parallelism to machines with separate address spaces.

HdpH refers to a computation that is available for parallel execution as a *spark*. This terminology and the underlying idea are similar to the sparks used by multicore GHC. In both cases, creating a spark identifies an opportunity for parallel execution without guaranteeing that the computation will actually execute in parallel. The execution mechanisms are, however, different. A spark in multicore GHC refers directly to a closure in the shared heap of one runtime system and may be converted into a thread by one of that runtime system's worker threads. An HdpH spark is instead represented by an explicit serialisable closure, allowing it either to be executed locally or to be transferred to a separate runtime system on another node. The programmer does not

assign an HdpH spark to a particular node. Instead, the scheduler distributes sparks among nodes, allowing idle nodes to obtain work from busy nodes.

Cloud Haskell processes communicate by sending messages to process identifiers and receiving them from their mailboxes. This supports general communication protocols among independently executing processes. HdpH tasks instead communicate information through `IVar`s. An `IVar` can be written exactly once, and reading it blocks until its value becomes available. A local `IVar` can be turned into a serialisable global reference called a `GIVar`. A remote task can write a result to this reference using `rput`, after which the task that owns the corresponding local `IVar` can read the result. There is no operation for reading a `GIVar` remotely. HdpH communication is therefore organised around dependencies between parallel tasks rather than messages exchanged between independent processes.

Like Cloud Haskell, HdpH uses explicit closures to represent remotely executable computations. HdpH extends Cloud Haskell's closure representation by storing both the actual thunk and a serialisable description of that thunk.

```
1  data Closure a = UnsafeMkClosure
2                     a
3                     (Static (ByteString -> a))
4                     ByteString
```

The three fields form two representations of the same computation. The first field contains the actual thunk and is used when the closure is evaluated on its current node. The remaining fields contain a static deserialiser and a serialised environment, which together form the representation used when the closure is transferred to another node. If the closure remains local, `unClosure` can return the first field directly without serialising and deserialising its environment. If the closure is transferred, only its static reference and environment are serialised, and the receiving node uses them to reconstruct the thunk lazily. This dual representation is particularly useful in HdpH because the scheduler may execute a spark locally or transfer it to another node. The constructor is unsafe because the actual thunk and its serialisable representation must describe the same computation.

The dual representation also allows HdpH to transform explicit closures while retaining their ability to be transferred. For example, HdpH provides the following polymorphic operation:

```
1  mapClosure
2    :: Closure (a -> b)
3    -> Closure a
4    -> Closure b
```

Because both the function and its argument are explicit closures, HdpH can construct both representations of the resulting closure without requiring the types `a` and `b` themselves to be serialisable. This makes it possible to define polymorphic evaluation strategies and algorithmic skeletons, such as a parallel map, on top of the primitive task interface.

Despite supporting distributed scheduling, HdpH does not provide code mobility. The static component of a closure identifies code that must already

be installed on every participating node. HdpH therefore transfers tasks and their environments, but not arbitrary executable code or runtime graphs.

The programming language Unison [46] avoids this shared-code assumption by making program definitions independently identifiable and transferable. Unison is a statically typed, purely functional language influenced by Haskell, and its compiler is implemented in Haskell. Each definition is identified by a hash of its syntax tree, including the hashes of the definitions on which it depends. When a computation is sent to another node, the recipient can identify the referenced definitions it does not possess and request them from the sender [47]. The transferred definitions can then be cached and reused by subsequent computations. A node can therefore execute a received computation without having been provisioned with the complete application in advance.

The systems described above demonstrate that *mobility* is a tricky problem that has captured the interest of many people. GUM, GdH, and Eden execute code in a setting where every participant has been provisioned with the same code. Data and work can move between these participants, but program-specific code cannot. Cloud Haskell and HdpH represent transferable computations as explicit closures, but the code identified by their static components must still be installed on every participating node. mHaskell and Unison remove this shared-code assumption in different ways. mHaskell can include portable bytecode in a transmitted graph, whereas Unison retrieves immutable definitions using their content hashes.

The papers in this thesis can be placed along the same axis. HasTEE identifies two locations at which computations should be placed, with the assumption that one of them is a TEE. The two components are compiled separately, deployed within their own address spaces, and communicate by sending serialised data. The code for each location is installed during deployment and does not change during runtime. Code cannot move from one location to another, only data.

The paper about program partitioning studies how to use compile-time techniques to derive which code should go into a specific location. It ensures that computations that should happen at a specific endpoint do not end up at another endpoint and that, again, only serialised data is transmitted between endpoints. Both HasTEE and the partitioning paper thus use language abstractions to talk about *where* code belongs, and how to communicate serialised data between endpoints.

As explained in section 2.1, the bytecode representation used by MicroHs allows *both* code and data to be serialised and sent between running instances. The only required shared software base is a compatible version of the MicroHs runtime system. The final paper in this thesis applies this property in CloudMicroHaskell. CloudMicroHaskell retains Cloud Haskell's nodes, lightweight processes, mailboxes, remote spawning, and process monitoring, but moves the serialisation boundary out of the source-level API and into the MicroHs runtime. A remotely spawned process can be written as an ordinary `ProcessM` computation that captures variables from its surrounding scope. When the process crosses a node boundary, the runtime serialises the reachable graph containing both its code and its captured environment. Messages are handled

by the same mechanism, allowing ordinary values, including functions, to be transmitted without programmer-written `Serializable` instances or explicit closure conversion.

Unison and CloudMicroHaskell both allow a node to execute a computation whose program-specific code was not installed in advance, but they operate on different program representations. Unison assigns immutable definitions stable content hashes and transfers the code definitions that the receiving node lacks, after which those definitions can be cached and reused. CloudMicroHaskell instead serialises the runtime graph reachable from each process or message, in which application code, data, and the captured environment share the same graph representation. This requires no separate mechanism for identifying and deploying definitions, but the amount transferred depends on the current runtime graph, and the same program-specific code may be transmitted again as part of a later graph.

Moving the boundary from the type system to the runtime system changes where errors are detected. Cloud Haskell can rule out erroneous programs at compile time, whereas CloudMicroHaskell will encounter them at runtime. It is also invisible to the programmer precisely what the price of laziness is. A small value might be represented by a large, unevaluated graph, which will be more costly to serialise, transmit, and deserialise.

The papers in this thesis therefore investigate two complementary ways of reaching execution environments for which an ordinary Haskell program was not originally prepared. HasTEE and the partitioning paper transform and divide the program before execution, installing only the code intended for each environment. CloudMicroHaskell instead changes what the runtime can transfer during execution by using a representation in which program-specific code and data can move together.

2.7 Embedded Haskell

This section considers the use of Haskell to program resource-constrained embedded systems. The term *embedded system* covers a wide range of devices, including both small microcontrollers and comparatively powerful computers running Linux. Here, we are primarily concerned with microcontrollers having at most a few hundred kilobytes of RAM and a similarly limited amount of program storage. Such devices commonly run either without an operating system or with a small real-time operating system such as Zephyr.

C remains a common implementation language for these systems because it provides direct access to hardware and gives the programmer precise control over data representation and memory use. However, the same mechanisms that provide this control, including pointers and manual memory management, also permit memory-safety errors. Haskell offers memory safety, abstraction, and an expressive type system, but a conventional GHC-compiled program is generally unsuitable for these devices. The generated executable and the GHC runtime system typically require substantially more memory and operating-system support than a small microcontroller provides.

Haskell is not the only language that offers memory safety. Rust, for example, uses an ownership-based type system to provide memory safety without a tracing garbage collector. Each value has an owner, and references temporarily borrow access to that value. The compiler ensures that references do not outlive the values they borrow and prevents incompatible mutable and shared references from being used simultaneously. Compared with Haskell, this approach exposes resource ownership and lifetimes more directly to the programmer, while providing precise control over memory use that is particularly valuable on microcontrollers. Low-level operations such as accessing memory-mapped hardware or calling foreign functions still require unsafe code somewhere in their implementation. These operations can be encapsulated behind safe interfaces, but the memory safety of the complete system depends on the unsafe implementations upholding the invariants assumed by the compiler.

Memory safety can also be established without changing the implementation language. Separation logic [48] extends Hoare logic with assertions that describe disjoint regions of the heap. This separation enables local reasoning about pointer-manipulating code and can be used to prove that only valid memory is accessed. Unlike Haskell and Safe Rust, separation logic is a program logic rather than a language-level safety guarantee. It provides guarantees only for code for which proofs have been supplied. These proofs can be obtained either manually or by using a verification tool.

Within the Haskell systems considered in this thesis, there are two broad approaches to targeting resource-constrained devices. In the code-generation approach, a Haskell program using an EDSL runs on a development machine and generates C code. A conventional C cross-compiler then compiles this code into a native executable for the embedded device. In the native-execution approach, a Haskell compiler produces a program representation that is deployed together with a sufficiently small Haskell runtime directly on the device. The two approaches are illustrated in fig. 2.11.

Among the EDSLs that generate C, Ivory [49] emphasises memory-safe systems programming. Ivory defines a restricted imperative language whose type and lifetime rules are encoded in Haskell's type system. The Haskell compiler can therefore reject certain unsafe target programs before Ivory generates any C code. For example, Ivory uses rank-2 types to prevent a reference to stack-allocated storage from escaping the scope in which that storage is valid. Consider the example below:

```
1  useLocal :: Def ('[] :-> Uint32)
2  useLocal = proc "useLocal" $ body $ do
3    r <- local (ival 0)   -- r :: Ref s (Stored Uint32), zero-initialised
4    incr r
5    incr r
6    deref r >>= ret
```

The call to `local` creates a zero-initialised, stack-allocated reference. The reference is tagged with a type variable `s` representing the scope in which it is valid. Ivory's use of rank-2 types prevents this scope variable, and hence the reference, from escaping. This rules out errors such as null-pointer dereferences and use-after-free within the corresponding fragment of Ivory.

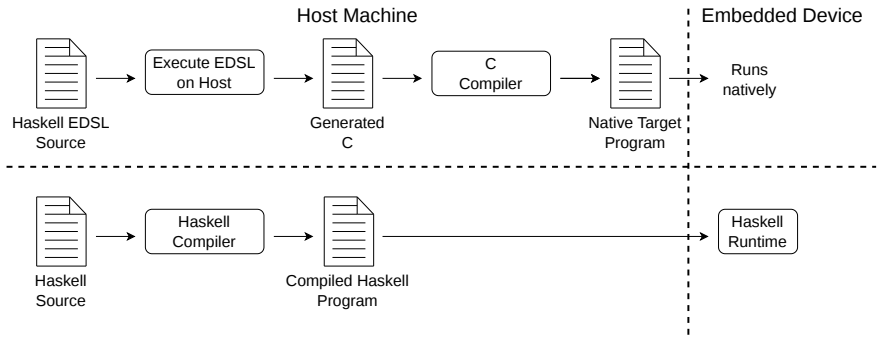


Figure 2.11: Two approaches to using Haskell for embedded programming. In the code-generation approach (top), a Haskell EDSL is executed on the host to produce C, which is compiled into a native target program. In the on-device execution approach (bottom), an ordinary Haskell program is compiled into an implementation-specific representation that is evaluated by a Haskell runtime on the embedded device.

Ivory was successfully used to implement an autopilot for an unpiloted aircraft, a project titled SMACCMPilot [50]. The application itself consisted of 13 kLOC of Ivory and 10 kLOC of machine-generated Ivory.

Copilot [11] is a stream-based Haskell EDSL for implementing runtime monitors for safety-critical systems. A Copilot program defines a collection of conceptually infinite streams whose successive values correspond to successive execution steps. Ordinary operations such as arithmetic and comparison are lifted to operate pointwise on these streams. In the following definition, the prefix `[1,1]` denotes the first two Fibonacci numbers, while the recursive expression defines each subsequent value as the sum of its two predecessors.

```
fib :: Stream Int32
fib = [1, 1] ++ (fib + drop 1 fib)
```

Copilot programs can also sample values supplied by the surrounding C system. The following example[1] monitors an externally provided temperature reading and invokes different procedures when the reading crosses two thresholds.

```
temp :: Stream Word8
temp = extern "temperature" Nothing

ctemp :: Stream Float
ctemp = (unsafeCast temp) * (150.0 / 255.0) - 50.0

spec = do
  trigger "heaton"  (ctemp < 18.0) [arg ctemp]
  trigger "heatoff" (ctemp > 21.0) [arg ctemp]
```

[1]The example is borrowed from: https://github.com/Copilot-Language/copilot

The stream `temp` samples the externally defined symbol `temperature`. The stream `ctemp` converts the raw byte value to degrees Celsius. The specification then declares two triggers. The first invokes `heaton` when the temperature is below 18 degrees, while the second invokes `heatoff` when it is above 21 degrees.

Copilot compiles the specification into C99 code that can be incorporated into the monitored system. The generated monitor has statically bounded memory use and execution time per evaluation step, making its resource requirements predictable. Copilot has been used successfully to monitor critical avionics hardware.

Haski [51] follows a similar code-generation approach. It deeply embeds Lustre in Haskell, allowing programs to be expressed using the nodes and equations described in section 2.3. Like Copilot, Haski compiles these programs into C code with bounded resource use per reaction. Whereas Copilot focuses on runtime monitoring, Haski is intended for resource- and energy-constrained IoT devices.

The first paper in this thesis places Scoria in the same broad design space. Scoria is a deeply embedded reactive language based on the Sparse Synchronous Model. Its compiler runs on the development machine and generates C code targeting a small runtime system on the embedded device. This allows programmers to use high-level temporal abstractions without requiring a Haskell runtime on the target. Scoria therefore demonstrates how Haskell's abstraction mechanisms and compiler infrastructure can bridge the gap to devices that cannot accommodate GHC.

In all these systems, Haskell is used to construct and compile the embedded program, but the deployed device executes generated C. This approach provides a small target footprint and permits close control over the generated program's resource use. Its principal limitation is that the target program is restricted to the operations and abstractions provided by the EDSL. The alternative is to retain Haskell's execution model by deploying a smaller Haskell runtime on the device.

The MicroHs compiler makes this native-execution approach possible on substantially smaller devices than GHC can ordinarily support. It compiles Haskell programs into a combinator representation evaluated by a compact graph-reduction runtime. In the configuration used in this thesis, the runtime occupies approximately 150 KiB of flash, while a few hundred kilobytes of RAM are sufficient for the graph heap. The runtime exposes a small platform-specific interface through which the operations required by a new target can be implemented.

This approach trades execution speed for a compact runtime and broad language support. Graph reduction and garbage collection contribute to this overhead, while the absence of compiler optimisations leaves scope for improving performance. The performance difference becomes especially important on microcontrollers, whose processors are themselves considerably slower than contemporary desktop processors. Whether this cost is acceptable depends on the workload. Garbage collection and graph reduction also make execution time harder to bound, limiting the suitability of MicroHs for applications with strict hard real-time requirements.

The fourth paper in this thesis uses MicroHs to run Haskell natively on embedded devices, implementing a framework for describing isolated processes that communicate via message passing. By running Haskell directly on the devices, the programmer can write idiomatic Haskell code without having to consider what kind of C code various definitions will produce. This brings us back to the questions asked by this thesis. The example in the paper demonstrates that Haskell code can execute on a heterogeneous network of devices. We argue that this is a much more impactful example than it might first appear. The example combines 32-bit and 64-bit runtimes built for platforms ranging from laptops to MCUs, while compiled Haskell code travels between the devices and can execute on any of them.

The native-execution approach must nevertheless justify its additional runtime and performance costs. Memory safety provides one reason to consider it, since it rules out broad classes of errors associated with unrestricted pointer manipulation and manual memory management. Microsoft and the Chromium project have both reported that memory-safety errors account for a substantial proportion of the security vulnerabilities they studied.[2] Although these studies do not concern embedded Haskell specifically, they illustrate the general value of replacing unchecked memory operations with safer abstractions.

Haskell's type system provides a second advantage by allowing libraries to encode domain invariants and reject violations statically. Because many types are erased during compilation, these checks need not require corresponding checks at runtime. The distinction between pure and effectful computations also makes interactions with hardware, communication channels, and other external resources visible in program types. Pure application logic can consequently be understood and tested independently of many of these interactions.

For this thesis, the strongest motivation is the possibility of retaining a common programming and execution model across heterogeneous devices. The same abstractions can describe computations on conventional computers and microcontrollers, while CloudMicroHaskell's graph representation allows code and data to move between them. These advantages do not make native Haskell appropriate for every embedded application. Rather, they establish a useful design point when memory safety, high-level abstraction, and computation mobility justify the additional runtime cost.

[2] Microsoft Security Response Center, "A proactive approach to more secure code," July 16, 2019, https://www.microsoft.com/en-us/msrc/blog/2019/07/a-proactive-approach-to-more-secure-code/. The Chromium Projects, "Memory safety," https://www.chromium.org/Home/chromium-security/memory-safety/.

Chapter 3

Contributions

This chapter returns to the thesis statement and explains how the papers contribute to answering it. The papers investigate two complementary ways of reaching execution environments for which Haskell was not originally designed. The first transforms or specialises a high-level program before deployment, producing code for a particular target or endpoint. The second adapts the language runtime and its representation of programs so that Haskell computations can execute or move within unconventional environments. These approaches overlap, most notably in HasTEE, which combines type-directed program partitioning with a runtime adapted for Intel SGX.

3.1 Transforming Programs Before Deployment

A resource-constrained microcontroller cannot ordinarily accommodate the GHC runtime, but Haskell can still be used to describe and compile programs for it. Paper 1 [13] explores this approach through Scoria, a deeply embedded language based on the Sparse Synchronous Model. Executing a Scoria program on the development machine constructs an abstract syntax tree, which the Scoria compiler translates into C code for a lightweight runtime on the target device. The microcontroller therefore executes generated C rather than Haskell, while the programmer still uses Haskell's type system, abstraction mechanisms, and libraries to construct the program.

Aside from embedding the Sparse Synchronous Model in Haskell, Scoria contributes a design and implementation of an interface that allows asynchronous hardware interrupts to enter the synchronous program state. Interrupt handlers timestamp incoming events and place them in a queue outside of the synchronous program state rather than modifying it directly. The runtime will take these events one by one and introduce them into the synchronous program, while the priority ordering of computations within each logical instant preserves deterministic execution. Output peripherals are also represented through Scoria variables, allowing their handlers to participate in the same priority structure as application code. This allows the programmer to control when an output event is actually realised on hardware. The design allows

a programmer to describe input and output events in Scoria without using interrupt-driven control flow.

The paper evaluates both the compiler and the resulting runtime. Experiments on physical microcontrollers measure response latency, timing stability, and behaviour under overload. The results show that Scoria can execute reactive programs on constrained hardware with predictable behaviour within a measured operating window. They do not establish performance parity with hand-written C, and the evaluation identifies substantial overheads and conditions under which the runtime can no longer keep model time aligned with physical time.

The correctness of Scoria is tested empirically using property-based testing techniques. A bespoke generator of Scoria programs is used to generate programs that are well-formed. Generated programs are compiled and executed, with their results then being compared against those of an interpreter written in Haskell, acting as the reference semantics. If a counterexample is found, it is shrunk to a minimal failing counterexample.

The strategy of transforming a program before deployment is similarly used by HasTEE, presented in Paper 2 [14]. A programmer can describe the trusted and untrusted computations in one Haskell program. Where a computation happens is identified in the type system by the use of two different monads. A HasTEE program is compiled twice, with different implementations of its API used for each compilation. The two implementations differ in which of the two monads is given meaning and which is left as a meaningless monad. This gives the programmer automatic partitioning without requiring modifications to GHC itself.

Paper 3 [3] builds upon this style of partitioning. It uses GHC rewrite rules and specialisation to control the projection of an endpoint, together with a Core plugin that verifies that partitioning has succeeded. For programs written using the Haste.App or HasChor frameworks, rewrite rules replace the operations of the multiparty abstraction with the operations required by one endpoint. In the HasChor case, the rules eliminate the intermediate choreographic representation and construct the projected endpoint program directly. Specialisation exposes location information that would otherwise remain hidden inside location-polymorphic functions, enabling further rewrite rules to apply. A Core plugin runs after optimisation and reports an error if operations belonging to the unpartitioned abstraction remain.

The paper demonstrates a practical consequence of this transformation with an application whose authenticator endpoint depends on the BlueZ Bluetooth stack. After projection, the client endpoint contains no calls to the Bluetooth interface and can be built without that platform-specific dependency. The example shows how endpoint projection can remove unnecessary code and dependencies, making an endpoint easier to build and deploy. Possible reductions in executable size and the trusted computing base follow from the same mechanism, but the paper does not evaluate those reductions quantitatively.

This form of partitioning is lightweight because it reuses GHC's existing optimisation infrastructure rather than introducing a new transformation pass. It nevertheless depends on the framework operations being rewritten and spe-

cialised as expected. Exported definitions, separate compilation, and insufficient location information can prevent unwanted code from being eliminated. The technique therefore establishes that GHC's existing mechanisms are sufficient under the conditions studied in the paper, rather than that every multiparty Haskell program can be partitioned automatically.

3.2 Adapting Runtime Systems

If we wish to execute Haskell code directly on the target, we must ensure that the runtime system meets the requirements of the platform. The second strategy is therefore to adapt the runtime system or exploit a runtime representation suited to the target environment. HasTEE and CloudMicroHaskell explore two different forms of this strategy.

HasTEE requires its trusted component to execute as Haskell inside an Intel SGX enclave. Paper 2 achieves this by combining a modified GHC runtime with the Gramine library operating system. The result is an application-facing programming model in a high-level, statically typed, and memory-managed language, although the underlying implementation still relies on low-level runtime and SGX infrastructure.

HasTEE also uses Haskell's type system to constrain information flow across the enclave boundary. The restricted enclave monad limits the effectful operations available to trusted computations, while the types of values crossing the boundary determine which data can be transferred. These restrictions prevent classes of accidental disclosure represented by the HasTEE API. Explicit declassification and endorsement remain trusted operations, and the guarantees do not cover side channels or weaknesses in the underlying SGX platform.

Three case studies demonstrate that the programming model can express applications from different domains: privacy-preserving machine learning, an encrypted password wallet, and a data clean room providing differential privacy. Together, the case studies show that type-directed partitioning and information-flow restrictions can be integrated into a practical client-server programming model. They demonstrate applicability rather than complete security, since the resulting system still trusts components including GHC, the modified runtime, Gramine, and the SGX hardware and software stack.

Paper 4 [5] explores a different runtime design through CloudMicroHaskell. MicroHs represents both code and data as a combinator graph evaluated by a compact graph-reduction runtime. In exchange for a small and portable runtime, MicroHs is substantially slower than mature native-code implementations such as GHC. CloudMicroHaskell uses the graph representation not only for execution, but also as the serialisation format for distributed computation.

Serialising the reachable graph allows a remotely spawned process to be written as an ordinary computation that captures values from its surrounding scope. Messages may similarly contain ordinary values, including functions, without programmer-written serialisers or closure conversion. Unlike Cloud Haskell, participating nodes do not need binaries containing the same collection of static symbols. They must instead agree on a compatible communication

protocol, graph format, and runtime representation.

The implementation provides a node runtime with remote spawning, message delivery, selective receive, process registration, monitors, exit propagation, and detection of node disconnection. Generic servers and supervisors are implemented as libraries using these primitives. These mechanisms show that runtime graph serialisation can support more than isolated examples of transmitting functions, and can form the basis of a conventional message-passing programming model.

The evaluation examines several aspects of this design. Process and message benchmarks quantify the overhead of the implementation and of graph serialisation. A distributed work-pool benchmark demonstrates that additional nodes can provide useful speedup despite the comparatively slow MicroHs runtime. The MicroSync case study exercises dynamic process creation, messaging, monitoring, generic servers, and supervisors in a larger application. A heterogeneous deployment connects two microcontrollers using 32-bit runtimes with a laptop using a 64-bit runtime, demonstrating that the same messaging abstractions and compatible graph format can operate across these platforms.

Moving the serialisation boundary into the runtime simplifies the source-level programming model, but also moves some guarantees out of the source-level type system. Messages can be checked against runtime type representations, while remote computations rely on stronger assumptions about the graph being reconstructed. Programmers must also avoid transmitting runtime-owned resources, unintentionally large graphs, or unevaluated computations whose work should have occurred on the sending node. Runtime graph serialisation therefore provides direct-style code and data mobility at the cost of weaker static guarantees and substantial runtime overhead.

Chapter 4

Conclusions

Together, the papers support a qualified answer to the thesis statement.

The common result is not that the boundary between Haskell and each target environment disappears. Instead, each paper represents that boundary in a form that can be managed by language abstractions, compiler transformations, or the runtime system. Scoria represents temporal and IO constraints through an embedded language. HasTEE represents placement and confidentiality constraints through types. The partitioning work represents endpoint projection through rewrite rules and specialisation. CloudMicroHaskell represents mobile code and data as serialisable runtime graphs.

The papers demonstrate these mechanisms separately. They do not provide a single program that is automatically partitioned, compiled, and deployed across microcontrollers, trusted execution environments, and cloud nodes. Nor do they statically enforce every safety property required by those environments. They establish that the constituent mechanisms are feasible and identify the guarantees and costs associated with each, while their integration remains an open problem.

Chapter 5

Chronology

The papers in this thesis are a segment of a longer line of work. Earlier projects led to them, and they have in turn led to projects that are not included here. This chapter follows that line, and I write it in the first person because the choices it describes were mine. Told in order, these projects may appear to follow a plan. In practice, my interests provided a direction, while opportunities and experiments determined the next problem.

I set out to program devices for the Internet of Things (IoT) in Haskell, using its high-level abstractions and keeping security in mind. Security could mean anything from relying on Haskell's memory safety to rule out a class of errors, to modelling security policies explicitly in the type system. IoT devices typically form networks and must communicate with each other. I began by contributing to Haski, an embedding of Lustre in Haskell, in which two devices communicated over Bluetooth Low Energy [51].

Haski illustrated how messy it can be to react to a spontaneous event in a system that is otherwise deterministic and ordered. My advisor and I kept thinking about this, and came across the Sparse Synchronous Model of Stephen Edwards and John Hui [24]. We thought the model might admit externally produced events into its ordered state more gracefully. We also suspected that its sparsity could save power, since a device that knows when its next scheduled event is due can sleep until then. The model had a runtime but no programming language, so together with Edwards and Hui we implemented one, Scoria, and wrote programs to see whether the model was what we were looking for.

Embedding Scoria in Haskell let us prototype rapidly. We implemented the compiler swiftly and reused GHC's parser and type checker. The model also admitted external events as we had hoped. The programmer benefited less than we had intended. An embedded language is only as powerful as the features we embed into it, and Scoria programs could use few of the abstractions that make Haskell attractive. In the end we did not evaluate power consumption, and Scoria ran on a single device and not on a network.

The second paper did not follow from Scoria. It began as an opportunity. Abhiroop Sarkar was studying Intel SGX and asked what a pure, statically

typed language could add to its threat model. My main lesson from HasTEE was how rigid GHC is. We spent a couple of months trying to get GHC to run in an SGX enclave. Its runtime relies on a rich operating system, which the enclave does not provide. We tried Hugs, and we tried running GHC's WebAssembly output in an interpreter inside the enclave, and both failed. In the end we patched the runtime to run on Gramine, a library operating system for enclaves.

We presented HasTEE at the Haskell Symposium in Seattle in 2023. At the co-located ICFP, a pearl presented HasChor, a library for choreographic programming in Haskell. HasChor is also a multiparty framework, but I observed that it did no partitioning at all. HasTEE's partitioning was type-directed, but in the end it was GHC's dead-code elimination that removed the code. What HasTEE had and HasChor lacked was one piece of information at compile time, namely which participant the binary being compiled is for. I thought that if I accepted compiling a HasChor program once per endpoint, and supplied that information each time, the compiler would have everything it needed to partition the program.

We had found GHC's optimiser reliable in partitioning HasTEE programs and never failing us, but I wanted a more deliberate mechanism. I asked how I could ensure that partitioning and endpoint projection happen at compile time. My answer was to turn the functions that do the partitioning into rewrite rules, which the compiler applies as it optimises. To determine whether partitioning succeeded, I implemented a Core plugin that inspects the optimised code and ensures that no reference to an unpartitioned component remains. This solution, presented in Paper 3, still relies on GHC's optimiser to do the partitioning. However, my intention is now explicit in the rules, and the plugin checks the result. The same mechanism applied to HasTEE with little effort.

The partitioning mechanism depended on GHC, whose runtime requirements prevented me from deploying these programs directly on the microcontrollers I wanted to target. I therefore turned to MicroHs, whose runtime is smaller and more portable. MicroHs does not support GHC's rewrite rules, so the mechanism could not carry over directly. The underlying idea still seemed applicable: if the location of a computation is known at compile time, a partial evaluator might use that information to remove computations intended for other participants. This led me to begin work on a partial evaluator for the combinator graphs that MicroHs uses. My hope was that it could partition programs by evaluating away the abstractions that describe their participants and locations.

Alongside the partial evaluator, I began work on a project with the working name MicroHasTEE, bringing HasTEE's multiparty programming model to microcontrollers with Armv8-M TrustZone. Using MicroHs, I implemented a prototype that executes Haskell in both the Secure and Non-secure states without an operating system. This brought me back to the problem of programming embedded devices, now with Haskell itself rather than an embedded language. To serialise data crossing between the two domains, I used a runtime primitive that serialises the combinator graph. While experimenting, I discovered that the primitives could serialise even functions, an unexpected and

exciting consequence of their design. My advisor and I thought this might let us build a simpler version of Cloud Haskell, without requiring the programmer to construct static closures explicitly. I began experimenting with a small prototype, which grew into CloudMicroHaskell.

The work continues beyond the papers included in this thesis. With MicroHasTEE, I am exploring how Haskell's type system can express the constraints of programming across a microcontroller's security boundary. With the partial evaluator, I am investigating whether partial evaluation can bring program partitioning to MicroHs. Both projects remain ongoing, and I return to the questions they raise in chapter 6.

Chapter 6

Future Work

The papers demonstrate their mechanisms in isolation, and the ongoing projects described in chapter 5 are first attempts to combine them. MicroHasTEE brings HasTEE's programming model to the portable MicroHs runtime, and the partial evaluator aims to bring compile-time partitioning to the same runtime. Neither project yet answers the question that opened this thesis. That question asks for a single system that automatically partitions, compiles, and deploys Haskell programs across microcontrollers, trusted execution environments, and cloud nodes. That system must accommodate the constraints of several execution environments while providing abstractions that span their boundaries. The following sections break this goal into concrete research questions.

6.1 Multiparty Haskell

CloudMicroHaskell supports a dynamic model of distributed computation inspired in part by Erlang. Processes can be created and terminated at runtime, and nodes can exchange both messages and executable code. This flexibility raises a natural question: what would multiparty programming add when one node can already send a computation to another?

Code mobility and multiparty programming address complementary concerns. Code mobility allows a computation selected at runtime to be moved to another node, while multiparty programming constrains how the participants interact. CloudMicroHaskell's current programming model does not prevent mismatched messages, unintended message ordering, or protocol deadlocks. A choreographic framework or protocol description language could rule out some of these errors while retaining CloudMicroHaskell's dynamic process creation and code mobility.

This leads to the following research question.

> *Can multiparty programming constrain the interactions among CloudMicroHaskell processes without giving up dynamic process creation, placement, and code mobility?*

Conventional multiparty programs generally describe a fixed set of named participants. In CloudMicroHaskell, however, processes may be created at runtime, several processes may perform the same role, and their host nodes may not be selected until the program is running. Combining the two models therefore raises several concrete questions:

- Can a choreography describe a role that may be filled by any member of a pool of interchangeable processes? For example, a server might send each request to an available worker selected at runtime.
- How should endpoint projection interact with runtime placement? Could the system project a program for a role and then use CloudMicroHaskell's graph serialisation to send it to the node selected to execute it?
- How should the protocol respond when a dynamically created participant terminates or its host node fails?

An initial investigation could focus on a server communicating with a dynamically sized pool of workers. The choreography would describe the protocol between the server and a worker, while CloudMicroHaskell would select and place the particular worker process at runtime. This case would test whether a static description of the communication protocol can coexist with dynamic process placement.

The resulting system should be compared with the same application written directly using CloudMicroHaskell. The comparison should determine which communication errors the multiparty description rules out, what restrictions it places on dynamic behaviour, and what runtime overhead it introduces.

The objective need not be a multiparty language supporting arbitrary changes to the set of participants. A useful first result would establish whether choreographic guarantees can be retained when instances of a role are selected and placed at runtime. This would connect the endpoint projection explored in Paper 3 with the code mobility provided by Paper 4.

6.2 Haskell on Microcontrollers with Trusted Execution Environments

The MicroHs compiler and runtime did not exist when the work in this thesis began. MicroHs now makes it possible to investigate whether HasTEE's programming model can be adapted to microcontrollers with Armv8-M TrustZone.

On such a microcontroller, an application can restrict access to a peripheral to the Secure domain or permit access from both domains. Conventional development involves writing separate Secure and Non-secure programs, typically in C. These programs must agree about resource assignments and the interfaces through which they communicate. Inconsistent assumptions may remain undetected until an operation triggers a hardware fault.

A multiparty Haskell program can describe both components together. The type system can then check each component's resource use and cross-domain

calls against a shared description of the security configuration. This leads to the following research questions.

> *Can Haskell, a high-level functional language, execute directly on a microcontroller in both the Secure and Non-secure domains of Armv8-M TrustZone, without an operating system? Can a multi-party programming model use Haskell's type system and abstractions to rule out inconsistent security configurations and cross-domain interactions? What additional security guarantees can this model provide under Armv8-M TrustZone's threat model?*

Writing both components in one program allows the framework to check their agreement, but does not make the Non-secure component trustworthy. The framework helps developers avoid mistakes in resource assignments and cross-domain interfaces. After deployment, however, the Non-secure component may be compromised. The Secure component must therefore remain protected even when calls through its exposed interface do not follow the original Haskell program. This raises the question of which guarantees can be established by type checking and which require validation at the security boundary.

Work on this topic is already underway in MicroHasTEE. A working prototype lets developers describe both components in one Haskell program. For programs written through its public interface, the prototype checks agreement about resource assignments and cross-domain interfaces at compile time. Further work should formalise the programming model and establish the soundness of these checks. Case studies should establish whether the programming model supports realistic applications and identify which configuration errors it rejects. Comparison with equivalent C implementations should quantify the memory requirements and cross-domain call latency introduced by running Haskell in both domains.

6.3 Performance and Resource Predictability in MicroHs

MicroHs allows Haskell programs to run natively on microcontrollers. Its combinator-graph representation also allows code and data to move between running instances. Together, these capabilities support the broader objective of using a common programming and execution model across heterogeneous devices. CloudMicroHaskell demonstrates this possibility by moving Haskell computations between laptops and microcontrollers running compatible 32-bit and 64-bit runtimes.

However, the performance and unpredictable resource use of MicroHs restrict the applications for which this approach is practical. Some conventional routes to higher performance would weaken the properties that make MicroHs useful in this setting. Compiling functions to platform-specific native code could reduce evaluation overhead, but would prevent those functions from moving between heterogeneous nodes in their current form. Extending the evaluator

with many specialised operations could similarly increase performance while enlarging the runtime and the graph format understood by each participating node.

This tension leads to the following research question.

> *How far can the execution performance and resource predictability of MicroHs be improved without sacrificing its small runtime and serialisable graph representation?*

This question has two related dimensions. The first is whether compiler and runtime improvements can reduce execution time while retaining the representation required for code mobility. The second is whether memory consumption and garbage-collection pauses can be characterised well enough to determine which microcontroller applications can safely use the runtime.

Work on execution performance should begin by profiling the evaluator on representative microcontroller workloads. These workloads should include computation-heavy programs, allocation-heavy programs, and CloudMicroHaskell processes that exchange computations between nodes. The profiling should distinguish the costs of combinator dispatch, graph allocation and rewriting, primitive operations, and garbage collection. It would thereby establish which costs dominate in practice before particular compiler or runtime changes are pursued.

Compiler optimisations could reduce the amount of work left to the evaluator by producing smaller or more efficient combinator graphs. One possibility is to specialise functions when their type-class dictionaries are statically known, as when a particular `Num` instance has been selected. Profiling should establish how much runtime work these dictionary operations require, and evaluation should determine whether specialisation provides sufficient benefit to justify any increase in graph size.

Runtime improvements offer a complementary route to better performance. The evaluator interprets a fixed set of combinators, while the compiler expresses each user-defined function in terms of those combinators. Consequently, a function application may require many small graph rewrites merely to arrange its arguments. Specialised combinators could perform common reduction sequences in fewer steps, while changes to graph allocation could reduce the cost of constructing intermediate nodes.

Such changes must be evaluated against the constraints expressed by the research question. Each new combinator may enlarge the runtime and extend the graph vocabulary that participating nodes must understand. A change that improves local execution performance could therefore increase the runtime footprint. The evaluation should measure these costs alongside any reduction in execution time.

Resource predictability presents a separate challenge. Lazy functional programs allocate frequently and create thunks whose evaluation depends on runtime demand. The amount of live memory and the time at which computations execute can therefore be difficult to predict. Reducing the initial graph through partial evaluation may help, but a smaller initial graph does not by itself guarantee lower peak memory consumption during execution.

Garbage collection must also be considered when programs have timing requirements. A stop-the-world collector with an unknown pause time prevents useful claims about worst-case response time. An initial evaluation should characterise collection pauses as a function of heap size, live data, and allocation behaviour. It should also determine the maximum memory demand and maximum observed pause for each representative workload. If the resulting pauses cannot be bounded or predicted usefully, alternative collection strategies could be investigated, with their additional runtime size and complexity included in the comparison.

The objective need not be performance parity with generated C or a guarantee that arbitrary Haskell programs satisfy hard real-time constraints. Instead, the work should identify the range of microcontroller applications for which native Haskell execution is practical and determine how that range can be enlarged without losing the compact and portable graph representation on which heterogeneous code mobility depends.

6.4 Partial Evaluation of Combinator Graphs

MicroHs enables us to run Haskell natively on embedded devices, but the program partitioning techniques used in this thesis rely on GHC's optimisation passes. MicroHs discards trivially unreachable code, but lacks the optimisation passes on which these techniques depend. Bringing program partitioning to MicroHs therefore requires another approach.

Endpoint projection suggests one possible approach. In HasChor, the function that projects a multiparty program onto an endpoint has a type similar to:

```
1 epp :: Program Multiparty -> Endpoint -> Program Endpoint
```

Given a multiparty program and an endpoint, this function returns a program describing that endpoint's actions. Projection itself is pure, even when the projected program performs effects. If the endpoint and enough of the program's structure are known before execution, partial evaluation could perform the projection while leaving input-dependent computations and effects for runtime. The question is whether this process can also eliminate the code belonging to other endpoints.

This leads to the following research questions.

> *Can partial evaluation of MicroHs combinator graphs partition multiparty programs while preserving the selected endpoint's behaviour? What information must be known statically, and how does partial evaluation affect graph size, runtime reduction count, and execution time?*

Work on a prototype partial evaluator is already underway. It can simplify computations with partially known inputs and produce residual IO computations whose effects remain to be performed at runtime. An experimental fusion mechanism can also eliminate an intermediate list in a producer–consumer

example, although it currently requires manual annotations. Further work should investigate whether these mechanisms suffice to partition representative multiparty programs, how to control unfolding so that partial evaluation terminates, and how to establish that the residual programs preserve the intended behaviour.

6.5 Preserving Scoria's Guarantees in a Common Runtime

Scoria currently bridges the gap to resource-constrained microcontrollers by constructing an explicit syntax tree and compiling it to C. The generated program executes on a small runtime designed specifically for Scoria. This approach avoids placing a Haskell runtime on the device, but it also gives Scoria a separate execution model from the Haskell computations used elsewhere in this thesis.

A first step towards a common execution platform is therefore to determine whether Scoria can execute within a Haskell runtime at all. The MicroHs runtime provides a suitable platform for this investigation because it can execute Haskell on the same class of resource-constrained devices targeted by Scoria. A shallow embedding can represent Scoria operations directly as Haskell computations executed by MicroHs. A prototype of this embedding exists, but its semantics and temporal behaviour have not yet been evaluated systematically.

This leads to the following research question.

> *Can Scoria's deterministic sparse-synchronous semantics and predictable timing behaviour be preserved when Scoria programs execute within a general Haskell runtime such as MicroHs?*

Moving Scoria into MicroHs changes more than its implementation language. The existing compiler controls the representation and execution of the complete Scoria program. A shallow embedding instead executes alongside lazy Haskell computations whose evaluation may allocate memory, trigger garbage collection, or perform unrelated runtime work. The central semantic question is whether this execution model can preserve Scoria's logical instants, deterministic priority ordering, future scheduling, and controlled introduction of asynchronous inputs.

Semantic correspondence could be tested by running generated Scoria programs through the existing interpreter, the C backend, and the shallow implementation. Each implementation could emit traces containing model times, scheduled-variable updates, thread activations, and observable outputs. Comparing these traces would test whether the shallow implementation preserves Scoria's behaviour, including when several events occur at the same logical instant or the system becomes overloaded.

The Sparse Synchronous Model may also provide information that helps the runtime preserve physical timing. At the end of an instant, the runtime knows the physical deadline associated with the next internally scheduled event.

Rather than immediately sleeping, it could use some of the intervening time for garbage collection and other runtime maintenance. It could also collect before entering the next instant and reserve enough free heap to complete that instant without another collection. This would make garbage collection part of the temporal scheduling policy rather than allowing it to interrupt reactions unpredictably.

This approach raises further questions. A collection must finish or be pre-empted before the next deadline, while an asynchronous hardware event may arrive earlier than any internally scheduled event. The runtime must therefore determine when maintenance can safely run, how much work can be completed in the available interval, and how incoming events are timestamped and buffered while that work is in progress. The evaluation should establish whether these policies reduce timing variation and prevent garbage-collection pauses from disrupting logical instants.

There are three relevant implementation points to compare. A pure shallow embedding would express Scoria entirely using ordinary MicroHs computations and require no Scoria-specific support from the evaluator. A second design could implement scheduling, scheduled variables, and other Scoria operations as primitive combinators handled directly by the runtime. This may reduce allocation and evaluation overhead, but would enlarge the runtime and tie the implementation more closely to MicroHs. The existing C backend provides the third point of comparison and offers greater control over execution at the cost of requiring a separate runtime.

The comparison should measure semantic correspondence, response latency, timing variation, overload behaviour, allocation, peak memory use, and garbage-collection pauses on representative Scoria programs. The objective need not be performance parity with generated C. It should instead establish whether a general Haskell runtime can support the properties that motivate Scoria, and identify the runtime mechanisms and resource limits under which those properties hold.

If this is feasible, later work can investigate how Scoria computations interact with the partitioning, security, and distributed abstractions developed elsewhere in the thesis. Establishing a compatible execution model is a necessary first step towards that integration.

Bibliography

[1] S. P. Jones, C. Hall, K. Hammond, W. Partain and P. Wadler, "The Glasgow Haskell compiler: A technical overview," in *Proc. UK Joint Framework for Information Technology (JFIT) Technical Conference*, 1993, pp. 249–257 (cit. on p. 8).

[2] I. Duncan. "A Couple Million Lines of Haskell: Production Engineering at Mercury," Accessed: 18th Jul. 2026. [Online]. Available: https://blog.haskell.org/a-couple-million-lines-of-haskell/ (cit. on p. 8).

[3] R. Krook and S. Hammersberg, "Welcome to the Parti(tioning) (Functional Pearl): Using Rewrite Rules and Specialisation to Partition Haskell Programs," in *Proceedings of the 17th ACM SIGPLAN International Haskell Symposium, Haskell 2024, Milan, Italy, September 6-7, 2024*, N. Vazou and J. G. Morris, Eds., ACM, 2024, pp. 27–40. DOI: 10.1145/3677999.3678276 [Online]. Available: https://doi.org/10.1145/3677999.3678276 (cit. on pp. 9, 13, 46).

[4] L. Augustsson, "Microhs: A Small Compiler for Haskell," in *Proceedings of the 17th ACM SIGPLAN International Haskell Symposium, Haskell 2024, Milan, Italy, September 6-7, 2024*, N. Vazou and J. G. Morris, Eds., ACM, 2024, pp. 120–124. DOI: 10.1145/3677999.3678280 [Online]. Available: https://doi.org/10.1145/3677999.3678280 (cit. on p. 9).

[5] R. Krook and L. Augustsson, "Cloudmicrohaskell: Direct-Style Distributed Haskell via Runtime Graph Serialisation," in *Proceedings of the 19th ACM SIGPLAN International Haskell Symposium, Haskell 2026, Indianapolis, IN, USA, August 24-29, 2026*, L. Kuper and N. Xie, Eds., ACM, 2026, pp. 34–64. DOI: 10.1145/3830439.3831272 [Online]. Available: https://doi.org/10.1145/3830439.3831272 (cit. on pp. 9, 13, 47).

[6] M. Schönfinkel, "Über die Bausteine der mathematischen Logik," *Mathematische annalen*, vol. 92, no. 3, pp. 305–316, 1924 (cit. on p. 9).

[7] H. B. Curry, "Grundlagen der kombinatorischen Logik," *American journal of mathematics*, vol. 52, no. 4, pp. 789–834, 1930 (cit. on p. 9).

[8] D. A. Turner, "A New Implementation Technique for Applicative Languages," *Softw. Pract. Exp.*, vol. 9, no. 1, pp. 31–49, 1979. DOI: 10.1002/SPE.4380090105 [Online]. Available: https://doi.org/10.1002/spe.4380090105 (cit. on p. 9).

[9] P. Hudak, "Domain-specific languages," *Handbook of programming languages*, vol. 3, no. 39-60, p. 21, 1997 (cit. on p. 12).

[10] K. Claessen and J. Hughes, "Quickcheck: A Lightweight Tool for Random Testing of Haskell Programs," in *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming (ICFP '00), Montreal, Canada, September 18-21, 2000*, M. Odersky and P. Wadler, Eds., ACM, 2000, pp. 268–279. DOI: 10.1145/351240.351266 [Online]. Available: https://doi.org/10.1145/351240.351266 (cit. on p. 12).

[11] L. Pike, A. Goodloe, R. Morisset and S. Niller, "Copilot: A Hard Real-Time Runtime Monitor," in *Runtime Verification - First International Conference, RV 2010, St. Julians, Malta, November 1-4, 2010. Proceedings*, H. Barringer et al., Eds., ser. Lecture Notes in Computer Science, vol. 6418, Springer, 2010, pp. 345–359. DOI: 10.1007/978-3-642-16612-9_26 [Online]. Available: https://doi.org/10.1007/978-3-642-16612-9_26 (cit. on pp. 12, 41).

[12] J. Svenningsson and E. Axelsson, "Combining Deep and Shallow Embedding for EDSL," in *Trends in Functional Programming - 13th International Symposium, TFP 2012, St. Andrews, UK, June 12-14, 2012, Revised Selected Papers*, H. Loidl and R. Peña, Eds., ser. Lecture Notes in Computer Science, vol. 7829, Springer, 2012, pp. 21–36. DOI: 10.1007/978-3-642-40447-4_2 [Online]. Available: https://doi.org/10.1007/978-3-642-40447-4_2 (cit. on p. 13).

[13] R. Krook, J. Hui, B. J. Svensson, S. A. Edwards and K. Claessen, "Creating a Language for Writing Real-Time Applications for the Internet of Things," in *20th ACM-IEEE International Conference on Formal Methods and Models for System Design, MEMOCODE 2022, Shanghai, China, October 13-14, 2022*, IEEE, 2022, pp. 1–20. DOI: 10.1109/MEMOCODE57689.2022.9954383 [Online]. Available: https://doi.org/10.1109/MEMOCODE57689.2022.9954383 (cit. on pp. 13, 45).

[14] A. Sarkar, R. Krook, A. Russo and K. Claessen, "Hastee: Programming Trusted Execution Environments with Haskell," in *Proceedings of the 16th ACM SIGPLAN International Haskell Symposium, Haskell 2023, Seattle, WA, USA, September 8-9, 2023*, T. L. McDonell and N. Vazou, Eds., ACM, 2023, pp. 72–88. DOI: 10.1145/3609026.3609731 [Online]. Available: https://doi.org/10.1145/3609026.3609731 (cit. on pp. 13, 46).

[15] G. Kahn, "The Semantics of a Simple Language for Parallel Programming," in *Information Processing, Proceedings of the 6th IFIP Congress 1974, Stockholm, Sweden, August 5-10, 1974*, J. L. Rosenfeld, Ed., North-Holland, 1974, pp. 471–475 (cit. on p. 14).

[16] E. A. Ashcroft and W. W. Wadge, "Lucid - A Formal System for Writing and Proving Programs," *SIAM J. Comput.*, vol. 5, no. 3, pp. 336–354, 1976. DOI: 10.1137/0205029 [Online]. Available: https://doi.org/10.1137/0205029 (cit. on p. 14).

[17] W. W. Wadge and E. A. Ashcroft, *Lucid, the Dataflow Programming Language* (APIC Studies in Data Processing 22). London: Academic Press, 1985, ISBN: 978-0-12-729650-0 (cit. on p. 14).

[18] G. Berry, S. Moisan and J.-P. Rigault, "Esterel: Towards a synchronous and semantically sound high-level language for real-time applications," in *Proc. IEEE Real-Time Systems Symposium*, IEEE Computer Society Press, 1983, pp. 30–40 (cit. on p. 15).

[19] G. Berry and G. Gonthier, "The Esterel Synchronous Programming Language: Design, Semantics, Implementation," *Sci. Comput. Program.*, vol. 19, no. 2, pp. 87–152, 1992. DOI: 10.1016/0167-6423(92)90005-V [Online]. Available: https://doi.org/10.1016/0167-6423(92)90005-V (cit. on p. 15).

[20] P. Le Guernic, A. Benveniste, P. Bournai and T. Gautier, "SIGNAL-A Data Flow-Oriented Language for Signal Processing," *IEEE Trans. Acoust. Speech Signal Process.*, vol. 34, no. 2, pp. 362–374, 1986. DOI: 10.1109/TASSP.1986.1164809 [Online]. Available: https://doi.org/10.1109/TASSP.1986.1164809 (cit. on p. 16).

[21] P. Caspi, D. Pilaud, N. Halbwachs and J. Plaice, "LUSTRE: A Declarative Language for Programming Synchronous Systems," in *Conference Record of the Fourteenth Annual ACM Symposium on Principles of Programming Languages, Munich, Germany, January 21-23, 1987*, ACM Press, 1987, pp. 178–188. DOI: 10.1145/41625.41641 [Online]. Available: https://doi.org/10.1145/41625.41641 (cit. on p. 16).

[22] N. Halbwachs, P. Caspi, P. Raymond and D. Pilaud, "The Synchronous Data Flow Programming Language LUSTRE," *Proc. IEEE*, vol. 79, no. 9, pp. 1305–1320, 1991. DOI: 10.1109/5.97300 [Online]. Available: https://doi.org/10.1109/5.97300 (cit. on p. 16).

[23] P. Caspi and M. Pouzet, "Synchronous Kahn Networks," in *Proceedings of the 1996 ACM SIGPLAN International Conference on Functional Programming, ICFP 1996, Philadelphia, Pennsylvania, USA, May 24-26, 1996*, R. Harper and R. L. Wexelblat, Eds., ACM, 1996, pp. 226–238. DOI: 10.1145/232627.232651 [Online]. Available: https://doi.org/10.1145/232627.232651 (cit. on p. 17).

[24] S. A. Edwards and J. Hui, "The Sparse Synchronous Model," in *Forum for Specification and Design Languages, FDL 2020, Kiel, Germany, September 15-17, 2020*, IEEE, 2020, pp. 1–8. DOI: 10.1109/FDL50818.2020.9232938 [Online]. Available: https://doi.org/10.1109/FDL50818.2020.9232938 (cit. on pp. 18, 51).

[25] M. Lohstroh, C. Menard, S. Bateni and E. A. Lee, "Toward a Lingua Franca for Deterministic Concurrent Systems," *ACM Trans. Embed. Comput. Syst.*, vol. 20, no. 4, 36:1–36:27, 2021. DOI: 10.1145/3448128 [Online]. Available: https://doi.org/10.1145/3448128 (cit. on p. 20).

[26] Intel. "Intel Software Guard Extension. "[Online]. Available: https://www.intel.com/content/www/us/en/developer/tools/software-guard-extensions/overview.html (accessed: 16.02.2023) (cit. on p. 21).

[27] Arm Limited, *Arm TrustZone Technology for the Armv8-M Architecture*, 100690, version 2.1, 30th Oct. 2018. Accessed: 20th Jul. 2026. [Online]. Available: https://developer.arm.com/documentation/100690/0201 (cit. on p. 21).

[28] C. Tsai, D. E. Porter and M. Vij, "Graphene-SGX: A Practical Library OS for Unmodified Applications on SGX," in *2017 USENIX Annual Technical Conference, USENIX ATC 2017, Santa Clara, CA, USA, July 12-14, 2017*, D. D. Silva and B. Ford, Eds., USENIX Association, 2017, pp. 645–658. [Online]. Available: https://www.usenix.org/conference/atc17/technical-sessions/presentation/tsai (cit. on p. 22).

[29] D. E. Denning, "A Lattice Model of Secure Information Flow," *Commun. ACM*, vol. 19, no. 5, pp. 236–243, 1976. DOI: 10.1145/360051.360056 [Online]. Available: https://doi.org/10.1145/360051.360056 (cit. on p. 22).

[30] J. A. Goguen and J. Meseguer, "Security Policies and Security Models," in *1982 IEEE Symposium on Security and Privacy, Oakland, CA, USA, April 26-28, 1982*, IEEE Computer Society, 1982, pp. 11–20. DOI: 10.1109/SP.1982.10014 [Online]. Available: https://doi.org/10.1109/SP.1982.10014 (cit. on p. 22).

[31] A. Sabelfeld and D. Sands, "Dimensions and Principles of Declassification," in *18th IEEE Computer Security Foundations Workshop, (CSFW-18 2005), 20-22 June 2005, Aix-en-Provence, France*, IEEE Computer Society, 2005, pp. 255–269. DOI: 10.1109/CSFW.2005.15 [Online]. Available: https://doi.org/10.1109/CSFW.2005.15 (cit. on p. 23).

[32] D. Hedin and A. Sabelfeld, "A Perspective on Information-Flow Control," in *Software Safety and Security - Tools for Analysis and Verification*, ser. NATO Science for Peace and Security Series - D: Information and Communication Security, T. Nipkow, O. Grumberg and B. Hauptmann, Eds., vol. 33, IOS Press, 2012, pp. 319–347. DOI: 10.3233/978-1-61499-028-4-319 [Online]. Available: https://doi.org/10.3233/978-1-61499-028-4-319 (cit. on pp. 23, 24).

[33] A. Russo, K. Claessen and J. Hughes, "A Library for Light-Weight Information-Flow Security in Haskell," in *Proceedings of the 1st ACM SIGPLAN Symposium on Haskell, Haskell 2008, Victoria, BC, Canada, 25 September 2008*, A. Gill, Ed., ACM, 2008, pp. 13–24. DOI: 10.1145/1411286.1411289 [Online]. Available: https://doi.org/10.1145/1411286.1411289 (cit. on p. 24).

[34] S. Giallorenzo, F. Montesi, M. Peressotti, D. Richter, G. Salvaneschi and P. Weisenburger, "Multiparty Languages: The Choreographic and Multitier Cases (Pearl)," in *35th European Conference on Object-Oriented Programming, ECOOP 2021, Aarhus, Denmark (Virtual Conference), July 11-17, 2021*, A. Møller and M. Sridharan, Eds., ser. LIPIcs, vol. 194, Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2021, 22:1–22:27. DOI: 10.4230/LIPICS.ECOOP.2021.22 [Online]. Available: https://doi.org/10.4230/LIPIcs.ECOOP.2021.22 (cit. on p. 25).

[35] E. Cooper, S. Lindley, P. Wadler and J. Yallop, "Links: Web Programming Without Tiers," in *Formal Methods for Components and Objects, 5th International Symposium, FMCO 2006, Amsterdam, The Netherlands, November 7-10, 2006, Revised Lectures*, F. S. de Boer, M. M. Bonsangue, S. Graf and W. P. de Roever, Eds., ser. Lecture Notes in Computer Science, vol. 4709, Springer, 2006, pp. 266–296. DOI: 10.1007/978-3-540-74792-5_12 [Online]. Available: https://doi.org/10.1007/978-3-540-74792-5_12 (cit. on p. 25).

[36] M. Serrano, E. Gallesio and F. Loitsch, "Hop: A Language for Programming the Web 2.0," in *Companion to the 21th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications, OOPSLA 2006, October 22-26, 2006, Portland, Oregon, USA*, P. L. Tarr and W. R. Cook, Eds., ACM, 2006, pp. 975–985. DOI: 10.1145/1176617.1176756 [Online]. Available: https://doi.org/10.1145/1176617.1176756 (cit. on p. 25).

[37] F. Montesi, "Choreographic Programming," Ph.D. dissertation, IT University of Copenhagen, 2014 (cit. on p. 25).

[38] G. Shen, S. Kashiwa and L. Kuper, "Haschor: Functional Choreographic Programming for All (Functional Pearl)," *Proc. ACM Program. Lang.*, vol. 7, no. ICFP, pp. 541–565, 2023. DOI: 10.1145/3607849 [Online]. Available: https://doi.org/10.1145/3607849 (cit. on p. 28).

[39] P. W. Trinder, K. Hammond, J. S. M. Jr., A. S. Partridge and S. L. P. Jones, "GUM: A portable parallel implementation of Haskell," in *Proceedings of the ACM SIGPLAN'96 Conference on Programming Language Design and Implementation (PLDI), Philadephia, Pennsylvania, USA, May 21-24, 1996*, C. N. Fischer, Ed., ACM, 1996, pp. 79–88. DOI: 10.1145/231379.231392 [Online]. Available: https://doi.org/10.1145/231379.231392 (cit. on p. 29).

[40] S. L. P. Jones, A. D. Gordon and S. Finne, "Concurrent Haskell," in *Conference Record of POPL'96: The 23rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, Papers Presented at the Symposium, St. Petersburg Beach, Florida, USA, January 21-24, 1996*, H. Boehm and G. L. S. Jr., Eds., ACM Press, 1996, pp. 295–308. DOI: 10.1145/237721.237794 [Online]. Available: https://doi.org/10.1145/237721.237794 (cit. on p. 30).

[41] R. F. Pointon, P. W. Trinder and H. Loidl, "The Design and Implementation of Glasgow Distributed Haskell," in *Implementation of Functional Languages, 12th International Workshop, IFL 2000, Aachen, Germany, September 4-7, 2000, Selected Papers*, M. Mohnen and P. W. M. Koopman, Eds., ser. Lecture Notes in Computer Science, vol. 2011, Springer, 2000, pp. 53–70. DOI: 10.1007/3-540-45361-X_4 [Online]. Available: https://doi.org/10.1007/3-540-45361-X_4 (cit. on pp. 30, 31).

[42] R. Loogen, Y. Ortega-Mallén and R. Peña-Marí, "Parallel functional programming in Eden," *J. Funct. Program.*, vol. 15, no. 3, pp. 431–475, 2005. DOI: 10.1017/S0956796805005526 [Online]. Available: https://doi.org/10.1017/S0956796805005526 (cit. on p. 31).

[43] A. R. D. Bois, P. W. Trinder and H. Loidl, "Implementing Mobile Haskell," in *Revised Selected Papers from the Fourth Symposium on Trends in Functional Programming, TFP 2003, Edinburgh, United Kingdom, 11-12 September 2003*, S. Gilmore, Ed., ser. Trends in Functional Programming, vol. 4, Intellect, 2003, pp. 79–94 (cit. on p. 32).

[44] J. Epstein, A. P. Black and S. L. P. Jones, "Towards Haskell in the Cloud," in *Proceedings of the 4th ACM SIGPLAN Symposium on Haskell, Haskell 2011, Tokyo, Japan, 22 September 2011*, K. Claessen, Ed., ACM, 2011, pp. 118–129. DOI: 10.1145/2034675.2034690 [Online]. Available: https://doi.org/10.1145/2034675.2034690 (cit. on p. 34).

[45] P. Maier and P. W. Trinder, "Implementing a High-Level Distributed-Memory Parallel Haskell in Haskell," in *Implementation and Application of Functional Languages - 23rd International Symposium, IFL 2011, Lawrence, KS, USA, October 3-5, 2011, Revised Selected Papers*, A. Gill and J. Hage, Eds., ser. Lecture Notes in Computer Science, vol. 7257, Springer, 2011, pp. 35–50. DOI: 10.1007/978-3-642-34407-7_3 [Online]. Available: https://doi.org/10.1007/978-3-642-34407-7_3 (cit. on p. 36).

[46] P. Chiusano, *Unison: A New Distributed Programming Language*, Presentation at Strange Loop, Sep. 2019. Accessed: 20th Aug. 2026. [Online]. Available: https://www.thestrangeloop.com/2019/unison-a-new-distributed-programming-language.html (cit. on p. 38).

[47] Unison Computing. "The Big Idea," Accessed: 20th Aug. 2026. [Online]. Available: https://www.unison-lang.org/docs/the-big-idea/ (cit. on p. 38).

[48] J. C. Reynolds, "Separation Logic: A Logic for Shared Mutable Data Structures," in *17th IEEE Symposium on Logic in Computer Science (LICS 2002), 22-25 July 2002, Copenhagen, Denmark, Proceedings*, IEEE Computer Society, 2002, pp. 55–74. DOI: 10.1109/LICS.2002.1029817 [Online]. Available: https://doi.org/10.1109/LICS.2002.1029817 (cit. on p. 40).

[49] T. Elliott et al., "Guilt Free Ivory," in *Proceedings of the 8th ACM SIGPLAN Symposium on Haskell, Haskell 2015, Vancouver, BC, Canada, September 3-4, 2015*, B. Lippmeier, Ed., ACM, 2015, pp. 189–200. DOI: 10.1145/2804302.2804318 [Online]. Available: https://doi.org/10.1145/2804302.2804318 (cit. on p. 40).

[50] P. C. Hickey, L. Pike, T. Elliott, J. Bielman and J. Launchbury, "Building Embedded Systems with Embedded DSLs," in *Proceedings of the 19th ACM SIGPLAN international conference on Functional programming, Gothenburg, Sweden, September 1-3, 2014*, J. Jeuring and M. M. T. Chakravarty, Eds., ACM, 2014, pp. 3–9. DOI: 10.1145/2628136.2628146

[Online]. Available: https://doi.org/10.1145/2628136.2628146 (cit. on p. 41).

[51] N. Valliappan, R. Krook, A. Russo and K. Claessen, "Towards Secure IoT Programming in Haskell," in *Proceedings of the 13th ACM SIGPLAN International Symposium on Haskell, Haskell@ICFP 2020, Virtual Event, USA, August 7, 2020*, T. Schrijvers, Ed., ACM, 2020, pp. 136–150. DOI: 10.1145/3406088.3409027 [Online]. Available: https://doi.org/10.1145/3406088.3409027 (cit. on pp. 42, 51).

Part II

Appended Papers

Paper I

Creating a Language for Writing Real-Time Applications for the Internet of Things

Robert Krook, John Hui, Bo Joel Svensson, Stephen A. Edwards, Koen Claessen

Proceedings of the 20th ACM-IEEE International Conference on Formal Methods and Models for System Design (MEMOCODE), 2022

Creating a Language for Writing Real-Time Applications for the Internet of Things

ROBERT KROOK, Chalmers University of Technology, Sweden
JOHN HUI, Columbia University, USA
BO JOEL SVENSSON, Chalmers University of Technology, Sweden
STEPHEN A. EDWARDS, Columbia University, USA
KOEN CLAESSEN, Chalmers University of Technology, Sweden

We describe the development of a new programming language Scoria and its compiler. Scoria is a high-level reactive real-time language based on the sparse synchronous model (SSM), designed to produce time- and power-efficient low-level C code that can run on small IoT devices. While the compiler is not yet in a state where it is meaningful to measure power usage, we carefully profile the timing behaviour and identify bottlenecks that can improve performance. The language and compiler are implemented as an Embedded Domain-Specific Language (EDSL) on top of Haskell.

Additional Key Words and Phrases: Real-time, IoT, Compilers, Embedded Domain-Specific Languages

1 Introduction

Devices for the Internet of Things (IoT) typically contain hardware for sensors, actuators, and wireless communication, and often need to run on batteries whose life expectancy is a couple of years. These peripherals and resources need to be carefully managed in order to maximize lifetime and reliability. The devices are typically programmed in C because of their low-level nature, timing, and power requirements. They are coded in a *reactive* style as the software needs to react to external events, such as a triggered sensor. Such programs are often written to register a myriad of callback functions to handle these events and communicate with other parts of the program. The resulting asynchronous code is often error-prone and hard to maintain, a situation often dubbed "callback hell."

Our language, Scoria, eschews callbacks in favor of lightweight threads that can block on external and internal events and provides precise control over the time at which code executes. We embed [14] Scoria in Haskell—which explains its syntactic idiosyncrasies—to enable rapid development of its compiler, which generates C code that uses our runtime system (RTS) to interface with hardware timers and other peripherals.

Figure 1 is a simple Scoria program: a remote-controlled square wave signal generator whose frequency can be adjusted by Bluetooth Low Energy (BLE) radio

Authors' Contact Information: Robert Krook, krookr@chalmers.se, Chalmers University of Technology, Gothenburg, Sweden; John Hui, j-hui@cs.columbia.edu, Columbia University, New York, USA; Bo Joel Svensson, joels@chalmers.se, Chalmers University of Technology, Gothenburg, Sweden; Stephen A. Edwards, sedwards@cs.columbia.edu, Columbia University, New York, USA; Koen Claessen, koen@chalmers.se, Chalmers University of Technology, Gothenburg, Sweden.

```
1  sigGen :: (?out0 :: Ref GPIO) => Ref Word64 -> SSM ()
2  sigGen hperiod = routine $ while true (do
3    after (ns (deref hperiod)) ?out0 (not' (deref ?out0))
4    wait ?out0)
5
6  remoteControl :: (?ble :: BLE) => Ref Word64 -> SSM ()
7  remoteControl hperiod = routine $ do
8    enableScan ?ble
9    while true (do
10     wait (scanref ?ble)
11     if deref (scanref ?ble) ==. 0
12       then hperiod <~ deref hperiod * 2
13       else hperiod <~ max' (deref hperiod /. 2) 1)
14
15 entry :: (?ble :: BLE, ?out0 :: Ref GPIO) => SSM ()
16 entry = routine $ do
17   hperiod <- var (time2ns (secs 1))
18   fork [sigGen hperiod, remoteControl hperiod]
19
```

Fig. 1. A remote-control signal generator in Scoria. `sigGen` produces a square wave on the `out0` GPIO pin that cycles every two `hperiod` nanoseconds. When a Bluetooth packet arrives, `remoteControl` doubles or halves the period.

packets. The `entry` routine (lines 15–18) expects a BLE receiver and GPIO pin in the environment (line 15), creates a shared variable `hperiod`, and runs `sigGen` and `remoteControl` concurrently (line 18).

The `sigGen` routine (lines 1–4) generates a precisely timed square wave on the GPIO pin by scheduling future output events and blocking on them. It takes a reference to the `hperiod` variable and enters an infinite loop that schedules a future toggle update to the GPIO pin (line 3), then blocks until the update occurs (line 4). The delay between toggle events is exactly the *model time* set by the `hperiod` variable, regardless of the loop's execution time—Scoria's model time only advances at blocking statements like `wait`. The `after` directive schedules the update according to model time.

The `remoteControl` routine (lines 6–13) waits for a BLE packet (line 10), and doubles or halves `hperiod` in response (lines 12 and 13). The `enableScan` call (line 8) enables the Bluetooth receiver, which then generates events on the variable given by `scanref ?ble`.

Figure 2 shows code that runs at compile time to initialize the signal generator program's environment. References to a GPIO pin and the BLE controller are obtained in lines 3 and 4, and added to the environment (indicated by the question marks) in lines 5 and 6. In lines 7–9, the SSM ready queue is populated with the `entry` routine and I/O handlers for the GPIO pin and BLE. I/O handlers are described in Section 4.

This small example illustrates a key feature of Scoria needed by most IoT applications: concurrent programming with lightweight threads that can block on events. In C, it is easy to write a program that waits for an event from a single source by entering a polling loop, but waiting for multiple events is difficult. A central event-handler loop works but discards control state between events; the programmer must explicitly maintain any state, i.e., what to do next.

Multiple events are typically handled by OS-provided threads, which allows the programmer to write sequential code that only blocks on single events, but IoT devices often do not have the memory to provide a separate stack per thread. Protothreads [11] provides an extremely lightweight thread package in the form of C macros that only require storage for one additional program counter per thread, but the onus remains on the programmer to store all other state globally.

The Sparse Synchronous Model (SSM) of Edwards & Hui [12] integrates this blocking-thread concurrency model with a model of time that results in programs with deterministic functional and temporal behavior. A program running under SSM behaves like a discrete-event simulation: time is divided into instants in which the program executes. At any time, the program only knows what instant it is in and may schedule variable updates for a future instant. The SSM scheduler does its best to keep model time synchronized with real time, but the behaviour of the program is not affected by any deviation. Thread execution within a single instant is totally ordered to eliminate races and guarantee deterministic behavior.

Built atop SSM, Scoria is a programming language designed to simplify prototyping new language features and experiments with language design. Our compiler implements concurrency with C functions that store their state—control state and all local variables—in heap-resident activation records, instead of on the stack. Such code is like what a programmer might write with Protothreads, but Scoria provides more familiar abstractions such as local variables and blocking `wait` statements.

This paper describes the current status of Scoria and the choices we made during its development:

- We implement Scoria as an embedded domain-specific language (EDSL) in Haskell. It is mature enough that we can write programs that use I/O peripherals and run them on SoC-boards, but more work is needed before we can write complicated examples such as mesh networks.
- Programs in Scoria do not have to explicitly manage memory; the compiler and RTS allocate and deallocate memory, sparing us from a garbage collector.
- Scoria's language interface for interacting with I/O peripherals is the same as for interacting with regular Scoria references. Our RTS provides bindings for Zephyr OS [23].
- We test Scoria's compiler with QuickCheck [9], a property-based Haskell tester. We check for memory leaks and errors; compare the behaviour of compiled programs with an interpreter; and our domain-specific shrinker reduces errant programs a minimal test case.

```
compiler :: Compile ()
compiler = do
  (gpio0, gpiohandler)  <- output 0
  (ble, _, scanhandler) <- enableBasicBLE
  let ?out0 = gpio0
      ?ble  = ble
  schedule scanhandler
  schedule entry
  schedule gpiohandler

```

Fig. 2. The top-level of the signal generator program in Figure 1. This requests a GPIO pin and a BLE radio handler, then runs the `entry` routine along with I/O handlers.

```
#include <zephyr/zephyr.h>
#include <zephyr/drivers/gpio.h>
static const struct gpio_dt_spec led =
    GPIO_DT_SPEC_GET(DT_ALIAS(led0), gpios);
void main(void) {
  int ret;
  if (!device_is_ready(led.port)) return;
  ret = gpio_pin_configure_dt(&led, GPIO_OUTPUT_ACTIVE);
  if (ret < 0) return;
  while (1) {
    ret = gpio_pin_toggle_dt(&led);
    if (ret < 0) return;
    k_msleep(1000); // sleep for 1000 ms
  }
}
```

Fig. 3. The “blinky” example distributed with Zephyr RTOS, which purports to generate a signal with a 2 s period.

- We evaluate the performance of Scoria’s runtime system and generated code by stress testing it on real hardware. We precisely measure latencies to identify bottlenecks.

2 Motivation

To motivate real-time programming languages like Scoria, consider the “blinky” application commonly used as “hello world” in embedded systems programming. This toggles an LED at a specified frequency; Figure 3 shows the sample distributed with the Zephyr real-time operating systems (RTOS).

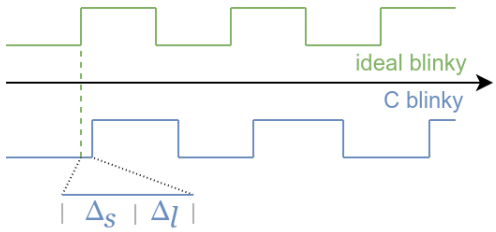


Fig. 4. The C blinky program of Figure 3 drifts due to latency from the RTOS (Δ_s) and the user program (Δ_l).

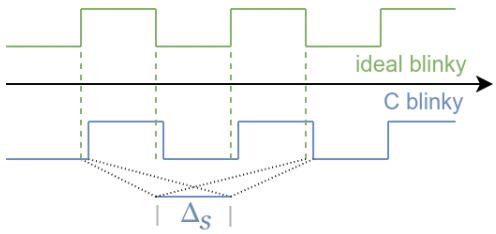


Fig. 5. Blinky without drift.

While blinky (written in C) is adequate for showcasing the basic I/O and timing facilities of an embedded programming framework, it misrepresents its timing behavior: blinky does not actually blink once every two seconds! This discrepancy is due to two sources of latency: the program spends some time each loop iteration to toggle the LED, between invocations of `k_msleep` and once the timer set by `k_msleep` has expired, Zephyr takes some time to reschedule and resume the blinky program. If the time toggling the LED is Δ_l, and the time spent rescheduling the blinky program is Δ_s, then each loop iteration will take $1000 + \Delta_l + \Delta_s$ ms rather than just 1000 ms. Over time, the latency will accumulate into *drift* as the program continues to drag behind its ideal behavior (Figure 4).

To compensate for this latency and eliminate drift, the blinky program should only sleep for as long as it needs to keep up with the ideal timing behavior. To do this, our "corrected" implementation (Listing 2) maintains a *logical* clock of how much time *should* have elapsed, and shortens its sleep time according to the difference between that logical time and the *actual* time. The resulting temporal behavior, shown in Figure 5, does not suffer from drift, only incurring a stable phase error of Δ_s.

However, this technique comes at the cost of complexity: the implementation shown in Listing 2 is 45 LOC, and is deeply entwined with Zephyr's API. Though certain configuration code may be simplified or omitted, the program's control flow and timing logic is obfuscated by the need to respond to the timer via interrupt handlers. Furthermore, this technique alone does not scale well beyond sequential

```
// include and defines
// declaration of button0, button1, led0 \& led1

uint32_t last = -1;
void button_cb(const struct device *dev,
               struct gpio_callback *cb, uint32_t pins) {
  if (last == button0_pins && pins == button1_pins)
    gpio_pin_set_dt(&led0, 1);
  else if (last == button1_pins && pins == button0_pins)
    gpio_pin_set_dt(&led1, 1);
  else {
    gpio_pin_set_dt(&led0, 0);
    gpio_pin_set_dt(&led1, 0);
  }
  last = pins;
}

// callback initialization and registration
```

Fig. 6. While the program above is semantically simple, the implementation has to install interrupt handlers. Knowledge of which events have occurred must be lifted to the global scope, which must then be consulted before an LED can be lit.

```

seq :: Ref SW -> Ref SW -> Ref GPIO -> Ref GPIO -> SSM ()
seq b0 b1 l0 l1 = routine $ do
    wait (b0, b1)
    if changed b0 -- was b0 the button that was pressed?
        then do wait b1
                l0 <~ ON -- we know the sequence was b0, b1
        else do wait b0
                l1 <~ ON -- we know the sequence was b1, b0
```

Fig. 7. This Scoria routine implements the same behaviour as Figure 6, but each branch knows the source of its event and which LED to light without consulting a global “last” variable.

programs, since any number of concurrent threads may need to synchronize their use of a finite number of hardware timers.

Scoria overcomes this complexity by incorporating the notion of logical time into its concurrent programming model. In doing so, Scoria allows the programmer to express the *intended* timing behavior, abstracting over platform-specific timer APIs. Scoria relies on its runtime to approximate the intended behavior to the extent possible by making use of available and supported timing capabilities. For comparison, Figure 1 shows blinky in Scoria, which is only 15 LOC.

```
routine                                    -- Define a Scoria routine body
var          :: Exp a -> SSM (Ref a)                -- New local var
deref        :: Ref a -> Exp a                      -- Access value
(<~)         :: Ref a -> Exp a -> SSM ()            -- Assignment
after        :: Time -> Ref a -> Exp a -> SSM ()    -- Delayed assignment
changed      :: Ref a -> Exp Bool                   -- Was written?
wait         :: Waitable a => a -> SSM ()           -- Block on write
if-then-else :: Exp Bool -> SSM () -> SSM () -> SSM () -- Conditional
while        :: Exp Bool -> SSM () -> SSM ()        -- Loop
fork         :: [SSM ()] -> SSM ()                  -- Concurrency
```

Fig. 8. The Core API of Scoria.

Another example that illustrates how using interrupt handlers can complicate code unnecessarily is shown in Figure 6. It is an interrupt handler of a simple program that lights up one of two LEDs depending on which sequence of presses from two different buttons is registered. This program needs to remember which button was pressed previously, so that when the next press arrives it can inspect the complete sequence and decide which LED to light up. In order to remember the previous press, the program needs to use global variables, opening itself up to the possibility of complicated control flow where global variables are modified by several concurrently running threads.

The Scoria version of this does not rely on global variables; it knows which events happened based on which code is executing and thus uses *local* information rather than global.

3 Overview of Scoria

Scoria is an embedded domain-specific language for IoT applications. It is domain-specific in that it is tailored to the demands of IoT applications through its facilities for concurrency and timing control. This makes the language easier to learn, compile, and optimize at the expense of making programming tasks outside its scope more difficult. It is embedded in a host language, Haskell, which allows us to benefit from the existing Haskell tools and compilers while presenting a convenient, new abstraction for the user. Haskell provides e.g the parser and a type checker, and we use host-level evaluation as a kind of macro system (see Section 5). Such an approach enabled us to develop Scoria much more rapidly, making it easy to experiment with new language features.

We modeled Scoria on the proposal by Edwards & Hui [12], with adaptations to embed it in Haskell.

Figure 8 lists the core API of Scoria. These may be thought of as Scoria's primitives, but each are actually Haskell functions composed using Haskell's **do**-notation. Values that are embedded (Scoria values) are of type `Exp a`, while values that belong to the host language are of type `a`. Host-level values are evaluated at Scoria compile-time.

The `routine` keyword introduces the body of a Scoria procedure. The Scoria compiler simply executes any Haskell code outside a `routine` block. See Section 5 for details.

A reference to a variable is created with the `var` keyword, can be assigned a new value with the `<~` ("assign") operator, and can be read with `deref`. The `changed` function returns true if a given variable (reference) has been written in the current instant, e.g., by an assignment in a concurrent thread.

The `after` operation schedules an assignment to a reference in a future instant. Its first argument is the time delay (relative to the current instant) before performing the assignment, and the other two arguments are the variable to update and the future value it should receive (which is evaluated immediately).

The `wait` statement blocks until at least one of the given variables (references) have been written, either by an immediate assignment `<~` or a scheduled assignment (`after`). `wait` takes either a single reference or tuples of references of varying sizes. Haskell's *type classes*[13] allow us to overload the operator to accept either.

`if-then-else` and `while` are the usual conditional and looping constructs. The code in their branches may block or call functions that do.

Lastly, `fork` is a parallel function call construct that spawns concurrently running child processes and blocks until all children have terminated (and not just blocked). The order of children is significant: it prescribes the order in which the children run in any instant, enforcing deterministic concurrency with a total execution order. In particular, an earlier child may write to a shared variable that a later child may read in the same instant, but not vice-versa (earlier children may only read data from a later child that was assigned in an earlier instant or in a scheduled update).

As shown in Figure 2, Scoria also provides a `Compile` monad that runs code at compile time. This is used to perform static initialization of e.g peripherals, and make them accessible to the rest of the Scoria program. As an example, a user might want to create a global variable `gv` and supply a reference to it to the `main` routine. This can be done as follows:

```
1  program :: Compile ()
2  program = do
3    gv <- global @Word64
4    let ?gv = gv
5    schedule main
6
7  main :: (?gv :: Ref Word64) => SSM ()
8  main = routine $ ?gv <~ 5
```

To start the Scoria program, at least one routine must be added to the ready queue with the `schedule` directive, which populates the ready queue when the program begins. Each successive invocation of `schedule` adds another routine at a lower priority level: those scheduled first will run first in any instant, much like a `fork`.

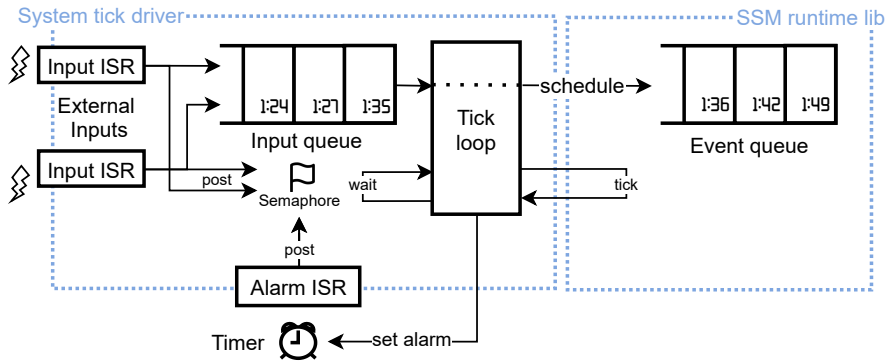


Fig. 9. The SSM runtime's input handling architecture.

Code generation from Scoria. Edwards & Hui [12] provide an SSM RTS coded in C. We have written a code generator that takes the abstract syntax of a Scoria program and produces C code that calls this RTS. The generated C code is platform agnostic; additional platform-specific code is necessary, e.g., to set a future wake-up alarm. We have written such bindings for Zephyr OS [23] and for something we call the trace platform, which we describe in Section 6.

4 I/O in Scoria

A Scoria program communicates with its environment through variables. The Scoria runtime collects input events from interrupt service routines (ISRs)—effectively the callback functions of traditional asynchronous C implementations—and delivers them to the SSM event queue. Writes to output variables are observed by special I/O handler routines that typically call external C functions to perform the output.

Handling External Inputs and Time. Figure 9 shows the structure of the Scoria runtime that handles input events, manages the system timer, and runs the system in each instant. The main tick loop (Figure 11) retrieves events enqueued by ISRs (Figure 10), calls the SSM runtime's `tick()` of Edwards and Hui [12] to run the system (i.e., the code generated by the Scoria compiler) for a single instant, sets an alarm to wake it up at the next scheduled event, and then sleeps until it is awakened by the alarm or input from a peripheral.

Input events are gathered by ISRs and ultimately appear as scheduled variable updates to the Scoria program, but this process operates in several steps for safety and efficiency.

The Scoria runtime maintains two distinct event queues to avoid excessive synchronization costs. The input queue is loaded asynchronously by ISRs and emptied by the main tick loop, but it is a simple ring buffer that is easy to make thread-safe. By contrast, the main event queue is a priority queue to which events are added

```
1  void input_isr(device *dev) {
2    int key = irq_lock();
3    ssm_time_t input_time = timer_read();
4    ssm_input_event *input = ssm_input_alloc();
5    irq_unlock(key);
6    if (input == NULL)  // No space in input queue
7      return;
8    input->time = input_time;
9    input->data = ssm_input_read(dev); // Device-specific
10   input->ref = ssm_input_binding(dev);
11   ssm_input_commit(); // Enqueue input queue
12   sem_post(ssm_sem);  // Wake up tick thread
13 }
```

Fig. 10. The structure of an input interrupt handler, which enqueues an event in the input queue. Each peripheral has its own device-specific handler.

and sometimes removed out-of-order by both the tick loop and the various concurrent routines. However, because these run synchronously, the event queue is never accessed asynchronously and need not be thread-safe.

There are several reasons why we need two queues instead of just one. The synchronous hypothesis states that computations are instantaneous, meaning that an invocation of `tick` returns instantly. In reality this is not the case, and physical time will progress. If an external event is delivered to the system as soon as it arrives, `tick()` would look into the future, determinism would break, and we would be susceptible to data races. Furthermore, without time stamping external events, a second external event might arrive and overwrite the previous event before the system has had a chance to observe the first one, leading to events essentially being dropped. The SSM runtime allows only one outstanding event per reference at a single time.

Figure 10 shows an input interrupt handler. It first records the current system time, then attempts to enqueue an event with that timestamp and any new data from the peripheral in the input queue: a ring buffer large enough to accommodate a modest backlog of input events before having to drop or overwrite events. Enqueue and dequeue operations are performed in-place via an allocate/commit and peek/release protocol to minimize copying.

In an ISR, recording the current system time as quickly as possible and allocating space in the input queue is critical to ensure enqueued input events appear with nondecreasing timestamps. Other higher-priority interrupts may occur while processing a lower-priority interrupt, but only after the timestamp of the lower priority interrupt has been captured.

Figure 11 shows the main tick loop, which waits on a OS-provided semaphore (`ssm_sem`) that is posted by an interrupt service routine from either a peripheral or the system clock. At each iteration of the tick loop, the runtime checks the input

```
struct semaphore ssm_sem;
void alarm_isr(void) { sem_post(&ssm_sem); }
void ssm_tick_loop(void) {
  ssm_init();
  ssm_tick();
  for (;;) {
    real_time = timer_read();
    model_time = ssm_next_time();
    ssm_input_event *input = ssm_input_peek();
    if (input && input->time <= model_time) {
      ssm_schedule_input(input); // Add to event queue
      ssm_input_release(); // Remove from input queue
    } else if (model_time <= real_time) {
      ssm_tick(); // Run system; advance model time
    } else if (model_time == NO_EVENT_SCHEDULED) {
      sem_wait(ssm_sem); // Wait for input
    } else {
      if (timer_set_alarm(model_time, alarm_isr))
        continue; // Alarm already expired
      sem_wait(ssm_sem); // Wait for alarm or input
      timer_cancel_alarm();
      sem_reset(ssm_sem);
    }
  }
}
```

Fig. 11. The Scoria runtime's main tick loop, which gathers events from the input queue, runs ọdetick() to advance SSM model time, then sleeps until the next scheduled event or an interrupt.

queue for events to schedule in the SSM event queue. Events in the input queue will be in increasing order since we assume the system timer advances monotonically. However, if the logical time of the SSM program is running behind physical time when some external input is received—`input->time` > `model_time`—or if there is no pending input event, the runtime executes the SSM program by calling `ssm_tick()`. Finally, if there are neither input events to process, nor internal events ready to execute, the main thread goes to sleep, blocking on a semaphore until either its timer expires or some fresh input appears.

Handling External Outputs. Just as with inputs, output peripherals are bound to regular Scoria references; writes to those references are emitted to the environment as output. Under the hood, those writes eventually trigger a call to some system-provided C output function that actualizes the output.

One of the design considerations we faced when implementing outputs was deciding exactly when in an instant the system output function should be invoked. The SSM model says all the code of an instant executes in zero time, but in practice any execution takes time and the code for an instant is effectively spread out over a finite time interval. If an output function is invoked too early, subsequent updates to the output variable in the same instant might be ignored, whereas if it is invoked too late, there will be output latency and perhaps even jitter arising from the execution speed of unrelated computation. For instance, consider the following three concurrent processes, shown in order of decreasing priority:

```
w1 :: Ref Button -> Ref LED -> SSM ()
w1 button led = routine $ do
  while true (do
    wait button
    led <~ true)

w2 :: Ref Button -> Ref LED -> SSM ()
w2 button led = routine $ do
  while true (do
    wait button
    led <~ false)

r :: Ref Button -> SSM ()
r button led = routine $ do
  while true (do
    wait button
    -- some expensive computation
  )
```

If the LED is lit immediately when it is first assigned—after the assignment of `true'` by `w1`—then the assignment of `false'` by `w2` will be missed, leading to a stale value being emitted. But if the LED is lit as late as possible—at the end of the instant—the system must wait until `r` has completed executing, incurring some delay. Depending on the particular application, either one of these outcomes may be unacceptable.

Our design leaves it to the user to decide when the physical output should take place by encapsulating it within an *output handler* process that the user must schedule. This handler is obtained when the user asks for the output device. If a process performs an immediate assignment to a reference, that assignment will only be visible to lower-priority processes. Note that delayed assignments do not have this behaviour; they always wake up every waiting process. For instance, if the user would like to schedule the handler after any possible assignments by `w1` and `w2`, but before `r`, the user may specify:

```
initProgram :: Compile ()
```

```
2  initProgram = do
3    button <- input 0
4    (led, handler) <- output 0
5
6    schedule $ w1 button led
7    schedule $ w2 button led
8    schedule $ handler
9    schedule $ r button led
```

While this precludes `r` from performing any effectful *instantaneous* assignments, it still accommodates effectful *delayed* assignments by `r`.

This gives the developer some control over two sources of jitter. The first is due to varying computation time—if an output handler is scheduled to run after a higher priority process in the same instant, then the actual output time will depend on the speed of the first process. This source of jitter can be eliminated by scheduling the output handler so it runs at the beginning of the instant at the expense of any instantaneous assignments—it is up to the user to balance the reliability versus the expressiveness of their application.

Discrepancies in input response times is a second source of jitter. The runtime system is able to handle inputs slightly faster when it is already awake than when it is asleep. If output is to be instantaneously triggered by some input, then the input-to-output latency will differ depending on whether an input is received when the system is asleep or awake.

5 Leveraging the Host Language

Scoria is implemented as a Haskell library that exposes an embedded language interface which allows us to reuse Haskell's parser and type checker, and otherwise take advantage of Haskell language features for rapid language development. In this section, we describe some of the embedding techniques we use to ensure that our language interface is robust, expressive, and easy to use.

Type Checking. Function Application in Haskell is checked to guarantee the type safety of the application. The value `routine body` has the same type as `body`, so Scoria procedures still look like ordinary Haskell functions and are invoked by performing normal Haskell function application, with the effect that the Haskell compiler will type check Scoria procedure invocations for us, for free. This kind of type checking happens in every corner of the language API, making it harder to construct illegal programs.

Providing New Abstractions. The host language of an embedded language can be used as a macro system, enabling effortless experimentation with language design. As an example, consider the `delay` statement, which blocks for a fixed period of time. This operation is not primitive to Scoria or SSM, but the behaviour can be recovered using existing primitives.

```
delay :: Exp Time -> SSM ()
delay x = do
  sync <- var event
  after x sync event
  wait sync
```

Because this Haskell function is not decorated with the `routine` keyword, it is not treated as a callable Scoria routine. Instead, when a program invokes `delay`, Scoria will inline `delay`'s body.

Yet this implementation has the downside of introducing a new local variable `sync` in the activation record of its call site each time `delay` is invoked. An alternative implementation which turns `delay` into a Scoria procedure might look like:

```
delay' :: Exp Time -> SSM ()
delay' x = fork [ delayRoutine x ]

delayRoutine :: Exp Time -> SSM ()
delayRoutine x = routine $ delay x
```

When a program invokes `delay'`, the inlined statement will instead be a `fork` statement that calls the auxiliary `delayRoutine` routine.

Plugin for Source-to-Source Transformations. Scoria uses the `routine` keyword to distinguish between a Haskell function and a Scoria procedure. This distinction is necessary because in the latter case, the compiler must construct AST nodes corresponding to the routine definition, rather than directly evaluating its body. It is this indirection that allows Scoria programs to perform recursion without endlessly inlining the routine body.

Yet routine definitions cannot be trivially embedded in Haskell, because it requires functions to reflect on their name and the names of their arguments. Case in point: an early iteration of our compiler used `make_routine` to construct routines, which expects additional arguments conveying the names of the routine and its arguments. This led to an awkward language interface with plenty of syntactic redundancy:

```
f = make_routine "f" ["x", "y"] $ \x y -> body
```

To overcome this burden, Scoria leverages plugin support from the Haskell compiler, GHC, to infer the arguments for `make_routine`. Our plugin looks for invocations of `routine` and replaces it with an invocation of `make_routine` with the necessary source information. For example, this source-to-source transformation produces the above boilerplate from the following:

```
f x y = routine body
```

6 Testing

Interpretation. Following Edwards & Hui [12], we implemented an interpreter in Haskell that steps through instants of model time but does not attempt to synchronize

Bug location	Number of bugs
RTS	1
C generator	8
Interpreter	11

Table 1. Breakdown of the bugs we identified developing of our compiler.

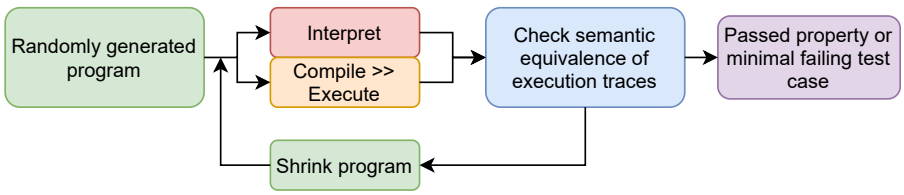


Fig. 12. Overview of how the second property is tested, asserting the semantic behaviour of SSM programs.

with wall-clock time. We use our interpreter as a reference to validate our compiler's output; see below.

Testing. To debug the Scoria compiler, we used QuickCheck [9] (QC), a property-based testing framework that generates *random* values and tests them against a property. If the property is falsified, QC shrinks the test to find a smaller failing case. The QC-generated random programs are typically too large to diagnose without shrinking.

We test two properties: the first is that generated C code compiles and runs without memory errors or leaks, according to Valgrind. This helped us find a bug related to our event queue: when a variable was deallocated, scheduled events were not being unscheduled and the RTS would attempt to perform updates using these stale pointers, corrupting the program state. We fixed this bug by ensuring that all routines unschedule outstanding updates to local variables before returning.

We also test semantic correctness of the generated program as shown in Figure 12. When testing, our compiler adds print statements to the generated code to produce an event trace, which we compare to one from the reference interpreter. This property helped us identify C generation problems. Most notably, we initially implemented Scoria signed integers with C's signed integers. Our tester found cases that branched on overflowing arithmetic to produce different behaviour because of the undefined overflow behavior of C's integers. We fixed this by switching to C's better-defined unsigned integers.

We documented 20 bugs, mostly semantic errors (Table 1).

```
freqCount :: Ref GPIO -> SSM ()
freqCount sw = routine $ do
  count <- var $ u32 0
  gate  <- var event
  after (secs 1) gate event
  while true (do
    if changed gate
      then do
        -- Print count to console
        if changed sw
          then count <~ 1
          else count <~ 0
        after (secs 1) gate event
        wait gate -- Sleep for 1 sec
        after (secs 1) gate event
      else
        count <~ deref count + 1
    wait (gate, sw))

freqCountProg :: Compile ()
freqCountProg = input 0 >>= schedule . freqCount
```

Fig. 13. A frequency counter program.

7 Experimental Results

We test the performance of our Zephyr-based runtime system by subjecting Scoria programs to varying loads. All experiments are performed using a Nordic Semi NRF52840-DK board with a 64 MHz Cortex-M4 processor, 256 kB RAM, 1 MB flash, BLE support, and a 16 MHz crystal time base. We apply loads with a pulse generator driving a GPIO input pin and measure the results with an oscilloscope on an output pin.

Frequency Counter. To assess our implementation's resilience to high input load, we test a small Scoria frequency counter, shown in Figure 13. This program measures the frequency of button presses by counting the number of input events each second. The reported count is double the frequency, corresponding to the two input events of a single button press. For benchmarking purposes, we insert a print statement in the generated code to report the frequency; the program alternates between counting and reporting to ensure that the overhead of reporting the frequency does not interfere with the frequency counting at high loads.

The reference `sw` is bound to `input 0` on the NRF52840-DK; we connect a function generator to the corresponding GPIO pin and generate pulses at various frequencies. We found that our frequency counter is capable of measuring up events of up to

```
struct semaphore sem;
void alarm_isr(void) { sem_post(&sem); }

uint32_t count = 0;
// Executed each time button is pressed or released
void input_isr(void) { count++; }

void main_loop(void) {
  for (time_t time = timer_read(); true; count = 0) {
    timer_set_alarm((time += secs(1)), alarm_isr);
    sem_wait(&sem);
    printk("frequency * 2: %u Hz\r\n", count);
    timer_set_alarm((time += secs(1)), alarm_isr);
    sem_wait(&sem);
  }
}
```

Fig. 14. Frequency counter program implemented in C.

29 kHz (input frequency of 14.5 kHz, each rising or dropping edge creates one event), with error within 2 Hz. Beyond this, the input queue fills up faster than inputs are consumed, leading to events being dropped and the program thrashing. The frequency counter is able to recover from thrashing after we lower the input frequency below 14 kHz.

To measure the overhead of SSM event scheduling, we compare our Scoria frequency counter to the functionally equivalent manually coded C in Figure 14. The C implementation omits the SSM scheduler and directly manages synchronous control flow in `main_loop` using the same semaphore API provided by Zephyr OS. We also forego the input queue and directly increment `count` in the input ISR.

We find that this optimized version can accurately measure input frequencies of up to 34 kHz with 5 Hz of error, but is more verbose and error-prone, and less expressive and flexible.

Button-to-blink. To better understand the system sans synchronous logic, we first evaluate the "button-to-blink" program in Figure 15. This lights an LED when a button is pressed and turns it off when released. We schedule the LED handler after the main `b2b` program so that this instantaneous assignment is written to the LED at the end of the instant.

The instantaneous assignment in `b2b` incurs a practically avoidable but theoretically significant amount of latency, which we call the *at-rest input latency*, δ_r. This value represents the time it takes for the system to wake *from sleep* and respond to external input. From profiling, we find that δ_r for our NRF52840-DK is approximately 60 µs.

```
b2b :: (?sw :: Ref Switch, ?led :: Ref GPIO) => SSM ()
b2b = routine $ while true $ do
    wait ?sw
    ?led <~ deref ?sw

b2bc :: Compile ()
b2bc = do
    sw <- input 0
    (led, handler) <- output 0
    let ?sw = sw
        ?led = led
    schedule b2b
    schedule handler
```

Fig. 15. Button-to-blink in Scoria; generated C in Listing 9

Frequency (kHz)	Latency (µs)	Jitter (µs)
2	60.0	0.7
4	60.6	0.7
6	49.7	0.5
8	52.1	13.1
10	47.3	34.0
12	45.6	24.0

Table 2. Latency and jitter of button-to-blink (Figure 15)

When the system receives inputs at a frequency beyond $1/\delta_r$, the system will not be able to process an input before receiving the next, making δ_r a significant number. As the system falls farther and farther behind wall clock time, its behaviour will degrade into an asynchronous system with none of the temporal behaviour of the underlying Scoria program.

A related metric is the *in-flight input latency*, δ_f: the time it takes for a busy system to respond to external input. When events arrive separated by an amount of time within δ_f, the input queue will be populated with new events by the time the main tick loop thread checks it again, meaning it can resume ticking without putting itself to sleep. δ_f is shorter than δ_r because it eliminates the time spent sleeping and waking. We measured δ_f on our board to be about 45 µs.

Sparse bursts of events do not overload the system if it can catch up between bursts. The system may even be able to keep up if it is consistently stimulated with a period between δ_r and δ_f. As shown in Table 2, the "button-to-blink" program is able to sustain activity when stimulated by a pulse generator with frequencies of 12 kHz; beyond 12 kHz, the program begins to thrash and drop input events. However, when the delay between successive events is less than δ_r, the computation time per instant

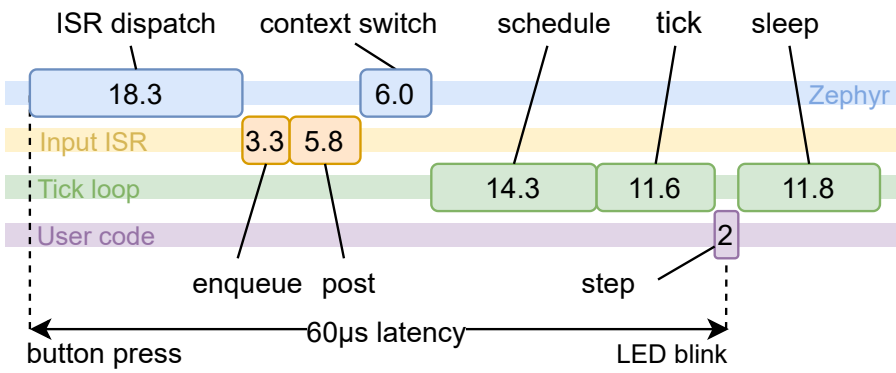


Fig. 16. Timeline in µs of a single button press of "button-to-blink" (Figure 15), reconstructed from GPIO profiling.

becomes less predictable, even as the system remains responsive. In Table 2, this manifests in increased jitter at input frequencies beyond 8 kHz, at which a square wave has a half-period of 62.5 µs.

To determine the source of our system's δ_r of 60 µs, we profile a single button press of `b2b` to obtain the timeline in Figure 16. We add code that emits 4-bit codes on GPIO pins for certain events and record them with a logic analyzer. Analyzing the recorded data determines time between events. While this approach is intrusive, we found each GPIO write took only 60 ns, negligible compared to what we are measuring. We find that 24.3 µs is directly introduced by Zephyr's own ISR and process scheduling facilities. A further 5.8 µs is introduced by Zephyr's semaphore implementation, when we call `sem_post` from the ISR to wake up the tick loop thread. The δ_r of 60 µs is from Table 2; Figure 16 shows slightly different numbers from a different Zephyr version.

We also measure δ_r for two Zephyr programs written in C to compare with our Scoria implementation. One of our implementations, Figure 2, unrealistically constantly polls for button presses, and represents the lowest theoretical at-rest response time for any Zephyr program. Our other implementation is more realistic, responding to input via an interrupt handler that reports the button press event by placing it in a ring buffer identical to the one in the Scoria runtime and resuming the main thread by releasing a semaphore. The main thread then unblocks, retrieves the event from the ring buffer, and updates the LED. Listing 1 shows this version, which models the Scoria runtime without the SSM "tick" scheduler. We standardized the input event frequency at 2 kHz across all implementations; our results are shown in Table 3.

The Scoria version is the slowest of the three. Figure 16 shows the amount of time spent by Zephyr locating and running the input ISR was roughly 33 µs. This is comparable to the response time measured by the more realistic Zephyr example

Program	Mean δ_r (µs)	Min. δ_r (µs)	Max. δ_r (µs)
Polling	1.519	0.88	2.17
Interrupt + sem	31.46	31.32	31.59
Scoria	57.96	57.86	58.07

Table 3. The at-rest response time of three button-to-blink implementations fed a 2 kHz input. Data gathered with a logic analyzer sampling at 100 MHz.

Program	Generated Frequency (kHz)	LOC
C	12.2	54
Scoria	6.57	32

Table 4. Implementations' maximum frequencies and sizes

(Table 3). The overhead imposed by the Scoria runtime system is mainly the parts in green (the *Tick loop* bar), where the Scoria runtime is retrieving the event from the input queue, inserting it in the Scoria environment, and scheduling processes to invoke the Scoria tick loop.

Signal Generator. With these measurements made we can return to the signal generator from Figure 1. We measure the highest frequency (shortest period) it can generate. For the purposes of evaluation, we adapted the example to adjust the half-period `hperiod` in linear increments of 2 µs by the press of a button. Listing 8 shows the complete code. We find that the signal generator from Figure 1 can reliably generate signals with half-period as low as 76 µs, corresponding to a frequency of about 6.6 kHz, with less than 500 ns of jitter—within the level of precision of our oscilloscope. At half-periods between 70 µs–76 µs, we observe degradation in signal accuracy and consistency: the output signal's half-period fluctuates between 60.2 µs–83.4 µs. At lower half-periods, the system ceases to output any signal as it is overwhelmed by the computation load. Once the half-period is increased beyond 76 µs, the system eventually recovers after a burst of high-frequency output, emitted as it catches up with wall-clock time.

Table 4 compares our Scoria implementation with a comparable signal generator written in C (Listing 7). We measure the maximum frequency for each implementation. Like the Scoria runtime, the C implementation populates a ring buffer when the alarm goes off; the main thread is then unblocked by a semaphore release, after which it toggles the LED.

The Zephyr program outperforms the Scoria program, as can be seen in table 4. As soon as the half period becomes lower than the time it takes for the program to handle one wave event, the signal becomes unstable. Since the Scoria runtime adds some overhead when reacting to events, as can be seen in Figure 16, the Scoria program can not handle as low half periods as the Zephyr program. The Zephyr program can generate about twice as high of a frequency.

BLE Frequency Mime. As a slightly more complicated example, we wrote an application where two boards communicate over BLE. One board will advertise varying frequencies using BLE, and the other board will scan for such messages. When a message is intercepted, the board will generate a signal of the detected frequency as it continues to scan for messages.

The C version needs to manage such things as callbacks, clocks, counters, and global state. Any of these areas could hide bugs, and they are all absent from the Scoria version. The Scoria version specifies only application logic, not the details of how it is realized. Listings 3 to 6 show both versions.

8 Related Work

Synchronous Languages. Scoria's programming model SSM [12] adopts the *synchrony hypothesis*: concurrent execution proceeds as a sequence of synchronized, zero-time instants [1]. While digital hardware designers have used this model for over half a century, French researchers in the 1990s brought the concept to software languages Lustre, Esterel, and Signal [2, 4, 7]. Synchrony simplifies reasoning about concurrent timing behavior by eliminating the non-determinism of asynchronony, and has since featured in languages such as Timber [6] and Lingua Franca [15], and EDSLs like Haski [24] and Copilot [20].

Despite their application in real-time programming, the "French school" of programming languages—Lustre, Esterel, and Signal [2, 4, 7]—do not include time as a first-class language construct. Instead, they specify computation through *successive* instants, and compile to tick functions that must be invoked periodically by a cyclic executive for every instant, even those where no computation takes place. Meanwhile, Scoria's `after` statement allows users to specify (or compute) concrete time delays, and its sparse execution model does not obligate its runtime to compute inert instants.

Many of the synchronous languages are dataflow-oriented, including Lustre, Signal, and Lingua Franca [2, 7, 15], where programs specify a static, finite dataflow graph where processes (nodes) communicate along fixed channels (edges). In contrast, Scoria is imperative (programs specify control flow); processes are spawned dynamically using `fork`, and the dataflow between them is implicit. Though Esterel is also imperative and includes concurrent control-flow constructs like Scoria's `fork` and `wait`, it lacks function calls, limiting Esterel programs to a finite number of processes [4].

Like Scoria, Haski [24] and Copilot [20] are deeply embedded Haskell DSLs [22] that generate C code for microcontrollers. Haski is an embedding of Lustre in Haskell for secure IoT programming; Copilot is an EDSL for monitoring software that promises constant-time and -space C code. Both are dataflow languages, so their embedding in Haskell constructs abstract streams instead of Scoria's syntax trees.

Compiler Testing. While manually written regression test suites remain the standard for compiler development, many have also considered randomly generating and

shrinking compiler test cases [5, 8]. Our work is most similar to RandIR [18], which uses ScalaCheck [17] to test LMS [21], a code generation framework for embedded DSLs. RandIR uses Scala as its test oracle, translating its test candidates to Scala to compare against other language backends; in comparison, Scoria's test oracle directly interprets generated Scoria IR.

For well-established languages, differential testing can take advantage of multiple compiler implementations, e.g., CSmith for C [25], or multiple optimization levels, e.g., Palka et al. [19] for the Glasgow Haskell Compiler. While Scoria cannot afford such an opportunity as a nascent language, we look forward to adapting our test-driven compiler development to help identify bugs as we begin to implement optimizations that take advantage of Scoria's synchronous semantics.

Discrete-Event Systems. Ptides[10] is a programming model used to implement discrete-event real-time systems that inspired SSM and Scoria. Unlike Scoria/SSM, however, Ptides implements a *distributed* discrete-event system that cautiously determines when it is safe to process a timestamped message, i.e., when a message with an earlier timestamp cannot appear. We plan to implement distributed Scoria following Ptides.

9 Future Work

Types. Scoria currently only supports a limited, predefined set of data types. We will add support for more complex types, such as arrays and user-defined algebraic data types (ADTs). Building on McDonnell et al. [16], we can expose a pattern-matching language interface that is familiar to Haskell programmers. We believe ADTs will be a good fit for writing IoT programs; the richness they bring will be especially helpful for developing safe interfaces for peripherals.

Linear Types. Linear types [3] were recently added to GHC, which allows certain invariants to be encoded in the types and checked by the compiler. For instance, the compiler can enforce that we schedule our I/O handlers, if `output` and `schedule` are given more precise linear types:

```
1    output :: Int
2             -> (Ref LED -> SSM () %1 -> Compile ())
3             -> Compile ()
4    output i continuation = ...
5
6    schedule :: SSM () %1 -> Compile ()
7    schedule handler = ...
```

`output`'s type indicates that it takes an integer identifying the LED, as well as a *continuation* that takes the LED reference and handler as parameters, where the `%1` indicates that handler must be consumed exactly once. These functions can be invoked as follows:

```
1  program :: Compile ()
```

```
2  program = output 0 $ \ref handler -> do
3    schedule handler -- followed by rest of program setup
```

Since the handler is scheduled inside of the continuation, forgetting to schedule the handler (or scheduling it more than once) is a GHC compile time error. This could otherwise only be checked at code-generation time (host-language run time).

Performance. Experimentally, we identified a few sources of inefficiency that make Scoria significantly slower than counterparts implemented directly in C and Zephyr. While the C and Zephyr versions process an event as soon as the associated interrupt handler is invoked, Scoria has to enqueue the message and wake up the main tick-thread that retrieves it and actually runs the program, which add overhead.

The benchmarked programs can perform significantly better if the required OS facilities were developed directly on the bare metal rather than acquired by Zephyr. While Zephyr gives quick access to many convenient abstractions, they are developed in a very general way to be suitable for many kind of applications. If we could tailor these abstractions to just the needs of Scoria, we might be able to get some speedup. We will try this, as well as trying out other operating systems.

10 Conclusions

While the Scoria language is still a simple language, Scoria programs are short and clutter-free, and the compiler generates resource-efficient code. Implementing Scoria as an EDSL has led to several advantages for language and compiler development. Taking advantage of features and tooling provided by the host language, we can rapidly prototype our language. Using primitives to derive new operations such as `delay` is straightforward and transparent. Since the EDSL itself is simply a Haskell library whose API constructs an AST, we can easily experiment with exposing and modifying compiler internals with little coordination with the language front-end.

We were able to confidently iterate on our compiler due to our Scoria interpreter, which we developed early on. Using the interpreter as a language oracle, we used QuickCheck to thoroughly and efficiently test corner cases in our language, identifying several bugs that would have otherwise taken much longer to identify. To help us better understand what went wrong, our shrinker produced minimal failing test programs which easily pointed to the source of each bug we found.

We extended the RTS proposed by Edwards & Hui [12] with Zephyr OS bindings to run Scoria programs on real hardware. I/O peripherals are exposed via Scoria references so that Scoria programs can converse with peripherals natively, avoiding the callback hell typical of IoT applications. This interaction model has allowed us to easily add support for a handful of peripherals, including GPIO and BLE. Scoria allows users to precisely specify the priority of I/O handling code relative to concurrent user code and make application-specific reactivity and sensitivity trade-offs. Our stress tests confirm we can write real-time applications with robust overflow behaviour.

While Scoria remains an experimental language, our successes with it to date have illustrated how implementing an EDSL provides a faster route to language

implementation for experimental languages. As expected, Scoria has raised many questions about how any sparse synchronous language should be designed, notably how its interface with the outside world should be specified, and given us a quick route to experimenting with solutions to these problems. Work on Scoria continues and will, at the very least, inform language design in similar domains.

Acknowledgments

Hui and Edwards were supported in part by the National Institutes of Health (NIH) under grant 1RF1MH120034-01.

Krook and Claessen were supported by the Swedish Foundation for Strategic Research (SSF) under the project Octopi (Ref. RIT17-0023) and by the Swedish research agency Vetenskapsrådet under the project SyTeC (Ref. 2016-06204).

An additional thanks goes to Agustín Mista for help writing the plugin described in this paper.

References

[1] Albert Benveniste, Paul Caspi, Stephen A. Edwards, Nicolas Halbwachs, Paul Le Guernic, and Robert de Simone. 2003. The Synchronous Languages 12 Years Later. *Proc. IEEE* 91, 1 (2003), 64–83. doi:10.1109/JPROC.2002.805826

[2] Albert Benveniste, Paul Le Guernic, and Christian Jacquemot. 1991. Synchronous Programming with Events and Relations: the SIGNAL Language and Its Semantics. *Sci. Comput. Program.* 16, 2 (1991), 103–149. doi:10.1016/0167-6423(91)90001-E

[3] Jean-Philippe Bernardy, Mathieu Boespflug, Ryan R. Newton, Simon Peyton Jones, and Arnaud Spiwack. 2018. Linear Haskell: practical linearity in a higher-order polymorphic language. *Proc. ACM Program. Lang.* 2, POPL (2018), 5:1–5:29. doi:10.1145/3158093

[4] Gérard Berry and Georges Gonthier. 1992. The Esterel Synchronous Programming Language: Design, Semantics, Implementation. *Sci. Comput. Program.* 19, 2 (1992), 87–152. doi:10.1016/0167-6423(92)90005-V

[5] Abdulazeez S. Boujarwah and Kassem Saleh. 1997. Compiler Test Case Generation Methods: a survey and assessment. *Inf. Softw. Technol.* 39, 9 (1997), 617–625. doi:10.1016/S0950-5849(97)00017-7

[6] Magnus Carlsson, Johan Nordlander, and Dick Kieburtz. 2003. The Semantic Layers of Timber. In *Programming Languages and Systems, First Asian Symposium, APLAS 2003, Beijing, China, November 27-29, 2003, Proceedings (Lecture Notes in Computer Science, Vol. 2895)*, Atsushi Ohori (Ed.). Springer, 339–356. doi:10.1007/978-3-540-40018-9_22

[7] Paul Caspi, Daniel Pilaud, Nicolas Halbwachs, and John Plaice. 1987. LUSTRE: A Declarative Language for Programming Synchronous Systems. In *Conference Record of the Fourteenth Annual ACM Symposium on Principles of Programming Languages, Munich, Germany, January 21-23, 1987*. ACM Press, 178–188. doi:10.1145/41625.41641

[8] Junjie Chen, Jibesh Patra, Michael Pradel, Yingfei Xiong, Hongyu Zhang, Dan Hao, and Lu Zhang. 2021. A Survey of Compiler Testing. *ACM Comput. Surv.* 53, 1 (2021), 4:1–4:36. doi:10.1145/3363562

[9] Koen Claessen and John Hughes. 2000. QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs. In *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming (ICFP '00), Montreal, Canada, September 18-21, 2000*, Martin Odersky and Philip Wadler (Eds.). ACM, 268–279. doi:10.1145/351240.351266

[10] Patricia Derler, Thomas H Feng, Edward A Lee, Slobodan Matic, Hiren D Patel, Yang Zheo, and Jia Zou. 2008. *PTIDES: A Programming Model for Distributed Real-Time Embedded Systems*. Technical Report. CALIFORNIA UNIV BERKELEY DEPT OF ELECTRICAL ENGINEERING AND COMPUTER SCIENCE.

[11] Adam Dunkels, Oliver Schmidt, Thiemo Voigt, and Muneeb Ali. 2006. Protothreads: Simplifying Event-Driven Programming of Memory-Constrained Embedded Systems. In *Proceedings of the 4th International Conference on Embedded Networked Sensor Systems, SenSys 2006, Boulder, Colorado, USA, October 31 - November 3, 2006*, Andrew T. Campbell, Philippe Bonnet, and John S. Heidemann (Eds.). ACM, 29–42. doi:10.1145/1182807.1182811

[12] Stephen A. Edwards and John Hui. 2020. The Sparse Synchronous Model. In *Forum for Specification and Design Languages, FDL 2020, Kiel, Germany, September 15-17, 2020*. IEEE, 1–8. doi:10.1109/FDL50818.2020.9232938

[13] Cordelia V. Hall, Kevin Hammond, Simon L. Peyton Jones, and Philip Wadler. 1996. Type Classes in Haskell. *ACM Trans. Program. Lang. Syst.* 18, 2 (1996), 109–138. doi:10.1145/227699.227700

[14] Paul Hudak. 1996. Building Domain-Specific Embedded Languages. *ACM Comput. Surv.* 28, 4es (1996), 196. doi:10.1145/242224.242477

[15] Marten Lohstroh, Christian Menard, Soroush Bateni, and Edward A. Lee. 2021. Toward a Lingua Franca for Deterministic Concurrent Systems. *ACM Trans. Embed. Comput. Syst.* 20, 4 (2021), 36:1–36:27. doi:10.1145/3448128

[16] Trevor L. McDonell, Joshua D. Meredith, and Gabriele Keller. 2022. Embedded Pattern Matching. In *Haskell '22: 15th ACM SIGPLAN International Haskell Symposium, Ljubljana, Slovenia, September 15 - 16, 2022*, Nadia Polikarpova (Ed.). ACM, 123–136. doi:10.1145/3546189.3549917

[17] R. Nilsson. 2014. *ScalaCheck: The Definitive Guide*. Artima Press. https://books.google.se/books?id=AOvcoQEACAAJ

[18] Georg Ofenbeck, Tiark Rompf, and Markus Püschel. 2016. RandIR: Differential Testing for Embedded Compilers. In *Proceedings of the 7th ACM SIGPLAN Symposium on Scala, SCALA@SPLASH 2016, Amsterdam, Netherlands, October 30 - November 4, 2016*, Aggelos Biboudis, Manohar Jonnalagedda, Sandro Stucki, and Vlad Ureche (Eds.). ACM, 21–30. doi:10.1145/2998392.2998397

[19] Michal H. Palka, Koen Claessen, Alejandro Russo, and John Hughes. 2011. Testing an Optimising Compiler by Generating Random Lambda Terms. In *Proceedings of the 6th International Workshop on Automation of Software Test, AST 2011, Waikiki, Honolulu, HI, USA, May 23-24, 2011*, Antonia Bertolino, Howard Foster, and J. Jenny Li (Eds.). ACM, 91–97. doi:10.1145/1982595.1982615

[20] Lee Pike, Alwyn Goodloe, Robin Morisset, and Sebastian Niller. 2010. Copilot: A Hard Real-Time Runtime Monitor. In *Runtime Verification - First International Conference, RV 2010, St. Julians, Malta, November 1-4, 2010. Proceedings (Lecture Notes in Computer Science, Vol. 6418)*, Howard Barringer, Yliès Falcone, Bernd Finkbeiner, Klaus Havelund, Insup Lee, Gordon J. Pace, Grigore Rosu, Oleg Sokolsky, and Nikolai Tillmann (Eds.). Springer, 345–359. doi:10.1007/978-3-642-16612-9_26

[21] Tiark Rompf and Martin Odersky. 2012. Lightweight Modular Staging: a pragmatic approach to runtime code generation and compiled DSLs. *Commun. ACM* 55, 6 (2012), 121–130. doi:10.1145/2184319.2184345

[22] Josef Svenningsson and Emil Axelsson. 2012. Combining Deep and Shallow Embedding for EDSL. In *Trends in Functional Programming - 13th International Symposium, TFP 2012, St. Andrews, UK, June 12-14, 2012, Revised Selected Papers (Lecture Notes in Computer Science, Vol. 7829)*, Hans-Wolfgang Loidl and Ricardo Peña (Eds.). Springer, 21–36. doi:10.1007/978-3-642-40447-4_2

[23] The Zephyr Project. 2021. The Zephyr Project. https://www.zephyrproject.org/.

[24] Nachiappan Valliappan, Robert Krook, Alejandro Russo, and Koen Claessen. 2020. Towards Secure IoT Programming in Haskell. In *Proceedings of the 13th ACM SIGPLAN International Symposium on Haskell, Haskell@ICFP 2020, Virtual Event, USA, August 7, 2020*, Tom Schrijvers (Ed.). ACM, 136–150. doi:10.1145/3406088.3409027

[25] Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. 2011. Finding and Understanding Bugs in C Compilers. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2011, San Jose, CA, USA, June 4-8, 2011*, Mary W. Hall and David A. Padua (Eds.). ACM, 283–294. doi:10.1145/1993498.1993532

```
{-# LANGUAGE ImplicitParams #-}
{-# OPTIONS_GHC -fplugin=SSM.Plugin
                -fplugin-opt=SSM.Plugin:mode=routine #-}
module Blinky where

import SSM.Language
import SSM.Frontend.Peripheral.LED

program :: (?pin :: Ref GPIO) => SSM ()
program = routine $ while true $ do
  after (msecs 1000) ?pin (not $ deref ?pin)
  wait ?pin

main :: Compile ()
main = do
  (pin, handler) <- output 1
  let ?pin = pin
  schedule handler
  schedule program
```

Fig. 1. The “blinky” example from Listing 2, implemented in Scoria.

```
1  #include <zephyr.h>
2  #include <drivers/gpio.h>
3
4  static const struct gpio_dt_spec button =
5      GPIO_DT_SPEC_GET_OR(DT_ALIAS(sw0), gpios, {0});
6
7  static struct gpio_dt_spec led =
8      GPIO_DT_SPEC_GET_OR(DT_ALIAS(led0), gpios, {0});
9
10 void main() {
11     gpio_pin_configure_dt(&button, GPIO_INPUT);
12     gpio_pin_configure_dt(&led, GPIO_OUTPUT);
13
14     while (1)
15         gpio_pin_set_dt(&led, gpio_pin_get_dt(&button));
16 }
```

Fig. 2. The code of the polling button-to-blink code. 100% of the CPU is allocated to reflecting the button state on the LED, which gives a kind of theoretical minimum for how fast the response time *could* be.

Listing 1. The button-to-blink program implemented using Zephyr, where input events are delivered to an input ISR (line 14). When an event arrives, it is placed in a ring buffer (line 17-20), and the main thread is woken up by releasing a semaphore (line 22). The main thread retrieves the event from the ring buffer (line 39-40) and toggles the LED.

```
1  #include <zephyr.h>
2  #include <drivers/gpio.h>
3  #include "rb.h"
4
5  K_SEM_DEFINE(my_sem, 0, 1);
6  SSM_RB_DEFINE(int, ssm_input_buffer, 12);
7
8  static const struct gpio_dt_spec button =
9      GPIO_DT_SPEC_GET_OR(DT_ALIAS(sw0), gpios, {0});
10 static struct gpio_dt_spec led =
11     GPIO_DT_SPEC_GET_OR(DT_ALIAS(led0), gpios, {0});
12 static struct gpio_callback button_cb_data;
13
14 void button_pressed(const struct device *dev
15                     , struct gpio_callback *cb
16                     , uint32_t pins) {
17     int *msg = ssm_rb_writer_alloc(ssm_input_buffer);
```

```
18      if(msg) {
19          *msg = gpio_pin_get_dt(&button);
20          ssm_rb_writer_commit(ssm_input_buffer);
21      }
22      k_sem_give(&my_sem);
23  }
24
25  void main(void)
26  {
27      gpio_pin_configure_dt(&button, GPIO_INPUT);
28      gpio_pin_interrupt_configure_dt(&button, GPIO_INT_EDGE_BOTH);
29      gpio_init_callback(&button_cb_data
30                         , button_pressed
31                         , BIT(button.pin));
32      gpio_add_callback(button.port, &button_cb_data);
33
34      gpio_pin_configure_dt(&led, GPIO_OUTPUT);
35
36      while(true) {
37          k_sem_take(&my_sem, K_FOREVER);
38          int *msg = ssm_rb_reader_claim(ssm_input_buffer);
39          if(msg) {
40              gpio_pin_set_dt(&led, *msg);
41              ssm_rb_reader_free(ssm_input_buffer);
42          }
43      }
44  }
```

Listing 2. “Blinky” implementation with correct temporal behavior, implemented using Zephyr RTOS. To achieve more precise timing, the code configures the NRF52840-DK board to use its external crystal oscillator as its clock source. This implementation does not suffer from drift due to its use of logical time, though this comes at the complexity of (1) maintaining the value of the logical clock, and (2) decomposing the application into interrupt handlers.

```
1  #include <zephyr.h>
2  #include <drivers/gpio.h>
3  #include <drivers/counter.h>
4  #include <drivers/clock_control.h>
5  #include <drivers/clock_control/nrf_clock_control.h>
6  static struct gpio_dt_spec led =
7      GPIO_DT_SPEC_GET_OR(DT_ALIAS(ssm_led), gpios, {0});
8  const struct device *timer_dev, *clock_dev;
9  struct counter_alarm_cfg alarm_cfg;
```

```
10  uint32_t current; // Logical time
11
12  int configure_led(void) {
13    if (led.port && !device_is_ready(led.port))
14      return -ENODEV;
15    gpio_pin_configure(&led, GPIO_OUTPUT);
16  }
17
18  int configure_timer(void) {
19    timer_dev = device_get_binding(DT_LABEL(DT_ALIAS(ssm_timer)));
20    if (!timer_dev)
21      return -ENODEV;
22    counter_start(timer_dev);
23
24    clock_dev = device_get_binding(DT_LABEL(DT_INST(0, nordic_nrf_clock)));
25    if (!clock_dev)
26      return -ENODEV;
27    clock_control_on(clock_dev, CLOCK_CONTROL_NRF_SUBSYS_HF);
28  }
29
30  void set_alarm(uint64_t delay_in_us) {
31    alarm_cfg.ticks = (current += counter_us_to_ticks(timer_dev, delay_in_us));
32    counter_set_channel_alarm(timer_dev, 0, &alarm_cfg);
33  }
34
35  void counter_interrupt_fn(const struct device *counter_dev
36                            , uint8_t chan_id
37                            , uint32_t ticks, void *user_data) {
38    gpio_pin_toggle_dt(&led);
39    set_alarm(1000000); // 1000000 us = 1000 ms
40  }
41
42  int configure_alarm(void) {
43    alarm_cfg.flags = COUNTER_ALARM_CFG_ABSOLUTE |
44                      COUNTER_ALARM_CFG_EXPIRE_WHEN_LATE;
45    alarm_cfg.callback = counter_interrupt_fn;
46    alarm_cfg.user_data = &alarm_cfg;
47
48    counter_get_value(timer_dev, &current);
49  }
50
51  void main(void) {
52    configure_led();
```

```
    configure_timer();
    configure_alarm();
    set_alarm(1000000); // 1000000 us = 1000 ms
}
```

Listing 3. The C version of the BLE mime broadcaster. This application broadcasts one of two frequencies, switching between the two every five seconds.

```
#include <zephyr.h>
#include <device.h>
#include <drivers/gpio.h>
#include <drivers/counter.h>
#include <drivers/clock_control.h>
#include <drivers/clock_control/nrf_clock_control.h>
#include <bluetooth/bluetooth.h>
#include <bluetooth/hci.h>
static void counter_interrupt_fn( const struct device *counter_dev
                                , uint8_t chan_id
                                , uint32_t ticks, void *user_data);

int ssm_timer_configure_clock(void) {
    const struct device *clock;
    clock = device_get_binding(DT_LABEL(DT_INST(0, nordic_nrf_clock)));
    clock_control_on(clock, CLOCK_CONTROL_NRF_SUBSYS_HF);
    return 0;
}

const struct device *timer_dev;
struct counter_alarm_cfg alarm_cfg;

void set_alarm(uint64_t delay_in_us) {
    alarm_cfg.ticks = counter_us_to_ticks(timer_dev, delay_in_us);
    counter_set_channel_alarm(timer_dev, 0, &alarm_cfg);
}

uint64_t broadcast_data = 0;
static struct bt_data ad[] = {
    BT_DATA(BT_DATA_MANUFACTURER_DATA, &broadcast_data, 8)};
K_THREAD_STACK_DEFINE(my_stack_area, 512);
struct k_work_q my_work_q;
#define SHORT_FREQ 200000
#define LONG_FREQ  500000

```

```
36  void alternate_frequency_broadcast(struct k_work *item) {
37      bt_le_adv_stop();
38      broadcast_data = broadcast_data == SHORT_FREQ ? LONG_FREQ : SHORT_FREQ;
39      bt_le_adv_start(BT_LE_ADV_NCONN_IDENTITY, ad, ARRAY_SIZE(ad), NULL, 0);
40  }
41
42  K_WORK_DEFINE(my_work_item, alternate_frequency_broadcast);
43
44  static void counter_interrupt_fn( const struct device *counter_dev
45                                  , uint8_t chan_id
46                                  , uint32_t ticks, void *user_data) {
47      k_work_submit(&my_work_item);
48      set_alarm(5000000);
49  }
50
51  void main() {
52      bt_enable(NULL);
53      k_work_queue_start(&my_work_q, my_stack_area
54                         , K_THREAD_STACK_SIZEOF(my_stack_area), 5, NULL);
55      timer_dev = device_get_binding(DT_LABEL(DT_ALIAS(ssm_timer)));
56      counter_start(timer_dev);
57      ssm_timer_configure_clock();
58      alarm_cfg.flags = 0;
59      alarm_cfg.callback = counter_interrupt_fn;
60      alarm_cfg.user_data = &alarm_cfg;
61      set_alarm(5000000);
62  }
```

Listing 4. The C version of the BLE mime generator. This application scans for broadcasted messages, and generates a frequency on LED 0. If a broadcasted message is detected, the generated frequency is the one specified in the advertisement payload.

```
1   #include <zephyr.h>
2   #include <device.h>
3   #include <drivers/gpio.h>
4   #include <drivers/counter.h>
5   #include <drivers/clock_control.h>
6   #include <drivers/clock_control/nrf_clock_control.h>
7   #include <bluetooth/bluetooth.h>
8   #include <bluetooth/hci.h>
9   static struct gpio_dt_spec led =
10      GPIO_DT_SPEC_GET_OR(DT_ALIAS(led0), gpios, {0});
11  static void counter_interrupt_fn( const struct device *counter_dev
```

```
                                    , uint8_t chan_id
                                    , uint32_t ticks, void *user_data);

int ssm_timer_configure_clock(void) {
    const struct device *clock;
    clock = device_get_binding(DT_LABEL(DT_INST(0, nordic_nrf_clock)));
    clock_control_on(clock, CLOCK_CONTROL_NRF_SUBSYS_HF);
    return 0;
}
const struct device *timer_dev;
struct counter_alarm_cfg alarm_cfg;

void set_alarm(uint64_t delay_in_us) {
    alarm_cfg.ticks = counter_us_to_ticks(timer_dev, delay_in_us);
    counter_set_channel_alarm(timer_dev, 0, &alarm_cfg);
}

uint64_t period_in_us = 1000000;
uint64_t next_value = 0;

static void counter_interrupt_fn( const struct device *counter_dev
                                    , uint8_t chan_id
                                    , uint32_t ticks, void *user_data) {
    gpio_pin_set_dt(&led, next_value);
    next_value = next_value ^ 1UL;
    set_alarm(period_in_us);
}

static void device_found( const bt_addr_le_t *addr, int8_t rssi
                            , uint8_t type
                            , struct net_buf_simple *ad) {
    if (type != BT_GAP_ADV_TYPE_ADV_NONCONN_IND) return;
    if (ad->data[0] == 9 && ad->data[1] == BT_DATA_MANUFACTURER_DATA) {
        uint64_t res = 0;
        for (int i = 7; i >= 0; i--) { res = (res << 8) |
                                            (uint64_t)ad->data[2 + i]; }
        period_in_us = res;
    }
}

void main() {
    bt_enable(NULL);
    timer_dev = device_get_binding(DT_LABEL(DT_ALIAS(ssm_timer)));
```

```
55      counter_start(timer_dev);
56      ssm_timer_configure_clock();
57      alarm_cfg.flags = 0;
58      alarm_cfg.callback = counter_interrupt_fn;
59      alarm_cfg.user_data = &alarm_cfg;
60      bt_le_scan_start(BT_LE_SCAN_PASSIVE, device_found);
61      gpio_pin_configure_dt(&led, GPIO_OUTPUT);
62      set_alarm(period_in_us);
63  }
```

Listing 5. The Scoria version of the BLE mime broadcaster. It implements the same application as the one in Listing 3. See Section 5 for a description of what `delay` does.

```
1   {-# LANGUAGE ImplicitParams #-}
2   {-# OPTIONS_GHC -fplugin=SSM.Plugin -fplugin-opt=SSM.Plugin:mode=routine #-}
3   module FrequencyMime where
4
5   import SSM.Language
6   import SSM.Frontend.Peripheral.BasicBLE
7
8   freqCount :: Ref Time -> SSM () -- concurrent process dictating which
9                                   -- frequency to broadcast
10  freqCount period = routine $ while true $ do
11      period <~ (msecs 200)
12      delay (secs 5)
13      period <~ (msecs 500)
14      delay (secs 5)
15
16  broadcastCount :: (?ble :: BBLE) => Ref Time -> SSM ()
17  broadcastCount count = routine $ while true $ do
18      enableBroadcast $ time2ns $ deref count
19      delay (secs 5)
20      disableBroadcast
21
22  counterEntry :: (?ble :: BBLE) => SSM ()
23  counterEntry = routine $ do
24      count <- var $ secs 1 -- create the reference that is shared between the
25                            -- concurrent processes
26      fork [ freqCount count, broadcastCount count ]
27
28  counter :: Compile ()
29  counter = do
30      (ble, broadcast, scanning) <- enableBasicBLE
```

```
31
32      let ?ble = ble
33
34      schedule counterEntry
35      schedule broadcast
36      schedule scanning
```

Listing 6. The Scoria version of the BLE mime generator. It implements the same application as the one in Listing 4. See Section 5 for a description of what delay does.

```
1   {-# LANGUAGE ImplicitParams #-}
2   {-# OPTIONS_GHC -fplugin=SSM.Plugin -fplugin-opt=SSM.Plugin:mode=routine #-}
3   module FrequencyMime where
4
5   import SSM.Language
6   import SSM.Frontend.Peripheral.LED
7   import SSM.Frontend.Peripheral.BasicBLE
8
9   -- scan for broadcasted frequencies and write them to the shared reference
10  bleHandler :: (?ble :: BBLE) => Ref Time -> SSM ()
11  bleHandler period = routine $ do
12      enableScan
13      while true $ do
14        wait scanref
15        period <~ nsecs (deref scanref)
16        delay (secs 5)
17
18  -- generate a frequency with a half period dictated by the value of
19  -- the shared reference
20  freqGen :: (?led0::Ref LED) => Ref Time -> SSM ()
21  freqGen period = routine $ while true $ do
22    after (deref period) ?led0 (not' $ deref ?led0)
23    wait ?led0
24
25  entry :: (?ble :: BBLE, ?led0::Ref LED) => SSM ()
26  entry = routine $ do
27    period <- var $ secs 1 -- create reference that is shared between the
28                           -- concurrent processes
29    fork [freqGen period, bleHandler period]
30
31  generator :: Compile ()
32  generator = do
33    (led, handler) <- output 0
```

```
34    (ble, broadcast, scanning) <- enableBasicBLE
35
36    let ?led0 = led
37        ?ble  = ble
38
39    schedule handler
40    schedule entry
41    schedule broadcast
42    schedule scanning
```

Listing 7. The frequency generator implemented using Zephyr. When the timer goes off, the associated event (the new state of the LED) is placed in the ring buffer (line 38-41), and the main thread is woken up by releasing a semaphore (line 43). The main thread retrieves the event (line 63-66), and schedules the next wake-up point (line 67).

```
1   #include <zephyr.h>
2   #include <drivers/gpio.h>
3   #include <drivers/counter.h>
4   #include <drivers/clock_control.h>
5   #include <drivers/clock_control/nrf_clock_control.h>
6   #include "rb.h"
7   K_SEM_DEFINE(my_sem, 0, 1);
8   SSM_RB_DEFINE(uint8_t, ssm_input_buffer, 12);
9   static struct gpio_dt_spec led =
10      GPIO_DT_SPEC_GET_OR(DT_ALIAS(led0), gpios, {0});
11  static void counter_interrupt_fn( const struct device *counter_dev
12                                  , uint8_t chan_id
13                                  , uint32_t ticks, void *user_data);
14  uint32_t current;
15  const struct device *timer_dev;
16  struct counter_alarm_cfg alarm_cfg;
17  int ssm_timer_configure_clock(void) {
18      const struct device *clock;
19      clock = device_get_binding(DT_LABEL(DT_INST(0, nordic_nrf_clock)));
20      clock_control_on(clock, CLOCK_CONTROL_NRF_SUBSYS_HF);
21      return 0;
22  }
23
24  void set_alarm(uint64_t delay_in_us) {
25      alarm_cfg.ticks = (current += counter_us_to_ticks(timer_dev, delay_in_us));
26      counter_set_channel_alarm(timer_dev, 0, &alarm_cfg);
27  }
28  uint8_t next;
```

```

static void counter_interrupt_fn( const struct device *counter_dev
                                , uint8_t chan_id
                                , uint32_t ticks, void *user_data) {
    next = next ? 0 : 1;
    uint8_t *msg = ssm_rb_writer_alloc(ssm_input_buffer);
    if(msg) { *msg = next; ssm_rb_writer_commit(ssm_input_buffer); }
    k_sem_give(&my_sem);
}

void main() {
    gpio_pin_configure_dt(&led, GPIO_OUTPUT);
    timer_dev = device_get_binding(DT_LABEL(DT_ALIAS(ssm_timer)));
    counter_start(timer_dev);
    ssm_timer_configure_clock();
    counter_get_value(timer_dev, &current);
    alarm_cfg.flags = COUNTER_ALARM_CFG_ABSOLUTE |
                      COUNTER_ALARM_CFG_EXPIRE_WHEN_LATE;
    alarm_cfg.callback = counter_interrupt_fn;
    alarm_cfg.user_data = &alarm_cfg;
    uint32_t half_period = 41;
    set_alarm(half_period);
    while(true) {
        k_sem_take(&my_sem, K_FOREVER);
        uint8_t *msg = ssm_rb_reader_claim(ssm_input_buffer);
        if(msg) {
            gpio_pin_set_dt(&led, *msg);
            ssm_rb_reader_free(ssm_input_buffer);
            set_alarm(half_period);
        }
    }
}
```

Listing 8. A frequency generator implemented in Scoria. The half period of the generated frequency is adjusted in steps of 2 µs when button 0 or button 1 is pressed.

```
{-# LANGUAGE ImplicitParams #-}
{-# LANGUAGE FlexibleContexts #-}
{-# LANGUAGE RebindableSyntax #-}
{-# OPTIONS_GHC -fplugin=SSM.Plugin -fplugin-opt=SSM.Plugin:mode=routine #-}
module Generator where

import Prelude
```

```
8   import SSM.Language
9   import SSM.Frontend.Peripheral.GPIO
10
11  -- signal generator, generating a signal on ?out0 with a half period
12  -- specified by hpr
13  sig_gen :: (?out0 :: Ref GPIO) => Ref Time -> SSM ()
14  sig_gen hpr = routine $ while true $ do
15      after (deref hpr) ?out0 (not' $ deref ?out0)
16      wait ?out0
17
18  -- increments or decrements the half period by 2 us, depending on which of
19  -- two buttons was pressed
20  button_handler :: (?b1 :: Ref Switch, ?b2 :: Ref Switch) => Ref Time -> SSM ()
21  button_handler hpr = routine $ while true $ do
22      wait (?b1, ?b2)
23      if changed ?b1 -- here we woke up, so either b1 or b2 was pressed
24          then hpr <~ deref hpr + usecs 2
25          else hpr <~ deref hpr - usecs 2
26
27  -- create shared half period reference and initialize it to 1 second, before
28  -- spawning the concurrent child processes
29  entry :: (?out0 :: Ref GPIO, ?b1 :: Ref Switch, ?b2 :: Ref Switch) => SSM ()
30  entry = routine $ do
31      hp <- var $ usecs 1000000
32      fork [ sig_gen hp, button_handler hp]
33
34  -- Specify what inputs and outputs the program uses, and specify the initial
35  -- contents on the ready queue
36  main :: Compile ()
37  main = do
38      (led, handler) <- output 0
39      button1 <- input 0
40      button2 <- input 1
41
42      let ?out0 = led
43          ?b1 = button1
44          ?b2 = button2
45
46      schedule handler
47      schedule entry
```

Listing 9. The generated C code from the Scoria program in Figure 15. For more details of code such as this, see [12].

```
#include "ssm-platform.h"
#ifndef SSM_DEBUG_TRACE
#define SSM_DEBUG_TRACE(...) do ; while (0)
#endif
#ifndef SSM_DEBUG_MICROTICK
#define SSM_DEBUG_MICROTICK(...) do ; while (0)
#endif
ssm_bool_t input0;
ssm_bool_t output1;
typedef char event;
typedef struct {
    struct ssm_act act;
    struct ssm_trigger trig1;
} act_b2b_t;
struct ssm_act *enter_b2b( struct ssm_act *caller, ssm_priority_t priority
                         , ssm_depth_t depth);
void step_b2b(struct ssm_act *actg);
struct ssm_act *enter_b2b( struct ssm_act *caller, ssm_priority_t priority
                         , ssm_depth_t depth) {
    struct ssm_act *actg =
        ssm_enter(sizeof(act_b2b_t), step_b2b, caller, priority, depth);
    act_b2b_t *acts = container_of(actg, act_b2b_t, act);
    acts->trig1.act = actg;
    return actg;
}
void step_b2b(struct ssm_act *actg) {
    act_b2b_t *acts = container_of(actg, act_b2b_t, act);
    SSM_DEBUG_TRACE("ActStepBegin \"b2b\"");
    SSM_DEBUG_MICROTICK();
    switch (actg->pc) {
    case 0:;
        while (true) {
            SSM_DEBUG_MICROTICK();
            SSM_DEBUG_TRACE("ActSensitize \"input0\"");
            ssm_sensitize(&(&input0)->sv, &acts->trig1);
            actg->pc = 1;
            return;
        case 1:;
            ssm_desensitize(&acts->trig1);
            ssm_assign_bool(&output1, actg->priority, (&input0)->value);
        }
    default: break;
    }
```

```
44      ssm_leave(actg, sizeof(act_b2b_t));
45  }
46  int ssm_program_initialize(void) {
47      ssm_initialize_bool(&input0);
48      ssm_assign_bool(&input0, 0, 0);
49      ssm_initialize_bool(&output1);
50      ssm_assign_bool(&output1, 0, 0);
51      bind_static_input_device((ssm_sv_t *)&input0.sv, 0U);
52      ssm_activate(enter_b2b(&ssm_top_parent
53          , SSM_ROOT_PRIORITY + 0 * (1 << SSM_ROOT_DEPTH - 1), SSM_ROOT_DEPTH - 1));
54      ssm_activate(bind_static_output_device(&ssm_top_parent
55          , SSM_ROOT_PRIORITY + 1 * (1 << SSM_ROOT_DEPTH - 1)
56          , SSM_ROOT_DEPTH - 1, &output1.sv, 0U));
57      return 0;
58  }
```

Paper II

HasTEE - Programming Trusted Execution Environments with Haskell

Abhiroop Sarkar, **Robert Krook**, Alejandro Russo, Koen Claessen

Proceedings of the 16th ACM SIGPLAN International Haskell Symposium, 2023

HasTEE: Programming Trusted Execution Environments with Haskell

ABHIROOP SARKAR, Chalmers University, Sweden
ROBERT KROOK, Chalmers University, Sweden
ALEJANDRO RUSSO, Chalmers University DPella AB, Sweden
KOEN CLAESSEN, Chalmers University, Sweden

Trusted Execution Environments (TEEs) are hardware enforced memory isolation units, emerging as a pivotal security solution for security-critical applications. TEEs, like Intel SGX and ARM TrustZone, allow the isolation of confidential code and data within an untrusted host environment, such as the cloud and IoT. Despite strong security guarantees, TEE adoption has been hindered by an awkward programming model. This model requires manual application partitioning and the use of error-prone, memory-unsafe, and potentially information-leaking low-level C/C++ libraries.

We address the above with *HasTEE*, a domain-specific language (DSL) embedded in Haskell for programming TEE applications. HasTEE includes a port of the GHC runtime for the Intel-SGX TEE. HasTEE uses Haskell's type system to automatically partition an application and to enforce *Information Flow Control* on confidential data. The DSL, being embedded in Haskell, allows for the usage of higher-order functions, monads, and a restricted set of I/O operations to write any standard Haskell application. Contrary to previous work, HasTEE is lightweight, simple, and is provided as a *simple security library*; thus avoiding any GHC modifications. We show the applicability of HasTEE by implementing case studies on federated learning, an encrypted password wallet, and a differentially-private data clean room.

Additional Key Words and Phrases: Trusted Execution Environment, Haskell, Intel SGX, Enclave

1 Introduction

Trusted Execution Environments (TEEs) are an emerging design of hardware-enforced memory isolation units that aid in the construction of security-sensitive applications [39, 52]. TEEs have been used to enforce a strong notion of *trust* in areas such as confidential (cloud-)computing [11, 65], IoT [34] and Blockchain [10]. Intel and ARM each have their own TEE implementations known as Intel SGX [29] and ARM TrustZone [4], respectively. Principally, TEEs provide a *disjoint* region of code and data memory that allows for the physical isolation of a program's execution and state from the underlying operating system, hypervisor, and I/O peripherals. For terminology, we shall use the term *enclave* (adopted from Intel) to refer to the isolated region of code and data and its trusted computing base (TCB).

TEEs, despite their strong security guarantees, have seen limited adoption in software development owing to several challenges. Firstly, TEEs often present *an*

Authors' Contact Information: Abhiroop Sarkar, Chalmers University, Gothenburg, Sweden, sarkara@chalmers.se; Robert Krook, Chalmers University, Gothenburg, Sweden, krookr@chalmers.se; Alejandro Russo, Chalmers University and DPella AB, Gothenburg, Sweden, russo@chalmers.se; Koen Claessen, Chalmers University, Gothenburg, Sweden, koen@chalmers.se.

awkward and low-level programming model [14]. For instance, Intel provides a C/C++ interface to program SGX that requires *partitioning* the program's state into trusted and untrusted components and dividing the entire logic into two separate software projects (Section 2)—a complex and error-prone process that could lead to data leakage. From a security perspective, the use of C/C++ APIs can open further opportunities to exploit well-known memory-unsafe vulnerabilities such as return-oriented programming (ROP) [53] in applications running inside TEEs [40]. Secondly, *current TEE programming models are insufficient to enforce security policies.* Applications should be written in a way such that they do not accidentally reveal confidential information. Furthermore, inputs and outputs to an enclave must be correctly encrypted, signed, decrypted, and verified to protect against malicious hosts. Thirdly, *little support is given to migrate legacy applications inside enclaves.* Applications inside enclaves often rely on their own Operating System (OS) since they cannot trust the one in the host machine. Library OS-based approaches exist to provide this functionality. However, for legacy applications written in high-level languages relying on non-trivial runtimes, the porting of the runtime becomes a challenging task.

Efforts have been made to address these challenges. The work by [21] introduces GoTEE, a modification of the Go programming language with support for *secure routines* that are executed inside enclaves. In GoTEE, the authors heavily modify the Go compiler and extend the language to support new TEE-specific abstractions that helps to automatically partition an application. GoTEE does not provide any control over how sensitive information moves within the application, which could enable accidental data leaks. In a similar spirit, [43] introduce J_E, a subset of Java with support for enclaves. J_E focuses on providing information-flow control (IFC) to ensure that the code does not leak sensitive data by accident or by coercion of a malicious host. J_E uses a sophisticated compilation pipeline to first partition the application and then uses another compiler to check that sensitive information is not leaked. Virtualization-based solutions, such as AMD SEV [2], attempt to alleviate the effort required to port legacy applications. However, the trade-off is that the TCB becomes larger and the granularity to identify sensitive data becomes much coarser.

Our contribution through this paper is *HasTEE*, a domain-specific language (DSL) embedded in Haskell for programming TEE applications. HasTEE integrates TEE-specific abstraction and semantics while hiding low-level hardware intricacies making it hardware neutral! Additionally, HasTEE offers IFC to prevent accidental leakage of sensitive data. Owing to its embedding in Haskell, developers can use familiar abstractions such as high-order functions, monads, and a limited set of I/O operations to write applications in a conventional manner. This design choice enables seamless integration with all of the existing Haskell features. Compared to the previous work, HasTEE is lightweight, simple, and is provided as a *simple security library*; thus avoiding any GHC [31] compiler modifications!

1.1 HasTEE by Example

Listing 1 presents a sample password checker application written using HasTEE.

```
1  pwdChkr :: Enclave String -> String -> Enclave Bool
2  pwdChkr pwd guess = fmap (== guess) pwd
3
4  passwordChecker :: App Done
5  passwordChecker = do
6    passwd <- inEnclaveConstant "secret"
7    efunc  <- inEnclave $ pwdChkr passwd
8    runClient $ do -- Client code
9      liftIO $ putStrLn "Enter your password"
10     userInput <- liftIO getLine
11     res       <- gateway (efunc <@> userInput)
12     liftIO $ putStrLn ("Login returned " ++ show res)
```

Listing 1. A password checker written in HasTEE

The distinction between the trusted and untrusted parts of the application is done via the type system that encodes the former as the `Enclave` type (line 1) and the latter as the `Client` type (type inferred in line 8).

The function `pwdChkr` takes a sensitive string located in the enclave, of type `Enclave String`, a public string from the client host (`String`) and produces a sensitive Boolean in the enclave (`Enclave Bool`). Line 6 holds the secret string that we want to protect (`inEnclaveConstant`). Line 7 uses the `inEnclave` call to obtain a reference to the function `pwdChkr` located in the enclave. The function `gateway` (line 11) is responsible for transmitting the collected arguments to the enclave function, and bringing the result back to the client. The `gateway` function acts as an interface between the enclave and non-enclave environment. The untrusted *host client* is in charge of driving the application, while the *enclave* is assigned the role of a computational and/or storage resource that services the client's requests. HasTEE connects an application (`passwordChecker`) to Haskell's `main` method using the `runApp :: App a -> IO a` function that executes the application. From an IFC perspective, lines 6 and 7 correspond to labelling, i.e., establishing, which inputs are sensitive for the program—an activity that is part of the TCB. In general, HasTEE code starts by labelling the sensitive input with the `inEnclave` primitive. Subsequently, the client code is compelled to manipulate secrets in a *secure* manner. In this setting, secure means that no sensitive information in the enclave gets leaked except that it has been obtained via the primitive `gateway`. The HasTEE API is explained in Section 4.2, and the semantics are discussed in Section 4.3.

1.2 Contributions

A type-safe, secure, high-level programming model. The HasTEE library enables developers to program a TEE environment, such as Intel SGX, using Haskell - a type-safe, memory-managed language whose expressive type system can be leveraged to enforce various security constraints. Additionally, HasTEE allows programming in a

familiar client-server style programming model (Section 4.2 and 5.2), an improvement over the low-level Intel SGX APIs.

Automatic Partitioning. A key part of programming TEEs, partitioning the trusted and untrusted parts of the program is done automatically using the type system (details in Section 3 and 4.3). Crucially, our approach does not require any modification of the GHC compiler and can be adapted to other programming languages, as long as their runtime can run on the desired TEE infrastructure.

Information Flow Control. Drawing inspiration from restricted IO monad families in Haskell, we designed an Enclave monad that prevents accidental leaks of secret data by TEE programmers (Section 5.3). Hence, our Enclave monad enables writing applications with a relatively low level of trust placed on the enclave programmer.

Portability of Haskell's runtime. We modify the GHC runtime, without modifying the compiler, to run on SGX enclaves. This enables us to host the complete Haskell language, including extensions, supported by GHC 8.8 (Section 5.1).

Demonstration of expressiveness. We illustrate the practicality of the HasTEE through three case studies across different domains: (1) a Federated Learning example (Section 6.1), (2) an encrypted password wallet (Section 6.2) and (3) a *differentially-private* data clean room (Section 6.3). The examples also demonstrate the simplicity of TEE development enabled by HasTEE.

2 Background

Intel Software Guard Extensions (SGX). Intel Software Guard Extensions (SGX) [29] is a set of security-related instructions supported since Intel's sixth-generation Skylake processor, which can enhance the security of applications by providing a *secure enclave* for processing sensitive data. The enclave is a disjoint portion of memory separate from the DRAM, where sensitive data and code reside, beyond the influence of an untrusted operating system and other low-level software.

Intel offers an SGX SDK for programming enclaves. The SDK requires dividing the application into trusted and untrusted parts, where sensitive data resides in the trusted project. It provides specialized function calls called *ecall* for enclave access and an *ocall* API for communication with the untrusted client. The boundary between the client and enclave is defined using an *Enclave Description Language (EDL)*. The SDK utilizes a tool called *edger8r* to parse EDL files and generate two *bridge* files. These files ensure secure data transfer between projects through *copying* instead of sharing via pointers, preventing potential manipulation of the enclave's state. Fig 1 shows the SDK's programming model.

Application developers working with enclaves aim to minimize the Trusted Computing Base (TCB) by keeping the operating system and system software outside the enclave. The SGX SDK offers a restricted C standard library implementation (`tlibc`) for essential system software. Programming SGX enclaves involves understanding the complex control flow between trusted and untrusted components. Enforcing

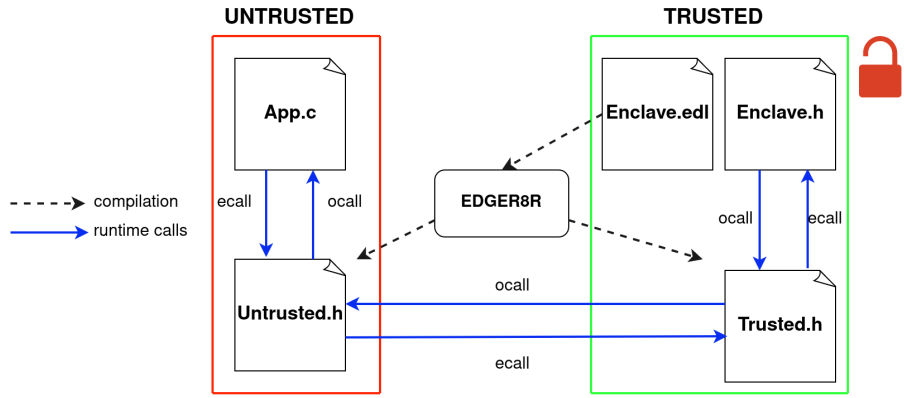


Fig. 1. Intel SGX SDK Programming Model

SGX's programming model on typical software projects can be challenging, and the limited `tlibc` library restricts running applications beyond those written in vanilla C/C++.

3 Key Idea: A Typed DSL for Enclaves

3.1 The Programming Model and Partitioning

HasTEE supports the automatic partitioning of programs by utilizing a combination of the type system to identify the enclave and a conditional compilation tactic to provide different semantics to each component. The compilation tactic was first used in Haste.App [18], to partition a single program into a `Client` and `Server` type. Fig 2 shows the partitioning procedure at a high level.

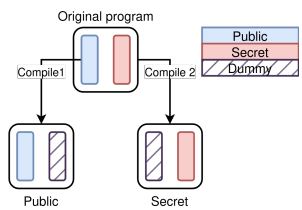


Fig. 2. The HasTEE partitioning scheme

Importantly, this approach does not require any compiler extensions or elaborate dependency analysis passes to distinguish between the underlying types. The codebase involved in other complex partitioning approaches [21, 43] becomes part of the Trusted Computing Base (TCB), creating a larger TCB. In contrast, our approach does not add any partitioning code to the TCB. Fig 3 shows the partitioned software stack in the HasTEE approach.

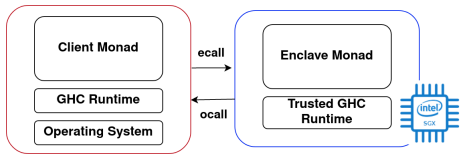


Fig. 3. The untrusted (left) and trusted (right) software

Post-partitioning, the client-server-style programming model is used for programming the enclave. In this model, the client takes on the primary role of driving the program and utilizes the enclave as a computational and/or storage resource. The source program, written in Haskell, benefits from type safety, while HasTEE internally handles the message transfer between the client and enclave memory at runtime.

3.2 Information Flow Control on Enclaves

Being a Haskell library enables HasTEE to tap into the library-based Information Flow Control techniques in Haskell [12, 49, 50]. The IFC literature distinguishes between sensitive and non-sensitive computations via monads indexed with security levels [50], e.g., `Sec H` and `Sec L`, where security levels `H` and `L` are assigned to sensitive and public information, respectively. Public information can flow into sensitive entities but not in the other way around. We have a similar security-level hierarchy between the `Enclave` and `Client` monads, respectively. Accordingly, we design the `Enclave` monad such that it restricts the possible variants of I/O operations. Internally, the `Enclave` monad constrains the scope of side-effecting operations to protect the confidentiality of data within the enclave (details in Section 5.3). Furthermore, HasTEE demands to explicitly mark where information is being sent back to the client (`gateway`), thus clearly indicating where to audit and control information leakages. Due to the security-critical nature of the Enclave monad, we include a `trust` operator, which is similar to the `endorse` function found in IFC literature.

3.3 Trusted GHC Runtime

One of the key challenges in allowing Haskell programs to run on TEE platforms is to provide support for the GHC Haskell Runtime [36] itself. A Haskell program relies on the runtime for essential tasks such as memory allocation, concurrency, I/O management, etc. The GHC runtime heavily depends on well-known C standard libraries, such as `glibc` on Linux [23] and `msvcrt` on Windows [38]. In contrast, the Intel SGX SDK provides a much more restricted `libc` known as `tlibc`.

This results in the fact that several `libc` calls used by the GHC runtime such as `mmap`, `madvise`, `epoll`, `select` and 100+ other functions become unavailable. Even the core threading library used by the GHC runtime, `pthread`, has a much more restricted API on the SGX SDK. To solve this conundrum, we have patched portions

of the GHC runtime and used functionalities from a library OS, Gramine [57], to enable the execution of GHC-compiled programs on the enclave.

3.4 TEE Independence

Finally, HasTEE provides an abstraction over low-level system APIs offered by TEEs. As a result, the principles applied in programming Intel SGX should translate to the programming of other popular TEEs, such as the ARM TrustZone.

4 Design of HasTEE

4.1 Threat Model

We begin by discussing the threat model of the HasTEE DSL. HasTEE has the very same threat model as that of Intel SGX. In this model, only the software running inside the enclave memory is trusted. All other application and system software, such as the operating system, hypervisors, driver firmware, etc., are considered compromised by an attacker. A very similar threat model is shared by a number of other work based on Intel SGX [5, 11, 21, 35].

In this work, we enhance the application-level security firstly by using a memory-safe language, Haskell, and secondly use the `Enclave` monad to introduce information flow control. Our implementation strategy of loading the GHC runtime on the enclave allows us to handle Iago attacks [13] (see Section 5.1). We trust the underlying implementation of the SGX hardware and software stack (such as `tlibc`) as provided by Intel. Known limitations of Intel SGX such as denial-of-service attacks and side-channel attacks [61] are beyond the scope of this paper.

An ideally secure development process should include auditing the code running on the enclave either through static analyses or manual code reviews or both. The conciseness of Haskell codebases should generally facilitate the auditing process. However, the mechanisms for fail-proof audits are beyond the scope of this paper as well.

4.2 HasTEE API

We show the core API of HasTEE in Fig 4. The functions presented operate over three principal Haskell data types: (1) `Enclave`, (2) `Client`, and (3) `App`. All three types are instances of the `Monad` typeclass, which allows for the use of `do` notation when programming with them. One of the key differences in functionality provided by the `Client` and `Enclave` monads is that `Client` allows for arbitrary I/O, whereas `Enclave` only provides restricted I/O. More on the latter in Section 5.3. The `App` monad sets up the infrastructure for communication between the `Client` and `Enclave` monad. We show a simple secure counter written using most of the API in Listing 2.

Listing 2 internally gets partitioned into the trusted and untrusted components via conditional compilation. In line 3, `liftNewRef` is used to create a secure reference initialised to the value 0. Followed by that, the computation to increment this value

```
-- mutable references
liftNewRef :: a -> App (Enclave (Ref a))
readRef    :: Ref a -> Enclave a
writeRef   :: Ref a -> a -> Enclave ()

-- get reference to function inside enclave
inEnclave :: Securable a => a -> App (Secure a)

-- runs the Client monad
runClient :: Client () -> App Done

-- used for function application on the enclave
gateway :: Binary a => Secure (Enclave a) -> Client a
(<@>) :: Binary a => Secure (a -> b) -> a -> Secure b

-- call this from `main` to run the App monad
runApp :: App a -> IO a
```

Fig. 4. The core HasTEE API

```
1  app :: App Done
2  app = do
3    enclaveRef <- liftNewRef 0 :: App (Enclave (Ref Int))
4    count <- inEnclave $ do
5      r <- enclaveRef
6      v <- readRef r
7      writeRef r (v + 1) >> return v :: Enclave Int
8    runClient $ gateway count >>=
9                \v -> liftIO $ print $ "Counter's #" ++ show v
10
11 main = runApp app
```

Listing 2. A secure counter written in HasTEE (types annotated for clarity)

inside the enclave is given in lines 4 - 7. Applying `inEnclave` on the enclave computation (line 4) yields the type `App (Secure (Enclave Int))`. The `Secure` type is HasTEE's internal representation of a closure. Line 8 uses the critical `gateway` function to actually execute the enclave computation within the enclave memory and get the result back in the client memory. This resulting value, `v`, is displayed to the user.

The only function from Fig. 4 not used in Listing 2 is the `<@>` operator, used to collect arguments that are sent to the enclave. For example, an enclave function, `f`, that accepts two arguments, `arg1` and `arg2`, would be executed as `gateway (f <@> arg1 <@> arg2)`. Parameters to secure functions are copied to the enclave before the function is invoked, and results are copied from the enclave to the client before the client resumes execution. To do this copying, `gateway` and `<@>` has a `Binary` constraint on the types involved. This specifies that the values of the types involved have to be serialisable. Listing 1 in Section 1 shows a concrete usage of the operator. We have larger case studies in Section 6.

4.3 Operational Semantics of HasTEE

We provide big-step operational semantics of the HasTEE DSL. Note, we illustrate the semantics using an interpreter written in Haskell that shows the transition of the client as well as the enclave memory as each operators gets interpreted. We show our *expression language* and the abstract machine values to which we evaluate below:

```
type Name = String

data Exp = Lit Int | Var Name | Fun [Name] Exp
  | App Exp [Exp] | Let Name Exp Exp | Plus Exp Exp
  | InEnclave Exp | Gateway Exp | EnclaveApp Exp Exp --HasTEE

data Value = IntVal Int | Closure [Name] Exp Env
  | SecureClosure Name [Value] | ArgList [Value] | Dummy
  | Err ErrState -- Error conditions
```

The `Exp` language above is a slightly modified version of lambda calculus with the restriction of allowing only fully applied function application. This restriction is done to reflect the nature of the HasTEE API, which through the type system, only permits fully saturated function applications for functions residing in the enclave. The lambda calculus language is then extended with the core HasTEE operators.

In the `Value` type, the `Closure` constructor, owing to saturated function application, captures a list of variable names and the environment. Notable in the `Value` type is the `SecureClosure` constructor that represents a closure residing in the enclave memory. This constructor does not capture the body of the closure as the body could hold any hidden state that lies protected within the enclave memory. The `SecureClosure` value is used by the `Gateway` function to invoke functions residing in the enclave.

The `ArgList` constructor supports the `<@>` operator that collects enclave function arguments. Lastly, the `Dummy` value is used as a placeholder for operators lacking semantics depending on the client or the enclave memory. For instance, the `Gateway` function has no meaning inside the `Enclave` monad, it is only usable from the `Client` monad. The `Dummy` crucially enables the conditional compilation trick in HasTEE by

```
1  testProgram = let m = 3 in
2                let f = λ x -> x + m in
3                let y = inEnclave f in
4                gateway (y <@> 2)
```

Listing 3. A simple program for illustrating the operational semantics of HasTEE

acting as a placeholder for meaningless functions in the respective client and enclave memory.

Our evaluators will show transition relations operating on two distinct memories that maps variable names to values - the enclave memory and the client memory.

```
type ClientEnv  = [(Name, Value)]
type EnclaveEnv = [(Name, Value)]
```

Accordingly, we define two evaluators - `evalEnclave` (Fig. 5) and `evalClient` (Fig. 6). The complete evaluator run in two passes. In the first pass, it runs a program and loads up the necessary elements in the enclave memory and then in the second pass, the loaded enclave memory is additionally passed to the client's evaluator.

Two helper functions, `genEncVar` and `evalList` are not shown for concision. They generate unique variable names and fold over a list of expressions respectively. Appendix A contains the complete, typechecked semantics as runnable Haskell code..

We use Listing 3 to illustrate how the enclave, as well as the client memory, evolves as a program gets evaluated. Our semantic evaluator operates in two passes. In the first pass, the `evalEnclave` evaluator from Fig. 5 is run. Fig. 7a shows the state of the enclave environment after the evaluator has completed evaluating Listing 3. Notably, the variable `y` maps to a value with no semantic meaning, as the evaluator is already running in the secure memory.

In the second pass, the environment from Fig. 7a is additionally passed as a state variable to the evaluator `evalClient` from Fig. 6. Note the different value mapped to the variable `y` in Fig 7b. `EnclaveApp` is evaluated on lines 25-34 in Fig 6. It generates the value `SecureClosure "`$EncVar_0$`" [Lit 2]`.

Notable is the evaluation of the `gateway` call on line 4 of Listing 3. The semantics for this evaluation are in lines 12-24 of Fig 6. The evaluator upon finding a reference $EncVar_0$ with no semantics in the client memory (Fig 7b) looks up $EncVar_0$ in the enclave environment (Fig 7a) and finds a `Closure` with a body. Crucially, **it evaluates the `Closure` by invoking the `evalEnclave` function on line 21 of Fig. 6 using the enclave environment.** This part models how the SGX hardware switches to the enclave memory when executing the secure function `f` rather than the client memory. An important point is generating an identical fresh variable name, $EncVar_0$, that the client uses to identify and call the functions in the enclave memory.

```
evalEnclave :: (MonadState StateVar m)
            => Exp -> EnclaveEnv -> m (Value, EnclaveEnv)
evalEnclave (Lit n) env    = pure (IntVal n, env)
evalEnclave (Var x) env    = pure (lookupVar x env, env)
evalEnclave (Fun xs e) env =
  pure (Closure xs e env, env)
evalEnclave (Let name e1 e2) env = do
  (e1', env') <- evalEnclave e1 env
  evalEnclave e2 ((name,e1'):env')
evalEnclave (App f args) env     = do
  (v1, env1)   <- evalEnclave f env
  (vals, env2) <- evalList args env1 []
  case v1 of
    Closure xs body ev ->
      evalEnclave body ((zip xs vals) ++ ev)
    _ -> pure (Err ENotClosure, env2)
evalEnclave (Plus e1 e2) env = do
  (v1, env1) <- evalEnclave e1 env
  (v2, env2) <- evalEnclave e2 env1
  case (v1, v2) of
    (IntVal a1, IntVal a2) -> pure (IntVal (a1 + a2), env2)
    _                      -> pure (Err ENotIntLit, env2)
evalEnclave (InEnclave e) env = do
  (val, env') <- evalEnclave e env
  varname     <- genEncVar
  let env''   = (varname, val):env'
  pure (Dummy, env'')
-- the following two are essentially no-ops
evalEnclave (Gateway e) env = evalEnclave e env
evalEnclave (EnclaveApp e1 e2) env = do
  (_, env1) <- evalEnclave e1 env
  (_, env2) <- evalEnclave e2 env1
  pure (Dummy, env2)
```

Fig. 5. Operational Semantics of the Enclave

4.4 Practical Security Analysis

In what follows, we perform a security analysis of HasTEE. We start by making explicit that the only communication from the enclave back to the host client is

```
evalClient :: (MonadState StateVar m)
           => Exp -> ClientEnv -> m (Value, ClientEnv)

-- evalClient for Lit, Var, Fun, Let, App, Plus not
-- shown as they behave the same as evalEnclave above
evalClient (InEnclave e) env = do
  (_, env') <- evalClient e env
  varname   <- genEncVar
  let env'' = (varname, Dummy):env'
  pure (SecureClosure varname [], env'')
evalClient (Gateway e) env = do
  (e', env1) <- evalClient e env
  case e' of
    SecureClosure varname vals -> do
      enclaveEnv <- gets encState
      let func = lookupVar varname enclaveEnv
      case func of
        Closure vars body encEnv -> do
          (res,enclaveEnv') <-
            evalEnclave body ((zip vars vals) ++ encEnv)
          pure (res, env1)
        _ -> pure (Err ENotClosure, env1)
    _ -> pure (Err ENotSecClos, env1)
evalClient (EnclaveApp e1 e2) env = do
  (v1, env1) <- evalClient e1 env
  (v2, env2) <- evalClient e2 env1
  case v1 of
   SecureClosure f args ->
    case v2 of
     ArgList vals ->
          pure (SecureClosure f (args ++ vals), env2)
     v -> pure (SecureClosure f (args ++ [v]), env2)
   v -> pure (ArgList [v,v2], env2)
```

Fig. 6. Operational Semantics of the Client

primitive `gateway`. In this regard, we have the following claim capturing a (progress-insensitive [7]) non-interference property. Intuitively, this property states that (side-effectful) programs do not leak information except via their termination behavior.

Proposition 4.1 (Non-interference). *Given a HasTEE program* `p` *of type* `Enclave a -> App Done`, *where* `p` *does not use primitive* `gateway`, *and two enclave*

$$m \longmapsto 3$$
$$f \longmapsto Closure\ [\text{"}x\text{"}]\ (x + m)\ [m \mapsto 3]$$
$$EncVar_0 \longmapsto Closure\ [\text{"}x\text{"}]\ (x + m)\ [m \mapsto 3]$$
$$y \longmapsto Dummy$$

(a) Enclave Environment

$$m \longmapsto 3$$
$$f \longmapsto Closure\ [\text{"}x\text{"}]\ (x + m)\ [m \mapsto 3]$$
$$EncVar_0 \longmapsto Dummy$$
$$y \longmapsto SecureClosure\ \text{"}EncVar_0\text{"}\ []$$

(b) Client Environment

Fig. 7. (a) gets loaded during the first evaluator pass, and the Client Environment remains empty. In the second pass, (b) gets loaded while having access to the memory (a), as can be seen in Fig 6.

computations `e1` *and* `e2` *of type* `Enclave a`*, then* `p e1` *and* `p e2` *perform the same side-effects in the host client.*

This proposition states that in `p` the public effects on the host client cannot depend on the content of the argument of type `Enclave a`. The veracity of this proposition can be proven from the semantics of `gateway`, which is the only primitive calling `evalEnclave` from `evalClient` Fig. 6. If non-interference does not hold in the context of developing HasTEE, it could indicate the presence of vulnerabilities in the system. For example, it could suggest that data is being leaked into the host environment due to an error in the partitioning process of the HasTEE compiler. Alternatively, it might imply that certain side effects within the enclave are unintentionally revealing data back to the host, contrary to our expectations. Non-interference serves as an important initial security condition in the development of HasTEE as it helps identify and address numerous vulnerabilities that may arise during the process.

When it comes to reason about programs with the primitive `gateway`, we need to reason about IFC *declassification* primitives (or intended ways to release sensitive information) [51] and how to avoid exploiting it to reveal more information than intended. [25] utilizes *delimited release* as the security policy. This security policy extends information-flow control beyond non-interference. It allows for explicit points of controlled information release, called *escape hatches*, where sensitive information can be sent to public channels. This policy stipulates that information may only be released through escape hatches and no additional information is leaked. The function `gateway` is our escape hatch. If we apply delimited release to HasTEE, then host clients can always learn what the function `gateway e` returns, given that expression `e` evaluates to the same value in the initial states `st1, st2 :: Enclave a` given to a program `p`—a condition to avoid misusing escape hatches to reveal more information than intended. Our case studies (Section 6) satisfy delimited release.

Automatically enforcing delimited release or robust declassification [42] imposes severe restrictions in either the information being declassified or how declassification primitives are used. Hence, we leave enforcing such security policies as future work. Instead, our DSL explicitly requires marking the points where information is sent back to the client (i.e., `gateway`), making it clear where to audit and control information leakages.

5 Implementation of HasTEE

5.1 Trusted GHC Runtime

One of the crucial challenges in implementing the HasTEE library is enabling Haskell programs to run within an Intel SGX enclave. All Haskell programs compiled via the Glasgow Haskell Compiler (GHC), rely on the GHC runtime [36] for crucial operations such as memory allocation and management, concurrency, I/O management, etc. As such, it is essential to port the GHC runtime in order to run Haskell programs on the enclave.

The GHC runtime is a complex software that is heavily optimized for specific platforms, such as Linux and Windows, to maximize its performance. For instance, on Linux, the runtime relies on a wide variety of specialised low-level routines from a C standard library, such as `glibc` [23] or `musl` [19], to provide essential facilities like memory allocation, concurrency, and more. The challenge lies in porting the runtime due to the limited and constrained implementation of the C standard library in the SGX SDK, called `tlibc` [30]. Specifically, `tlibc` does not support some of the essential APIs required by the GHC runtime, including `mmap`, `madvise`, `munmap`, `select`, `poll`, a number of `pthread` APIs, operations related to timers, file reading, writing, and access control, and 100+ other functions.

Given the magnitude of engineering effort required to port the GHC runtime, we fall back on a library OS called Gramine [57]. Gramine internally intercepts all `libc` system calls within an application binary and maps them to a Platform Abstraction Layer (PAL) that utilizes a smaller ABI. In Gramine's case, this amounts to only 40 system calls that are executed through dynamic loading and runtime linking of a larger `libc` library, such as `glibc` or `musl`. Importantly, to protect the confidentiality and integrity of the enclave environment, Gramine uses a concept known as *shielded execution*, pioneered by the Haven system [11], where a library is only loaded if its hash values are checked against a measurement taken at the time of initialisation. Shielded execution further protects applications against *Iago attacks* [13] in Gramine.

However, there are additional difficulties in loading the GHC runtime on the SGX enclave via Gramine. Owing to Gramine's diminished system ABI, it has a dummy or incomplete implementation for several important system calls that the runtime requires. For instance, the absence of the `select`, `pselect`, and `poll` functions, which are used in the GHC IO manager, required us to modify the GHC I/O manager to manually manage the polling behavior through experimental heuristics. Similarly, the critical `mmap` operation in GHC uses specific flags (`MAP_ANONYMOUS`) that require modification. In addition, other calls, such as `madvise`, `getrusage`, and timer-based system calls, also require patching. We hope to quantify these modifications' performance in the future.

After the GHC runtime is loaded onto an enclave, communication between the untrusted and trusted parts of the application effectively occurs between two disjoint address spaces. Communication between them can happen over any binary interface, emulating a remote procedure call. Our early prototype stage implementation uses

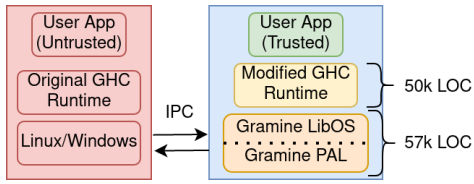


Fig. 8. The high-level overview of communication between the untrusted and trusted parts of the app

an inter-process communication (IPC) call to copy the serialised data (Fig 8). A production implementation should communicate via the C ABI using Haskell's Foreign Function Interface (FFI), as this would be significantly faster than an IPC.

The Gramine approach requires 57,000 additional lines of code in the Trusted Computing Base (TCB) [57]. However, this is still an improvement over traditional operating systems, like Linux, with a TCB size of 27.8 million lines of code [33].

5.2 HasTEE Library

The API of the HasTEE library was already shown (Figure 4) and discussed in Section 4.2. The principal data types, `Enclave` and `Client`, have been implemented as wrappers around the IO monad, as shown below:

```
1  newtype Enclave a = Enclave (IO a) -- data constructor not exported
2  type Client = IO
```

A key distinction is that the `Enclave` data type does not instantiate the `MonadIO` typeclass, as a result of which arbitrary IO actions cannot be lifted inside the `Enclave` monad. This is to ensure that the enclave does not perform leaky IO operations such as writing to the terminal. These are effectful operations that may leak information, which may not be rolled back. However, the Enclave monad does instantiate a `RestrictedIO` typeclass that will be discussed in the following section. The conditional-compilation-based partitioning technique is achieved by having dummy implementations of certain data types in one of the modules, while the concrete implementation of those types is defined in the second module. We give an example of this using two different data types from the API.

```
1  -- Enclave.hs
2  data Secure a = SecureDummy
3
4  type Ref a = IORef a
```

```
1  -- Client.hs
2  data Secure a =
3   Secure CallID [ByteString]
4  type Ref a = RefDummy
```

A notable aspect of the API is the `Securable` typeclass, which constrains the `inEnclave` function and enables it to label functions with any number of arguments as residents of the enclave memory. The `Securable` typeclass accomplishes this using a well-known typeclass trick in Haskell, used to represent statically-typed

variadic functions such as `printf` [8]. In general, `Securable` characterises functions of the form $a_1 \rightarrow ... \rightarrow a_n \rightarrow Enclave\ b$.

The operational semantics presented in Section 4.3 should provide an intuition for the core implementation techniques used in the library. The complete HasTEE project has been open-sourced[1]. More implementation details can be found in the Haste.App paper [18].

5.3 Information Flow Control for Enclaves

The HasTEE library, being written in Haskell, allows using language-based Information Flow Control (IFC) techniques available in Haskell [50]. IFC approaches in Haskell aim to protect the confidentiality of data by encapsulating computations within a `Sec` monad. Typically, the monad employs a lattice of *labels* [15] to model various security levels and then enforces policies on how data can flow between the levels. For a two-label lattice, where confidential data is marked with `H` and public data with `L`, a security policy known as *non-interference* is to prevent information flow from the secret to public channels [24]. In other words, $L \sqsubseteq L$, $H \sqsubseteq H$, $L \sqsubseteq H$, but $H \not\sqsubseteq L$, where $\sqsubseteq$ indicates the *flows to* relation.

A similar scenario arises in HasTEE, where the `Enclave` monad can be compared to a security-critical `Sec H` monad that attempts to prevent information leakage to a public `Sec L` channel represented by the `Client` monad. Enforcing the non-interference policy in this scenario would imply that no data can flow out of the `Enclave` monad to the `Client`, which would make the enclave very restrictive for any real-world use cases. As such, the IFC literature relaxes the non-interference policy by the means of *declassification* [51], to allow controlled data leak from `H` to `L`.

In the HasTEE API, the function

```
gateway :: (Binary a) => Secure (Enclave a) -> (Client a)
```

is an *escape hatch* [28] that allows the enclave to leak *any* data to the client. We prioritise the usability of the API and trust that the enclave programmer will make the `gateway` call when they are certain they want to intentionally leak information to a public channel. However, there is a hidden line of defence in the `gateway` function. If the programmer wishes to send any user-defined data type to the untrusted client, they need to provide an instance of the `Binary` typeclass. Writing this typeclass instance for some confidential data type, such as a private key, equips the confidential data with the capacity to leave the enclave boundary, which should be done in a highly controlled manner.

Besides the `gateway` function, the `Enclave` monad has occasional requirements to interact with general I/O facilities like file reading/writing or random number generation. For such operations, the `Enclave` monad would need a `MonadIO` instance in Haskell to perform any I/O operations. However, as discussed in the previous section, we do not provide the lenient `MonadIO` instance to the Enclave monad but

[1] https://github.com/Abhiroop/EnclaveIFC

instead, use a `RestrictedIO` typeclass to limit the types of I/O operations that an `Enclave` monad can do.

`RestrictedIO`, shown in Listing 4, is a collection of typeclasses that constrains the variants of I/O operations possible inside an `Enclave` monad. For instance, if a programmer, through the usage of a malicious library, mistakenly attempts to leak confidential data through a network call, the typeclass would not allow this.

```
type RestrictedIO m = (EntropyIO m, UnsafeFileIO m) -- other typeclasses
   not shown

class EntropyIO (m :: Type -> Type) where
    type Entropy m:: Type
    genEntropyPool :: m (Entropy m)

class UnsafeFileIO (m :: Type -> Type) where
    untrustedReadFile  :: FilePath -> m (Untrusted String)
```

Listing 4. The Restricted IO typeclass

This approach is invasive in that it restricts how a library (malicious or otherwise) that interacts with a HasTEE program conducts I/O operations. For instance, we had to modify the HsPaillier library [59] that used the `genEntropy` function for random number generation. Initially, the library could use the Haskell IO monad freely, but to interact with a package written in HasTEE, it had to be modified to use the more restricted type class constraint (`EntropyIO`) for its *effectful* operations. This limits potential malicious behaviour within the library. Notably, our changes involve only five lines of code that instantiate the type class and generalize the type signature of effectful operations.

Another aspect of IFC captured in our system is the notion of *endorsement* [28], which is the dual of declassification. Endorsement is concerned with the integrity, i.e., trustworthiness, of information. In HasTEE, we utilize endorsement to ensure that the integrity of secrets is not compromised by data being introduced into the enclave.

HasTEE allows file reading operations inside the `Enclave` monad, which can potentially corrupt the enclave's data integrity. To control this, HasTEE provides two forms of file reading operation - (1) untrusted file read and (2) trusted encrypted file reads. For (1), data read from untrusted files require manual endorsement via the `trust :: Untrusted a -> a` operator (where `Untrusted a` is a wrapper over the data read). This provides an additional check before untrusted data interacts with the trusted domain.

For point (2), HasTEE relies on an Intel SGX feature known as *sealing*. Every Intel SGX chip is embedded with a unique 128 bit key known as the Root Seal Key (RSK). The SGX enclave can use this RSK to encrypt trusted data that it wishes to persist

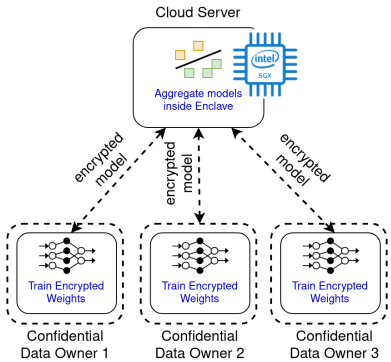


Fig. 9. A Federated Learning setup where the data owners are protecting their data and the ML model owner is protecting their model. The training with encrypted weights can be done using homomorphic encryption.

on untrusted media. This process is known as sealing; HasTEE provides a simple interface to seal as well as unseal the trusted data being persisted, as shown below:

```
1  data SecurePath = SecurePath String
2
3  securefile :: FilePath -> SecurePath
4  securefile fp = "/secure_location/" <> fp -- path hidden
5
6  readSecure  :: SecurePath -> Enclave String
7  writeSecure :: SecurePath -> String -> Enclave ()
```

In the above, the `writeSecure` operation corresponds to *ciphertext declassification* [6], while `readSecure` to an operation that applies automatic endorsement if the file can be decrypted successfully by the enclave RSK. If an attacker were to locate the secure location, the worst possible outcome would be the deletion of the file. However, the contents of the file cannot be read or modified outside the enclave, so the attacker would not be able to access the sensitive information stored within.

6 Case Studies

6.1 Federated Learning

Federated Learning is an emerging *privacy-preserving* machine learning [1] approach that allows multiple parties to train a model without sharing the raw training data. A typical federated learning setup involves multiple decentralized edge devices holding local datasets, training a model locally and then aggregating the trained model on a cloud server. Fig. 9 shows the desired setup.

The setup in Fig. 9 above is facilitated by a combination of TEEs and homomorphic encryption. Homomorphic Encryption (HE) [20] is a form of encryption that enables

```
data SrvSt =
  SrvSt { publicKey :: PubKey, privateKey :: PrvKey
        , updWts :: Vector Double, numClients :: Int
        , wtsDict :: Map Epoch [Vector CipherText]) }
```

Listing 5. The Federated Learning server state

direct computation on encrypted data, revealing the computation result only to the decryption key owner. We emulate the very same setup for our case study where we have two mutually distrusting parties -

• **Confidential data owner.** This party wants to protect its confidential data. A real-life example would be a hospital containing confidential patient data.

• **ML model owner.** This party wants to protect their intellectual property (the ML model) from the data owners as well as the cloud provider. They encrypt their model when sending it to the data owners and allows them to use only homomorphic encryption for operating on the model.

The above setup only requires the cloud server supporting Intel SGX technology so that even mobile devices can participate in training as a worker role. We can very conveniently model this entire setup as three clients and a server with an enclave in HasTEE. For illustration purposes, we will use GHC's threads to represent the three clients instead of three separate data owner machines.

Listing 5 models the server's state. Note that the weights are kept in plaintext form. The enclave state holds both its public and private keys. However, only the public key should be allowed to move to the client. We enforce this by not providing an instance of the `Binary` typeclass for the private key. If untrusted modules try to attack such enforcement by adding new instances to `Binary`, or even providing overlapping ones to override the behaviour of overloaded methods, then Safe Haskell [56] will indicate GHC to not compile the code. Haskell is unique in terms of having an extension like Safe Haskell. Safe Haskell enforces sandboxing for trusted code by banning extensions that introduce *loopholes* and compromise type-safety or module abstraction (often for the sake of performance). As discussed in Section 5.3, the lack of a `Binary` instance for the `privateKey` will prevent the enclave programmer from accidentally leaking the security-critical private key.

Listing 6 shows the API exposed to the client machine. Instead of the complex `SGX_ECALL` machinery, our API is expressed in idiomatic Haskell. Calling any function `f` from the record `api` with an argument `arg` in this API is expressed simply as `gateway ((f api) <@> arg)`.

```
type Accuracy = Double
type Loss     = Double
data API = API
  { aggregateModel :: Secure (Epoch -> Vector CipherText
```

```
5                     -> Enclave (Maybe (Vector CipherText)))
6     , validateModel  :: Secure (Enclave (Accuracy, Loss))
7     , getPublicKey   :: Secure (Enclave PubKey)
8     , reEncrypt      :: Secure (CipherText -> Enclave CipherText)
9     }
```

Listing 6. The Federated Learning client API

Listing 7 shows the main ML model training loop. A few functions have been elided for brevity, but the key portions of the client-server interaction in HasTEE should be visible. The `Config` type holds the state containing encrypted weights sent from the cloud server, the learning rate, the current epoch number and the public key. After each epoch it updates the weights to the new aggregated value (Line 12). The value `x'` is the data set that the data owners are protecting and `y` is the result of the learning algorithm. The `adjustModelWithLearningRate` function (body elided, line 6) takes the computed gradient (line 5) and tries to converge on the desired result.

On line 7 the server is communicated to aggregate models spread across different clients, with the server returning the encrypted updated weights `wt'`. We use a wrapper over `gateway`, called `retryOnEnclave` (body elided), to allow the server to move in lock step with all the clients. Then in line 8, the server is communicated again to collect the accuracy and loss in the ongoing epoch number, which gets displayed in line 9. Finally, the loop continues in line 10.

```
1  handleSingleEpoch :: API -> CurrentEpochNum -> MaxEpochNum  -> Matrix
     Double -> Vector Int -> Config -> Client Config
2  handleSingleEpoch api n m x' y cfg'
3    | n == m    = return cfg'
4    | otherwise = do
5      grad   <- computeGradient api cfg' x' y
6      cfgNew <- adjustModelWithLearningRate api
7                                  (cfg' { iterN = n }) grad
8      wt' <- retryOnEnclave $ (aggregateModel api) <@> n
9                                       <@> (weights cfgNew)
10     (acc, loss) <- gateway (validateModel api)
11     printClient $ " Iteration no: " <> show n <>
12       " Accuracy: "  <> show acc <> " Loss : " <> show loss
13     handleSingleEpoch api (n+1) m x' y
14                                (cfgNew { weights = wt' })
```

Listing 7. The key model training loop

Listing 7 above features a complex control flow with at least two interactions visible in the loop itself. Internally, `computeGradient` and `adjustModelWithLearning` both co-mmunicate with the enclave, calling the `reEncrypt` function to remove noise

from the homomorphic encryption operation. HasTEE can represent a fairly complex, asynchronous control flow as simple Haskell function calls.

In terms of Information Flow Control, there are two important aspects in this case study. Firstly, the `RestrictedIO` typeclass constrains potentially malicious libraries from misbehaving. For example, consider the library HsPaillier [59], which implements the Paillier Cryptosystem [44] for partial homomorphic encryption. All effectful operations from this library, such as `genKey :: Int -> IO (PubKey, PrvKey)`, need to be rewritten for them to be usable within the `Enclave` monad. The following snippet shows our typeclass instantiation and a sample type signature change needed inside the library.

```
instance (IO ~ m) => EntropyIO m where
    type Entropy m = EntropyPool
    genEntropyPool = createEntropyPool

-- genKey :: Int -> IO (PubKey, PrvKey) -- original type
genKey :: (Monad m, EntropyIO m) => Int -> m (PubKey, PrvKey)
```

The second aspect of IFC arises when the client machine queries the server for accuracy and loss by asking it to validate the model. On the server side, the enclave has to read a file with test data. This test data resides outside of the enclave and is potentially an attack vector. In order to not inadvertently trust such an exposed source, the enclave uses the `untrustedReadFile` function from the `RestrictedIO` typeclass (Listing 4). The file is read as an `Untrusted String` and requires explicit programmer *endorsement* via the `trust` operator for the compiler to typecheck the program.

Overall the case study constitutes only 500 lines of code. It naturally fits into the client-server programming model, and the usage of Haskell provides type safety and enables IFC-based security.

6.2 Encrypted Password Wallet

For this case study, we use HasTEE to implement a secure password wallet that stores authentication tokens in encrypted form on the disk. An authentication token can be retrieved from the wallet if the right master password is supplied. The definition of a password wallet used by the case study follows in Listing 8.

```
-- | A single entry of authentication tokens
data Item = Item { title    :: String
                 , username :: String
                 , password :: Password
                 }
  deriving (Show, Read)

-- | The secure wallet
data Wallet = Wallet { items          :: [Item]
```

```
-- | Secure file path to the wallet
  wallet :: SecureFilePath
  wallet = secureFile "wallet.seal"

  -- | Try to load the secure wallet into the enclave
  loadWallet :: Enclave (Maybe Wallet)
  loadWallet = do b <- doesSecureFileExist wallet
                  if b then do contents <- readSecure wallet
                               return $ readMaybe contents
                       else return Nothing

  -- | Store the wallet on disk in encrypted form
  saveWallet :: Wallet -> Enclave ReturnCode
  saveWallet w = writeSecure wallet (show w) >> return Success
```

Listing 9. The code that storing and loading the encrypted wallet. Programmer do not need to manage encryption keys.

```
                      , size           :: Int
                      , masterPassword :: Password
                      }
    deriving (Show, Read)
```

Listing 8. The definition of a password wallet as a regular Haskell data type.

The Show and Read instances are used to convert a wallet to and from a string. This allows us to write the wallet to disk, and by writing to a secure file path we ensure that the stored wallet is encrypted, as described in section 5.3. By omitting a Binary instance we ensure that the wallet is not inadvertently leaked to the client directly. The code in Listing 9 implements the functions that store and load the wallet. We emphasize that the code does not need to explicitly reason about encryption and decryption, except for defining the secure file path.

Our password wallet has the following features - (1) adding an authentication token, (2) retrieving a password, (3) deleting a token and (4) changing the master password. It is designed as a command-line utility where the commands are handled by an untrusted client and the passwords are protected by the enclave. The complete implementation is roughly 200 lines of Haskell code.

The hardware-enforced security provided by our secure wallet makes it a natural fit for designing wallets that are protected by biometrics. A similar approach is used on modern iPhones, where passwords are stored in a secure enclave [3] to ensure confidentiality, and the user's biometric data is used as the master password. In our case, the usage of a high-level language like Haskell enables expressing this relatively complex application concisely.

6.3 Data Clean Room with Differential Privacy

A *Data Clean Room* (DCR) [9] is a technology that provides aggregated and anonymised user information to protect user privacy while providing advertisers and analytic firms with non-personally identifiable information to target a specific demographic with advertising campaigns and analytics-based services.

DCRs compute and release aggregated results based on the user data. To prevent attackers from compromising individual user information from aggregate data (via statistical techniques), DCRs employ *differential privacy* [16]. Differential privacy adds calibrated noise to the aggregate data making it computationally hard for attackers to compromise individual data. The noise calibration can be adjusted for increased privacy (more noise) or increased accuracy (less noise).

Our third case study implements a *differentially-private* DCR within an SGX enclave using HasTEE. The DCR consists of record, `User`, containing fields such as `name, occupation, salary, gender, age, etc.` The `User` record is encrypted before being provisioned to the DCR, after which we use the *Laplace Mechanism* [17] when performing counting queries to add noise to the result. The mechanism introduces noise by sampling a Laplace distribution. The complete Laplace mechanism code can be found in Appendix B.

The DCR does not provide a `Binary` instance for the `User` record to ensure that it is not transferred to the enclave via plain serialisation. Instead, we expose functions that encrypt and decrypt users.

The Laplace Mechanism for adding noise requires a source of randomness. Here, we use Haskell's `System.Random` package, which internally reads from `/dev/urandom`. For production environments, a more cryptographically secure source of randomness is required. We extend the `RestrictedIO` (Section 5.3) interface to allow this operation as long as the programmer *endorses* the file read.

Consider a sample query to test how many individuals in a data set have a salary within a specific range.

```
1  salaryWithin :: Integer -> Integer -> User -> Bool
2  salaryWithin l h u = l <= salary u && salary u <= h
```

The HasTEE code for the DCR executing this query is shown in Listing 10. Lines 3 to 8 specify the API of the data clean room. The DCR's API supports (1) initialisation, (2) fetching of the public key, (3) provisioning user data to the enclave, and (4) executing the salary query. Line 8 is used to generate some arbitrary users (for testing), after which the client code takes over. The client initializes the DCR and fetches its public key. After this, the users are encrypted and sent to the DCR. On line 15 the salary query is executed in the DCR, and then the result is printed.

Generating arbitrary users to test the setup is done purely for illustration purposes. In a more faithful implementation, the client would relay the public key to data owners that would then send already encrypted user records to the client, which provisions them to the DCR. Owing to HasTEE's client-server programming model

```
app :: App Done
app = do
    ref    <- liftNewRef undefined
    initSt <- inEnclave $ initEnclave ref 0.1
    pkey   <- inEnclave $ getPublicKey ref
    prov'  <- inEnclave $ provisionUserEnclave ref
    lm     <- inEnclave $ laplaceMechanism ref $
                                salaryWithin 10000 50000
    dataset <- liftIO $ sequence $ replicate 500
                                    (generate arbitrary)
    runClient $ do
        gateway $ initSt      -- initialize enclave state
        key <- gateway pkey   -- enclaves public key
        mapM_ (\u -> do ct <- encryptUser u key
                        gateway $ prov' <@> ct) dataset
                              -- provision users
        result <- gateway lm  -- run the salary query
        liftIO $ putStrLn $ concat ["res: ", show result]
```

Listing 10. The client running the salaryWithin query over the data set in the data clean room.

and the use of a high-level language like Haskell, the implementation becomes very compact with roughly 200 LOC.

7 Evaluations

7.1 Discussion

In contrast to development on the Intel C/C++ SGX SDK, HasTEE's high-level programming model entirely abstracts away the complexity of dealing with the low-level `edl` files in the SGX SDK. The remote procedure calls that happen between the untrusted client and trusted enclave are typechecked in Haskell, unlike the SGX SDK. The benefits of high-level of abstraction can also be seen in the password wallet example, where functions `readSecure` and `writeSecure` (Listing 9) relieves developers from the burden of key management. Furthermore, HasTEE warns a program against accidental data leaks and can enforce stronger compile-time guarantees than Intel C/C++ SGX SDK. For instance, in all three case studies, the lack of the `Binary` type-class constraint would, by construction, prevent accidental leakage of the secret data from the enclave. All three case studies restrict the I/O operations possible in the `Enclave` monad by the type-class `RestrictedIO`. Notably, in the federated learning example, we adapted the homomorphic encryption library to limit the effects possible in the `IO` monad.

7.2 Performance Evaluations

Our evaluations were conducted on an Azure Standard DC1s v2 (1 vcpu, 4 GiB memory) SGX machine. We use the password wallet case study as the canonical example to present performance evaluations across different parameters. We chose this example as it covers all the major aspects of the HasTEE API, such as protecting the confidentiality of data across the memory as well as the disk.

Memory Overhead. We show the memory consumption of our modified GHC runtime, sampled across 100 runs, where a sample was collected every second.

Memory	RSS	Virtual Size	Disk Swap
At rest	19,132 KB	287,920 KB	0 KB
Peak	20,796 KB	290,032KB	0 KB

Although the memory usage of HasTEE will certainly vary across applications, these numbers provide a general estimate of the trusted GHC runtime's space usage. The Resident Set Size (RSS) indicates that the application fits within 20 MB at peak usage. RSS is an overestimate of memory usage as it includes the memory occupied by shared libraries as well. As a result, we can be certain that our application fits within the Enclave Page Cache limit (Section 2) of 93 MB.

Latency. We measure the latency and throughput for an instance of password retrieval, that includes - (i) an enclave crossing to call the trusted runtime, (ii) standard GHC execution time, (iii) encrypted file load, (iv) file decryption, (v) file read, and (v) a second enclave crossing to return the result.

Our measurements show that using the Linux `send/recv` call for enclave crossing results in a *60 milliseconds overall latency*. As our current socket-based communication is a proof-of-concept, it incurs a substantial overhead compared to native SGX enclave crossings. As a baseline, we measured the latency of an encrypted password retrieval in unmodified GHC (file encrypted with gpg [22]). The baseline number comes out to be 0.6 milliseconds showing an overall 100x slowdown. Note that an average SGX ECALL operation incurs at least a 10x slowdown via the native SDK [21]. We believe switching to native ECALLs has the potential to improve our latencies.

Throughput. In terms of throughput, HasTEE is able to handle on average 11 requests for password retrieval per second. Again, this number has the potential for further improvemnt by switching to native SGX ECALLs.

We currently present coarse-grained measurements of the various metrics but envision future work, where more fine-grained parameters, such as the correlation between the GC pauses across the two runtimes can be presented. Section 7.3 provides a qualitative comparison of HasTEE against GoTEE and J_E.

7.3 Comparing HasTEE to GoTEE and J_E

Table 1 presents a comparison between HasTEE and its two closest counterparts - GoTEE [21] and J_E [43]. While both GoTEE and J_E had to modify the respective compilers, HasTEE required no modifications to the compiler. The specific runtime used by J_E is not mentioned in the paper [43]; however, it suggests that no modification of

Table 1. Comparison of HasTEE, GoTee, and J_E. We specify the core components involved in the Trusted Computing Base in all three frameworks.

Framework	HasTEE	GoTEE	J_E
IFC support	Standard declassification	None	Robust declassification
Partitioning scheme	Type-based	Process-based	Annotation-based
Modified compiler	No	Yes	Yes
Modified runtime system	Yes	Yes	No
Trusted Components	GHC compiler, GHC runtime, Gramine	GoTEE compiler, GoTEE runtime	Java parser and partitioner, Jif compiler, JVM, SGX-LKL[45]
Programming model	Client-server	Synchronous Message-Passing	Using the object-framework provided by Java

the runtime was required, as it was run on a large virtualized host - SGX-LKL [45]. In contrast, the runtimes for HasTEE and GoTEE required modification. GoTEE required significant modifications to the Golang runtime system to enable communication between the trusted and untrusted memory.

Both GoTEE and J_E use sophisticated static analysis passes and program transformations to partition a program into its two components. In contrast, HasTEE's conditional compilation-based approach is much simpler, which is beneficial when it comes to security. Having less and simpler code makes it easier to verify for correctness. Notably, the purity of Haskell enables the user to inspect the type of a function and infer that it is naturally confined whenever a function is side-effect free. Inferring the confinement of a pure function is much more challenging in imperative languages like Java and Go.

8 Related Work

Managed programming languages. While there are imperative and object-oriented languages with TEE support (e.g., Go-based [21], and Java-based[43, 58], HasTEE is (to the best of our knowledge) the first functional language running on a TEE environment. The Rust-SGX [62] project provides foreign-function interface (FFI) bindings to the C/C++ Intel SGX SDK. Different from HasTEE, Rust-SGX does not aim to introduce any programming model or IFC to protect against leakage of sensitive data. Instead, Rust-SGX's main goal is application-level memory safety when programming with the low-level SGX SDK. HasTEE provides memory safety by the

virtue of running Haskell, a memory-safe language, on the enclaves. TrustJS [27] takes a similar FFI-based approach as Rust-SGX for programming enclaves with JavaScript. An important project in this space is the WebAssembly (WASM) initiative [48]. There have been WASM projects, both academic, such as Twine [37], as well as commercial, such as Enarx [46], aimed at allowing WASM runtimes to operate within SGX enclaves. Our initial approach was to use the experimental Haskell WASM backend [60] to run Haskell on SGX enclaves. However, the aforementioned runtimes are not supported by GHC and lack several key features required for loading Haskell onto an enclave.

Automatic partitioning. HasTEE has a seamless program partitioning and familiar client-server-based programming model for enclaves. HasTEE's lightweight partitioning approach is inspired by the Haste.App library [18]—a library to write web applications in Haskell and deploy parts of it into JavaScript on the web browser. The most well-known automatic partitioning tool for C programs on an SGX enclave is Glamdring [35]. The general idea of partitioning a single program has been studied as multitier programming [64]. Among the existing approaches to multitier programming, HasTEE provides a lightweight alternative that does not require any compiler modification or elaborate dataflow analysis to partition the program.

Application development. There have been attempts to virtualize entire platforms within the enclave memory to reduce the burden of dealing with the two-project programming model of Intel SGX. Haven [11] virtualizes the entire Windows operating system as well as an entire SQL server application running on top of it. SCONE [5] virtualizes a Docker container instance within an SGX enclave. The libraryOS Gramine [57], which is used in this work, is an example of lightweight virtualization.

AMD's TEE system, AMD SEV [2], is a native virtualization based approach. While it eases development, virtualization can result in drastically increasing the size of the TCB. We chose to apply a libraryOS approach for HasTEE in order to have a TCB of 57k lines of code (Gramine). As a future work, we can move away from Gramine and make the GHC runtime a standalone library inside the SGX enclave.

Information Flow Control. HasTEE draws inspiration from the work on static IFC security libraries (e.g., [12, 49, 50]). Such approaches relies on the purity of Haskell to detect and stop malicious behaviour. HasTEE can support IFC in a dynamic manner [55] by adapting the interpretation of the `Enclave` type to be a runtime monitor rather than just a wrapper for IO, where `gateway` performs security checks when sending/receiving information—an interesting direction for future work.

The work on IMP_E [25] studies IFC non-interference for passive and active attackers on the host client. [26] present a calculus for reasoning about IFC for applications distributed across several enclaves. J_E [43] studies how compromised host clients can abuse gateway (declassification) primitives. Their security property and enforcement is based on the notion of robust declassification [42, 63]. Intuitively, this policy ensures that low-integrity data cannot influence the declassification of secret data. HasTEE enforces a simpler IFC policy for passive attackers—along the lines of [25]—and defer automatic analyses of the use of `gateway` for future work. Another interesting line

of work is Moat [54], which formally verifies enclave programs running on Intel SGX such that data confidentiality is respected. It uses IFC to enforce the policies and automated theorem proving to verify the policy enforcement mechanism.

9 Conclusion & Future Work

This paper presents HasTEE, a domain-specific language to write TEE programs while ensuring confidentiality of data by construction. Unlike previous work, HasTEE provides its partitioning of source code and IFC as a library! For HasTEE to work, we ported GHC's runtime to run within SGX enclaves by using the libraryOS Gramine. We demonstrate through three diverse case studies how HasTEE's IFC mechanism can help prevent accidental data leakage while producing concise code. We hope HasTEE opens future research avenues at the intersection of TEEs and functional languages.

There are several directions for future work. The IFC scheme we consider operates on two security levels - sensitive (`Enclave`) and public (`Client`) data. A natural extension is to enable multiple security levels [41, 55] to represent the concerns of different principals contributing data to enclaves. TEEs also provide a verifiable launch of the execution environment for the sensitive code and data, enabling a remote entity to ensure that it was set up correctly. *Remote attestation* [32] allows an SGX enclave to prove its identity to a challenger using the private key embedded in the enclave. HasTEE does not capture attestation at the programming language level since it a property of the system components layout. Nevertheless, remote attestation can facilitate secure communication between multiple enclaves, e.g., a distributed-enclave setting; so it would be interesting to incorporate language-level support for remote attestation. Finally, GHC runtime is extensively optimized for performance. Obtaining a more compact and portable runtime, e.g., by using a restricted set of `libc` operations, could result in a considerably smaller TCB. A more portable runtime would facilitate HasTEE experiments on other TEE infrastructures such as ARM TrustZone and RISC-V PMP [47].

10 Acknowledgements

This work was funded by the Swedish Foundation for Strategic Research(SSF) under the project Octopi (Ref. RIT17-0023) and VR. Special thanks to Mary Sheeran, Marco Vassena, Bo Joel Svensson, and Claudio Agustin Mista for their initial reviews, and Facundo Dominguez and Michael Sperber for guiding the published draft.

References

[1] Mohammad Al-Rubaie and J. Morris Chang. 2019. Privacy-Preserving Machine Learning: Threats and Solutions. *IEEE Secur. Priv.* 17, 2 (2019), 49–58. https://doi.org/10.1109/MSEC.2018.2888775

[2] AMD. 2018. *AMD.* https://developer.amd.com/sev/

[3] Apple. 2021. *Apple Platform Security.* https://support.apple.com/guide/security/face-id-and-touch-id-security-sec067eb0c9e/web

[4] ARM. 2004. *ARM TrustZone.* https://www.arm.com/technologies/trustzone-for-cortex-a

[5] Sergei Arnautov, Bohdan Trach, Franz Gregor, Thomas Knauth, André Martin, Christian Priebe, Joshua Lind, Divya Muthukumaran, Dan O'Keeffe, Mark Stillwell, David Goltzsche, David M. Eyers, Rüdiger Kapitza, Peter R. Pietzuch, and Christof Fetzer. 2016. SCONE: Secure Linux Containers with Intel SGX. In *12th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2016, Savannah, GA, USA, November 2-4, 2016*, Kimberly Keeton and Timothy Roscoe (Eds.). USENIX Association, 689–703. https://www.usenix.org/conference/osdi16/technical-sessions/presentation/arnautov

[6] Aslan Askarov, Daniel Hedin, and Andrei Sabelfeld. 2008. Cryptographically-Masked Flows. *Theor. Comput. Sci.* 402, 2-3 (2008), 82–101. https://doi.org/10.1016/j.tcs.2008.04.028

[7] Aslan Askarov, Sebastian Hunt, Andrei Sabelfeld, and David Sands. 2008. Termination-Insensitive Noninterference LLeaks More Than Just a Bit. In *Computer Security - ESORICS 2008, 13th European Symposium on Research in Computer Security, Málaga, Spain, October 6-8, 2008. Proceedings (Lecture Notes in Computer Science, Vol. 5283)*, Sushil Jajodia and Javier López (Eds.). Springer, 333–348. https://doi.org/10.1007/978-3-540-88313-5_22

[8] Lennart Augustsson and Bart Massey. 2013. *Text.Printf - printf in Haskell.* https://hackage.haskell.org/package/base-4.17.0.0/docs/Text-Printf.html#t:PrintfType

[9] Amazon AWS. 2022. *AWS Clean Room.* https://aws.amazon.com/clean-rooms/

[10] Zijian Bao, Qinghao Wang, Wenbo Shi, Lei Wang, Hong Lei, and Bangdao Chen. 2020. When Blockchain Meets SGX: An Overview, Challenges, and Open Issues. *IEEE Access* 8 (2020), 170404–170420. https://doi.org/10.1109/ACCESS.2020.3024254

[11] Andrew Baumann, Marcus Peinado, and Galen C. Hunt. 2015. Shielding Applications from an Untrusted Cloud with Haven. *ACM Trans. Comput. Syst.* 33, 3 (2015), 8:1–8:26. https://doi.org/10.1145/2799647

[12] Pablo Buiras, Dimitrios Vytiniotis, and Alejandro Russo. 2015. HLIO: Mixing Static and Dynamic Typing for Information-Flow Control in Haskell. In *Proceedings of the 20th ACM SIGPLAN International Conference on Functional Programming, ICFP 2015, Vancouver, BC, Canada, September 1-3, 2015*, Kathleen Fisher and John H. Reppy (Eds.). ACM, 289–301. https://doi.org/10.1145/2784731.2784758

[13] Stephen Checkoway and Hovav Shacham. 2013. Iago Attacks: Why the System Call API is a Bad Untrusted RPC Interface. In *Architectural Support for Programming Languages and Operating Systems, ASPLOS 2013, Houston, TX, USA, March 16-20, 2013*, Vivek Sarkar and Rastislav Bodik (Eds.). ACM, 253–264. https://doi.org/10.1145/2451116.2451145

[14] Decentriq. 2022. *Decentriq.* https://blog-decentriq-com.cdn.ampproject.org/c/s/blog.decentriq.com/swiss-cheese-to-cheddar-securing-amd-sev-snp-early-boot-2

[15] Dorothy E. Denning. 1976. A Lattice Model of Secure Information Flow. *Commun. ACM* 19, 5 (1976), 236–243. https://doi.org/10.1145/360051.360056

[16] Cynthia Dwork. 2006. Differential Privacy. In *Automata, Languages and Programming, 33rd International Colloquium, ICALP 2006, Venice, Italy, July 10-14, 2006, Proceedings, Part II (Lecture Notes in Computer Science, Vol. 4052)*, Michele Bugliesi, Bart Preneel, Vladimiro Sassone, and Ingo Wegener (Eds.). Springer, 1–12. https://doi.org/10.1007/11787006_1

[17] Cynthia Dwork and Aaron Roth. 2014. The Algorithmic Foundations of Differential Privacy. *Found. Trends Theor. Comput. Sci.* 9, 3-4 (2014), 211–407. https://doi.org/10.1561/0400000042

[18] Anton Ekblad and Koen Claessen. 2014. A Seamless, Client-Centric Programming Model for Type Safe Web Applications. In *Proceedings of the 2014 ACM SIGPLAN symposium on Haskell, Gothenburg, Sweden, September 4-5, 2014*, Wouter Swierstra (Ed.). ACM, 79–89. https://doi.org/10.1145/2633357.2633367

[19] Richard Felker. 2005. *musl C Library.* https://musl.libc.org/

[20] Craig Gentry. 2009. Fully Homomorphic Encryption Using Ideal Lattices. In *Proceedings of the 41st Annual ACM Symposium on Theory of Computing, STOC 2009, Bethesda, MD, USA, May 31 - June 2, 2009*, Michael Mitzenmacher (Ed.). ACM, 169–178. https://doi.org/10.1145/1536414.1536440

[21] Adrien Ghosn, James R. Larus, and Edouard Bugnion. 2019. Secured Routines: Language-based Construction of Trusted Execution Environments. In *2019 USENIX Annual Technical Conference,*

USENIX ATC 2019, Renton, WA, USA, July 10-12, 2019, Dahlia Malkhi and Dan Tsafrir (Eds.). USENIX Association, 571–586. https://www.usenix.org/conference/atc19/presentation/ghosn

[22] GNU. 1999. *GNU Privacy Guard.* https://gnupg.org/

[23] GNUDevs. 1991. *glibc - The GNU C Library.* https://www.gnu.org/software/libc/

[24] Joseph A. Goguen and José Meseguer. 1982. Security Policies and Security Models. In *1982 IEEE Symposium on Security and Privacy, Oakland, CA, USA, April 26-28, 1982*. IEEE Computer Society, 11–20. https://doi.org/10.1109/SP.1982.10014

[25] Anitha Gollamudi and Stephen Chong. 2016. Automatic Enforcement of Expressive Security Policies Using Enclaves. In *Proceedings of the 2016 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications, OOPSLA 2016, part of SPLASH 2016, Amsterdam, The Netherlands, October 30 - November 4, 2016*, Eelco Visser and Yannis Smaragdakis (Eds.). ACM, 494–513. https://doi.org/10.1145/2983990.2984002

[26] Anitha Gollamudi, Stephen Chong, and Owen Arden. 2019. Information Flow Control for Distributed Trusted Execution Environments. In *32nd IEEE Computer Security Foundations Symposium, CSF 2019, Hoboken, NJ, USA, June 25-28, 2019*. IEEE, 304–318. https://doi.org/10.1109/CSF.2019.00028

[27] David Goltzsche, Colin Wulf, Divya Muthukumaran, Konrad Rieck, Peter R. Pietzuch, and Rüdiger Kapitza. 2017. TrustJS: Trusted Client-Side Execution of JavaScript. In *Proceedings of the 10th European Workshop on Systems Security, EUROSEC 2017, Belgrade, Serbia, April 23, 2017*, Cristiano Giuffrida and Angelos Stavrou (Eds.). ACM, 7:1–7:6. https://doi.org/10.1145/3065913.3065917

[28] Daniel Hedin and Andrei Sabelfeld. 2012. A Perspective on Information-Flow Control. In *Software Safety and Security - Tools for Analysis and Verification*, Tobias Nipkow, Orna Grumberg, and Benedikt Hauptmann (Eds.). NATO Science for Peace and Security Series - D: Information and Communication Security, Vol. 33. IOS Press, 319–347. https://doi.org/10.3233/978-1-61499-028-4-319

[29] Intel. 2015. *Intel Software Guard Extension.* https://www.intel.com/content/www/us/en/developer/tools/software-guard-extensions/overview.html

[30] Intel. 2018. *tlibc - An Alternative to glibc.* https://github.com/intel/linux-sgx/tree/master/common/inc/tlibc

[31] SL Peyton Jones, Cordy Hall, Kevin Hammond, Will Partain, and Philip Wadler. 1993. The Glasgow Haskell compiler: a technical overview. In *Proc. UK Joint Framework for Information Technology (JFIT) Technical Conference*. 249–257.

[32] Thomas Knauth, Michael Steiner, Somnath Chakrabarti, Li Lei, Cedric Xing, and Mona Vij. 2018. Integrating Remote Attestation with Transport Layer Security. *CoRR* abs/1801.05863 (2018). https://doi.org/10.48550/arXiv.1801.05863 arXiv:1801.05863

[33] Michael Larabel. 2020. *Linux Stats.* https://www.phoronix.com/news/Linux-Git-Stats-EOY2019

[34] Christian M. Lesjak, Daniel M. Hein, and Johannes Winter. 2015. Hardware-Security Technologies for Industrial IoT: TrustZone and Security Controller. In *IECON 2015 - 41st Annual Conference of the IEEE Industrial Electronics Society, Yokohama, Japan, November 9-12, 2015*. IEEE, 2589–2595. https://doi.org/10.1109/IECON.2015.7392493

[35] Joshua Lind, Christian Priebe, Divya Muthukumaran, Dan O'Keeffe, Pierre-Louis Aublin, Florian Kelbert, Tobias Reiher, David Goltzsche, David M. Eyers, Rüdiger Kapitza, Christof Fetzer, and Peter R. Pietzuch. 2017. Glamdring: Automatic Application Partitioning for Intel SGX. In *2017 USENIX Annual Technical Conference, USENIX ATC 2017, Santa Clara, CA, USA, July 12-14, 2017*, Dilma Da Silva and Bryan Ford (Eds.). USENIX Association, 285–298. https://www.usenix.org/conference/atc17/technical-sessions/presentation/lind

[36] Simon Marlow, Simon L. Peyton Jones, and Satnam Singh. 2009. Runtime Support for Multicore Haskell. In *Proceeding of the 14th ACM SIGPLAN international conference on Functional programming, ICFP 2009, Edinburgh, Scotland, UK, August 31 - September 2, 2009*, Graham Hutton and Andrew P. Tolmach (Eds.). ACM, 65–78. https://doi.org/10.1145/1596550.1596563

[37] Jämes Ménétrey, Marcelo Pasin, Pascal Felber, and Valerio Schiavoni. 2021. Twine: An Embedded Trusted Runtime for WebAssembly. In *37th IEEE International Conference on Data Engineering, ICDE 2021, Chania, Greece, April 19-22, 2021*. IEEE, 205–216. https://doi.org/10.1109/ICDE51399.2021.00025

[38] Microsoft. 1994. *glibc - The GNU C Library*. Cruntime(CRT)andC++standardlibrary(STL)

[39] Dominic P. Mulligan, Gustavo Petri, Nick Spinale, Gareth Stockwell, and Hugo J. M. Vincent. 2021. Confidential Computing - A Brave New World. In *2021 International Symposium on Secure and Private Execution Environment Design (SEED), Washington, DC, USA, September 20-21, 2021*. IEEE, 132–138. https://doi.org/10.1109/SEED51797.2021.00025

[40] Antonio Muñoz, Ruben Rios, Rodrigo Román, and Javier López. 2023. A Survey on the (In)Security of Trusted Execution Environments. *Comput. Secur.* 129 (2023), 103180. https://doi.org/10.1016/j.cose.2023.103180

[41] Andrew C. Myers and Barbara Liskov. 2000. Protecting privacy using the decentralized label model. *ACM Trans. Softw. Eng. Methodol.* 9, 4 (2000), 410–442. https://doi.org/10.1145/363516.363526

[42] Andrew C. Myers, Andrei Sabelfeld, and Steve Zdancewic. 2004. Enforcing Robust Declassification. In *17th IEEE Computer Security Foundations Workshop, (CSFW-17 2004), 28-30 June 2004, Pacific Grove, CA, USA*. IEEE Computer Society, 172–186. https://doi.org/10.1109/CSFW.2004.9

[43] Aditya Oak, Amir M. Ahmadian, Musard Balliu, and Guido Salvaneschi. 2021. Language Support for Secure Software Development with Enclaves. In *34th IEEE Computer Security Foundations Symposium, CSF 2021, Dubrovnik, Croatia, June 21-25, 2021*. IEEE, 1–16. https://doi.org/10.1109/CSF51468.2021.00037

[44] Pascal Paillier. 1999. Public-Key Cryptosystems Based on Composite Degree Residuosity Classes. In *Advances in Cryptology - EUROCRYPT '99, International Conference on the Theory and Application of Cryptographic Techniques, Prague, Czech Republic, May 2-6, 1999, Proceeding (Lecture Notes in Computer Science, Vol. 1592)*, Jacques Stern (Ed.). Springer, 223–238. https://doi.org/10.1007/3-540-48910-X_16

[45] Christian Priebe, Divya Muthukumaran, Joshua Lind, Huanzhou Zhu, Shujie Cui, Vasily A. Sartakov, and Peter R. Pietzuch. 2019. SGX-LKL: Securing the Host OS Interface for Trusted Execution. *CoRR* abs/1908.11143 (2019). https://doi.org/10.48550/arXiv.1908.11143 arXiv:1908.11143

[46] Inc. Red Hat. 2019. *Enarx: Confidential Computing with WebAssembly*. https://enarx.dev/

[47] RISC-V. 2017. *RISC-V: Physical Memory Protection*. https://riscv.org/blog/2022/04/xuantie-virtualzone-risc-v-based-security-extensions-xuan-jian-alibaba/

[48] Andreas Rossberg. 2019. *WebAssembly Core Specification*. Technical Report. W3C. https://www.w3.org/TR/wasm-core-1/

[49] Alejandro Russo. 2015. Functional Pearl: Two Can Keep a Secret, If One of Them Uses Haskell. In *Proceedings of the 20th ACM SIGPLAN International Conference on Functional Programming, ICFP 2015, Vancouver, BC, Canada, September 1-3, 2015*, Kathleen Fisher and John H. Reppy (Eds.). ACM, 280–288. https://doi.org/10.1145/2784731.2784756

[50] Alejandro Russo, Koen Claessen, and John Hughes. 2008. A Library for Light-Weight Information-Flow Security in Haskell. In *Proceedings of the 1st ACM SIGPLAN Symposium on Haskell, Haskell 2008, Victoria, BC, Canada, 25 September 2008*, Andy Gill (Ed.). ACM, 13–24. https://doi.org/10.1145/1411286.1411289

[51] Andrei Sabelfeld and David Sands. 2005. Dimensions and Principles of Declassification. In *18th IEEE Computer Security Foundations Workshop, (CSFW-18 2005), 20-22 June 2005, Aix-en-Provence, France*. IEEE Computer Society, 255–269. https://doi.org/10.1109/CSFW.2005.15

[52] Moritz Schneider, Ramya Jayaram Masti, Shweta Shinde, Srdjan Capkun, and Ronald Perez. 2022. SoK: Hardware-Supported Trusted Execution Environments. *CoRR* abs/2205.12742 (2022). https://doi.org/10.48550/arXiv.2205.12742 arXiv:2205.12742

[53] Hovav Shacham. 2007. The Geometry of Innocent Flesh on the Bone: Return-Into-Libc Without Function Calls (on the x86). In *Proceedings of the 2007 ACM Conference on Computer and Communications Security, CCS 2007, Alexandria, Virginia, USA, October 28-31, 2007*, Peng Ning, Sabrina De Capitani di Vimercati, and Paul F. Syverson (Eds.). ACM, 552–561. https://doi.org/10.1145/1315245.1315313

[54] Rohit Sinha, Sriram K. Rajamani, Sanjit A. Seshia, and Kapil Vaswani. 2015. Moat: Verifying Confidentiality of Enclave Programs. In *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security, Denver, CO, USA, October 12-16, 2015*, Indrajit Ray, Ninghui Li, and

Christopher Kruegel (Eds.). ACM, 1169–1184. https://doi.org/10.1145/2810103.2813608

[55] Deian Stefan, Alejandro Russo, John C. Mitchell, and David Mazières. 2011. Flexible Dynamic Information Flow Control in Haskell. In *Proceedings of the 4th ACM SIGPLAN Symposium on Haskell, Haskell 2011, Tokyo, Japan, 22 September 2011*. 95–106. https://doi.org/10.1145/2034675.2034688

[56] David Terei, Simon Marlow, Simon L. Peyton Jones, and David Mazières. 2012. Safe Haskell. In *Proceedings of the 5th ACM SIGPLAN Symposium on Haskell, Haskell 2012, Copenhagen, Denmark, 13 September 2012*, Janis Voigtländer (Ed.). ACM, 137–148. https://doi.org/10.1145/2364506.2364524

[57] Chia-che Tsai, Donald E. Porter, and Mona Vij. 2017. Graphene-SGX: A Practical Library OS for Unmodified Applications on SGX. In *2017 USENIX Annual Technical Conference, USENIX ATC 2017, Santa Clara, CA, USA, July 12-14, 2017*, Dilma Da Silva and Bryan Ford (Eds.). USENIX Association, 645–658. https://www.usenix.org/conference/atc17/technical-sessions/presentation/tsai

[58] Chia-che Tsai, Jeongseok Son, Bhushan Jain, John McAvey, Raluca Ada Popa, and Donald E. Porter. 2020. Civet: An Efficient Java Partitioning Framework for Hardware Enclaves. In *29th USENIX Security Symposium, USENIX Security 2020, August 12-14, 2020*, Srdjan Capkun and Franziska Roesner (Eds.). USENIX Association, 505–522. https://www.usenix.org/conference/usenixsecurity20/presentation/tsai

[59] Li-Ting Tsai and Abhiroop Sarkar. 2016. *HsPaillier - Partial Homomorphic Encryption in Haskell*. https://hackage.haskell.org/package/Paillier

[60] Tweag.io. 2022. *Haskell WASM backend*. https://www.tweag.io/blog/2022-11-22-wasm-backend-merged-in-ghc/

[61] Stephan van Schaik, Alex Seto, Thomas Yurek, Adam Batori, Bader AlBassam, Christina Garman, Daniel Genkin, Andrew Miller, Eyal Ronen, and Yuval Yarom. 2022. SoK: SGX.Fail: How Stuff Get eXposed. https://sgx.fail

[62] Huibo Wang, Pei Wang, Yu Ding, Mingshen Sun, Yiming Jing, Ran Duan, Long Li, Yulong Zhang, Tao Wei, and Zhiqiang Lin. 2019. Towards Memory Safe Enclave Programming with Rust-SGX. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, CCS 2019, London, UK, November 11-15, 2019*, Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz (Eds.). ACM, 2333–2350. https://doi.org/10.1145/3319535.3354241

[63] Lucas Waye, Pablo Buiras, Dan King, Stephen Chong, and Alejandro Russo. 2015. It's My Privilege: Controlling Downgrading in DC-Labels. In *Security and Trust Management - 11th International Workshop, STM 2015, Vienna, Austria, September 21-22, 2015, Proceedings (Lecture Notes in Computer Science, Vol. 9331)*, Sara Foresti (Ed.). Springer, 203–219. https://doi.org/10.1007/978-3-319-24858-5_13

[64] Pascal Weisenburger, Johannes Wirth, and Guido Salvaneschi. 2021. A Survey of Multitier Programming. *ACM Comput. Surv.* 53, 4 (2021), 81:1–81:35. https://doi.org/10.1145/3397495

[65] Dmitry P. Zegzhda, E. S. Usov, V. A. Nikol'skii, and Evgeny Pavlenko. 2017. Use of Intel SGX to Ensure the Confidentiality of Data of Cloud Users. *Autom. Control. Comput. Sci.* 51, 8 (2017), 848–854. https://doi.org/10.3103/S0146411617080284

A Typechecked Operational Semantics of HasTEE in Haskell

```haskell
{-# LANGUAGE FlexibleContexts #-}
module HasTEEOrig where

import Control.Monad.State.Class
import Control.Monad.State.Strict

type Name = String

```

```haskell
data Exp = Lit Int
         | Var Name
         | Fun [Name] Exp
         | App Exp [Exp]
         | Let Name Exp Exp
         | Plus Exp Exp

         -- HasTEE operators
         | InEnclave  Exp
         | Gateway  Exp
         | EnclaveApp Exp Exp -- (<@>)
         deriving (Show)

data Value = IntVal Int
           | Closure [Name] Exp Env
           -- HasTEE values
           | SecureClosure Name [Value]
           | ArgList [Value]
           | Dummy

           -- Error values
           | Err ErrState
           deriving (Show)

data ErrState = ENotClosure
              | EVarNotFound
              | ENotSecClos
              | ENotIntLit

instance Show ErrState where
  show ENotClosure  = "Closure not found"
  show EVarNotFound = "Variable not in environment"
  show ENotSecClos  = "Secure Closure not found"
  show ENotIntLit   = "Not an integer literal"

type Env = [(Name, Value)]

type ClientEnv  = Env
type EnclaveEnv = Env


type VarName = Int

```

```
52
53  data StateVar =
54    StateVar { varName  :: Int
55             , encState :: EnclaveEnv
56             }
57
58  initStateVar :: EnclaveEnv -> StateVar
59  initStateVar = StateVar 0
60
61
62  eval :: Exp -> Value
63  eval e =
64    let newEnclaveEnv = snd $
65                        evalState (evalEnclave e initEnclaveEnv)
66                        (initStateVar initEnclaveEnv)
67    in fst $ evalState (evalClient e initClientEnv) (initStateVar newEnclaveEnv)
68    where
69      initEnclaveEnv = []
70      initClientEnv  = []
71
72  genEncVar :: (MonadState StateVar m) => m String
73  genEncVar = do
74    n <- gets varName
75    modify $ \s -> s {varName = 1 + n}
76    pure ("EncVar" <> show n)
77
78  evalList :: (MonadState StateVar m)
79           => [Exp] -> Env -> [Value] -> m ([Value], Env)
80  evalList []     e vals = pure (reverse vals, e)
81  evalList (e1:es) env xs = do
82    (v, e) <- evalEnclave e1 env
83    evalList es e (v:xs)
84
85
86  evalEnclave :: (MonadState StateVar m)
87              => Exp -> EnclaveEnv -> m (Value, EnclaveEnv)
88  evalEnclave (Lit n) env = pure (IntVal n, env)
89  evalEnclave (Var x) env = pure (lookupVar x env, env)
90  evalEnclave (Fun xs e) env =
91    pure (Closure xs e env, env)
92  evalEnclave (Let name e1 e2) env = do
93    (e1', env') <- evalEnclave e1 env
94    evalEnclave e2 ((name,e1'):env')
```

```
95  evalEnclave (App f args) env = do
96    (v1, env1) <- evalEnclave f env
97    (vals, env2) <- evalList args env1 []
98    case v1 of
99      Closure xs body ev ->
100       evalEnclave body ((zip xs vals) ++ ev)
101     _ -> pure (Err ENotClosure, env2)
102 evalEnclave (Plus e1 e2) env = do
103   (v1, env1) <- evalEnclave e1 env
104   (v2, env2) <- evalEnclave e2 env1
105   case (v1, v2) of
106     (IntVal a1, IntVal a2) -> pure (IntVal (a1 + a2), env2)
107     _ -> pure (Err ENotIntLit, env2)
108
109 evalEnclave (InEnclave e) env = do
110   (val, env') <- evalEnclave e env
111   varname     <- genEncVar
112   let env'' = (varname, val):env'
113   pure (Dummy, env'')
114 -- the following two are the essentially no-ops
115 evalEnclave (Gateway e) env = evalEnclave e env
116 evalEnclave (EnclaveApp e1 e2) env = do
117   (_, env1) <- evalEnclave e1 env
118   (_, env2) <- evalEnclave e2 env1
119   pure (Dummy, env2)
120
121 evalList2 :: (MonadState StateVar m)
122           => [Exp] -> Env -> [Value] -> m ([Value], Env)
123 evalList2 []      e vals = pure (reverse vals, e)
124 evalList2 (e1:es) env xs = do
125   (v, e) <- evalClient e1 env
126   evalList2 es e (v:xs)
127
128
129 evalClient :: (MonadState StateVar m)
130            => Exp -> ClientEnv -> m (Value, ClientEnv)
131 evalClient (Lit n) env = pure (IntVal n, env)
132 evalClient (Var x) env = pure (lookupVar x env, env)
133 evalClient (Fun xs e) env =
134   pure (Closure xs e env, env)
135 evalClient (Let name e1 e2) env = do
136   (e1', env') <- evalClient e1 env
137   evalClient e2 ((name,e1'):env')
```

```
evalClient (App f args) env = do
  (v1, env1) <- evalClient f env
  (v2, env2) <- evalList2 args env1 []
  case v1 of
    Closure xs body ev ->
      evalClient body ((zip xs v2) ++ ev)
    _ -> pure (Err ENotClosure, env2)
evalClient (Plus e1 e2) env = do
  (v1, env1) <- evalClient e1 env
  (v2, env2) <- evalClient e2 env1
  case (v1, v2) of
    (IntVal a1, IntVal a2) -> pure (IntVal (a1 + a2), env2)
    _ -> pure (Err ENotIntLit, env2)


evalClient (InEnclave e) env = do
  (_, env') <- evalClient e env
  varname     <- genEncVar
  let env'' = (varname, Dummy):env'
  pure (SecureClosure varname [], env'')
evalClient (Gateway e) env = do
  (e', env1) <- evalClient e env
  case e' of
    SecureClosure varname vals -> do
      enclaveEnv <- gets encState
      let func = lookupVar varname enclaveEnv
      case func of
        Closure vars body encEnv -> do
          (res,enclaveEnv') <- evalEnclave body ((zip vars vals) ++ encEnv)
          pure (res, env1)
        _ -> pure (Err ENotClosure, env1)
    _ -> pure (Err ENotSecClos, env1)
evalClient (EnclaveApp e1 e2) env = do
  (v1, env1) <- evalClient e1 env
  (v2, env2) <- evalClient e2 env1
  case v1 of
    SecureClosure f args ->
      case v2 of
        ArgList vals -> pure (SecureClosure f (args ++ vals), env2)
        v -> pure (SecureClosure f (args ++ [v]), env2)
    v -> pure (ArgList [v,v2], env2)


```

```
-- gateway (f == SecureClosure f [])
-- gateway (f <@> arg == EA f arg == SecureClosure f [arg])
-- gateway (f <@> arg1 <@> arg2 == EA f (EA arg1 arg2) == SC f [arg1, arg2])




lookupVar :: String -> [(String, Value)] -> Value
lookupVar _ [] = Err EVarNotFound
lookupVar x ((y, v) : env) =
  if x == y then v else lookupVar x env
```

Listing 11. Operational Semantics of HasTEE

B Data Clean Room Code

```
-- This function runs the query over the dataset and
-- returns the true result
countingQuery :: Enclave (Ref CleanRoomSt)
              -> (User -> Bool)
              -> Enclave Int
countingQuery refst q = do
    st <- readRef =<< refst
    return $ length $ filter id $ map q (users st)

-- Sample the laplace distribution
laplaceDistribution :: Enclave (Ref CleanRoomSt)
                    -> Double
                    -> Enclave Double
laplaceDistribution refst b = do
    z <- int2Double <$> getRandom refst (0,1)
    u <- ((/) 1000 . int2Double) <$> getRandom refst (1,1000)
    return $ (2 * z - 1) * (b * log u)

-- The laplace mechanism, assuming the server state is
-- already given, has type
-- (User -> Bool) -> Enclave Double
-- In the example usage in the paper, the salaryWithin
-- query is partially applied such that it is of type
-- User -> Bool
laplaceMechanism :: Enclave (Ref CleanRoomSt)
                 -> (User -> Bool)
                 -> Enclave Double
```

```
laplaceMechanism refst q = do
    st <- readRef =<< refst
    -- perform the true query
    true <- int2Double <$> countingQuery refst q
    -- sample noise from the laplace distribution
    noise <- laplaceDistribution refst (1 / (epsilon st))
    -- augment the true result with the noise
    return $ true + noise
```

Listing 12. Code for the laplace mechanism.

Received 2023-06-01; accepted 2023-07-04

Paper III

Welcome to the Parti(tioning) (Functional Pearl)

Robert Krook, Samuel Hammersberg

Proceedings of the 17th ACM SIGPLAN International Haskell Symposium, 2024

Welcome to the Parti(tioning) (Functional Pearl)

Using Rewrite Rules and Specialisation to Partition Haskell Programs

ROBERT KROOK, Chalmers University of Technology - Gothenburg University, Sweden
SAMUEL HAMMERSBERG, Gothenburg University, Sweden

Writing distributed applications is hard, as the programmer needs to describe the communication protocol between the different endpoints. If this is not done correctly, we can introduce bugs such as deadlocks and data races. Tierless and choreographic programming models aim to make this easier by describing the interactions of every endpoint in a single compilation unit. When such a program is compiled, ideally, a single endpoint is projected and the code for the other endpoints is removed. This leads to smaller binaries with fewer dependencies, and is called *program partitioning*.

In this pearl, we show how we can use rewrite rules and specialisation to get GHC to partition our Haskell programs (almost) for free, if they are written using the Haste App or HasChor framework.

As an example of why partitioning is useful, we show how an example application can be more easily built and deployed after being partitioned.

Additional Key Words and Phrases: Haskell, Program Partitioning, Choreographic Programming, Tierless Programming, Specialisation, Rewrite Rules

1 Introduction

Traditional distributed applications are implemented by writing several, separate programs (endpoints) that together implement a communication protocol using transmission primitives such as `send` and `receive`. Correctly matching send and receives is tricky to get right, and potential bugs may render an application unable to perform its function. Common problems are e.g. deadlocks and race conditions.

To ease the development of such code, primarily communications code, we may use specific programming models that are intended to make writing such apps easier. Popular examples of such programming models are *tierless* programming or *choreographic* programming.

Both tierless and choreographic programming models allow a developer to write the code for multiple endpoints in a single compilation unit. The advantages of this are many, but the foremost one is that the compiler has more information at hand during compilation, allowing it to e.g. type check communication protocols.

In both programming models, code can be tagged with a location, indicating which endpoint should execute a specific computation. The low-level communication primitives `send` and `receive` are replaced with high-level operations such as `communicate`. During execution, it is known which endpoint should be projected, and e.g. a call to `communicate` will be replaced with a call to `send`, `receive`, or `no-op`.

Authors' Contact Information: Robert Krook, Chalmers University of Technology - Gothenburg University, , Sweden, krookr@chalmers.se; Samuel Hammersberg, Gothenburg University, , Sweden, samuel.hammersberg@gmail.com.

While being able to describe multiple endpoints in a single compilation unit already gives us some guarantees from our code, such as the already mentioned freedom from deadlocks and race conditions, actually *partitioning* the code may help us even further. By program partitioning, we refer to the act of compiling a program several times, every time indicating which specific endpoint should be projected, and removing all code intended for other endpoints. A couple of benefits of this are

- When executing code inside a *Trusted Execution Environment* (TEE), it is important to minimize the amount of code that has to go into the TEE, as you have to *trust* this code. The amount of code that goes into the TEE is called the *Trusted Computing Base* (TCB). Typically, the code inside the TEE assumes the role of a server, with the client executing outside of the TEE. By using a tierless or choreographic programming model to write your application, you eliminate certain classes of bugs. However, without actually partitioning the program, there will be a lot of code going into the TEE that is not necessary. By partitioning the program such that only the necessary parts remain inside the TEE, we can reduce the TCB.
- If our program is proprietary and we do not wish to leak information about its function, partitioning may aid us. If an adversary gains access to the executable of a single endpoint, without partitioning, decompilation might yield a significant amount of information. In contrast, if the program was partitioned, decompilation would only yield information about that particular endpoint, as well as its communication protocol with other endpoints. The application in its entirety might still be relatively safe.
- If one endpoint is intended to run on a particular platform, partitioning may make building and deployment easier. Without partitioning, all the code for all endpoints has to go into the binary. Some dependencies may be absent from certain platforms, making deployment harder. If we partition a program, we will only retain those dependencies that we require for an endpoint, and deployment will be easier.
- The size of executables could decrease if we partition programs. By eliminating code that we know we will not need, we will save space on resource-constrained platforms. In the case of the *Glasgow Haskell Compiler* (GHC), where executables are already very large, partitioning might have very little effect on the overall executable size. If we partition a program and compile it with MicroHaskell[1], however, partitioning would have a larger impact on the size of executables. While this is not something we have experimented with, we discuss it in section 10.

Writing a compiler pass that correctly and thoroughly analyses a program to figure out how to partition it is not impossible, as shown by e.g. GoTEE[2] and J_e[5], but not trivial. The compiler pass becomes large and complicated, and errors can be hard

[1]https://github.com/augustss/MicroHs

to detect. In the case of GoTEE, modifications to the source language were required to facilitate the desired features.

In this pearl, we look at two methods partitioning Haskell programs. Both methods we show are type-driven, lightweight, and automatic, and require very little help from the developer. We use the optimisation phase of GHC to the max to get what we want, and one of our key insights is that we can use a combination of rewrite rules and specialisation to partition Haskell programs even in cases where the compiler does not have enough information to make the right choice[2].

2 Simple Partitioning à la Haste App

As a first example of program partitioning in Haskell, we will look at the technique used by Haste App[1] to write both the server and client endpoints of web applications. HasTEE[6] later adopted this technique to write the trusted and untrusted endpoints of applications running inside TEEs.

Haste App is used to specify the actions of a single server and a single client. The computations are organised around three monads, called `App`, `Client`, and `Server`. The `App` monad is used for staging, whereas the `Client` and `Server` monads describe the actions of the client and server, respectively.

To show how to use these monads in action, we will implement a classical example, the bookseller protocol. The application will describe the actions of a client who wishes to purchase a book from a store (the server). The server maintains a state of stock and prices, which we model as regular Haskell data types

```
type Price = Int
type Day = Int
type Store = Map String (Price, Day)
```

The server will keep a value of the `Store` in a mutable reference, to reflect stock changes. To enable the client to query the server for information about its stock, we define the following functions

```
storeLookup :: Server (Ref Store)
            -> String -> Server (Price, Day)
storeLookup secBooks book = do
    booksRef <- secBooks
    books    <- readRef booksRef
    return . fromJust $ Map.lookup book books

getPrice :: Server (Ref Store) -> String -> Server Price
getPrice books book = do
    (price,_ ) <- storeLookup books book
    return price

```

[2]The code is available here: https://github.com/Rewbert/HasChor

```
13 getDay :: Server (Ref Store) -> String -> Server Day
14 getDay books book = do
15     (_, deliveryDate) <- storeLookup books book
16     return deliveryDate
```

With these functions in place, how can the client communicate with the server? Below follows the rest of the bookseller program

```
1  -- liftNewRef :: a -> App (Server (Ref a))
2
3  bookseller :: App Done
4  bookseller = do
5    remoteRef <- liftNewRef defaultStock
6    remoteGetPrice  <- inServer (getPrice remoteRef)
7    remoteGetDay    <- inServer (getDay remoteRef)
8
9    runClient $ do
10     liftIO    $ putStrLn "Title of the book to buy"
11     userInput <- liftIO getLine
12     price     <- gateway (remoteGetPrice <@> userInput)
13
14     if price < clientBudget then do
15       day    <- gateway (remoteGetDay <@> userInput)
16       liftIO $ putStrLn $ "Delivery date: " <> show day
17     else do
18       liftIO $ putStrLn "Price is out of the budget!"
```

Lines 5,6, and 7 happen in the `App` monad, where staging is performed. We first use `liftNewRef` to create the mutable reference on the server, modeling the current stock. We then indicate that the client will want to invoke `getPrice` and `getDay`, by applying `inServer` to them. The result of `inServer` is a handle from which the client can invoke the corresponding function on the server. The client fully applies the handle by using the `<@>`-operator, after which the `gateway` function performs the remote function invocation. Even though the return type of the remote functions is in the `Server` monad, `gateway` brings the value into the `Client` monad. We emphasize that the client can only invoke remote functions by first acquiring a remote handle from `inServer`.

So, now that we've seen how we can write a program using Haste App, how is it compiled and executed? The trick is that there exist two distinct implementations of the Haste App API. One implements the client functionality, and the other the server functionality, and only one such module is included during compilation. We thus compile the same program twice – once for the client and once for the server. As a concrete example of what these two implementations look like, let us look closer at the `inServer` function.

During execution, the client will drive the computation, whereas the server enters a state where it is ready to receive synchronous requests from the client. This is made possible by *both* client and server executing the App code. When the client calls `inServer`, it generates an identifier that will be used to identify the remote function. The argument to `inServer` is actually discarded

```
inServer :: Remotable a => a -> App (Remote a)
inServer _ = App $ do
  (next_id, remotes) <- get
  put (next_id + 1, remotes)
  return $ Remote next_id []
```

When this remote handle is applied to an argument with `<@>`, the second field (a list) is populated with the arguments. `gateway` will take such a remote handle, serialise it, and transmit it to the server.

How does the server know what to do with such a closure-like value? Well, the implementation of `inServer` for the server generates *the same* identifier and then associates it to the given function in a map.

```
inServer :: Remotable a => a -> App (Remote a)
inServer f = App $ do
  (next_id, remotes) <- get
  put (next_id + 1, (next_id, \bs ->
    let Server n = mkSecure f bs in n) : remotes)
  return RemoteDummy
```

When an identifier and list of arguments arrive, the server will consult its map and fetch the associated function, apply it, and then return the result. Since the server will not apply any remote closures to parameters, a dummy remote is returned instead.

```
onEvent :: [(Int, Method)]
        -> ByteString'
        -> Socket
        -> IO ()
onEvent mapping incoming socket = do
  -- (Int, [ByteString])
  let (identifier, args) = decode incoming
      Just f = lookup identifier mapping
  result <- encode <$> f args
  -- the () type cannot be sent over wire
  let res = handleVoidTy result
  sendLazy socket (B.append (msgSize res) res)
  where
    msgSize r = encode $ B.length r
    -- the () type has msg length 0
    handleVoidTy r = if (B.length r == 0)
```

```
                    then encode '\0'
                    else r
```

When the server component is compiled, the client monad is replaced with a dummy monad, and vice versa.

```
-- when compiling the server endpoint
data Client a = Dummy
newtype Server a = Server (IO a)

-- when compiling the client endpoint
newtype Client a = Client (IO a)
data Server a = Dummy
```

This, together with the fact that `inServer` on the client side discards its argument, lets GHC optimise away the code intended for the other endpoint. If we look at the compilation for the server endpoint, every function returning a `Client a` computation will end up as `\_ -> \_ -> ... \_ -> Dummy` before machine code is generated, giving GHC a lot of opportunities to remove dead code.

3 Haste App and Rewrite Rules

Haste App manages to partition Haskell programs by compiling a program twice, replacing the semantics of the API with each compilation. To actually throw away code and partition the program we rely on GHCs optimisations. While it in practice seems that GHC can be trusted to optimise away the unwanted code, it is a bit shaky. The inner workings of the GHC optimiser can change at any time! We would like to take matters into our own hands (almost) to get stronger guarantees that partitioning actually happens.

One of GHCs optimisation phases is applying rewrite rules. The optimiser applies rewrite rules until no more rules can be applied, and if we can use rewrite rules to partition our program, we might be less at the mercy of GHC than before.

Rewrite Rules. Rewrite rules, as implemented in GHC, is a mechanism for library developers to add domain-specific optimisations to their code, often to do short-cut fusion (eliminate intermediate data structures). The canonical example is that of *map fusion*. Consider the code below

```
map f (map g xs)
```

We first map the function `g` over the input list `xs`, and then we map the function `f` over it. Not only do we traverse the list twice, but we also produce an intermediate list after the first traversal. We can convince ourselves that by composing `f` and `g`, we can write the semantically equivalent

```
map (f . g) xs
```

This version does only one pass over the input list, and the intermediate result is gone. While it is simple for us to understand that these two terms are equivalent, it

is difficult for a compiler to automatically figure that out. Using rewrite rules, we can *tell* the compiler that this is an equivalence, and have it apply the optimisation for us!

In GHC, we specify the corresponding rule as

```
{-# RULES "map/map" forall f g xs .
                    map f (map g xs) = map (f . g) xs #-}
```

We are saying that *for any functions f and g, and any input list xs, if you see the term denoted by the left-hand side, you should rewrite it to the right-hand side.* GHC checks that the rewrite rule does not change the type of a term, but does nothing to verify that the rule encodes an equivalence. It trusts you to write correct rules. GHC promises to apply the rule whenever the opportunity presents itself.

Now, one thing to remember is that GHC can only apply the rule if it can recognise the call to `map`. If we are unlucky, another optimisation will have made it impossible for us to apply the rule. If the code for `map` is *inlined* at the call site, our rewrite rule will not be able to fire. In the case of `map` this is no concern since GHC will not inline recursive functions (if it did, it would never stop!), but, as we will see, in the context of Haste App we do have to remember this. We can explicitly tell GHC to not inline a function by a `NOINLINE` pragma.

```
{-# NOINLINE map #-}
```

Another mechanism that can be used to control if and when terms are inlined or rewritten is *phase control*. While it is out of scope for this pearl, we mention it here for the curious reader to look up for themselves.

An important detail of rewrite rules is that *there can be no ambiguity of which rule to apply*. If more than one rule can be applied to a term, GHC is free to choose either of them. This means that we need to be very careful when we define our rules so that only one rule is applicable at a single time. Consider the function below

```
f :: Int -> Bool
f 0 = True
f _ = False
```

We may be tempted to write the two rules

```
{-# RULES
  "f/0"            f 0 = True
  "f/_" forall x . f x = False
#-}
```

and expect the first rule to be applied when we see `f 0`. However, the second rule also matches, and GHC is free to pick either. If our term has very few variants we can emit rules for *all* cases, but this very quickly becomes infeasible.

Haste App using Rewrite Rules. Now, knowing how GHC applies rewrite rules, we can modify our two different Haste App implementations to instead happen via rewrite rules. Circling back to the example of `inServer`, the corresponding rules become

```
{-# NOINLINE inServer #-}
{-# RULES "HASTEERULES inServer/Server" forall f .
      inServer f = App $ do
        (next_id, remotes) <- get
        put (next_id + 1, (next_id, \bs ->
          let Server n = mkSecure f bs in n) : remotes)
        return RemoteDummy #-}
inServer :: Remotable a => a -> App (Remote a)
inServer _ = error "inServer error"
```

for the server, and

```
{-# NOINLINE inServer #-}
{-# RULES "HASTEERULES inServer/Client" forall f .
      inServer f = App $ do
        (next_id, remotes) <- get
        put (next_id + 1, remotes)
        return $ Remote next_id [] #-}
inServer :: Remotable a => a -> App (Remote a)
inServer _ = error "inServer error"
```

for the client. We make the corresponding change for every function in the Haste App API, after which GHC will still be able to partition the program for us. When we compile the client component, in the case of `inServer`, the rewrite rules will have eliminated any call to functions in the `Server` monad, and dead-code elimination can eliminate those functions. This elimination happens because the rewrite rules have `f` appear on the left-hand side, but not on the right-hand side!

GHC Core Plugin for Verification. How can we be sure that partitioning actually happened? The compiler's representation of a program before machine code is emitted is called *core*. We have written a *Core Plugin* which runs after all optimisations have been applied. It traverses the final core and reports an error if it encounters any calls to the API functions of Haste App. If any call remains, the program was not completely partitioned. If partitioning happens as it should, our plugin will report

```
*** Partitioning verification pass running on Main
*** Verification for module Main: done
```

whereas if we remove a rewrite rule and run it again, it will report an error

```
*** Partitioning verification pass running on Main
<no location info>: error:
    Found indications of partitioning not occurring when
```

```
    inspecting module: Main

    The problematic symbol that was encountered: inServer
```

Scalability of Haste App. Haste App gives us a nice, simple framework for describing the interactions of a client and a server. What if we want to describe the interactions of a server and *two* clients? Then we would have to add yet another monad, with a third set of semantics. This does not scale very well. We could instead use a more general framework, such as HasChor[8].

4 HasChor

HasChor is a framework for describing *choreographic* programs. Choreographic and tierless programming are very similar, but there are some subtle differences. Primarily, in a choreographic framework there are no tiers, and every endpoint is considered an equal.

Rather than identifying an endpoint by a particular monad, the HasChor API defines endpoints via proxy values whose type parameters are string literals.

```
client :: Proxy "client"
client = Proxy
server :: Proxy "server"
server = Proxy
```

The rest of the API is fairly short, but very expressive

```
data a @ (l :: LocTy) -- values tagged with a location
type Choreo m a       -- choreographic monad

-- computations tagged with a location
locally :: KnownSymbol l
        => Proxy l -- location performing a computation
        -> ((forall a . a @ l -> a) -> m a)
        -> Choreo m (a @ l) -- result located at l

-- send a value from one location to another
(~>) :: (Show a, Read a, KnownSymbol l, KnownSymbol l')
     => (Proxy l, a @ l) -- value tagged with a location
     -> Proxy l'         -- the receiving location
     -> Choreo m (a @ l')

-- synchronised branching on a value
cond :: (Show a, Read a, KnownSymbol l)
     => (Proxy l, a @ l)
     -> (a -> Choreo m b)
     -> Choreo m b
```

How do we ensure that only the client endpoint unwraps and accesses the value inside of a tagged value? Everything happens in the context of a `Choreo` monad. The API function `locally` denotes computations that are local to a specific endpoint, and the `Choreo` monad supplies an unwrap function as a parameter that can be used to unwrap tagged values. The type system ensures that only computations local to a specific endpoint may unwrap values existing on that endpoint, by ensuring that the polymorphic location variable `l` is the same in the first and second argument. The code below, for instance, indicates that the client is reading a line from standard input. The result of the entire call to `locally` has type `Choreo IO (String @ "client")`.

```
client `locally` \_ -> getLine
```

The located string can be transmitted to the server endpoint by using the ~> function.

```
do strAtClient <- client `locally` \_ -> getLine
   strAtServer <- (client, strAtClient) ~> server
   server `locally` \uw -> putStrLn $ uw strAtServer
```

Here we can also see on line 3 that, after the message has been transmitted to the server on line 2, the server *unwraps* it (inside `locally`) and prints it. If the server tried to do this with `strAtClient`, instead, a type error would have been thrown.

The last API function of HasChor is cond. To motivate the need for cond, consider the code below

```
case x of
  0 -> (client, someval) ~> server
  1 -> do (client, someval)  ~> server
          (client, someval') ~> server
  _ -> ...
```

If both endpoints do not agree on what the value of the variable x is, they might pick different branches. The different branches have different numbers of calls to ~>, so a deadlock would occur if different branches were taken.

If we instead say that we want the endpoints to branch on *one* value, located at a *specific* endpoint, we are describing what cond does. Consider the code below

```
cond (client, xAtClient) $ \x -> case x of
  0 -> (client, someval) ~> server
  1 -> do (client, someval)  ~> server
          (client, someval') ~> server
  _ -> ...
```

cond will broadcast the scrutinee to all other endpoints, where they will use the received value to pick the correct branch. The bookseller example implemented in HasChor can be found in appendix A.

HasChor is not Partitioned. In contrast to Haste App, HasChor programs are not actually partitioned. To understand what we mean by this, let us look at how a HasChor program is executed. We omit certain details that are not important and look at the `Choreo` monad.

```
type LocTm = String

data ChoreoSig m a where
  Local :: (KnownSymbol l)
        => Proxy l
        -> (Unwrap l -> m a)
        -> ChoreoSig m (a @ l)

  Comm :: ( Show a, Read a
          , KnownSymbol l, KnownSymbol l')
       => Proxy l
       -> a @ l
       -> Proxy l'
       -> ChoreoSig m (a @ l')

  Cond :: (Show a, Read a, KnownSymbol l)
       => Proxy l
       -> a @ l
       -> (a -> Choreo m b)
       -> ChoreoSig m b

type Choreo m = Freer (ChoreoSig m)
```

HasChor is implemented using *freer monads*[4]. While the full details of freer monads are out of scope for this pearl, we emphasise that freer monads separate the interface of an effectful monad and the implementation. The code above specifies the interface, and we note that the constructors and their types precisely match those of the HasChor API. In fact, the HasChor API is implemented by constructing values of the `ChoreoSig` data type and wrapping them in the `Freer` monad.

```
locally l m    = toFreer (Local l m)
(~>) (l, a) l' = toFreer (Comm l a l')
cond (l, a) c  = toFreer (Cond l a c)
```

To project a single endpoint, we need to move from a high-level API, like the HasCHor API, to a more low-level API that has the primitives `send` and `receive`. For this, we have the `Network` monad

```
data NetworkSig m a where
  Run   :: m a -> NetworkSig m a
  Send  :: Show a => a -> LocTm -> NetworkSig m ()
```

```
4   Recv  :: Read a => LocTm -> NetworkSig m a
5   BCast :: Show a => a -> NetworkSig m ()
6
7 type Network m = Freer (NetworkSig m)
```

Note that there are no more `KnownSymbol` constraints here. A `NetworkSig` computation no longer has any location ambiguities to resolve. The function that takes a `Choreo` computation, as well as the location that should be projected, and returns a `Network` computation, is called *Endpoint Projection*. The implementation is shown below, with some bits omitted for brevity. We ask the reader to focus on the cases in the `handler` function

```
1  toLocTm :: KnownSymbol l => Proxy l -> LocTm
2  toLocTm Proxy = ...
3
4  epp :: Choreo m a -> LocTm -> Network m a
5  epp c l' = interpFreer handler c
6    where
7      handler :: ChoreoSig m a -> Network m a
8      handler (Local l m)
9        | toLocTm l == l' = wrap <$> run (m unwrap)
10       | otherwise       = return Empty
11     handler (Comm s a r)
12       | toLocTm s == l' = do send (unwrap a) (toLocTm r)
13                              return Empty
14       | toLocTm r == l' = wrap <$> recv (toLocTm s)
15       | otherwise       = return Empty
16     handler (Cond l a c)
17       | toLocTm l == l' = do broadcast (unwrap a)
18                              epp (c (unwrap a)) l'
19       | otherwise       = do x <- recv (toLocTm l)
20                              epp (c x) l'
```

Here is where the magic happens. When a choreographic operation is executed (by `handler`), the given `LocTm` is compared to the endpoints involved in the operation, and a decision is made on whether to do something or not.

A call to `locally` indicates that a computation only happens on a specific endpoint. On line 9 we can see that the given `LocTm` is compared to the location that is supposed to perform the operation. If they are the same, the computation is performed, and otherwise nothing is done. Similarly, when `~>` (`Comm`) is handled, we compare the given `LocTm` against both the sending and receiving location, to figure out if `Comm` should become a `Send`, `Recv`, or a no-op (`Empty`). When `cond` is handled, we either broadcast or receive the value before we apply the continuation to it, based on where the value is located.

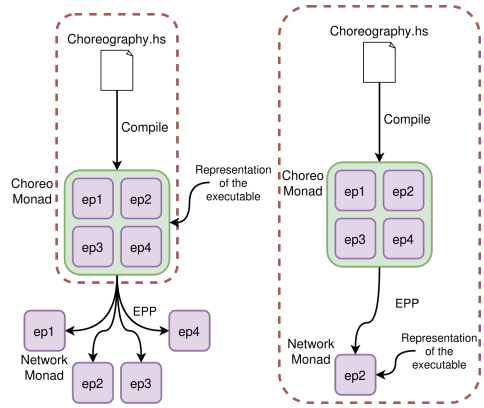


Fig. 1. A representation of how HasChor programs are compiled, and of what we would instead like to happen. The diagrams use a dotted line to show what is done during compilation. The compiled binary contains code for all endpoints, and then endpoint projection happens at runtime, where we can project any one of them. What we would like, instead, is to be able to make endpoint projection happen at compile time, yielding an executable that contains code for just one endpoint.

A key property of the `epp` function is that all these choices are *made at runtime*. It is *not* known at compile time which endpoint will be projected, meaning that all the code for all the endpoints has to be embedded in the compiled binary. Figure 1 illustrates what the problem is, and what we would like to do instead.

5 HasChor and Rewrite Rules

Well, we saw that in the case of Haste App, we could achieve true partitioning via rewrite rules! Can we apply the same trick to partition HasChor programs? There are two things that we need to do to achieve this.

- We need to *fix* the endpoint we wish to project at compile time, rather than delaying that decision to runtime
- We need to figure out what the rewrite rules *should say*

HasChor uses the `Freer` monad to build up the `ChoreoSig` structure, from which we can project a single endpoint in the `Freer` monad, but with the effects described in the
`NetworkSig` structure. What we want our rules to do is to achieve a kind of *short-cut fusion*[3] where we eliminate this intermediate `ChoreoSig` structure, and instead construct the projected `NetworkSig` structure directly. We redefine the `Choreo` monad to be defined as

```
1 type Choreo m = Freer (NetworkSig m)
```

Let us consider the bookseller protocol, with two endpoints. We are going to require two sets of rewrite rules – one specifying the behavior of the seller, and one specifying the behavior of the buyer. Let's consider the semantics of the API functions if we assume that we compile the buyer endpoint. We use the shorthand `Proxy "buyer"` and `Proxy "seller"` to mean `Proxy :: Proxy "buyer"` and `Proxy :: Proxy "seller"`.

```
locally (Proxy "buyer") f  == wrap <$> run (f unwrap)
locally (Proxy "seller") f == return Empty

(Proxy "buyer", v) ~> r == do send (unwrap v) (toLocTm r)
                              return Empty
(s, v) ~> (Proxy "buyer") == wrap <$> recv (toLocTm s)

cond (Proxy "buyer", v) c  == do broadcast (unwrap v)
                                 c (unwrap v)
cond (Proxy "seller", v) c ==
    do x <- recv (toLocTm (Proxy "seller"))
       c x
```

We will encode precisely these equations as rewrite rules. Only the rules for `locally` will be shown here for brevity, while the full set of rules can be found in appendix B.

```
{-# NOINLINE locally #-}
{-# RULES
"CHOREORULES locallybuyer"
    forall (f :: (forall a . a @ "buyer" -> a) -> m a)
           (buyer :: Proxy "buyer") .
           buyer `locally` f = wrap <$> run (f unwrap)

"CHOREORULES locallyseller"
    forall (f :: (forall a . a @ "seller" -> a) -> m a)
           (seller :: Proxy "seller") .
           seller `locally` f = return Empty
#-}
```

The first rule says that if a call to `locally` is seen where the location is known to be `Proxy "buyer"`, it should be rewritten to retain the computation. The second rule says that if the location is known to be `Proxy "seller"`, it should be written to not retain the computation.

When we compile the program, GHC applies the rules until no more rules can be applied. Again, here we rely on our trusted core plugin to verify that partitioning occurred as we expected it to. It seeks occurrences of the symbols `locally`, `~>`, and `cond`.

Now that we have compiled the buyer endpoint, we need to rewrite our rules to encode the seller semantics, and then recompile our program. What a drag!

Template Haskell. Luckily for us, we can use *Template Haskell* [7] (TH) to lessen our burden. With TH we can generate code at compile time. Not only that, but we can also generate compiler pragmas, such as rewrite rules!

We have written TH code that generates all of this boilerplate code for us. A programmer only needs to specify the names of the endpoints, and their location (IP address and port number)

```
$(compileFor 0 [ ("buyer",  ("localhost", 4224))
               , ("seller", ("localhost", 2424)) ]
```

This will generate first the proxies

```
buyer :: Proxy "buyer"
buyer = Proxy
seller :: Proxy "seller"
seller = Proxy
```

and then *all* the rewrite rules that are required to partition the program. The rules will be generated based on the first parameter to `compileFor`, which is an index indicating which of the endpoints in the list should be projected by the rules. To compile and run the bookseller example it is enough to compile the program twice, only changing the `0` to a `1` between the compilations.

6 Location Polymorphic Functions

Great, it seems as if we can also partition HasChor programs using rewrite rules! However, so far we have only tested our method on one example. Let us write another one, using more features from the type system, such as polymorphism.

```
node :: Proxy "node"
node = Proxy

server :: Proxy "server"
server = Proxy

addNumber :: KnownSymbol l => Proxy l -> Int @ l
          -> IORef Int @ "server"
          -> Choreo IO (() @ "server")
addNumber l vAtl ref = do
  vAtServer <- (l, vAtl) ~> server
  server `locally` \uw ->
    modifyIORef (uw ref) (\a -> a + uw vAtServer)

choreography :: Choreo IO (() @ "server")
choreography = do
```

```
  n1v <- node `locally` \_ -> return 50
  s1v <- server `locally` \_ -> return 100
  sref <- server `locally` \_ -> newIORef 0
  addNumber node n1v sref
  addNumber server s1v sref
  server `locally` \uw -> do
    v <- readIORef (uw sref)
    putStrLn $ show v
```

This program has two endpoints – `node` and `server`. The function `addNumber` takes a number at any location, and sends it to the server before it is added to the contents of a mutable reference. The function is *location polymorphic*, as it does not explicitly name which endpoint the supplied number resides at. The endpoint is identified via a `KnownSymbol` constraint, rather than via an explicit proxy type.

In the choreography, both `node` and `server` invoke this function, contributing an integer to the final content of the mutable reference.

When we compile this program we are at the mercy of GHC. Partitioning might work, but it might also not. The problem is that *no rule can be applied*. The rules require certain type information to be available to apply a rule.

Consider the rules for ~> when we compile the `node` endpoint

```
{-# RULES
"CHOREORULES sendfromnode"
    forall v
           r
           (node :: Proxy "node") .
           (node, v) ~> r
                = do (send (unwrap v) (toLocTm r))
                     return Empty

"CHOREORULES sendtonode"
    forall s
           v
           (node :: Proxy "node") .
           (s, v) ~> node = wrap <$> recv (toLocTm s)
#-}
```

For calls to ~> to be rewritten, the compiler must be able to see that the sender or receiver is `node`. Looking at the call to ~> in `addNumber`, the type of the sending location is actually `KnownSymbol l => Proxy l`, which we have no rule for! Should we rewrite ~> to a `send` or a `receive`? If nothing more is done, partitioning will fail. Our plugin verifies that partitioning indeed did not happen as we hoped it would

```
*** Partitioning verification pass running on Main
<no location info>: error:
```

```
    Found indications of partitioning not occurring when
    inspecting module: Main

    The problematic symbol that was encountered: ~>
```

However, if GHC chooses to *inline* the body of `addNumber`, it will in many cases be able to infer more type information and apply the appropriate rule.

We can make GHC very eager to inline functions by marking them as inlineable with a pragma, `{-# INLINE addNumber #-}`. However, GHC may *still* choose to not inline a function, so this is not a general solution. Furthermore, some functions will never be inlined. Consider the definition of `map` below

```
map :: (a -> b) -> [a] -> [b]
map _ []     = []
map f (x:xs) = f x : map f xs
```

if we start inlining `map`, we will never stop! It is a recursive function, and recursive functions are never inlineable. What can we do now? Are we beyond saving? We need a way of introducing the missing information, enabling more rules to fire. Luckily for us, we can introduce the missing information with *specialisation*.

Specialisation in GHC. Specialisation in GHC is a feature that lets a programmer tell GHC that it should create specialised versions of certain functions, to reduce indirections. Consider the code below

```
numOp :: Num a => a -> a -> a
numOp x y = x + y * x

main :: IO ()
main = do
  print $ numOp (5 :: Int) 10
  print $ numOp (5.0 :: Double) 10.0
```

The function is overloaded as it works for any type that is a member of the `Num` type class. We invoke it twice at different types, `Int` and `Double`. This kind of polymorphism is implemented by passing in a dictionary at runtime, from which the overloaded operations `(+)` and `(*)` can be fetched. In pseudo-Haskell

```
data Num a = Num
  { add :: a -> a -> a
  , mul :: a -> a -> a
  }

numInt :: Num Int
numInt = Num primIntAdd primIntMul

numDouble :: Num Double
```

```
numDouble = Num primDoubleAdd primDoubleMul

numOp :: Num a -> a -> a -> a
numOp dict x y = add dict x (mul dict y x)

main :: IO ()
main = do
  print $ numOp numInt (5 :: Int) 10
  print $ numOp numDouble (5.0 :: Double) 10.0
```

While this is a convenient way of implementing overloading, there is a price to pay at runtime. We have to pass in an extra parameter, and the type-specific functions must be fetched from the dictionary before they can be applied.

If we know that we have a lot of invocations of `numOp` at the type `Int`, we can tell GHC to *specialise* `numOp` at that type. We do this by writing

```
{-# SPECIALISE numOp :: Int -> Int -> Int #-}
```

after which GHC will do two things. First, it will create a copy of `numOp` that is specialised to the type `Int`, and the indirections are removed

```
numOpInt :: Int -> Int -> Int
numOpInt x y = primIntAdd x (primIntMul y x)
```

Second, a rewrite rule is emitted that rewrites calls to `numOp` to `numOpInt`, if the argument type is `Int`.

```
{-# RULES "numOp/Int" numOp = numOpInt -}
```

The cost of specialisation is that we duplicate code, so the compiled binaries might be larger.

Generalised specialisation. A recent feature of GHC is a generalisation of the existing specialisation mechanism[3]. What specialisation is really doing is fixing one of the type parameters of a function. The new specialisation allows a programmer to fix arbitrary expressions. How does this work? Consider the code below

```
process :: Bool -> Input -> IO Output
process debug input = do
  res <- computeSomething input
  -- imagine there are many debug calls like these
  if debug then putStrLn res else return ()
  computeSomethingElse res
```

Imagine that we have a function that processes some input. The function receives a boolean argument deciding whether debug information should be printed as the

[3]This feature is implemented but not yet merged into the main branch of GHC. The accepted proposal can be found here: https://github.com/ghc-proposals/ghc-proposals/blob/master/proposals/0493-specialise-expressions.rst

input is processed. If we know that many call sites where `process` is applied to the literal boolean `True`. We can say

```
{-# SPECIALISE forall input . process True input #-}
```

Which, again, will do two things. Firstly, a copy of `process` is created where the parameter `debug` is moved into the right-hand side, and fixed to the value `True`. We can illustrate this by `let`-binding the parameter

```
processTrue :: Input -> IO Output
processTrue input =
  let debug = True in do
    res <- computeSomething input
    -- imagine there are many debug calls like these
    if debug then putStrLn res else return ()
    computeSomethingElse res
```

Secondly, a rewrite rule is emitted to rewrite call sites where the first parameter is `True` to instead call the specialised version.

```
{-# RULES "process/True" forall input .
              process True input = processTrue input #-}
```

Let us consider what happens to a recursive function when we specialise it to concrete terms

```
data Event

renderVerbose :: Event -> String
renderConcise :: Event -> String

render :: Bool -> MVar Event -> IO ()
render b mv = do
  v <- takeMVar mv
  if b
    then putStrLn $ renderVerbose v
    else putStrLn $ renderConcise v
  render b mv
```

`render` will wait until an event is written to an `MVar`. An `MVar` is a mutable reference that can either be empty or hold a value. If a thread tries to read from an empty `MVar`, it is blocked until another thread writes a value to it. Once the thread wakes up an event is ready to be rendered. It is rendered either in great or little detail, after which we recurse. If we would specialise `render` to `True`, the body of the generated function is

```
renderTrue :: MVar Event -> IO ()
renderTrue mv =
  let b = True in do
```

```
  v <- takeMVar mv
  if b
    then putStrLn $ renderVerbose v
    else putStrLn $ renderConcise v
  render b mv
```

The function is no longer recursive! However, as the optimiser tackles the right-hand side of `renderTrue`, the generated rewrite rule is going to be applied to the call to `render`, and the new function will be

```
renderTrue :: MVar Event -> IO ()
renderTrue mv =
  let b = True in do
  v <- takeMVar mv
  if b
    then putStrLn $ renderVerbose v
    else putStrLn $ renderConcise v
  renderTrue mv
```

`renderTrue` is now once again a recursive function, like its ancestor `render`. After some further optimisation, the boolean is folded into the test and the test is removed altogether, yielding

```
renderTrue :: MVar Event -> IO ()
renderTrue mv = do
  v <- takeMVar mv
  putStrLn $ renderVerbose v
  renderTrue mv
```

This new generalised specialisation feature, together with the optimisations that GHC does, is something of a *poor man's partial evaluation*.

The older version of specialisation is used to fix types, whereas the new version is used to fix terms. Now, let us see how we can use specialisation to partition HasChor programs!

7 HasChor and Rewrite Rules + Specialisation

We remind ourselves of the code in section 6. We had no rewrite rule to apply when inspecting the call to `~>` in `addNumber`, as not enough type information was available. However, we see that we apply `addNumber` to `node` and `server`. Our key insight is that we can use specialisation to introduce the missing information, enabling more rewrite rules to be applied. The following text only illustrates the effects of specialising the `node` endpoint, but we must duplicate these specialisation pragmas for all possible endpoints the function is invoked at. While this is a bit tedious, we can generate the pragmas using Template Haskell.

If we use the older version of specialisation, we can *specialise* `addNumber` to the types we know we need it at.

```
{-# SPECIALISE addNumber :: Proxy "node" -> Int @ "node"
                         -> IORef Int @ "server"
                         -> Chore IO (() @ "server") #-}
```

This will generate the specialised function

```
addNumberNode :: Proxy "node" -> Int @ "node"
              -> IORef Int @ "server"
              -> Choreo IO (() @ "server")
addNumberNode l vAtl ref = do
  vAtServer <- (l, vAtl) ~> server
  server `locally` \uw ->
    modifyIORef (uw ref) (\a -> a + uw vAtServer)
```

where there are no longer any type ambiguities. Additionally, we can also choose to specialise the program using the new, generalised specialisation. We achieve the same effect by specialising `addNumber` to the term `node` that is generated by our rewrite rules. This term has the necessary type information for our rules to be applied. The specialised function would be

```
-- notice that `node` is not bound by the forall, and
-- refers to the node that is in scope already
{-# SPECIALISE forall v sref . addNumber node v sref #-}

addNumberNode :: Int @ "node"
              -> IORef Int @ "server"
              -> Choreo IO (() @ "server")
addNumberNode vAtl ref =
  let l = node in do
    vAtServer <- (l, vAtl) ~> server
    server `locally` \uw ->
      modifyIORef (uw ref) (\a -> a + uw vAtServer)
```

In this version the type of `l` is known to be `Proxy "node"`, enabling our rewrite rules to fire. It might be slightly more efficient as well, as we have to pass in one argument less. In both cases, a specialised function and a rewrite rule is emitted, which will replace our call to `addNumber` with a call to `addNumberNode` instead.

It might be a bit bothersome to write all these specialisation pragmas, but they can be generated using TH, enumerating all combinations of endpoints that a function might be applied at[4].

Specialisation, together with the rewrite rules described in section 5, lets us partition HasChor programs, and our plugin verifies that partitioning happened as it should.

[4]The authors have written the code that does everything up until the last stage, actually emitting the pragmas. At the time of writing this pearl, TH does not yet support outputting the new syntax of specialisation pragmas.

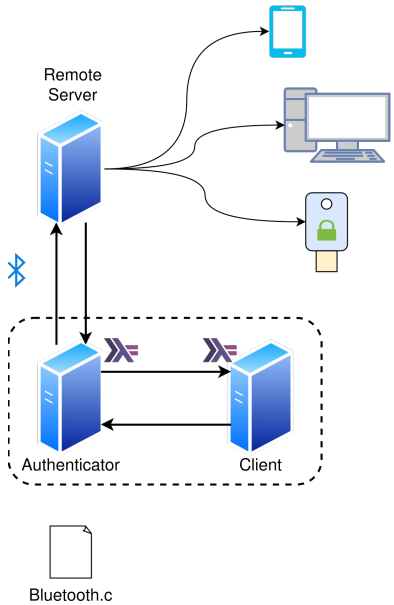


Fig. 2. The 3-factor authentication setup. The authenticator and client are endpoints in a HasChor program, whereas the remote server is a completely disjoint program written in C. The remote server can present an authentication token to the client in different ways – by sending an SMS, by sending an email, interacting with a hardware key, or as in this case, printing to the terminal. The border surrounding the authenticator and client is intended to emphasise that these two endpoints are written together in a choreographic Haskell program. They can still execute on different physical machines. The code for `bluetooth.c` can be found in the appendix, as well as the complete C code for the remote server.

8 Example - Third-Party Authentication

As an example of a scenario where partitioning makes building and deploying an application easier, we will implement a choreographic program that uses Bluetooth. The application as a whole implements a 3-factor authentication service, as depicted in figure 2.

The application is intended to authenticate a user against a remote server. The result of the interaction is an access token, granting access to some resource that is not shown here. The client invokes the authenticator, which will establish a bluetooth connection to a remote server. When a connection is established the remote server will present the user with a secret token somehow, e.g. via email or SMS. In this case, the token is printed to a terminal. The client sends the token to the authenticator, which relays it to the remote server. The remote server checks that the token is the same, after which it returns an access token.

The authenticator is going to need to make some foreign calls to C code[5] to establish a Bluetooth connection to the remote server. We will also declare the proxies that denote two endpoints – the authenticator and a client.

```
foreign import ccall "allocate_socket_"
    allocateSocket :: IO Int
foreign import ccall "connect_"
    connect :: Int -> IO ()
foreign import ccall "authenticate_"
    authenticate :: Int -> Word32 -> IO Word64
foreign import ccall "close_"
    close :: Int -> IO ()

authenticator :: Proxy "authenticator"
authenticator = Proxy

client :: Proxy "client"
client = Proxy
```

The choreography may include many endpoints which all might want to receive an access token, so we will make the authentication function location polymorphic

```
getAccessToken :: KnownSymbol l
               => Proxy l -> Choreo IO (Maybe Word64 @ l)
```

The authenticator will begin by allocating a socket and connecting to it. The `connect` FFI call knows the Bluetooth address of the remote server and tries to establish a connection.

```
getAccessToken p = do
    authSocket <- authenticator `locally` \_ -> do
        socket <- allocateSocket
        -- upon connection, the remote server is going to
        -- present the user with a secret.
        connect socket
        return socket
```

After successfully connecting to the remote server, the client will be presented with a token somehow. The client will enter this token, after which it will be sent to the authenticator

```
    secret <- p `locally` \_ -> do
        putStr "enter the secret>"
        hFlush stdout
```

[5]The C code that the Haskell code calls use `dlopen` to access the BlueZ Bluetooth stack, in order to avoid declaring this dependency in the Cabal file. The code for this example can be found here: https://github.com/Rewbert/HasChor/tree/rwr/examples/pearlexample

```
4         getLine
5
6     authSecret <- (p, secret) ~> authenticator
```

The authenticator will call the `authenticate` FFI call with the token, after which the server will reply with either a zero, indicating the token was wrong, or with a 64-bit value, representing the access token. The authenticator will close the socket, severing the connection to the remote server. Finally, the token is sent back to the client, who can now use it to access some guarded resource

```
1     authRes <- authenticator `locally` \uw -> do
2         r <- authenticate
3                (uw authSocket)
4                (read (uw authSecret) :: Word32)
5         close (uw authSocket)
6         if r == 0
7           then return Nothing
8           else return (Just r)
9
10    (authenticator, authRes) ~> p
```

Finally, we need to specialise the whole function to the endpoints we will invoke it at

```
1 {-# SPECIALISE getAccessToken client #-}
2 -- potentially more
```

Now, the C code that is called via FFI functions depends on the *BlueZ* Bluetooth stack for the Linux kernel. If the platform for which we are compiling the application does not have access to the `bluetooth.c` file that we've written, or the BlueZ library, compilation is going to fail in the linking stage. With standard HasChor, where no partitioning occurs, we always need to have these resources at hand when we are compiling the program. This is because we do not know until runtime if we will need it or not (when we choose which endpoint to project). The code in `bluetooth.c` uses the `dlopen` function from the C standard library to use the BlueZ Bluetooth stack at runtime.

With partitioning, however, there is a different story. We point out that all interactions with the FFI functions happen on the authenticator endpoint. When we compile the authenticator, we need to provide the `bluetooth.c` source. In contrast, when we compile the client endpoint, *after partitioning there are no calls to the FFI functions left.* We do not need to provide `bluetooth.c` when we compile our code, because it is not needed. The client machine does not need to unnecessarily install the Bluetooth libraries! Not only does this make building simpler, but we also don't need to rely on thousands of lines of code in vain.

This, of course, goes beyond eliminating simple C files. If a package is used only at a single endpoint, all usages of the package occur in calls to `locally`. Once these are partitioned away, GHC will choose to not link in the code for that package during

compilation. This has the potential to severely reduce the trusted computing base of an endpoint.

9 When does it Not Work

Firstly, when we want to partition a program to make sure that certain code is not linked in, we need to be very careful with what we export. If a module lacks an export list, everything in that module is going to be exported. In our Bluetooth example, we export only `main`, and that means that we are not exporting the FFI functions. If we did not have this explicit export list, all the FFI functions would be exported as well, and the code would be required during linking.

It might at first glance seem a bit arbitrary which form of specialisation we use to expose more type information for HasChor, and this is for the most part true. When we experimented with the examples that are shipped with HasChor, we did find a difference, however. Consider the following type

```
sort ::
  KnownSymbol a => Proxy a ->
  KnownSymbol b => Proxy b ->
  KnownSymbol c => Proxy c ->
  ([Int] @ a) ->
  Choreo IO ([Int] @ a)
```

It is a bit non-idiomatic to not collect all constraints in the beginning of the type, but legal nonetheless. If we naively try to specialise this as such

```
{-# SPECIALISE sort :: Proxy "primary"
                    -> Proxy "worker1"
                    -> Proxy "worker2"
                    -> ([Int] @ "primary")
                    -> Choreo IO ([Int] @ "primary") #-}
```

we get a type error, whereas if we naively specialise it to the concrete terms, everything goes well

```
{-# SPECIALISE forall . sort primary worker1 worker2 #-}
```

We can of course also just refactor the original type to specify all the constraints in the beginning. This might be the most ideal, as it is more idiomatic.

Finally, GHCs separate compilation may complicate things. If a function and its specialisations are defined in one module, but used in another, all variants of the function must be exported. Dead code elimination will in this case not be able to remove the original function, and it will remain even after linking. Care must be taken with when and where some functions are defined.

10 Conclusions & Future Work

In conclusion, perhaps unsurprisingly so, we can use rewrite rules to partition Haskell programs. Traditional short-cut fusion aims to eliminate intermediate data structures,

and that is precisely what we are trying to do in the case of HasChor. We eliminate the intermediate `ChoreoSig` structure that represents the whole program and use rewrite rules to immediately project a `NetworkSig` structure, modeling just one endpoint, and our plugin helps us verify that partitioning occurred.

We have shown by example that in some situations, partitioning a program can remove dependencies, yielding a program that is easier to build and deploy.

One effect of partitioning programs is that we are removing code, yielding smaller binaries. The binaries produced by GHC are already very large, so this effect is sometimes hardly noticeable. However, the new MicroHaskell compiler has a runtime small enough that it can execute Haskell code on microcontrollers. In this setting, we must make our binaries as small as possible, as memory can be a scarce resource. Today, MicroHaskell does not have enough functionality to partition programs the way that we describe it in this pearl, but the authors are discussing possible ways of making it happen with the MicroHaskell developers.

Acknowledgments

The authors would like to thank Simon Peyton Jones for taking the time to discuss some details of GHC with us. Further thanks go to Koen Claessen and John Hughes for helpful discussions and feedback.

A HasChor Bookseller

Below follow the implementation of the bookseller protocol using HasChor, as taken from the HasChor repository of examples[6]

```
buyer :: Proxy "buyer"
buyer = Proxy

seller :: Proxy "seller"
seller = Proxy

budget :: Int
budget = 100

priceOf :: String -> Int
priceOf "Types and Programming Languages" = 80
priceOf "Homotopy Type Theory"            = 120

deliveryDateOf :: String -> Day
deliveryDateOf "Types and Programming Languages" =
  fromGregorian 2022 12 19
deliveryDateOf "Homotopy Type Theory" =
  fromGregorian 2023 01 01
```

[6]https://github.com/gshen42/HasChor/blob/main/examples/bookseller-1-simple/Main.hs

```
19
20 bookseller :: Choreo IO (Maybe Day @ "buyer")
21 bookseller = do
22   title <- buyer `locally` \_ -> do
23       putStrLn "Enter the title of the book to buy"
24       getLine
25   title' <- (buyer, title) ~> seller
26
27   price <- seller `locally` \un ->
28     return $ priceOf (un title')
29   price' <- (seller, price) ~> buyer
30
31   decision <- buyer `locally` \un ->
32     return $ (un price') < budget
33
34   cond (buyer, decision) \case
35     True  -> do
36       deliveryDate  <- seller `locally` \un ->
37         return $ deliveryDateOf (un title')
38       deliveryDate' <- (seller, deliveryDate) ~> buyer
39
40       buyer `locally` \un -> do
41         putStrLn $ "The book will be delivered on "
42                       ++ show (un deliveryDate')
43         return $ Just (un deliveryDate')
44
45     False -> do
46       buyer `locally` \_ -> do
47         putStrLn "Too expensive"
48         return Nothing
```

B Bookseller Rewrite Rules

Below follows the rewrite rules that implement the semantics for the buyer endpoint in the bookseller choreography.

```
1 {-# NOINLINE locally #-}
2 {-# NOINLINE (~>) #-}
3 {-# NOINLINE cond #-}
4 {-# RULES
5 "CHOREORULES locallybuyer"
6     forall (f::(forall a. At a "buyer" -> a) -> m a)
7            (buyer::Proxy "buyer") .
8            buyer `locally` f = wrap <$> run (f unwrap)
```

```
9
10 "CHOREORULES locallyseller"
11     forall (f::(forall a. At a "seller" -> a) -> m a)
12            (seller::Proxy "seller") .
13            seller `locally` f = return Empty
14
15 "CHOREORULES sendfrombuyer"
16     forall v
17            r
18            (buyer::Proxy "buyer") .
19            (buyer, v) ~> r
20                 = do (send (unwrap v) (toLocTm r))
21                      return Empty
22
23 "CHOREORULES sendtobuyer"
24     forall s
25            v
26            (buyer::Proxy "buyer") .
27            (s, v) ~> buyer = wrap <$> recv (toLocTm s)
28
29 "CHOREORULES condbuyer"
30     forall v
31            c
32            (buyer::Proxy "buyer") .
33            (buyer, v) `cond` c = do broadcast (unwrap v)
34                                     c (unwrap v)
35
36 "CHOREORULES condseller"
37     forall v
38            c
39            (seller::Proxy "seller") .
40            (seller, v) `cond` c =
41                do x <- recv (toLocTm seller)
42                   c x
43 #-}
```

References

[1] Anton Ekblad and Koen Claessen. 2014. A Seamless, Client-Centric Programming Model for Type Safe Web Applications. In *Proceedings of the 2014 ACM SIGPLAN symposium on Haskell, Gothenburg, Sweden, September 4-5, 2014*, Wouter Swierstra (Ed.). ACM, 79–89. https://doi.org/10.1145/2633357.2633367

[2] Adrien Ghosn, James R. Larus, and Edouard Bugnion. 2019. Secured Routines: Language-based Construction of Trusted Execution Environments. In *2019 USENIX Annual Technical Conference,*

USENIX ATC 2019, Renton, WA, USA, July 10-12, 2019, Dahlia Malkhi and Dan Tsafrir (Eds.). USENIX Association, 571–586. https://www.usenix.org/conference/atc19/presentation/ghosn

[3] Andrew John Gill, John Launchbury, and Simon L. Peyton Jones. 1993. A Short Cut to Deforestation. In *Proceedings of the conference on Functional programming languages and computer architecture, FPCA 1993, Copenhagen, Denmark, June 9-11, 1993*, John Williams (Ed.). ACM, 223–232. https://doi.org/10.1145/165180.165214

[4] Oleg Kiselyov and Hiromi Ishii. 2015. Freer Monads, more Extensible Effects. In *Proceedings of the 8th ACM SIGPLAN Symposium on Haskell, Haskell 2015, Vancouver, BC, Canada, September 3-4, 2015*, Ben Lippmeier (Ed.). ACM, 94–105. https://doi.org/10.1145/2804302.2804319

[5] Aditya Oak, Amir M. Ahmadian, Musard Balliu, and Guido Salvaneschi. 2021. Language Support for Secure Software Development with Enclaves. In *34th IEEE Computer Security Foundations Symposium, CSF 2021, Dubrovnik, Croatia, June 21-25, 2021*. IEEE, 1–16. https://doi.org/10.1109/CSF51468.2021.00037

[6] Abhiroop Sarkar, Robert Krook, Alejandro Russo, and Koen Claessen. 2023. HasTEE: Programming Trusted Execution Environments with Haskell. In *Proceedings of the 16th ACM SIGPLAN International Haskell Symposium, Haskell 2023, Seattle, WA, USA, September 8-9, 2023*, Trevor L. McDonell and Niki Vazou (Eds.). ACM, 72–88. https://doi.org/10.1145/3609026.3609731

[7] Tim Sheard and Simon Peyton Jones. 2002. Template Meta-Programming for Haskell. In *Proceedings of the 2002 ACM SIGPLAN Workshop on Haskell, Haskell 2002, Pittsburgh, Pennsylvania, USA, October 3, 2002*, Manuel M. T. Chakravarty (Ed.). ACM, 1–16. https://doi.org/10.1145/581690.581691

[8] Gan Shen, Shun Kashiwa, and Lindsey Kuper. 2023. HasChor: Functional Choreographic Programming for All (Functional Pearl). *Proc. ACM Program. Lang.* 7, ICFP (2023), 541–565. https://doi.org/10.1145/3607849

Received 2024-06-03; accepted 2024-07-05

Paper IV

CloudMicroHaskell - Direct-Style Distributed Haskell via Runtime Graph Serialisation

Robert Krook, Lennart Augustsson

Proceedings of the 19th ACM SIGPLAN International Haskell Symposium, 2026

CloudMicroHaskell

Direct-Style Distributed Haskell via Runtime Graph Serialisation

ROBERT KROOK, Chalmers University of Technology and University of Gothenburg, Sweden
LENNART AUGUSTSSON, Unaffiliated, Sweden

Cloud Haskell brings Erlang-style distributed programming to Haskell, but its treatment of mobile code exposes a difficult boundary in the source-level API. Remote processes must be expressed as static closures, messages must satisfy serialisation constraints, and participating nodes are assumed to share the relevant code.

This paper explores a different design point. We present CloudMicroHaskell, a Cloud Haskell-style library built on MicroHaskell, whose runtime represents both code and data as a combinator graph. When a process or message crosses a node boundary, CloudMicroHaskell serialises the reachable graph directly. As a result, remote spawning can be written in direct style: process bodies may capture variables from their surrounding scope, and messages may contain ordinary values, including functions, without programmer-written closure conversion.

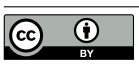
We describe the implementation of the CloudMicroHaskell node runtime, including remote spawn, message delivery, monitors, exit propagation, and library implementations of generic servers and supervisors. We evaluate the system with process/message benchmarks, a distributed work-pool benchmark, a file-synchronisation case study, and a heterogeneous deployment on microcontrollers. The results show that runtime graph serialisation makes the Cloud Haskell programming model substantially more direct, while also making the tradeoff explicit: some guarantees enforced by Cloud Haskell's source-level types become dynamic checks, and programmers must be aware of laziness and runtime-owned resources when moving graphs between nodes.

Additional Key Words and Phrases: Haskell, Distributed Haskell, Actor Model Concurrency, Concurrency, MicroHaskell, MicroHs, Serialisation, Graph Reduction, Combinators, Code Mobility

1 Introduction

Cloud Haskell [4] is a Haskell framework for writing distributed, concurrent programs in the style of Erlang. Erlang's programming model emphasises concurrency, with lightweight processes that share no memory. Instead, inter-process communication is

Authors' Contact Information: Robert Krook, Chalmers University of Technology and University of Gothenburg, Department of Computer Science and Engineering, Gothenburg, Sweden, krookr@chalmers.se; Lennart Augustsson, Unaffiliated, Gothenburg, Sweden, lennart@augustsson.net.



This work is licensed under a Creative Commons Attribution 4.0 International License.
Haskell '26, Indianapolis, IN, USA
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2869-3/2026/08
https://doi.org/10.1145/3830439.3831272

done via message passing. This programming model greatly aids in writing distributed programs that execute on and communicate over a network.

The primary benefit of Cloud Haskell over Erlang is Haskell's strong type system. With it, the API of Cloud Haskell can be described in a very clear and expressive way. It is short and (mostly) elegant. Furthermore, Cloud Haskell extends Erlang's untyped message passing machinery with typed channels, so that communicating processes can exchange messages whose format is known at compile time. Erlang, being dynamically typed, cannot statically enforce a message passing protocol between processes.

These advantages come from making distribution explicit in the API, and that explicitness has practical consequences. The implementation assumes that nodes in the network run a very uniform software environment, and the machinery required to handle closures is necessarily visible to the programmer. We illustrate this with the direct-style program one might like to write

```
sendFunc :: Pid -> Int -> ProcessM ()
sendFunc p x = send p (\y -> x + y + 1)
```

The above example sends a lambda to a recipient process, which captures a free variable (x). Cloud Haskell uses the type system to enforce that values are serialised before transmission, but note that the type of the function, `Int -> Int`, says nothing about the type of the free variable. It is therefore not possible to provide an ordinary serialiser for functions from the function type alone.

To get around this, a mechanism for `Static` values was devised. Explained briefly, a value is considered static if it is defined at the top-level and refers to only bound variables or other variables defined at the top-level. Furthermore, when a static function is transmitted to a remote node, it must be explicitly closure converted, whereby the environment of the function is populated with its (serialised) free variable. The function is responsible for deserialising its environment itself. To implement the above example, the programmer must write

```
sendFunc :: Pid -> Int -> ProcessM ()
sendFunc p x = send p clo
  where
    -- closures can be serialised
    clo :: Closure (Int -> Int)
    clo = MkClosure (static sfun) (encode x)

sfun :: ByteString -> Int -> Int
sfun bs y = let x = decode bs
            in x + y + 1
```

The previously intuitive and brief program has now been rewritten in a style with a significantly higher cognitive cost. Furthermore, a static value is assumed to exist on both machines (the one running the sending process and the one running the receiving process), and a closure only contains a static reference to it (together with

its environment). While it seems like a function is being sent, what is actually being sent is a static reference to it and its environment.

While this is a clever solution to a tricky problem, we note that there are strict requirements placed on participating machines in the network, namely that they must run binaries that contain the same static symbols.

In this paper, we ask what Cloud Haskell might look like if the serialisation boundary were moved out of the source-level API and into the runtime representation of the program. We present CloudMicroHaskell, an exploration of the Cloud Haskell programming model using the MicroHaskell compiler [2] (MicroHs for short). MicroHs is a Haskell compiler with a much smaller and more modest runtime system than GHC, the de facto Haskell compiler. MicroHs emits a *combinator graph* as its object code — a self-modifying byte code that represents both a program's data and code. While the cost of this decision is a significant reduction in performance, there are several benefits, which we hope to illustrate with this paper.

Using MicroHs changes the tradeoffs in several useful ways, namely

- We use a simple compiler primitive to serialise values of (almost) any kind, including functions. The first variant of `sendFunc` in the introduction is directly runnable in CloudMicroHaskell.
- Our implementation does not require every node in the network to agree beforehand on which static values should exist. Instead, nodes need only agree on a compatible CloudMicroHaskell runtime and the combinator format understood by that runtime.
- CloudMicroHaskell can serialise and transmit a message of (almost) any type, without requiring a serialisable constraint. This reduces the boilerplate required to prepare a data type for transmission over a network.

This does not remove the distribution boundary, but relocates it. The price is that this boundary is no longer enforced by Haskell's source-level types. Deserialisation is necessarily a dynamic claim that the received graph has the type expected by the receiver. If that claim is wrong, the program may fail at runtime. In that sense, CloudMicroHaskell trades some of Cloud Haskell's static safety for a much simpler programming model.

The contribution is therefore not the actor API itself, which is deliberately conventional, but the consequences of making runtime graph serialisation the mobility mechanism for that API.

2 Programming in CloudMicroHaskell

In this section we present CloudMicroHaskell. At the surface, CloudMicroHaskell looks much like Cloud Haskell. Programs create processes, identify remote nodes, and communicate by message passing. This is deliberate. The point of CloudMicroHaskell is not to propose a new process calculus, but to explore what happens when the serialisation boundary in Cloud Haskell is moved out of the source-level API and into the MicroHs runtime.

The most important differences therefore appear in the two operations that move computation and data between nodes, `spawn` and `send`. In Cloud Haskell, these operations expose distribution through `Closure` and `Serializable` constraints. In CloudMicroHaskell, these operations accept ordinary process computations and ordinary values, relying instead on MicroHs's graph serialiser (the serialiser and deserialiser are described in section 3).

We therefore focus on these two operations in the main text. The surrounding API follows the familiar Cloud Haskell and Erlang shape[1].

The core difference is visible in the types

```
spawn :: NodeId -> ProcessM () -> ProcessM Pid
send  :: Identifiable process => process -> a -> ProcessM ()
```

In Cloud Haskell, remote spawning requires a `Closure` and message passing requires a
`Serializable` constraint. In CloudMicroHaskell, the programmer supplies the computation or value directly. If the target process or message crosses a node boundary, the runtime will serialise the reachable MicroHs graph, capturing both the code and environment. This makes direct-style examples possible to execute, but it also means that some errors that Cloud Haskell eliminates via the type checker become runtime failures in CloudMicroHaskell.

The serialisation primitive is intentionally very broad, but not universal. Some things are owned by a particular runtime instance, such as `MVar`s and open file handles. These cannot be serialised. The exceptions are the standard handles *stdin*, *stdout*, *stderr*, and the *null pointer*. These handles are recognised by the MicroHs runtime system. They can be serialised and bound to the receiver's version during deserialisation. Similarly, the null pointer is always a zero-valued pointer, and can be serialised and deserialised as such.

The example below illustrates how `spawn` is used to spawn a concurrent process that captures free variables from the outer process:

```
data PingPong = Ping | Pong

f :: ProcessM ()
f = do
  s <- self
  n <- node
  n' <- (head . filter (/= n)) <$> nodes

  child <- spawn n' $ do
    Ping <- expect
    liftIO $ putStrLn "child received ping!"
    send s Pong
```

[1]The full API is described in section A

```
13
14    send child Ping
15    Pong <- expect
16    liftIO $ putStrLn "parent received pong!"
```

The parent process first fetches its own process identifier by calling `self`. It then spawns a child process on a remote node. The child refers to `s`, which is bound in the parent. When `spawn` crosses the node boundary, the child computation is serialised together with this reachable environment. Additionally, we can send the `PingPong` data type without saying how to serialise it.

To make it even clearer that code is serialised and sent to a remote node, we show yet another example:

```
1   f :: ProcessM ()
2   f = do
3     s <- self
4     n <- node
5     n' <- (head . filter (/= n)) <$> nodes
6
7     let interesting x = x `mod` 7 == 0 && x `mod` 11 == 0
8
9     _ <- spawn n' $ do
10      let r = length (filter interesting [1..1000000])
11      r `seq` send s r
12
13    r <- expect :: ProcessM Int
14    liftIO $ print r
```

We define a local function `interesting :: Int -> Bool`, and reference it from the spawned process. `interesting` will be included in the serialised graph that is sent to the remote node, and reconstructed on that node's heap. The call to `seq` is intentional. Since CloudMicroHaskell serialises graphs, not values produced by a `Binary` encoder, sending `r` without first evaluating it could send a thunk back to the original node. The result would still be correct, but the work would happen in the wrong place.

It is not just `spawn` that can be used to send functions to remote processes. `send` uses the same underlying serialiser, and will happily send functions. Consider the example below:

```
1   data Apply = Apply (Int -> Int) Int Pid
2
3   worker :: ProcessM ()
4   worker = do
5     Apply f x replyTo <- expect
6     let y = f x
7     y `seq` send replyTo y
```

```
8
9   example :: ProcessM Int
10  example = do
11    me <- self
12    n  <- node
13    n' <- (head . filter (/= n)) <$> nodes
14
15    workerPid <- spawn n' worker
16
17    let offset = 1
18    send workerPid (Apply (\x -> x + offset) 41 me)
19
20    expect
```

The message sent to `workerPid` contains a function, and that function captures `offset`. In Cloud Haskell, this is exactly the case that requires static closures and explicit environment serialisation. In CloudMicroHaskell, the function is just part of the message graph. The receiver expects an `Apply` message, extracts the function, and applies it.

The rest of the library follows the familiar process-programming structure. Nodes can be connected and queried; processes have identifiers and mailboxes; selective receive inspects messages by runtime type; monitors and exits make process failure observable; and a node-local registry lets long-running processes be found by name. These operations are not the novelty of CloudMicroHaskell, but they are necessary to make the core `spawn` and `send` design usable in larger programs. Their full API is listed in section A.

2.1 Process Behaviours as Libraries

The examples above use `spawn` and `send` directly. To test whether this style scales beyond our toy examples, we implement two familiar Erlang/OTP behaviours as ordinary CloudMicroHaskell libraries, generic servers and supervisors. These behaviours are not built into the runtime, but are implemented using the same process creation, message passing, monitors, and exits available to the programmer.

A generic server captures a common pattern whereby a process responds to requests and then enters a state where it is ready to respond to a subsequent request. Furthermore, the process may maintain a state which may be modified by a request, and the modified state must be used in subsequent requests. The application-specific part is described by a record of callbacks:

```
1   data ServerSpec st callReq castReq reply = ServerSpec
2     { setup      :: ProcessM st
3     , handleCall :: st -> callReq -> ProcessM (st, reply)
4     , handleCast :: st -> castReq -> ProcessM st
5     , tearDown   :: st -> ProcessM ()
```

```
    }

startServer :: NodeId -> ServerSpec st callReq castReq reply
            -> ProcessM (Server callReq castReq reply)
-- run the server in the *current* process
runServer :: ServerSpec st callReq castReq reply -> ProcessM ()

call :: Server callReq castReq reply -> callReq -> ProcessM reply
cast :: Server callReq castReq reply -> castReq -> ProcessM ()
```

`startServer` can start the server on a remote node, even though the server specification contains ordinary Haskell functions. Internally, a server loop is spawned together with the supplied callbacks. The programmer only implements the state transition logic, and the library handles the receive loop, reply routing, and state threading.

For example, a counter server can be described by giving its initial state and two handlers

```
data CallRequest = Get
data CastRequest = Inc | Reset

counterSpec :: ServerSpec Int CallRequest CastRequest Int
counterSpec = ServerSpec
  { setup      = return 0
  , handleCall = \st Get -> return (st, st)
  , handleCast = \st req ->
      case req of
        Inc   -> return (st + 1)
        Reset -> return 0
  , tearDown   = \_ -> return ()
  }
```

A client can then start the server remotely and interact with it through a typed handle:

```
do counter <- startServer n counterSpec
   cast counter Inc
   cast counter Inc
   i <- call counter Get
   liftIO $ print i
```

Supervisors capture a second common pattern, where a supervisor process keeps a set of child processes alive according to a restart policy. Again, the implementation is library code. A supervisor starts its children using `spawn`, monitors them, and reacts to `ProcessDied` messages by consulting its policy.

```
data ChildSpec = ChildSpec
```

```
  { childName    :: String
  , childStart   :: ProcessM ()
  , childNode    :: Maybe NodeId
  , childRestart :: Restart
  }

data Restart = Permanent | Transient | Temporary
data Strategy = OneForOne | OneForAll

data SupervisorSpec = SupervisorSpec
  { strategy  :: Strategy
  , intensity :: Int
  , period    :: Int
  , children  :: [ChildSpec]
  }

supervise :: SupervisorSpec -> ProcessM Pid
getChild  :: Pid -> String -> ProcessM (Maybe Pid)
```

The field `childStart` is an ordinary `ProcessM ()`. It may be a generic server, another supervisor, or an application-specific process. If `childNode` names a remote node, the child is spawned there. If the child dies and the restart policy says it should be restarted, the supervisor simply spawns the same computation again.

For instance, a supervised remote counter server is just a child whose start action runs the generic server

```
  do let child = ChildSpec
           { childName    = "counter"
           , childStart   = runServer counterSpec
           , childNode    = Just n
           , childRestart = Permanent
           }

     sup <- supervise SupervisorSpec
       { strategy  = OneForOne
       , intensity = 3
       , period    = 5000
       , children  = [child]
       }

     Just pid <- getChild sup "counter"
     let counter = serverFromPid pid
     cast counter Inc
```

This is the role of the Erlang-style parts of CloudMicroHaskell in this paper. They are evidence that the direct `spawn` and `send` interface is expressive enough to build structured distributed programs, while keeping the central tradeoff visible: process behaviour is easy to move because it is represented as a runtime graph, and the safety of that movement is correspondingly checked at runtime.

3 Implementation

3.1 MicroHs & Graph Reduction

The Haskell implementation used in this paper is MicroHs [2]. What matters for our purposes is not MicroHs itself, but the runtime support it provides: the serialisation primitives that make the framework work. The design of the framework is not tied to MicroHs, provided an alternative runtime exposes equivalent graph serialisation primitives.

Serialisation & Deserialisation. MicroHs uses graph reduction with combinators [12]. This means that a running program is represented by a graph in memory that is continuously rewritten. The leaf nodes in the graph are the combinators and the interior nodes are application nodes. There is sharing (which is necessary for lazy evaluation) and cycles (because of recursion). To serialise an arbitrary value, you simply have to traverse the sub-graph reachable from the starting node, *i.e.*, the value to serialise.

What can be serialised? In general, pure computations and values can be serialised without any problem. If something cannot be serialised, the runtime will throw an exception. Here is a more detailed breakdown of what works and what does not.

Can be serialised:

- application nodes
- primitive operations, *i.e.*, combinators, basic arithmetic, etc.
- and by extension, any Haskell data type
- basic values, *i.e.*, fixed-size integers, floating point, etc.
- byte strings
- boxed arrays
- `IORef`

Cannot be serialised:

- pointers to memory allocated with `malloc()`
- foreign pointers
- C function pointers
- `MVar` values, which could be serialised, but are unlikely to work since threads are not serialised
- thread IDs, which represent threads; we have chosen not to serialise these

Serialisation API. The underlying API exposed by the runtime system to serialise and deserialise values is simple

```
serialize   :: a -> IO ByteString
deserialize :: ByteString -> IO a
```

Taken in isolation, this API is unsafe. `deserialize` makes no attempt to verify that the deserialised graph represents a value of the requested type.

CloudMicroHaskell therefore treats these operations as low-level runtime primitives, not as the user-facing serialisation API. Their use is confined to protocol points where the runtime determines the expected shape of the graph. For remote spawn, the payload is deserialised as the process function type expected by the node runtime. For message delivery, a remote message is stored together with a `TypeRep`, and selective receive compares this representation before deserialising the message payload.

These checks do not make graph deserialisation statically type safe. They rely on compatible nodes following the CloudMicroHaskell protocol and agreeing on the runtime graph format and type representations. We return to richer boundary descriptors, such as a `TypeDescribable` constraint, in the Discussion.

Laziness. Any graph can be serialised, regardless of how evaluated it is. Thus unevaluated "thunks" can be serialised. This is a feature, but it is also often not what you want. It is easy to accidentally serialise a graph that was actually intended to be evaluated first. For data types, it is possible to control evaluation using `rnf` from the `NFData` class. For functions, the situation is more delicate. A function can only be forced to weak head normal form, which exposes a closure but does not force the values captured in its environment, nor any computation under the lambda. Sending functions therefore preserves the direct style, but it does not remove the programmer's responsibility to understand what the closure reaches. In practice, this means that values captured by mobile functions should be forced before the function is sent when their evaluation location matters.

Details of serialisation. The serialisation is implemented as a two-pass algorithm. The first pass identifies shared nodes via a depth-first traversal, whereas the second pass turns the graph into a byte string. The information from the first pass is used to guide the second pass on where to reference previously serialised nodes rather than serialising them again. Below follows an outline of the steps taken by the serialiser algorithm

- First, two bit arrays are allocated, called *visited* and *shared*, with as many elements as there are nodes in the heap. These arrays are used as mark bits during the two passes. We use two arrays rather than reserving bits in the nodes themselves for two main reasons: fewer writes to memory, and the possibility of concurrent traversals.
- When the first pass traverses the graph and sees a node, a previously unvisited node will have its *visited* bit set, whereas an already visited node will have its *shared* bit set. Any outgoing edges from the node are then recursively visited.

- The second pass flattens the graph using a post-order traversal. If a node is both *shared* and *visited*, we unset its *visited* bit, serialise the subparts, emit that node's textual representation, and finally emit a fresh label for that node. If a node is *shared* and not *visited*, it has already been serialised. In that case we emit the label that was associated with the node when it was previously serialised. If a node is not *shared*, it is serialised.
- Finally, the two temporary bit arrays are freed.

Labels can be represented in different ways. The easiest representation is the address of the node, or perhaps its offset into the heap. The representation doesn't matter as long as labels can be generated to be unique for each node.

Details of deserialisation. Deserialisation is the inverse operation of serialisation. The textual format from the serialiser is parsed, and an equivalent graph, preserving sharing and cycles, is reconstructed. Below follows an outline of the steps taken by the deserialiser algorithm

- First, we allocate an array that serves as a hash table, where labels are associated to node addresses.
- When a node is parsed, we check whether it is a label *reference* or not. If it is, we look up the label in the hash table. The parser might see a label reference before the label *definition*. In that case we allocate a placeholder indirection node and later update it to point at the parsed definition, and return the address of the indirection node.
- After a node has been parsed, if we see a label *definition*, we associate the label with the address of the just parsed node in the hash table.
- Finally, we free the memory of the hash table and return the last parsed node as the result of deserialisation.

Format. The serialisation format is textual. That makes it easier to read and debug, but it also avoids using fixed-width data for the basic types.

The format is portable across architectures in the sense that it does not expose machine byte order, pointer values, or binary code layout. Portability depends on the sender and receiver using compatible runtime versions and on primitive values being representable on the receiving runtime. In our current system, `Int` is an architecture-dependent primitive. If a 64-bit runtime produces and serialises a value of type `Int`, the receiving runtime might not be able to represent the value if it is a 32-bit runtime. Cross-architecture protocols that must rely on numeric ranges should use fixed-width types. An optional compression/decompression pass can also be used to reduce the size of the serialised data.

Example. Consider first the function `f` below

```
1  f :: Int -> Int
2  f x = x * x + 1
```

The serialised combinator representation of this function is

```
v8.4
0
C' + @ S * @ I @@ #1 @
```

Here, `v8.4` is the format version, and `0` is the number of labels used. This is followed by some combinators, with `@` being postfix application. Another example, illustrating a recursive function, is shown below

```
fac :: Int -> Int
fac n = if n==0 then 1 else n*fac(n-1)
```

The serialised representation of this function is

```
v8.4
1
C S C == @ #0 @@ S * @ B _49382130 @ C - @ #1 @@@@@#1 @ :49382130
```

Here, `_49382130` is a label reference and `:49382130` is a label definition (because the function is recursive).

Figure 1 visualises the two serialised examples above. The purpose of the figure is simply to relate the flat textual format to the runtime object it describes: each `@` becomes an application node, while the other tokens become leaves in the graph. In the factorial example, the label reference and label definition correspond to the recursive edge in the graph. This graph, rather than a source-level closure name, is what CloudMicroHaskell serialises when values or process bodies cross node boundaries.

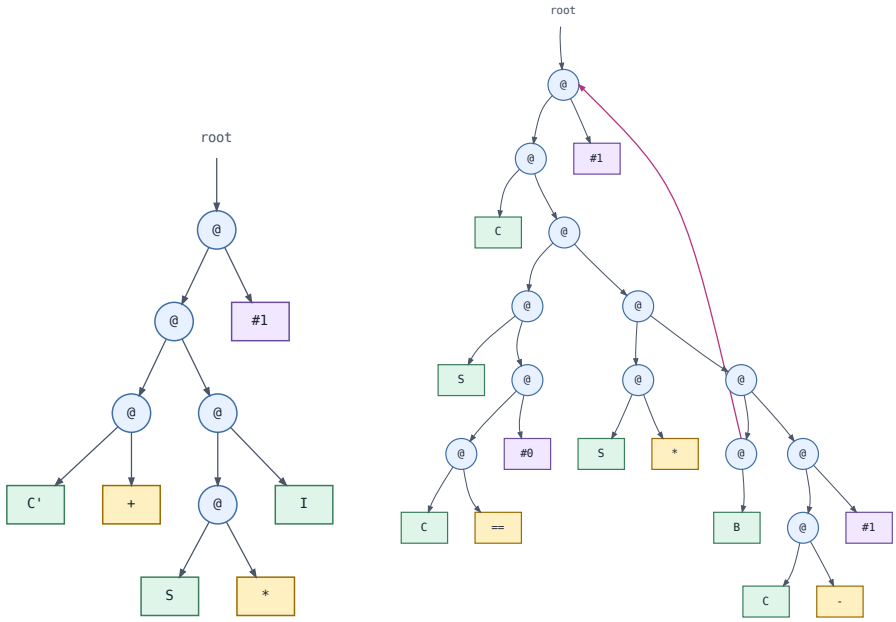
Having described the graph serialiser itself, we now describe where the distributed runtime invokes it: first in the node command loop, then in remote spawn and message delivery.

3.2 Node Runtime Architecture

A CloudMicroHaskell program always runs a *node*. The node is invoked from `main` with a `NodeConfig`, a structure holding the node's identity (an IP address and port number). The Haskell main thread creates an initial *node state*, spawns a *TCP listener* thread that awaits incoming connection requests, and then enters the main node loop. After this initial setup, the node continuously awaits commands from either a remote peer over the network, or from a process running on the same machine.

This gives the runtime a simple shape, where processes and peer-reader threads typically do not manipulate the distributed runtime directly. Instead, they communicate with the node loop by submitting commands. The rest of this subsection explains that boundary: which requests can arrive at a node, where they come from, and how the node turns them into local runtime effects.

We first conclude that the node clearly needs to be able to process commands. Examples of commands could be *install this monitor*, *send this message*, *connect to this remote node*, to name just a few. Some of the commands the node needs to



(a) A non-recursive value.

(b) A recursive value.

Fig. 1. Combinator graphs corresponding to the two serialised examples. Application nodes are drawn as circles; combinators, primitive operations, and literals are drawn as leaves. The recursive example requires a label in the textual serialisation because the graph contains a cycle. The figure shows multiple copies of primitive nodes for clarity; in the runtime they are allocated once and shared.

process come from processes that are running on the node in question, whereas some commands will be sent from remote nodes over the network. We model the commands without much flair

```haskell
-- | A command for the node to handle
data NodeCommand
  = Local Command          -- ^ A command from a process on this node
  | NonLocal PeerCommand -- ^ A command sent from another node

-- | Commands that a local process sends to the node
data Command where
  Connect :: SockAddr -> Maybe (MVar ()) -> Command
  Deliver :: Pid -> Mail -> Command
  -- more

```

```haskell
-- | A command that a remote node has asked the node to execute
data PeerCommand
  = PeerConnect SockAddr Socket [SockAddr]
  | PeerDeliver Pid Mail
  -- more


```

The node processes commands as they arrive in an `cmdQueue :: CommandQueue` maintained in its node state. The `CommandQueue` is, internally, a double-ended queue coupled with an `MVar` used to signal when an empty queue has been written to.

```haskell
data Queue a = Queue [a] [a]
data CommandQueue a = CommandQueue (MVar (Queue a)) (MVar ())

newCommandQueue :: IO (CommandQueue a)
enqueueCommand :: CommandQueue a -> a -> IO ()
takeCommand :: CommandQueue a -> IO a
```

A more robust implementation might assign different priorities to commands, but our implementation treats them all as equally important.

A local process will construct a `Local Command` and write it to the command queue, whereas a remote command will be received by a green thread monitoring a socket, as a `NonLocal PeerCommand`. While a command is travelling over the network, it is represented as yet another type of command, `WireCommand`.

```haskell
data WireCommand
  = WireConnect SockAddr [SockAddr]
  | WireDeliver Pid Int Int
  -- more
```

During transit, a command will have been turned primarily into a buffer of bytes. In the example constructors above, the `WireDeliver` variant carries a process identifier, and two sizes, denoting the length of the two following data packets (a serialised `TypeRep`, and the `Mail` itself). The TCP listener will convert a `WireCommand` to a `PeerCommand` by e.g. reading potential buffers and reconstructing values that have been serialised.

The node processes commands in the order in which they arrive. A figure that shows how the different types of commands flow through the node application is shown in fig. 2. The command interface is what lets the node remain the owner of the runtime state. Mailboxes, monitors, connections, pending receives, and the name registry are not exposed to processes directly, but are rather updated only by the node loop while handling commands. Instead of showing the entire node state definition at once, we will introduce the relevant pieces as they become necessary.

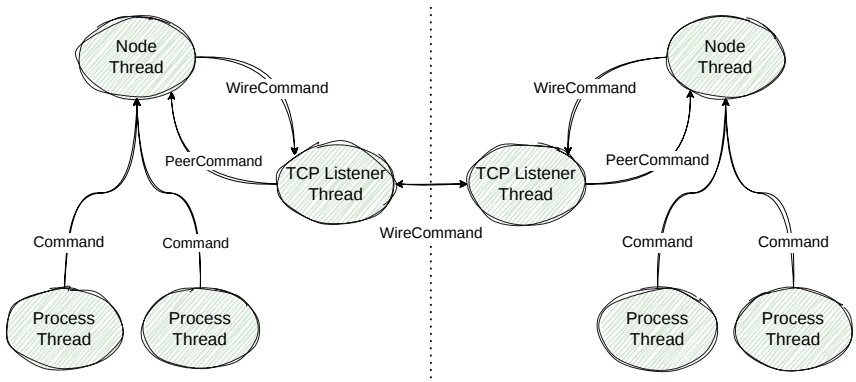


Fig. 2. The figure illustrates how the different types of commands flow through the node application. Every node in this diagram is a Haskell green thread, and the dotted line represents the network boundary. Process threads will send the node ordinary `Command` values, whereas the node thread needs to convert such a command to an appropriate `WireCommand` before the TCP listener thread can send it to a remote node. A TCP listener converts a received `WireCommand` to a `PeerCommand` before sending it to the node thread.

3.3 The ProcessM Monad

The `ProcessM` monad is a central part of CloudMicroHaskell, as it models a *process*. Conceptually, a process is an independent entity that performs computations regardless of what other processes do, and communicates with other processes only via message passing. It is modelled as a function that takes the node state and its own pid, and then produces an `IO` action.

```
1 data ProcessM a = ProcessM { unProcessM :: NodeState -> Pid -> IO a }
```

The process needs to know about the node state as the node state supplies, most notably, the `MVar` through which commands are sent. A common pattern in `ProcessM` primitives that needs to wait for a result from the node is that they allocate a fresh `MVar`, send its command to the node (which includes the `MVar`), and then block until the result is there. An example is shown below

```
1  -- | Register a Pid under a name
2  register :: Pid -> String -> ProcessM ()
3  register pid name = ProcessM $ \ns _ -> do
4    -- create a fresh MVar to block until the node has
5    -- registered the name
6    mv <- newEmptyMVar
7
8    -- send the command to the node
9    enqueueCommand (cmdQueue ns) (Local (Register pid name mv))
```

```
10
11      -- wait for the node to write to the MVar, indicating
12      -- that it is done
13      takeMVar mv
```

Additionally, functions that return a result fetched from the node state must force it. Consider the function `node` that returns the node a process is currently executing on.

```
1   node :: ProcessM NodeId
2   node = ProcessM $ \ns _ ->
3     let nd = NodeId $ nodeAddr ns
4     in rnf nd `seq` return nd
```

If we did not force the evaluation of the `NodeId`, there is a high risk that the result from `node` would be an unevaluated thunk. We illustrate this problem with the code snippet below, which shows a process that spawns another process that references the fetched `NodeId`.

```
1 do n <- node
2    ns <- filter ((/=) n) <$> nodes
3    mapM_ (\remoteNode ->
4               spawn remoteNode (spawn n (liftIO $ putStrLn "hi")))
5          ns
```

The above program fetches the current process's node, and then spawns a process on every other remote node. That process in turn spawns a process on the original node again, by referencing the free variable `n`. If `n` was an unevaluated thunk that references the node state, the outer call to `spawn` would not succeed, as the serialiser would follow the graph into the node state and try to serialise its inner `MVar`s. By ensuring the result is forced, we know that the node state is no longer referenced by `n` and serialising proceeds as expected.

This is also where the conceptual process and the runtime bookkeeping part ways. A `ProcessM` computation is given a `Pid`, but the resources associated with that pid are owned by the node. The process has a mailbox, may have pending receives, may be registered under a name, and may be monitored by other processes. However, none of these are fields of the process itself. They are entries in the node state, and the process can affect them only by sending commands to the node.

This distinction is important for distribution. When a process is spawned remotely, we do not serialise its mailbox, monitors, or other local runtime resources. We serialise only the computation to run. The receiving node allocates a fresh pid, creates the corresponding local runtime entries, and then runs the computation with its own node state.

3.4 Remote Spawn

The central contribution of CloudMicroHaskell is that `spawn` can take an ordinary `ProcessM` computation and run it on another node. We now describe how a call to `spawn` moves through the node runtime, crosses the network, and results in a fresh green thread running the supplied computation on a remote machine. The example program we begin with is shown below

```
do n <- node
   anotherNode <- (head . filter ((/=) n)) <$> nodes

   let x = 7

   _ <- spawn anotherNode $ do
          liftIO $ putStrLn $ show x
          let x = 10
          liftIO $ putStrLn $ show x

   return ()

```

We note that the spawned process references the outer `x`, a free variable.

The implementation of `spawn` has a "fast path" for node-local spawns, to achieve better performance. The rest of this section, however, describes how a remote spawn is implemented (the "slow path"). The fast path differs from the slow path in that the running process spawns the new process itself, bypassing the node loop/command queue. Additionally, the process body of a locally spawned process is not serialised, as the process body does not need to traverse the network. The slow path presented below describes a superset of the actions that the fast path takes.

As `spawn` returns the `Pid` of the spawned process, it blocks until the remote node has created the process and acknowledged its `Pid`. A fresh `MVar` is allocated where the node will write the `Pid` once it receives it together with a spawn confirmation from the remote node. Additionally, a unique token is generated and paired with the spawn request, so that an acknowledgement of another spawn request is not confused with the current request. The result `MVar` is associated with the unique token in a mutable map reachable from the node state.

```
spawn :: NodeId -> ProcessM () -> ProcessM Pid
spawn (NodeId targetAddr) action = ProcessM $ \ns _myPid -> do
  -- generate token and create MVar, to record in the node state
  reqId    <- newUniqueToken
  replyVar <- newEmptyMVar
  modifyIORef' (spawnRequests ns) (Map.insert reqId replyVar)

  -- send the spawn command to the node loop
```

```
enqueueCommand (cmdQueue ns)
    (SpawnOn targetAddr reqId (unProcessM action))

-- block here until the node loop publishes the process
-- identifier of the spawned process
pid <- takeMVar replyVar
modifyIORef' (spawnRequests ns) (Map.delete reqId)

return pid
```

When the node processes the `SpawnOn` command, it fetches the socket associated with the remote node and initiates the network communication.

```
-- this function pattern matches on many commands, but we just
-- show SpawnOn here
handleLocalCommand :: NodeState -> Command -> IO ()
handleLocalCommand ns (SpawnOn targetAddr reqId f) = do
  c <- getConn ns targetAddr
  sendSpawn c (nodeAddr ns) reqId f

-- remember that the ProcessM monad wrapped a function of type
-- NodeState -> Pid -> IO a
sendSpawn :: Socket -> SockAddr -> Int
          -> (NodeState -> Pid -> IO ()) -> IO ()
sendSpawn conn spawnerAddr reqId f = do
  (buf, len) <- enc f -- serialise the ProcessM function

  -- send first the serialised WireSpawn command, and then
  -- the serialised graph
  sendLine conn (show (WireSpawn spawnerAddr reqId len))
  sendAll conn buf len

  free buf
```

Here is where we invoke our magic serialisation function. The serialised graph will reference everything that was reachable from the expression, including the outer free variable. This is now a serialised function that contains both its code and environment.

After serialisation, we get a pointer to a memory buffer holding our ready-to-send bytes. The `WireCommand` that we send to the remote node includes the unique token that we have associated with the spawn request, as well as the length of the following data buffer. The remote node's TCP listener will first receive the `WireCommand`, look at it to figure out how many bytes to read for the subsequent graph, and then decode it when it turns it into a `PeerCommand`. We omit most of the listener code, but show how the received `WireSpawn` message is treated

```
handleWireCommand (WireSpawn spawnerAddr reqId payloadLen) = do
  buf <- mallocBytes payloadLen
  ok  <- recvExact conn buf payloadLen

  if not ok
    then do
      -- error handling code, omitted for brevity
    else do
      f <- dec buf payloadLen :: IO (NodeState -> Pid -> IO ())
      free buf
      enqueueCommand (cmdQueue ns)
        (NonLocal (PeerSpawn spawnerAddr reqId f))
```

After the `WireSpawn` command has already been parsed, we know that we should read a subsequent buffer of data, the graph. We allocate a buffer of the appropriate length and read exactly that many bytes into it. We then invoke our magic deserialiser, which reconstructs the graph on the heap of the remote CloudMicroHaskell node. Here we annotate the deserialiser with the type of the function that `ProcessM` wraps. There is no verification that the parsed graph represents a value of that specific type, so we must be very careful to properly line up the types of the values we serialise, and the values we deserialise. We then enqueue the `PeerSpawn` command with the node loop.

The node loop on the remote node must invoke its `PeerCommand` handler to deal with the command. It starts the local process, and sends the `Pid` together with the unique token back to the original node. We omit the code for `startLocalProcess`, but mention that it uses `forkIO` to spawn a green thread that runs the `ProcessM` function we just deserialised. The process's mailbox and associated metadata are then added to the node state.

```
handleNonLocalCommand ns (PeerSpawn spawnerAddr reqId f) = do
  pid <- startLocalProcess ns f
  c   <- getConn ns spawnerAddr
  sendLine c (show (WireSpawnAck reqId pid))
```

At the wire level the original node receives the `WireSpawnAck`, which the listener promptly converts into a `PeerSpawnAck`. The spawn request is then completed by invoking the
`completeSpawnRequest` function that was briefly shown in the code that described how the node handled the initial spawn request. `completeSpawnRequest` publishes the `Pid` into the `MVar` that is associated with the unique token. This unblocks the process that initiated the spawn request, which now has access to the process identifier of the new process. This concludes the handling of `spawn`.

3.5 Message Passing

The underlying mechanism for transmitting messages between communicating processes is the same. There is a fast and slow path, differing in exactly the same way that the fast and slow path did for `spawn`. As values are, like code, just subgraphs in the combinator graph, the serialiser and deserialiser work here as well. If a message has to traverse the network it will be encoded together with a runtime representation of its type. If a message does not have to traverse the network, and is being sent to a process running on the same node, the message is passed by its pointer into the graph. We capture these two kinds of messages in the following data type

```
data Mail where
  HeapMail    :: a -> Mail
  EncodedMail :: TypeRep -> Ptr Word8 -> Int -> Mail
```

We use `TypeRep` from the `Typeable` type class to observe the types of values at runtime. Both GHC and MicroHs derive `Typeable` instances for every type, so the programmer does not need to implement them.

Every process has a mailbox where its messages go, but the mailbox is owned by the node loop. In the fast path, the message will be delivered directly to the receiving process, whereas in the slow path, the action of sending the message will be delegated to the node. We omit the code for `deliverMailLocal`, which is responsible for doing the actions just described. The code for `send` itself is fairly brief

```
send :: Identifiable process => process -> a -> ProcessM ()
send targetPid x = ProcessM $ \ns _ -> do
  targetPid' <- toPid targetPid ns
  let Pid (NodeId targetAddr) _ = targetPid'
  if targetAddr == nodeAddr ns
    then deliverMailLocal Nothing ns targetPid' (HeapMail x)
    else
      enqueueCommand (cmdQueue ns)
        (Local (Deliver targetPid' (HeapMail x)))
```

There are two types that are `Identifiable`, `Pid` and `String`. `toPid` does a lookup in an `IORef` fetched from the node state if the instantiated type is `String`.

When the node handles the remote send, it will fetch the associated socket, encode both the message and the `TypeRep`, and then send everything through the socket.

```
handleLocalCommand ns (Deliver targetPid mail) = do
  let Pid (NodeId targetAddr) _ = targetPid
  deliverMailRemote ns targetPid mail

deliverMailRemote :: NodeState -> Pid -> Mail -> IO ()
deliverMailRemote ns targetPid (HeapMail x) = do
  let Pid (NodeId targetAddr) _ = targetPid
  c <- getConn ns targetAddr
```

```
9
10    (valueBuf, valueLen) <- enc x
11    (trBuf, trLen)       <- encodeTypeRep (typeOf x)
12
13    sendDeliver c targetPid trLen trBuf valueLen valueBuf
14
15    free valueBuf
16    free trBuf
17  deliverMailRemote _ _ (EncodedMail _ _ _) = -- should not happen
```

When a message is delivered, either to a local process or a remote one, the thread responsible for doing the delivery will check whether the receiving process is already blocked waiting for a message. If the receiving process is already blocked waiting for a message, it will be delivered immediately if the message can be delivered, and only placed in the mailbox if not.

Forcing values before sending. Something to keep in mind when sending messages between processes is that messages will be sent as they are, then and there. The serialiser walks the combinator graph, and if a value is still an unevaluated thunk, the unevaluated thunk will be serialised and transmitted. This might be the intended behaviour, but it will most likely be the intent of the programmer that an evaluated value should be sent. The problems with sending unintended unevaluated thunks are twofold; firstly, a seemingly small expression might make a large part of the sender's heap be serialised and sent to the remote node. Secondly, if we are using processes and message passing to achieve speedups by distributing our computation over a network, work might not be performed on those nodes where the programmer expects it to be done. We note that in Cloud Haskell, this concern does not arise. The `Binary` instance required by Cloud Haskell's `send` requires a value to be evaluated as it is serialised.

To give an example of misplacement of work, consider `countChunk` below. It takes a list of numbers as input and sends a message to the process identifier indicating how many of those numbers were prime. Since `n` is not forced, its thunk will point to `xs`, `filter`, `length`, and `isPrime`. Serialising `n` will serialise those functions as well, and when `n` is required at the parent, the work will be done there.

```
countChunk :: Pid -> [Int] -> ProcessM ()
countChunk parent xs = do
  let n = length (filter isPrime xs)
  send parent n
```

The fix is to ensure that `n` is forced before it is given to `send`.

```
countChunkStrict :: Pid -> [Int] -> ProcessM ()
countChunkStrict parent xs = do
  let n = length (filter isPrime xs)
  n `seq` send parent n
```

In this case we can use `seq` to force evaluation, since `n` is an `Int`, but more structured data would require `rnf` or `deepseq`.

This is the same discipline familiar to programmers of parallel Haskell. A spark created by `par` only buys parallelism if the sparked expression is itself evaluated by another core. In CloudMicroHaskell, the analogous failure mode is not lost parallelism, but misplaced computation, and possibly a much larger message than the programmer intended. The remedy is the same: ensure the message is in normal form before handing it to `send`, using `seq`, bang patterns, or a deep-force such as `deepseq`.

3.6 Failure Detection and Exit Propagation

The fault-tolerance primitives in CloudMicroHaskell are built around process death detection. The runtime does not try to recover a failed process by itself, but instead makes failure observable to other processes. Application-level code, such as supervisors, can be programmed to decide whether a process should restart or not.

One process can observe the death of another by, as described previously, installing a monitor. As the node owns all the state, it maintains a table which maps each watched process to the processes that watch it, and the monitor action each watcher has requested.

```
monitors :: IORef (Map.Map Pid [(Pid, MonitorAction)])
```

Unsurprisingly, a monitor is registered by sending a command to the node loop. If the process that should be watched is local, the node records the monitor in its `monitors` table. Instead, if the process lives on another node, the request is sent to the remote node as a `WireMonitor` command. The remote node will record the monitor in its own `monitors` table. The short summary is that a node is responsible for detecting process death and recording which watchers must be notified.

Every CloudMicroHaskell process is implemented by a Haskell green thread. When a process is started, the runtime forks the thread with a finaliser. This finaliser runs regardless of whether the process returns normally, calls `terminate`, dies because of an exception, or is killed by another process via `exit`.

```
startLocalProcess :: NodeState -> String
                  -> (NodeState -> Pid -> IO ()) -> IO Pid
startLocalProcess ns label f = do
  pid <- allocatePid ns
  tid <- forkFinally (f ns pid) $ \res -> do
    let reason = reasonFromResult res
    cleanupProcess ns pid
    notifyMonitors ns pid reason
  modifyIORef' (threadIds ns) (Map.insert pid tid)
  return pid
```

The finaliser turns the result of the thread into an `ExitReason`. Normal return and `terminate` become `ExitNormal`, whereas termination by `exit` preserves the `ExitReason`

supplied by the caller. An uncaught exception becomes `ExitOther`. The runtime will then remove the process-local state such as its mailbox, pending receives, registered name, and thread identifier, before finally notifying the monitors.

Notification is also routed through the node loop. For each registered watcher the node loop checks whether it is a local process or not. A local watcher is notified immediately, whereas a remote watcher is notified by sending a `WireProcessDied` command to the watcher's node.

```
applyMonitorAction ns watcherPid watcheePid TrapExit reason =
  deliverMailLocal ns
                   watcherPid
                   (HeapMail (ProcessDied watcheePid reason))

applyMonitorAction ns watcherPid watcheePid Succumb _ = do
  tids <- readIORef (threadIds ns)
  case Map.lookup watcherPid tids of
    Just tid -> throwTo tid (MonitoredProcessDied watcheePid)
    Nothing  -> return ()
```

A monitor registered with `TrapExit` will receive the notification via a `ProcessDied` message in its mailbox. The monitor process can then observe the termination and decide what to do. Supervisor processes use this behaviour to figure out whether a process should be restarted or not. The `Succumb` monitor action will make the monitor receive an asynchronous exception, making it die together with the watched process. This is useful to tie together the lifetimes of processes that are meant to exist at the same time.

Node disconnection is treated as process failure. If an open socket is closed the surviving nodes will assume that any monitored process on that node has terminated. The surviving nodes will consider any monitored process as terminated with an `ExitOther` reason. This makes the running processes able to detect network failure as if it were ordinary process failure.

Explicit `exit` signals use the same path. If the target process is running on the current node, the node raises a `ProcessExit` exception in its thread, whereas if the process is remote, the node instead sends a `WireExit` command to the node that owns the process. In either case, the target process eventually runs the same finaliser.

4 Evaluation

We evaluate CloudMicroHaskell as a prototype implementation of a design point, rather than as a mature high-performance actor runtime. The evaluation therefore asks four questions. First, what are the basic costs of process creation, message passing, and runtime graph serialisation? Second, can a distributed CloudMicroHaskell program obtain speedup on a parallel workload despite those costs? Third, is the programming model expressive enough to structure a nontrivial distributed application?

Finally, does the small MicroHs runtime make the same model usable on platforms outside the reach of a conventional GHC-based system?

The ring benchmark addresses the first question, while the distributed work-pool benchmark addresses the second. The MicroSync case study addresses expressiveness by exercising named processes, dynamic process creation, generic servers, supervisors, monitors, and cross-node messaging in one application. The heterogeneous-node case study addresses portability by running CloudMicroHaskell nodes across microcontrollers and an ordinary laptop.

4.1 Benchmarks

Ring Benchmark. We take inspiration from Erlang with our first evaluation. A classic Erlang example is to spawn a *ring* of processes, where a message is sent from one process to another, through the entire ring, until the message arrives again at the original process. We use the ring of processes to measure how long it takes, on average, to spawn a single worker, and how long it takes to send a message from one process to another. We build a ring of 3000 processes.

All processes in the ring are spawned on the same node. Node-local `spawn` and `send` have fast-paths whereby the actions are performed directly by the process rather than the node, avoiding the need to hand over to the node thread. We can, however, instrument CloudMicroHaskell to take the long path through the node loop, simulating what would happen if the `spawn` or `send` targeted another node. Additionally, even for node-local `spawn` or `send`, we can force serialisation and deserialisation, to measure how long those operations take. We measure the performance both with and without serialisation enabled, to gain some insight into how much time encoding and decoding the graph takes. The serialised process is roughly 32 kilobytes of data, whereas the serialised message sent through the ring is roughly 300 bytes.

We run this experiment on a laptop with an Intel i5-8365U CPU, and 16 GB of RAM.

fig. 3 and fig. 4 presents the elapsed time when spawning a process on the same node, using either the fast path or the slow path. The fast path takes roughly 60 µs, whereas the slow path took, for this example, 550 µs. The elapsed time is reported as segments, to try and measure what precisely it is that is taking time. The reported segments are

- **dispatch**: The running process handing off to the thread responsible for doing the `spawn`. In the fast path this is roughly figuring out that the spawning process itself should do it, whereas in the slow path it is a context switch to the node thread.
- **register**: The time it takes to create and initialise process resources, such as mailboxes and the process identifier, and to register them in the node state.
- **fork**: The time it takes to actually spawn the green thread that will execute the new process. Additionally, the node state is updated to include the fresh thread ID of the spawned thread.

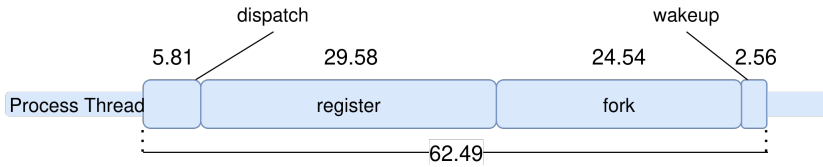


Fig. 3. Breakdown of node-local process spawn time on the fast path, where the spawning process creates the child directly without routing through the node loop.

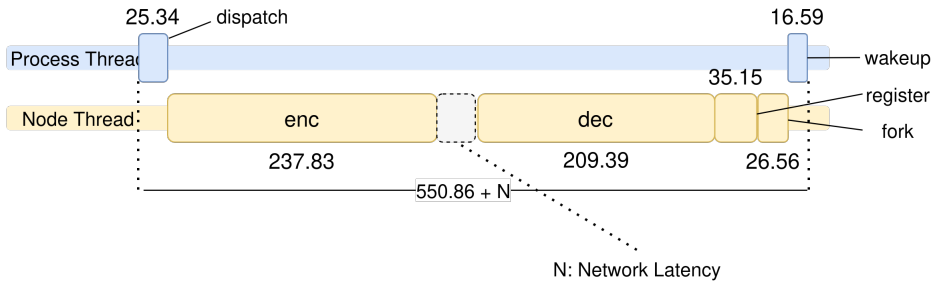


Fig. 4. Breakdown of process spawn time on the slow path, where the process body is serialised, routed through the node loop, deserialised, and then started locally. A remote spawn on comparable hardware would add the network round-trip and transmission time.

- **wakeup**: The time it takes to hand control back to the initial process, which is done by writing the fresh process identifier to an `MVar` that the initial process reads from.
- **enc**: The time the runtime takes to serialise the reachable subgraph representing the process body to be spawned.
- **dec**: The time it takes for the runtime to deserialise and reconstruct the graph from the serialised representation.

It takes CloudMicroHaskell 371,347 µs to send a message through the entire ring.

For comparison, a state of the art actor model concurrency system, Erlang, takes on average 1.27 µs to spawn a single process. Sending a message through a ring of 3000 processes takes on average 1456 µs.

Distributed Work Pool Benchmark. This benchmark measures whether adding CloudMicroHaskell nodes yields proportional speedup on an embarrassingly parallel workload. We count the primes in a fixed integer interval, dividing it into equally sized segments that a work pool distributes as units of work across remote workers. The work pool is implemented as a CloudMicroHaskell application, where a master process takes N units of work, spawns M worker processes, and distributes the units of work to the workers one at a time. A unit of work is modelled as a `ProcessM a` computation.

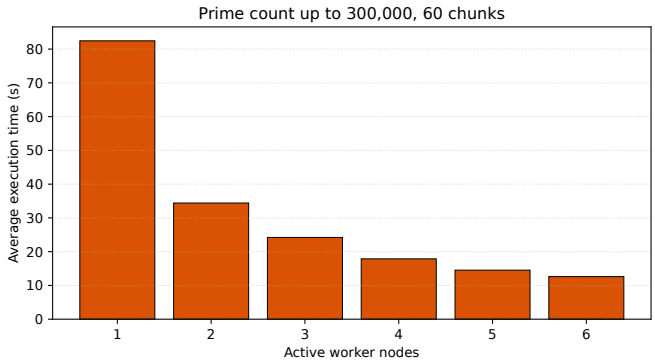


Fig. 5. Distributed work-pool benchmark for counting primes up to 300,000, split into 60 work units. Bars show mean execution time over 10 runs on one to six Google Cloud E2-standard-2 worker nodes; runtime drops from 82.5 s to 12.6 s, a 6.5× speedup over the one-node baseline.

We distribute the workload over 6 remote Google Cloud E2-standard-2 machines, each equipped with 8 GB of RAM and 2 vCPUs. Each instance runs a CloudMicroHaskell node; these nodes are connected together. We run the workload using between one and six nodes, averaging over 10 results. The results can be seen in fig. 5.

4.2 Case Study 1: MicroSync

To illustrate the programming model beyond small code snippets and toy examples, we implement a file syncing application, MicroSync. The case study is modest in scope, but deliberately nontrivial. Several machines work together to keep the contents of a directory in sync, discover each other, broadcast local edits, exchange metadata, exchange partial chunks of files, and recover from failing helper processes. The case study shows that this application is expressed fairly naturally as a collection of communicating processes.

We dive straight in, and illustrate how an edit of a local file on machine A is advertised and applied to a remote copy of the same file on machine B; fig. 6 shows the corresponding message flow.

On machine A, a user modifies a local file. The *modify detector* process assigned to monitor that particular file detects this edit by observing the last modified timestamp, and reports this fact to its parent *file tracker* process. The file tracker process is MicroSync's designated owner of one tracked file. It receives the message from the modification detector that the file has likely been modified, and asks the *file accessor* for the file manifest. A file accessor process is the only process that is allowed to modify a file on the disk, and is implemented as a server.

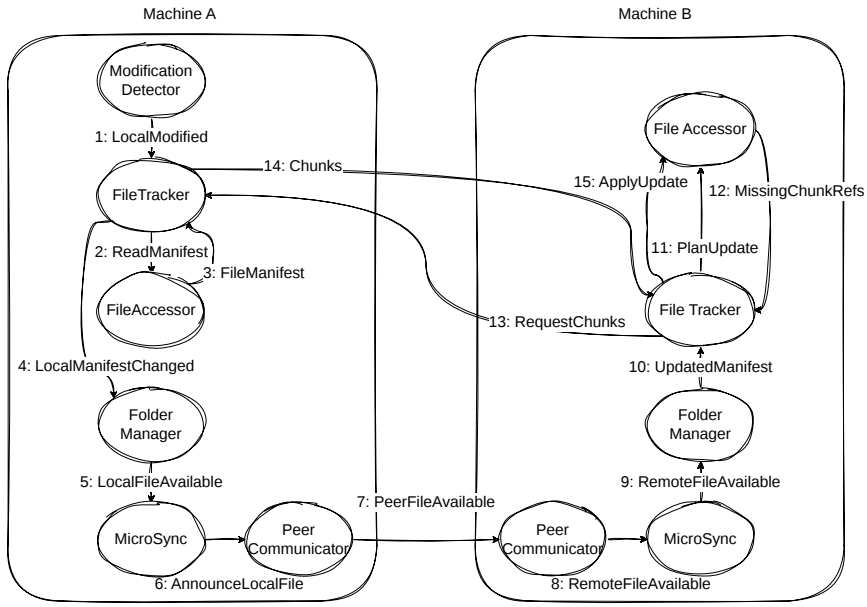


Fig. 6. Message flow for synchronising a local edit from machine A to machine B. The numbered arrows show the messages described in the walkthrough: the update is first advertised through the peer processes, after which the receiving file tracker requests missing chunks directly from the file tracker that advertised the version.

A file manifest is a compact description of a file version. It records, among other things, the path to the file, the file size, and most importantly, a list of chunk references. Each chunk reference contains an offset into the file, the size of the chunk, and a hash of the chunk contents. MicroSync uses *Content-Defined Chunking* to divide a file into chunks. The chunks are chosen based on the contents of the file, rather than using fixed byte offsets. The effect of this is that a small edit tends to change only chunks close to where the edit happened, allowing a remote MicroSync instance to request only the chunks that it does not already have.

Once the file accessor has supplied the file manifest, the file tracker sends it to the *folder manager* process. The folder manager process is the designated owner of the entire folder being synchronised. For this local-modification case, its main role is to turn the tracker's manifest into a file-version announcement that is sent upwards, to the top-level *MicroSync* process. The top-level MicroSync process routes the announcement to its *peer communicator* process. The peer communicator process informs the remote peer communicator on machine B what machine A's version of the file looks like. On machine B, the information flows in a similar way from the peer to the MicroSync top-level process, from there to the folder manager, which

will in turn inform the responsible file tracker process that there exists a potentially new version of the file.

The file tracker on machine B will ask its file accessor for the local file manifest, figure out that the version on machine A is newer, and then compute which chunks of the file it should request from machine A. The broadcast message from machine A included the process identifier of the specific file tracker on machine A, so the equivalent file tracker on machine B will now request the missing chunks directly from the file tracker on machine A. This process identifier is only assumed valid for this one exchange, and is not retained on machine B for subsequent use. Once the chunks have been delivered, the file tracker on machine B instructs its file accessor to rebuild the file according to the new file manifest. It will supply the new chunks, and together with the chunks that already exist on machine B, the file is updated to have the same contents as the version on machine A.

fig. 6 shows two machines for clarity. However, in general, a MicroSync instance may be connected to several remote peers. The peer process broadcasts file-version announcements to all of them, and each receiving instance independently decides how to react to the announcement. The described application is built entirely from CloudMicroHaskell primitives such as named processes for discovery, message passing for routing, generic servers for stateful services, supervisors for recovery, etc. The following paragraph gives some details on the implementation, but naturally omits most of the application code.

The Process Tree. We have already touched upon several processes that are part of a MicroSync instance, but here we give a complete description of the different kinds of processes. A MicroSync instance is implemented as a process tree, with the top level process being the MicroSync process. It registers itself under the name `"microsync"` and starts a supervisor for the long-running services of the application. The supervisor uses a one-for-one strategy to permanently supervise four child processes. The child processes in question are the *Event Logger* process, the *Folder Manager* process, the *Peer Communication* process, and finally the *Peer Connector* process.

The event logger is a generic server, registered under the name `"eventlogger"`, whose service is to accept events and log them to a persistent file on disk. It gives the rest of the application a single named process which events can be sent to for logging.

The peer connector process discovers remote MicroSync instances by querying connected CloudMicroHaskell nodes for a registered `"peer"` process. This peer process is the control-plane process for remote MicroSync instances: it tracks connected peers, monitors them for disconnection, and forwards file-version announcements between machines.

The folder manager process owns the state of the synchronised folder. It manages the set of files being tracked at a point in time, and starts or stops file tracker processes as necessary. Additionally, it manages a folder scanner process, which scans the folder

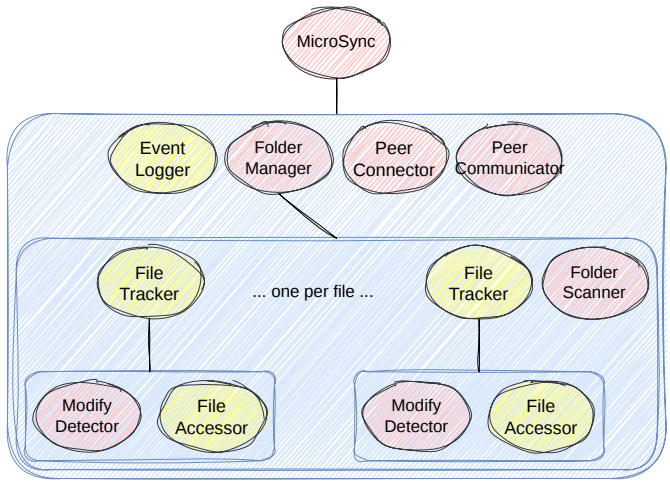


Fig. 7. Process tree of one MicroSync instance. Blue containers are supervisors, yellow processes are generic servers, and red processes are ordinary processes. File trackers are created dynamically, one per tracked file, and each tracker supervises its own modify detector and file accessor.

for new and deleted files, while each tracked file is represented by a dynamically added file tracker process.

A file tracker process is created dynamically for each tracked file. It owns the synchronisation protocol for that file and presents the rest of the application with a single process to address. The tracker itself is implemented as a generic server, registered under the file path, and it starts a private supervision tree for the helper processes associated with the file. Its modify detector reports suspected local changes, while its file accessor serialises all reads and writes to the underlying file.

fig. 7 illustrates how the processes are structured, and who supervises whom.

4.3 Case Study 2: Heterogeneous Nodes

The second case study exercises a different motivation for building CloudMicroHaskell on MicroHs: portability rather than performance. We run a CloudMicroHaskell node on each of two STM32H563ZI microcontrollers running Zephyr RTOS, and a third node on the laptop used for the ring benchmark. The microcontrollers have 2 MB of flash memory, 640 KB of SRAM, and run at up to 250 MHz. All three devices are connected by Ethernet through a network switch.

This setup is deliberately heterogeneous. The laptop node runs a 64-bit MicroHs runtime, whereas the microcontroller nodes run a 32-bit runtime built for Zephyr. The nodes are therefore not executing the same binary, but they do share a compatible CloudMicroHaskell protocol, graph format, and runtime representation. This

exercises the claim that CloudMicroHaskell nodes need not agree on a common set of static symbols in the style of Cloud Haskell. Instead, they must agree on the runtime format used to serialise and deserialise graphs. The messages used in this case study contain primitive values that are representable on both the 64-bit and 32-bit runtimes. The case study therefore demonstrates portability of the runtime format and programming model across heterogeneous nodes, not that every possible value of every architecture-dependent primitive can be transported unchanged.

The application is small but end-to-end. One microcontroller runs a generic server that controls LEDs. The second microcontroller runs a process that reacts to button presses and sends requests to the LED server. The laptop runs a third CloudMicroHaskell node that accepts keyboard commands and sends the same kind of requests. Thus, both an embedded node and a laptop node communicate with a server process running on another embedded node, using the ordinary CloudMicroHaskell message-passing interface.

This case study is not a performance benchmark. Its purpose is to show that the implementation is not tied to GHC-sized machines or to a homogeneous cluster of identical binaries. The same process abstraction, message format, and generic-server library used in the earlier examples can also be deployed across resource-constrained devices.

5 Discussion

Making Serialisation Safer. The implementation of CloudMicroHaskell uses the unsafe primitives for serialisation and deserialisation. This should not be read as the authors claiming that static checks are undesirable. The goal of this project is to explore a programming model where the static checks by Cloud Haskell are turned into runtime checks.

A safer API can be built on top of the unsafe primitives, eliminating some of the risk. A natural refinement is to add a lightweight type-description constraint, for example

```
1  serializeHL   :: TypeDescribable a => a -> IO ByteString
2  deserializeHL :: TypeDescribable a => ByteString -> IO (Maybe a)
```

This constraint would not be used to define how to serialise a value. That task is still delegated to the runtime system. Instead, the constraint would be used to advertise the boundary type, so that a receiver can reject deserialisation of a serialised value where the boundary types don't match. Descriptors like this could be derived generically, for instance via `Data.Data`, and be made to include additional structural information. This is a more explicit version of the check CloudMicroHaskell already performs for messages: remote messages are sent together with a `TypeRep`, and selective receive compares that representation before deserialising the message payload.

This would sit between the solution of Cloud Haskell and CloudMicroHaskell. There is no need for manual closure conversion or for the programmer to write serialisers, but the communication boundary still becomes explicit in the source

program. This solution would not rule out serialisation of runtime-owned resources such as `MVar`s or foreign pointers, but it would turn some type mismatches that are currently detected only at use sites into dynamic checks at deserialisation time. These safer functions would still enable the programmer to accidentally serialise thunks or unintentionally large graphs, however.

Benchmarks. We begin our discussion by observing that the slow path takes significantly longer, increasing the required time from 60 µs to some 550 µs. Of this, roughly 30 µs is added by going through the node loop, and roughly 450 µs comes from serialisation and deserialisation. This is not unexpected, but ideally the cost would be smaller. The serialiser is implemented via two passes to enable sharing of cycles. A one-pass serialiser would be faster, but would not be as capable. We would not be able to serialise recursive graphs, and shared sub-graphs would be replicated as many times as they were referenced, increasing the size of the reconstructed graph.

The serialised process closure occupies roughly 32 KB. The serialised graph is emitted in a textual ASCII format, which is not particularly compact. A new output format could drastically reduce the size of the serialised graph. There would perhaps be a slight gain in performance, but nothing extraordinary.

The results for the ring benchmark indicate that CloudMicroHaskell is significantly slower than Erlang. There are several reasons for this, but a notable one is that Erlang has been optimised for this very task over several decades, whereas CloudMicroHaskell is running on MicroHs, a non-optimising compiler in its infancy. Additionally, even if MicroHs were an optimising compiler, the underlying choice of reducing a mutable combinator graph at runtime will always suffer performance penalties. While not reported in this paper, the authors point out that MicroHs's runtime is generally around two orders of magnitude slower than GHC's.

The heterogeneous-node case study explains why this tradeoff may still be attractive. MicroHs is much slower than mature native-code systems, but its runtime is small and portable. In our prototype, the compiled runtime is roughly 300 KB, small enough to run on the STM32H563ZI microcontrollers used in the case study. The case study therefore supports a portability claim rather than a performance claim: CloudMicroHaskell makes it possible to use the same distributed Haskell programming model across machines that would not normally be able to run a GHC-based distributed Haskell system.

The distributed prime counter shows significant speedups compared to sequential runtime. We point out that there seems to be a superlinear speedup when going from one to two remote machines, and we are not entirely sure why. The remote instances are separate instances, but it is impossible to say precisely where they are hosted and what else is hosted on the same hardware. While we cannot say for sure, we speculate that the phenomenon may be due to caching, and that using more than one MicroHs instance reduces the frequency with which the garbage collector has to run.

MicroSync. The purpose of MicroSync is not to evaluate file synchronisation performance, but to test whether the CloudMicroHaskell programming model is usable in a larger application. The case study exercises named processes, dynamic process creation, supervisors, monitors, generic servers, and cross-node message passing in one program. It therefore supports an expressiveness claim rather than a performance claim, namely that CloudMicroHaskell is sufficient to structure a nontrivial distributed application while preserving the direct style illustrated in the smaller examples.

6 Related Work

Cloud Haskell. The work described in this paper is strongly related to Cloud Haskell. Cloud Haskell presented an elegant API, as evidenced by the work described in this paper reusing many parts of it. The primary difference is the treatment of `spawn`, where Cloud Haskell required the programmer to manually apply lambda-lifting to their process bodies. Furthermore, the programmer had to apply explicit closure-conversion when spawning such functions, adding cognitive overhead. CloudMicroHaskell removes this source-level closure conversion step. A process body can be written directly at the call to `spawn`, including references to variables in scope, and the runtime serialises the reachable graph when the process crosses a node boundary.

Cloud Haskell explicitly discusses the style of implementation used by CloudMicroHaskell, where the compiler or runtime provides primitives for serialising arbitrary values, and argues against it. The concerns raised there still apply. Some runtime-owned values, such as open file handles, should not be serialisable. A programmer may also accidentally serialise a much larger graph than intended, or serialise an unevaluated computation whose work was expected to happen before transmission.

Runtime-Level Code Mobility. CloudMicroHaskell relies on a runtime representation in which code and data can both be serialised as graph nodes. This approach is less natural for implementations such as GHC, which compile Haskell to native machine code. Machine code is neither portable nor desirable as a communication format. The same idea is more plausible for implementations based on bytecode or another portable intermediate representation. In such systems, the code representation can be included in the serialised data and reconstructed, interpreted, or JIT-compiled by the receiver. This is how the Mu implementation at Standard Chartered worked [1].

Glasgow Distributed Haskell. Glasgow Distributed Haskell [7] (GDH) was an earlier effort to extend Haskell for distributed programming. GDH preceded Cloud Haskell, but the general idea was the same, that a distributed application should be described as a collection of communicating processes. GDH was elegant and well-designed, but practical concerns led to the project slowing down. It required substantial modification to GHC's runtime system of the time (the GUM [11] runtime), which limited its adoption and maintainability.

Elixir. The de facto implementation of actor model concurrency is Erlang, a dynamically typed language. Elixir [10] is a language that is executed with the same underlying runtime system, but whose syntax is more approachable. At the fundamental level Elixir is not much different from Erlang, but it offers a more modern tooling and ecosystem.

Tierless and Choreographic Haskell. Recent work has explored several ways to describe distributed Haskell programs from a single source program. In tierless programming, the behaviours of multiple tiers are written and type-checked together, then compiled into separate executables. The framework of Ekblad and Claessen [3] follows this style for client-server web applications. The client and server are described in one program, and the compiler separates the code so that client code does not appear in the server, and vice versa.

The idea was later adopted by HasTEE [8], a framework for programming *Trusted Execution Environments* in Haskell. They observed that a trusted computing unit running on Intel SGX hardware assumes the role of a server, and that the programming model of Ekblad and Claessen [3] was a good fit because of its separation of client code and server code.

Choreographic programming [6] takes a related whole-program view, but describes communication protocols globally rather than from one endpoint's perspective. A communication action is written once, naming both sender and receiver. When the choreography is interpreted or projected for a particular endpoint, that action becomes a send, a receive, or no local action. HasChor [9] embeds this style of choreographic programming in Haskell. Its presentation is elegant and simple, but the original implementation produced executables that contained the code for every endpoint in the network. Krook and Hammersberg [5] address this limitation by specialising HasChor programs with respect to a known endpoint, producing executables that contain only the code needed by that endpoint.

7 Conclusions & Future Work

We set out to explore what the Cloud Haskell programming model looks like when serialisation is provided by the runtime system rather than exposed as `Closure` and `Serializable` constraints in the source-level API. CloudMicroHaskell shows that this design point is practical: remote processes can be spawned in direct style, messages can contain ordinary values and functions, and familiar Erlang-style abstractions such as generic servers and supervisors can be implemented as ordinary libraries.

The benefit is ergonomic: programs can be written much like local communicating processes, with the runtime serialising the reachable MicroHs graph when computation or data crosses a node boundary. The cost is that this boundary moves to runtime: deserialisation depends on the receiver's expected type, and programmers must avoid moving runtime-owned resources or unevaluated work unintentionally.

Future work should reduce what crosses the wire. One direction is to give nodes a shared vocabulary of known code and send references where possible, falling back to

graph serialisation for dynamic code and environments. Another is to move low-level spawning and message delivery into the MicroHs runtime, reducing overhead while centralising dynamic checks for graph types, runtime-owned resources, and version compatibility.

Acknowledgments

The authors would like to thank Koen Claessen, Mary Sheeran, John Hughes, and Simon Peyton-Jones for their helpful discussions and feedback.

A API Reference

This appendix lists the core CloudMicroHaskell API used throughout the paper.

```
data Node a
data NodeId
data NodeConfig -- = (ip, port)
data ProcessM a; instance MonadIO ProcessM
data Pid
runNode :: NodeConfig -> Node () -> IO ()
```

A node says *where* something is: a location in a network. When a node connects to an existing network of nodes, they form a mesh, whereby every node is connected to every other node.

```
connect   :: NodeConfig -> ProcessM ()
```

Each node can host zero or more processes, which are independent computation units. Processes (almost) don't share any memory with other processes, with the exception(s) described in section 3. There are some basic operations for processes to identify themselves, their node, the nodes that are connected to the network, and to terminate.

```
self      :: ProcessM Pid
node      :: ProcessM NodeId
nodes     :: ProcessM [NodeId]
terminate :: ProcessM a
```

A process that wishes to retrieve a list of every node in the network, aside from its own host node, may execute

```
do mynode <- node
   allnodes <- nodes
   return $ filter (/= mynode) allnodes
```

Processes are started either by running an initial process on a node, or by spawning a new process from an existing one. A derived `call` operation spawns a process and waits for its result.

```
runProcessM :: ProcessM () -> Node ()
spawn :: NodeId -> ProcessM () -> ProcessM Pid
call  :: NodeId -> ProcessM a -> ProcessM a
```

As the processes are independent and don't share any memory, they cannot perform any meaningful task without interacting with each other. Interactions are done via message passing, where processes can `send` and `expect` messages to and from one another. `send` is an asynchronous call that returns immediately, whereas `expect` is a synchronous call that doesn't return until it has received a message of the correct type. It is appropriate to think of it as if every process had a mailbox where messages

of any type can go in, and where `expect` goes through the letters in the order they arrived, looking for one of the right type.

```
send   :: Identifiable process => process -> a -> ProcessM ()
expect :: ProcessM a
```

For selective receive, CloudMicroHaskell follows the matcher interface introduced by Cloud Haskell [4]. A matcher specifies one acceptable message type, optionally guarded by a predicate, and `receiveWait` blocks until any matcher succeeds. There is additionally a `matchUnknown` handler that will match a message of any type, removing it from the mailbox.

```
data MatchM q a
match          :: (a -> ProcessM q) -> MatchM q ()
matchIf        :: (a -> Bool) -> (a -> ProcessM a) -> MatchM q ()
matchUnknown   :: ProcessM q -> MatchM q ()
receiveWait    :: [MatchM q ()] -> ProcessM q
receiveTimeout :: Int -> [MatchM q ()] -> ProcessM (Maybe q)
```

A matcher is one branch of a selective receive. It recognises a message and runs a handler producing a value of type `q`. `receiveWait` blocks until one of the supplied matchers succeeds, while `receiveTimeout` bounds the wait and returns `Nothing` if no message matches in time.

For example, a process can wait for either a request to divide two numbers, or for a command to shut down

```
data Div       = Div Pid Int Int
data Shutdown = Shutdown

server :: ProcessM ()
server = receiveWait
  [ matchIf (\(Div _ _ y) -> y /= 0) $ \(Div sender x y) ->
      send sender (x `div` y)
  , match $ \Shutdown -> terminate
  ]
```

The two matchers receive messages of different types, but both branches are part of the same blocking receive. The first branch accepts only non-zero division requests and replies to the sender. The second branch accepts a shutdown command and terminates the process. Messages that do not match either branch remain in the mailbox.

So far, the API describes how live processes communicate. For fault-tolerant programs, processes must also be able to observe one another's lifetime, where a missing reply may mean that another process has terminated, crashed, or disappeared with its node. CloudMicroHaskell exposes this through monitors.

```
data ProcessDied   = ProcessDied Pid ExitReason
```

```haskell
data MonitorAction = TrapExit | Succumb

monitor :: MonitorAction -> Pid -> ProcessM ()
```

When process p monitors process q, p is notified if q terminates. With `TrapExit`, the notification is delivered as a `ProcessDied` message in p's mailbox. With `Succumb`, p terminates as well.

Aside from observing process termination, processes can also cause process termination, using the `exit` function. This makes it possible to designate application-specific roles to processes, such as workers and supervisors. Supervisor processes might send an exit signal to a process that has entered an unhealthy state.

```haskell
data ExitReason
  = ExitNormal       -- ^ Process exited normally
  | ExitShutdown     -- ^ Process exited upon request of another
                     --   process, e.g. a supervisor
  | ExitKill         -- ^ Process was forcibly killed, equivalent
                     --   to kill -9.
  | ExitOther String -- ^ Other custom reasons why a process was
                     --   terminated

exit :: Pid -> ExitReason -> ProcessM ()
```

An example that illustrates how `monitor` and `exit` can be used to control behaviour is shown below

```haskell
do  child <- spawn n childBody

    _ <- spawn n $ do
      monitor TrapExit child
      ProcessDied _ _ <- expect
      liftIO (putStrLn "the child died!")

    linked <- spawn n $ do
      monitor Succumb child
      expect

    monitor TrapExit linked
    exit child ExitKill
    ProcessDied _ _ <- expect
```

The first monitor survives the child's death and receives a `ProcessDied` message. The second monitor uses `Succumb`, so it dies together with the child. By monitoring the second process with `TrapExit`, the parent observes this cascading termination as an ordinary mailbox message.

The process registry maps node-local names to process identifiers. In the example above, the two monitoring processes had to have a way of knowing the process identifier of the child process. Sometimes it is not ergonomic to distribute process identifiers to everyone who has to know them. Furthermore, a process may be restarted by a supervisor process, in which case the pre-distributed process identifier will no longer be valid. The process registry lets a process register itself under a name on the node it is running on.

```
register   :: Pid -> String -> ProcessM ()
unregister :: String -> ProcessM ()
whois      :: String -> ProcessM (Maybe Pid)
```

Since `String` is an instance of `Identifiable`, `send` may target a registered name.

```
send :: Identifiable i => i -> a -> ProcessM ()
```

Types that are `Identifiable` are process identifiers and strings. If a message is sent to a named process, the process identifier is fetched from the process registry before the message is sent. If the name is not registered, the `send` fails with an exception, much like it would in Erlang.

We emphasise that the name registry is node-local, meaning that messages can only be sent to named processes on the same node. The process identifier of a remote named process can be retrieved without much fuss, however. We can simply issue a synchronous call, reading the remote registry

```
-- n' is the NodeId of some remote node
remoteProcess <- call n' (whois "child")
case remoteProcess of
  Just pid -> send pid "hello"
  Nothing -> return ()
```

References

[1] Lennart Augustsson. 2011. Keynote: Pragmatic Haskell. In *Proceedings of the 8th ACM SIGPLAN Workshop on Commercial Users of Functional Programming (CUFP)*. ACM, Tokyo, Japan. Discussions on the design and deployment of the Mu language at Standard Chartered Bank.

[2] Lennart Augustsson. 2024. MicroHs: A Small Compiler for Haskell. In *Proceedings of the 17th ACM SIGPLAN International Haskell Symposium, Haskell 2024, Milan, Italy, September 6-7, 2024*, Niki Vazou and J. Garrett Morris (Eds.). ACM, 120–124. doi:10.1145/3677999.3678280

[3] Anton Ekblad and Koen Claessen. 2014. A Seamless, Client-Centric Programming Model for Type Safe Web Applications. In *Proceedings of the 2014 ACM SIGPLAN symposium on Haskell, Gothenburg, Sweden, September 4-5, 2014*, Wouter Swierstra (Ed.). ACM, 79–89. doi:10.1145/2633357.2633367

[4] Jeff Epstein, Andrew P. Black, and Simon L. Peyton Jones. 2011. Towards Haskell in the Cloud. In *Proceedings of the 4th ACM SIGPLAN Symposium on Haskell, Haskell 2011, Tokyo, Japan, 22 September 2011*, Koen Claessen (Ed.). ACM, 118–129. doi:10.1145/2034675.2034690

[5] Robert Krook and Samuel Hammersberg. 2024. Welcome to the Parti(tioning) (Functional Pearl): Using Rewrite Rules and Specialisation to Partition Haskell Programs. In *Proceedings of the 17th ACM SIGPLAN International Haskell Symposium, Haskell 2024, Milan, Italy, September 6-7, 2024*, Niki Vazou and J. Garrett Morris (Eds.). ACM, 27–40. doi:10.1145/3677999.3678276

[6] Fabrizio Montesi. 2014. *Choreographic Programming*. Ph. D. Dissertation. IT University of Copenhagen.

[7] Robert F. Pointon, Philip W. Trinder, and Hans-Wolfgang Loidl. 2000. The Design and Implementation of Glasgow Distributed Haskell. In *Implementation of Functional Languages, 12th International Workshop, IFL 2000, Aachen, Germany, September 4-7, 2000, Selected Papers (Lecture Notes in Computer Science, Vol. 2011)*, Markus Mohnen and Pieter W. M. Koopman (Eds.). Springer, 53–70. doi:10.1007/3-540-45361-X_4

[8] Abhiroop Sarkar, Robert Krook, Alejandro Russo, and Koen Claessen. 2023. HasTEE: Programming Trusted Execution Environments with Haskell. In *Proceedings of the 16th ACM SIGPLAN International Haskell Symposium, Haskell 2023, Seattle, WA, USA, September 8-9, 2023*, Trevor L. McDonell and Niki Vazou (Eds.). ACM, 72–88. doi:10.1145/3609026.3609731

[9] Gan Shen, Shun Kashiwa, and Lindsey Kuper. 2023. HasChor: Functional Choreographic Programming for All (Functional Pearl). *Proc. ACM Program. Lang.* 7, ICFP (2023), 541–565. doi:10.1145/3607849

[10] The Elixir Team. 2026. The Elixir Programming Language. Accessed: 2026-05-21. https://elixir-lang.org/

[11] Philip W. Trinder, Kevin Hammond, James S. Mattson Jr., Andrew S. Partridge, and Simon L. Peyton Jones. 1996. GUM: A portable parallel implementation of Haskell. In *Proceedings of the ACM SIGPLAN'96 Conference on Programming Language Design and Implementation (PLDI), Philadephia, Pennsylvania, USA, May 21-24, 1996*, Charles N. Fischer (Ed.). ACM, 79–88. doi:10.1145/231379.231392

[12] D. A. Turner. 1979. A New Implementation Technique for Applicative Languages. *Softw. Pract. Exp.* 9, 1 (1979), 31–49. doi:10.1002/SPE.4380090105

Received 2026-05-22; accepted 2026-06-30